A FIELD GUIDE

Rust Lifetimes

A Field Guide to Regions, Elision, Variance, 'static and Higher-Ranked Bounds

Luciano Di Cocco

Rust 1.99 · edition 2024

First edition · October 2026

Rust Lifetimes — A Field Guide to Regions, Elision, Variance, 'static and Higher-Ranked Bounds

Copyright © 2026 Luciano Di Cocco

First edition, October 2026. This book targets **Rust 1.99** (edition 2024). The code uses only the standard library.

Every program shown in this book is part of the accompanying `code/` directory, a Cargo workspace. Code excerpts are included verbatim from those files when the book is built. Program outputs and compiler diagnostics were captured from real runs of the pinned compiler and are checked again every time the examples are verified: a program that the book says compiles must compile without warnings and print exactly the output shown, and a program that the book says is rejected must fail with exactly the diagnostics shown.

The text of this book and its diagrams are licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License (CC BY-NC-SA 4.0). The code, including the excerpts shown in the book, is available under the MIT license or the Apache License, Version 2.0, at your option.

Rust is a trademark of the Rust Foundation. This book is an independent work and is not affiliated with or endorsed by the Rust project.

Contents

Preface ix

Who this book is for ix

How this book is verified ix

How to read the listings x

How the book is organized xi

Acknowledgments xi

Part I – The Model

1 Regions, Not Durations 2

1.1 A lifetime is not a run-time thing 2

1.2 Regions: sets of program points 4

1.3 Non-lexical lifetimes: a borrow ends at its last use 5

1.4 Shared and mutable borrows: aliasing XOR mutation 7

1.5 Lifetimes in branches and loops 10

1.6 Annotations describe; they never extend 12

1.7 Reading a borrow-checker diagnostic 16

1.8 Notation 17

1.9 Common confusions 20

1.10 Summary 20

1.11 Exercises 21

2 Whose Borrow Is It? Lifetimes in Types and Methods 22

2.1 A struct that holds a reference 22

2.2 Two lifetimes in every method 24

2.3 Iterators whose items borrow from the data 29

2.4 Enums with references 31

2.5 How many lifetime parameters? 33

2.6 Traits with a lifetime parameter 35

2.7 The `&'a mut self` trap 37

2.8 Common confusions 39

2.9 Summary 40

2.10 Exercises 40

3 Lifetime Elision 42

3.1 Elision is a rewrite of the signature 42

3.2 The three rules 43
3.3 You write, the compiler reads 52
3.4 When elision fails: E0106 55
3.5 Why a struct field cannot be elided 57
3.6 Candidate rules that would not work 60
3.7 Common confusions 65
3.8 Summary 65
3.9 Exercises 66

4 Subtyping and Variance 67
4.1 The outlives order 67
4.2 Subtyping: a longer borrow is a shorter one 68
4.3 Variance: how subtyping passes through types 69
4.4 Covariance in action 71
4.5 Invariance: the safety net for writes 72
4.6 Contravariance: function arguments 74
4.7 The variance of your own types 76
4.8 Invariance in practice: `&'a mut T<'a>` 78
4.9 Variance in diagnostics 80
4.10 For the type-theoretically inclined 80
4.11 Common confusions 81
4.12 Summary 82
4.13 Exercises 82

5 Borrows in Practice: Reborrows, Temporaries and the Limits of NLL 84
5.1 Reborrowing 85
5.2 Two-phase borrows 88
5.3 Splitting borrows 90
5.4 Moves while borrowed, and borrows that cannot leave 93
5.5 Temporaries 95
5.6 Drop order and the drop check 102
5.7 The limits of NLL 104
5.8 Common confusions 107
5.9 Summary 108
5.10 Exercises 108

Part II – Lifetimes in Signatures

6 The Anonymous Lifetime `'_` 111
6.1 `'_` is not a name 111
6.2 Job 1: making hidden lifetime parameters visible 113
6.3 Job 2: `impl` headers 116
6.4 Job 3: letting trait objects borrow 119

6.5 `'_` inside function bodies 122
6.6 What `'_` cannot do 122
6.7 Checklist: `'a`, `'_`, or nothing? 124
6.8 Common confusions 125
6.9 Summary 126
6.10 Exercises 126

7 The `'static` Lifetime 128
7.1 Two meanings of `'static` 128
7.2 Where `&'static` references come from 130
7.3 `T: 'static` does not mean "lives forever" 133
7.4 Why APIs ask for `'static` 136
7.5 Myths and anti-patterns 143
7.6 A decision ladder 145
7.7 Common confusions 145
7.8 Summary 146
7.9 Exercises 146

8 Trait Objects, `impl Trait` and Captured Lifetimes 148
8.1 The lifetime bound of a trait object 149
8.2 `Box<dyn Error>` and `?` 155
8.3 Unsizing coercions: where the bound is checked 157
8.4 Trait objects in structs 159
8.5 Return-position `impl Trait`: what an opaque type captures 161
8.6 Over-capturing and precise capturing with `use<..>` 162
8.7 `impl Trait + 'a` is not `use<'a>` 164
8.8 Edition 2021: E0700 and the outlives trick 166
8.9 `impl Trait` in traits and `async fn` 167
8.10 `impl Trait` in argument position 169
8.11 Common confusions 171
8.12 Summary 172
8.13 Exercises 173

9 Higher-Ranked Trait Bounds 174
9.1 Who picks the lifetime? 175
9.2 Where the compiler writes `for<'a>` for you 177
9.3 When you have to write it yourself 178
9.4 Higher-ranked types are more general 183
9.5 Closures that return references 184
9.6 Async callbacks that borrow 186
9.7 Early-bound and late-bound lifetimes 188
9.8 The limits of `for<..>` 189

9.9 Reading higher-ranked errors 191
9.10 Common confusions 191
9.11 Summary 192
9.12 Exercises 192

Part III – Going Further

10 Generic Associated Types and Lending Iterators 195
10.1 The problem: items that borrow from the iterator 196
10.2 Generic associated types 198
10.3 Generic code over a lending iterator 205
10.4 Other uses of GATs 211
10.5 Common confusions 213
10.6 Summary 214
10.7 Exercises 214

11 Lifetimes in Async Code 216
11.1 What an `async fn` returns 217
11.2 Running futures with the standard library 220
11.3 Spawning: why executors require `'static` 221
11.4 Why scoped tasks are hard 225
11.5 Async callbacks: `AsyncFn` and async closures 227
11.6 Self-referential futures and `Pin` 229
11.7 `async fn` in traits 231
11.8 Common confusions 233
11.9 Summary 234
11.10 Exercises 234

12 Interior Mutability, Guards and Lifetimes 236
12.1 Two kinds of checking 237
12.2 `RefCell` and its guards 237
12.3 Guards and temporaries 242
12.4 `Mutex`, `RwLock` and their guards 245
12.5 `Cell`, `OnceCell` and invariance 248
12.6 `thread_local!` and closure-scoped borrows 251
12.7 Common confusions 253
12.8 Summary 253
12.9 Exercises 254

13 Lifetimes and `unsafe` Code 255
13.1 Raw pointers carry no lifetime 256
13.2 `PhantomData` 259
13.3 Self-referential structs 264
13.4 The drop check and `#[may_dangle]` 267

13.5 Leaking is safe 269
13.6 Checking `unsafe` code with Miri 272
13.7 Common confusions 273
13.8 Summary 273
13.9 Exercises 274

14 Designing APIs with Lifetimes 275
14.1 Own or borrow? 275
14.2 Alternatives to a lifetime parameter 278
14.3 Zero-copy parsing: borrowed tokens, owned tokens 281
14.4 How many lifetime parameters does a struct need? 284
14.5 The `&'a mut T<'a>` antipattern: borrowed forever 286
14.6 Callbacks 289
14.7 Iterators that return references 291
14.8 Errors that borrow and errors that own 293
14.9 Builders 295
14.10 A refactoring walkthrough: from lifetime soup to a clear design 297
14.11 Common confusions 302
14.12 Summary 303
14.13 Exercises 304

Appendices

A Setting Up and Verifying the Examples 306
A.1 Installing the toolchain 306
A.2 Running the examples 307
A.3 Compiling the programs that must be rejected 307
A.4 Verifying everything 307
A.5 Trying a newer (or older) compiler 308
A.6 Nightly-only experiments 308
A.7 Building the book 309
A.8 Summary 309

B Error Catalogue 310
B.1 Errors with a code 310
B.2 Diagnostics without a code 315
B.3 Lints 318
B.4 Run-time panics 318

C Quick Reference 319
C.1 I want to… 320
C.2 What the compiler writes for you 322
C.3 Rules of thumb 322

D Glossary **323**
D.1 Symbols 323
D.2 A–D 323
D.3 E–L 325
D.4 N–R 326
D.5 S–Z 327
E Further Reading **329**
E.1 Official documentation 329
E.2 The compiler 330
E.3 RFCs 330
E.4 The Rust blog 330
E.5 Unstable features mentioned in the book 331
E.6 Beyond the official documentation 331

Preface

Most Rust programmers learn lifetimes the same way: by fixing errors. The compiler says that something "does not live long enough", you add an `&`, a `.clone()` or a `'a`, and the error goes away. After a while the fixes become reflexes, but the reasons stay vague, and every so often an error appears that no reflex fixes: "implementation of `Fn` is not general enough", "returning this value requires that `'1` must outlive `'static`", "hidden type captures lifetime that does not appear in bounds". This book is about the reasons.

It starts from a small, precise model: a lifetime is a *region*, a set of points in a function body, and a signature is a *contract* that the body and every caller are checked against. Almost everything else follows from those two ideas plus a handful of defaults that the compiler applies when you leave a lifetime out. The book makes every one of those defaults visible: what elision writes for you, what `'_` means in each position, why `Box<dyn Trait>` means `Box<dyn Trait + 'static>`, what an `impl Trait` return type captures, and why `F: Fn(&str)` is secretly a higher-ranked bound. Then it applies the model to the places where lifetimes are hardest to see: generic associated types, async code, interior mutability, `unsafe` code and API design.

Who this book is for

You write Rust: ownership and borrowing, structs and enums, traits and generics, closures and iterators are familiar, and you have met lifetime errors that you could fix without being able to explain. You do not need to know anything about type theory or the internals of the compiler. When the book uses a concept such as variance, a higher-ranked bound or an early-bound lifetime, it explains it from the beginning, with programs.

How this book is verified

Every program in this book is a file in the accompanying `code/` directory, a Cargo workspace that uses only the standard library, and every listing is included from those files when the book is built. The compiler is pinned to **Rust 1.99.0** with **edition 2024**.

The book makes two kinds of claims about programs, and both are checked mechanically:

- "**This compiles.**" The program is a runnable example in `code/examples/`. It must build without a single warning and run to completion, and its output must match the output shown in the book, which was captured from a real run.
- "**This is rejected.**" The program is in `code/compile_fail/`. It must fail with the error codes or messages it declares, and the diagnostics shown in the book are the compiler's own output,

captured verbatim. Nothing in a diagnostic is retyped or edited, with two exceptions: the trailing "aborting due to ..." lines are removed, and paths into your local copy of the standard library are written in the machine-independent form `/rustc/<commit>/library/...` that rustc uses when that copy is not installed.

A script, `tools/run_examples.py`, recompiles every program and compares every output and diagnostic with the snapshots in the book. Appendix A explains how to run it, and how to use it to see what changes when you try a newer compiler.

This matters more for lifetimes than for most topics. The rules have changed over time (non-lexical lifetimes in 2018, new capture rules for `impl Trait` in edition 2024, new lints and elision fixes in recent releases), and diagnostics improve with nearly every release. Much of what you will find online describes an older compiler or an older edition. Everything in this book describes Rust 1.99 with edition 2024; when an older edition behaves differently and you are likely to meet code written for it, the text says so and shows both.

Some features are not stable yet: closure lifetime binders, Polonius (the next borrow checker), bounds on higher-ranked lifetimes. The book describes them where they matter, says that they are unstable, and gives the date of the nightly compiler that was checked. Their status will change; check it before you rely on it.

How to read the listings

Each listing is introduced in the text, and the text says whether it compiles. Listings come in three kinds:

Fence	Comes from	Means
`rust` from `code/examples/`	a runnable example	compiles without warnings; its output follows in a plain block
`rust` from `code/compile_fail/`	a program that must be rejected	does not compile; the diagnostics follow
`rust` marked as not compiled	written in the text	an expansion ("the compiler reads this as ..."), a signature from another crate, or unstable syntax

The first time a listing appears, the text names its file. Listings often show only part of a file (the part that matters); the line numbers in a diagnostic always refer to the whole file, so they may not match the excerpt above them. Open the file when you want the full context, and run it:

```
cd code
cargo run --example ch09_callee_picks
```

Diagnostics are part of the lesson. The book reads them with you, label by label, because rustc's lifetime diagnostics are precise once you know their vocabulary: `'1` and `'2` are names the compiler invents for anonymous lifetimes, "for any lifetime" marks a requirement and "for some specific lifetime" marks what you have, and the arrows point at the exact places where a borrow starts, is used and must end.

How the book is organized

Part I, The Model (Chapters 1–5), builds the foundations: regions and non-lexical lifetimes; whose borrow a method returns; the elision rules and why the compiler refuses to guess; subtyping and variance; and the everyday mechanics of reborrows, temporaries, drops and the limits of today's borrow checker.

Part II, Lifetimes in Signatures (Chapters 6–9), covers the lifetimes that appear in real APIs: the anonymous lifetime `'_`; the two meanings of `'static`; trait objects and `impl Trait`, whose lifetime defaults and captures changed in edition 2024; and higher-ranked trait bounds, `for<'a>`.

Part III, Going Further (Chapters 10–14), applies the model to generic associated types and lending iterators, async code, interior mutability and its guards, `unsafe` code, and finally to the design of APIs: when to borrow, when to own, and how many lifetime parameters a type needs.

The **appendices** are for reference: setting up and verifying the examples, a catalogue of every lifetime-related error in the book, a quick reference, a glossary and a list of further reading.

Three threads run through all of it. *The signature is the contract*: a body is checked against its signature, and callers are checked against the signature and nothing else. *Who picks the lifetime?*: a lifetime parameter is picked by the caller, a higher-ranked lifetime by the callee, at each use. And *what the compiler writes for you*: every default is shown expanded, so that you can see the lifetime that was there all along.

Every chapter opens with a list of what you will learn and closes with a summary and exercises. The exercises are meant to be done with the compiler at hand: predict, then check.

Acknowledgments

This book grew out of an earlier, much shorter field guide on lifetimes, the anonymous lifetime and `'static`, written for rustc 1.75 and edition 2021. Its material was reviewed against Rust 1.99, corrected where the language or the compiler had moved on, and extended into the chapters that follow.

The book relies on the work of the people who document Rust: *The Rust Programming Language*, *The Rust Reference*, *The Rustonomicon*, the *rustc development guide* and the Rust blog. Where it describes a rule, it checks the rule against the compiler, and where the documentation and the compiler disagree, it says so. Appendix E lists the sources.

PART I

The Model

Part I builds the model that every later chapter relies on. A lifetime is a region of a function body, not a span of time; a signature is a contract that the body and every caller are checked against; and the compiler fills in the lifetimes you leave out according to a small set of fixed rules. Chapter 1 introduces regions and non-lexical lifetimes, Chapter 2 asks whose borrow a method returns, Chapter 3 takes the elision rules apart, Chapter 4 explains how lifetimes of different lengths fit together through subtyping and variance, and Chapter 5 looks at the everyday mechanics that the model has to account for: reborrows, temporaries, drops, and the few places where today's borrow checker is stricter than it needs to be.

CHAPTER 1

Regions, Not Durations

The word *lifetime* suggests a duration: how long something lives, measured on a clock that starts when a value is created and stops when it is destroyed. That picture is close enough to be useful and wrong enough to mislead. It cannot explain why a reference to a vector stops blocking a `push` a few lines before the end of the block, why the same borrow can be fine on one branch of an `if` and an error on the other, or why writing `'static` on a reference to a local variable produces an error instead of a reference that lives forever.

The borrow checker does not think in durations. For the compiler, a lifetime is a **region**: a set of places in the code of a function where a reference must be valid. A region has no length in seconds and no existence at run time. It is computed from the code, checked, and thrown away before the program is built. Once you see lifetimes as regions, most borrow-checker errors read as what they are: two sets of program points that overlap where they must not, or a set that must contain a point it cannot reach.

This chapter builds that model with small programs, each one either compiled and run or rejected by the compiler, and reads the diagnostics with you. Everything else in the book is built on it.

In this chapter you will learn:

- what a lifetime is to the borrow checker: a region of a function body, never a run-time value, erased before code generation;
- the difference between the scope of a value, the scope of a variable that holds a reference, and the region of a borrow;
- how non-lexical lifetimes end a borrow at its last use, path by path, through branches and loops;
- the rule "aliasing XOR mutation" behind errors E0502, E0499 and E0506;
- why lifetime annotations describe relationships and never extend anything, and why the signature is the contract (E0597, "lifetime may not live long enough");
- how to read a borrow-checker diagnostic label by label, and turn it into a fix;
- the lifetime notation used in the rest of the book, including implied bounds.

1.1 A lifetime is not a run-time thing

Start with what a lifetime is not. A reference in a running Rust program is a pointer (plus a length, for a slice or a `str`). There is no field that records how long it may be used, no counter, no check at

the moment you dereference it. All of that work happens at compile time, and its result is a yes or a no: the program is accepted, or it is rejected with a diagnostic.

You can see the erasure from inside a program. The full program is in `code/examples/ch01_erased.rs`:

```
fn first_word(text: &str) -> &str {
    text.split(' ').next().unwrap_or("")
}

fn main() {
    let literal: &'static str = "regions not durations";
    let owned = String::from("borrowed from a local");
    let local: &str = &owned;

    // One function, one piece of machine code, both kinds of borrow.
    println!("{} / {}", first_word(literal), first_word(local));

    // The name of the type that code generation sees has no lifetime in it...
    let name = type_name::<&'static str>();
    println!("type_name::<&'static str>() = {name}");
    // ...and the two kinds of reference have the same size and layout.
    assert_eq!(size_of::<&'static str>(), size_of::<&str>());
}
```

```
regions / borrowed
type_name::<&'static str>() = &str
```

`first_word` is called once with a string literal, of type `&'static str`, and once with a borrow of a local `String`, whose region ends with `main`. The compiler generates one piece of machine code for `first_word` and uses it for both calls: nothing in the generated code depends on the lifetime. The two kinds of reference have the same size and layout (the `assert_eq!` passes), and when we ask for the name of the type `&'static str`, the compiler answers `&str`: by the time code is generated, the `'static` is gone. (The documentation of `std::any::type_name` warns that its exact text is a best-effort description that may change between compiler versions, so take that line as an illustration rather than a guarantee. The point does not depend on it.)

Three consequences follow, and they shape the rest of the book:

1. **A lifetime cannot be inspected or tested at run time.** There is no `if 'a == 'static`. Anything that needs to know about lifetimes while the program runs must be expressed in types instead. This is why `std::any::TypeId` works only for `'static` types (Chapter 7).
2. **Lifetimes cost nothing.** A program whose lifetimes are written out in full and the same program with every lifetime elided compile to the same code.
3. **The borrow checker must be conservative.** Since nothing is checked while the program runs, the compiler rejects every program it cannot prove safe, including some that would in fact run correctly. Chapter 5 shows the best-known example and the work under way to accept it. When you need checks that depend on run-time behavior, you move them to run time explicitly, with types such as `RefCell` (Chapter 12).

1.2 Regions: sets of program points

If a lifetime is not a duration, what is it? The model that rustc's borrow checker implements comes from RFC 2094, which introduced non-lexical lifetimes and states it in one line: "we will consider lifetimes as a set of points in the control-flow graph" (https://rust-lang.github.io/rfcs/2094-nll.html).

The **control-flow graph** of a function is its body seen as a graph: each statement is a point, and an edge goes from one point to every point that can run next. A straight sequence of statements is a chain; an `if` splits the chain in two and joins it again; a loop adds an edge back to its start. A **region** is a set of points of that graph, and a reference of type `&'a T` comes with a promise:

> At every point in the region `'a`, the value the reference points to is still there (not dropped, not moved), and nobody accesses it in a way that conflicts with the reference.

What "conflicts" means depends on the kind of reference, and we will come back to it below. For now, notice the shape of the promise: it is a statement about a set of points, not about an interval of time. Rustc actually computes regions on a simplified form of the function body (called MIR, for mid-level intermediate representation) whose points are finer than source lines. In this book we reason with lines and statements, which is precise enough for almost every error you will meet; Chapter 5 shows one of the few places where the finer points matter (two-phase borrows).

Three different extents

Three things in a program are easy to confuse because all three are informally called "how long something lives". This program, in `code/examples/ch01_scope_vs_region.rs`, compiles:

```
fn main() {
    let config = String::from("port=8080"); // `config` comes into scope
    let key = &config[..4]; // the borrow is created
    println!("key: {key}"); // last use of `key`: the borrow's region ends here

    let moved = config; // fine: no borrow of `config` is live any more
    println!("moved: {moved}");
} // `key` and `moved` go out of scope here
```

```
key: port
moved: port=8080
```

Look at the three extents involved:

What	From	To
the value `config`	`let config = ...`	`let moved = config` (moved out)
the variable `key`	`let key = ...`	the closing brace of `main`
the borrow `&config[..4]`	the borrow expression	the `println!`, its last use

The **scope of a value** is where it exists: from its creation until it is dropped or moved away. The **scope of a variable** is the block it is declared in, a purely syntactic notion. The **region of a borrow** is where the reference created by the borrow may still be used. Only the third is a lifetime in the borrow

checker's sense, and it is the smallest of the three. The variable `key` is still in scope when `config` is moved, but the borrow it holds is dead, because nothing uses `key` again. So moving `config` is fine.

This is also why "the lifetime of a reference" and "the lifetime of the value it points to" are different things. The region of a borrow must fit inside the scope of the value it borrows, since a reference must never outlast its referent. But it is usually much smaller, and the compiler keeps it as small as the uses of the reference allow. If you used `key` after the move, the region would have to reach past the point where `config` is moved away, which is impossible. That error (E0505, "cannot move out of `config` because it is borrowed") is covered with the other move-related errors in Chapter 5.

1.3 Non-lexical lifetimes: a borrow ends at its last use

The example that made non-lexical lifetimes famous is a vector, a reference to its first element, and a `push`. This version compiles (`code/examples/ch01_nll.rs`):

```
fn main() {
    let mut v = vec![1, 2, 3];
    let first = &v[0];
    println!("{first}"); // last use of `first`: the borrow ends here
    v.push(4); // so this mutable borrow does not overlap it
    println!("{v:?}");
}
```

```
1
[1, 2, 3, 4]
```

The borrow `&v[0]` is created on the second line and used for the last time by the first `println!`. Its region is those two points and nothing more. The `push` on the next line needs a mutable borrow of `v`, which would conflict with a live shared borrow; but the shared borrow is no longer live, so there is no conflict.

#	statement	lexical scope of first (old rule)	NLL region of the borrow &v[0]	access to v
1	let mut v = vec![1, 2, 3];			
2	let first = &v[0];	live	created	shared
3	println!("{first}");	live	last use	
4	v.push(4);	overlap: rejected	no overlap: accepted	mutable
5	println!("{v:?}");	live		shared

The borrow `&v[0]` is live only until its last use, so `v.push(4)` does not overlap it; a rule based on the lexical scope of `first` would keep the borrow alive to the end of the block and reject the program.

The figure compares the region with the scope of the variable `first`. Until the 2018 edition, Rust used the scope: a borrow stored in a variable was live until the end of the block that declared the variable, and this program was rejected. Non-lexical lifetimes, NLL for short, shipped in Rust 1.31 together with the 2018 edition (https://blog.rust-lang.org/2018/12/06/Rust-1.31-and-rust-2018/), and the borrow checker of every edition works this way today. "Non-lexical" means exactly that the region is computed from the uses of the reference, not from the blocks of the source text.

Move the last use of `first` after the `push`, and the region must stretch across it. The compiler rejects this version (`code/compile_fail/ch01_nll_conflict.rs`):

```
pub fn caller() {
    let mut v = vec![1, 2, 3];
    let first = &v[0];
    v.push(4); // needs `&mut v` while `first` is still live
    println!("{first}"); // this later use keeps the shared borrow alive
}
```

```
error[E0502]: cannot borrow `v` as mutable because it is also borrowed as immutable
  --> compile_fail/ch01_nll_conflict.rs:9:5
   |
 8 |     let first = &v[0];
   |                  - immutable borrow occurs here
 9 |     v.push(4); // needs `&mut v` while `first` is still live
   |     ^^^^^^^^^ mutable borrow occurs here
10 |     println!("{first}"); // this later use keeps the shared borrow alive
   |                ----- immutable borrow later used here
```

Read the diagnostic from top to bottom, as a short story told in three labels:

- `immutable borrow occurs here`, under `v` on line 8: the shared borrow starts here. Its region begins at this point.
- `mutable borrow occurs here`, under the whole `v.push(4)` on line 9: this is the conflicting access, and the primary label of the error (the one underlined with `^^^`). `push` takes `&mut self`, so the call borrows `v` mutably.
- `immutable borrow later used here`, under `first` on line 10: the reason the shared borrow is still live on line 9. Because `first` is used here, the region of the shared borrow must contain every point between line 8 and line 10, including the `push`.

The error is not a quirk of the checker. `push` may need more room than the vector has, in which case it allocates a new buffer, moves the elements there and frees the old one. `first` would then point into freed memory, and the `println!` would read it. The borrow checker cannot know whether this particular `push` reallocates, so it rejects every program in which a reference into the vector is live across a call that may.

Liveness follows the value, not the variable

"Last use" needs one refinement. A borrow is live at a point if the reference it produced, *or any reference derived from it*, may be used later. Copying the reference into another variable or storing it in a data structure moves the liveness along with it. In this program (`code/compile_fail/ch01_nll_derived.rs`), the variable `first` is never used after the line that builds `shortlist`:

```rust
pub fn caller() {
    let mut words = vec![String::from("regions")];
    let first = &words[0]; // last use of the variable `first`...
    let shortlist = vec![first]; // ...but the borrow now lives in `shortlist`
    words.push(String::from("durations"));
    println!("{shortlist:?}");
}
```

```
error[E0502]: cannot borrow `words` as mutable because it is also borrowed as immutable
  --> compile_fail/ch01_nll_derived.rs:10:5
   |
 8 |     let first = &words[0]; // last use of the variable `first`...
   |                  ----- immutable borrow occurs here
 9 |     let shortlist = vec![first]; // ...but the borrow now lives in `shortlist`
10 |     words.push(String::from("durations"));
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
11 |     println!("{shortlist:?}");
   |                ---------- immutable borrow later used here
```

The compiler still reports the conflict, and its third label points at the use of `shortlist`, not of `first`. The type of `shortlist` is `Vec<&String>`, and the reference inside it is the very borrow created on line 8. In the language of regions, storing `first` into the vector creates a constraint: the region of the borrow must contain every point where the reference in `shortlist` may be used.

How the compiler finds a region

You never write the regions of borrows inside a function body; the compiler infers them. A simplified description of what it does is enough to predict its answers:

1. **Liveness.** For each reference, it finds the points where the reference (or a copy of it) may still be used later, along some path of the control-flow graph. The region of the reference must contain all of them.
2. **Flow.** When a reference is copied, assigned, stored in a structure or passed to a function, the region of the source must contain the region of the destination. This is how the use of `shortlist` above extends the region of the borrow made on line 8.
3. **Smallest solution.** It computes the smallest regions that satisfy all these constraints.
4. **Check.** It walks the function body and, at every access to a variable (a read, a write, a move, a borrow, a drop at the end of a scope), checks that no live borrow conflicts with it. Every conflict is an error.

The borrow conflicts in this chapter are all reported in step 4, but their cause is always in steps 1 and 2: some later use, direct or derived, made a region bigger than the program allows. (The other kind of error you will meet below, "lifetime may not live long enough", comes from step 3: the smallest solution would need a relation between two lifetimes that nothing provides.)

1.4 Shared and mutable borrows: aliasing XOR mutation

Step 4 needs a definition of "conflict". Rust's rule is short: at every point of the program, a value may have **any number of shared borrows, or exactly one mutable borrow**, but not both. It is often summarized as *aliasing XOR mutation*: a value can be reachable through several paths (aliased), or it

can be changed (mutated), but never both at the same point. The owner of the value counts as one of the paths: while a mutable borrow is live, even the variable that owns the value cannot be used.

Since regions are sets of points, the rule allows a lot. This program (`code/examples/ch01_aliasing.rs`) compiles, even though it creates two mutable borrows of the same string:

```
fn main() {
    let mut log = String::from("== report ==");

    // Any number of shared borrows may be live at the same time.
    let whole = &log;
    let title = &log[3..9];
    println!("{whole} / {title}");

    // One mutable borrow at a time: each one ends before the next begins.
    let header = &mut log;
    header.push_str("\nstatus: ");
    let body = &mut log;
    body.push_str("all good");

    println!("{log}");
}
```

```
== report == / report
== report ==
status: all good
```

`whole` and `title` are two shared borrows, live at the same time, which is always allowed. `header` and `body` are two mutable borrows, but `header` is last used before `body` is created, so their regions do not overlap. Finally `println!("{log}")` reads `log` after both have ended.

In `code/compile_fail/ch01_two_mutable.rs` the two mutable regions overlap, and the compiler rejects the program with E0499:

```
pub fn caller() {
    let mut log = String::new();
    let header = &mut log;
    let body = &mut log; // `header` is still live: it is used below
    header.push_str("== report ==\n");
    body.push_str("all good\n");
}
```

```
error[E0499]: cannot borrow `log` as mutable more than once at a time
 --> compile_fail/ch01_two_mutable.rs:8:16
  |
7 |     let header = &mut log;
  |                  -------- first mutable borrow occurs here
8 |     let body = &mut log; // `header` is still live: it is used below
  |                ^^^^^^^^ second mutable borrow occurs here
9 |     header.push_str("== report ==\n");
  |     ------ first borrow later used here
```

The labels follow the same pattern as before: where the first borrow starts, where the conflicting second borrow happens (the primary label), and the later use that keeps the first borrow live (`header.push_str` on line 9).

The rule is symmetric. While a mutable borrow is live, the value cannot even be read through its owner (`code/compile_fail/ch01_mutable_then_shared.rs`):

```rust
pub fn caller() {
    let mut log = String::from("start");
    let writer = &mut log;
    println!("{log}"); // reading `log` needs a shared borrow
    writer.push_str(" end");
}
```

```
error[E0502]: cannot borrow `log` as immutable because it is also borrowed as mutable
  --> compile_fail/ch01_mutable_then_shared.rs:9:16
   |
 8 |     let writer = &mut log;
   |                  -------- mutable borrow occurs here
 9 |     println!("{log}"); // reading `log` needs a shared borrow
   |               ^^^ immutable borrow occurs here
10 |     writer.push_str(" end");
   |     ------ mutable borrow later used here
```

Reading `log` creates a shared borrow (the `println!` macro borrows its arguments), and the mutable borrow held by `writer` is live, because `writer` is used on the next line. The error code is again E0502; only the order of the two borrows changed, and the headline says so: "cannot borrow `log` as *immutable* because it is also borrowed as *mutable*".

Borrowing is not the only access that can conflict. Assigning a new value to a variable drops the old value, so it conflicts with any live borrow of it (`code/compile_fail/ch01_assign_borrowed.rs`):

```rust
pub fn caller() {
    let mut title = String::from("draft");
    let shown = &title;
    title = String::from("final"); // would drop the string `shown` points to
    println!("{shown}");
}
```

```
error[E0506]: cannot assign to `title` because it is borrowed
  --> compile_fail/ch01_assign_borrowed.rs:9:5
   |
 8 |     let shown = &title;
   |                 ------ `title` is borrowed here
 9 |     title = String::from("final"); // would drop the string `shown` points to
   |     ^^^^^ `title` is assigned to here but it was already borrowed
10 |     println!("{shown}");
   |                ----- borrow later used here
```

If the assignment were allowed, the old string `"draft"` would be freed while `shown` still pointed to it. Here is the full set of conflicts this chapter shows, with the code rustc uses for each:

A live borrow…	…conflicts with	Code
shared (`&v`)	a mutable borrow of `v`	E0502
mutable (`&mut v`)	a shared borrow of `v` , including a read	E0502
mutable (`&mut v`)	a second mutable borrow of `v`	E0499
shared or mutable	an assignment to `v`	E0506
shared or mutable	a move or drop of `v`	E0505, E0597

A shared borrow never conflicts with another shared borrow. The last row is about the value disappearing while borrowed: E0597 appears later in this chapter, and E0505 in Chapter 5, together

with two refinements of the rule that make common code compile (two-phase borrows, which accept `v.push(v.len())`, and reborrows).

1.5 Lifetimes in branches and loops

Because a region is a set of points in a graph, and not an interval of lines, a borrow can be live on one path through a function and dead on another. This function (`code/examples/ch01_branches.rs`) borrows the first word, mutates the vector on one branch of an `if`, and uses the borrow on the other:

```
fn describe_or_extend(words: &mut Vec<String>) {
    let first = &words[0];
    if first.len() <= 5 {
        words.push(String::from("durations")); // `first` is dead on this path...
    } else {
        println!("first word is long: {first}"); // ...and used on this one
    }
}

fn main() {
    let mut short = vec![String::from("not")];
    let mut long = vec![String::from("regions")];
    describe_or_extend(&mut short);
    describe_or_extend(&mut long);
    println!("{short:?} {long:?}");
}
```

```
first word is long: regions
["not", "durations"] ["regions"]
```

The condition uses `first`, so the borrow is live there. On the `then` branch, nothing uses `first` again: from the start of that branch, no path leads to a use. On the `else` branch, `first` is printed, so the borrow is live there. The region of the borrow therefore contains the condition and the `else` branch, and not a single point of the `then` branch, where `push` is free to borrow `words` mutably. A rule based on intervals of source text, "from the borrow to its last use", would cover every line from `let first` down to the `println!`, including the `push` written between them, and would reject this function.

Add one use of `first` after the `if`, and the borrow becomes live on both branches, because both lead to that use (`code/compile_fail/ch01_branch_conflict.rs`):

```
pub fn describe_or_extend(words: &mut Vec<String>, verbose: bool) {
    let first = &words[0];
    if verbose {
        words.push(String::from("durations"));
    }
    println!("first word: {first}");
}
```

```
error[E0502]: cannot borrow `*words` as mutable because it is also borrowed as immutable
  --> compile_fail/ch01_branch_conflict.rs:9:9
   |
 7 |     let first = &words[0];
   |                 ----- immutable borrow occurs here
 8 |     if verbose {
 9 |         words.push(String::from("durations"));
   |         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
10 |     }
```

```
11 |     println!("first word: {first}");
   |                            ----- immutable borrow later used here
```

The diagnostic names `*words`, the vector that `words` points to, because `words` is itself a reference (`&mut Vec<String>`): the borrows are of the vector behind it. The rest reads as before.

Loops add a back edge to the graph, and with it the possibility that a borrow made in one iteration is live in the next. This program (`code/examples/ch01_loop.rs`) keeps a reference to the word appended in the previous iteration, and compiles:

```
fn main() {
    let mut transcript = String::new();
    let mut previous: &str = "(start)";
    for word in ["regions", "not", "durations"] {
        println!("{previous} -> {word}"); // last use of the previous borrow
        if !transcript.is_empty() {
            transcript.push(' '); // `previous` is dead here: it is overwritten below
        }
        let start = transcript.len();
        transcript.push_str(word);
        previous = &transcript[start..]; // a new borrow of `transcript`
    }
    println!("last word: {previous}; transcript: {transcript}");
}
```

```
(start) -> regions
regions -> not
not -> durations
last word: durations; transcript: regions not durations
```

Follow the borrow `&transcript[start..]` made at the end of the first iteration. Through `previous`, it is live at the start of the second iteration, where the `println!` uses it. After that use, it is dead: the next thing that happens to `previous` is that a new borrow overwrites it, so its old value can never be read again. The region of the first borrow therefore covers the end of the first iteration and the beginning of the second, up to the `println!`, and does not contain the `push` and `push_str` that come after. Each iteration's mutation of `transcript` falls in the gap between the last use of one borrow and the creation of the next. The borrow made in the last iteration is different only in where its uses are: it stays live after the loop ends, because the final `println!` uses `previous`.

Now move the `println!` after the `push_str`, and the previous borrow must survive the mutation (`code/compile_fail/ch01_loop_conflict.rs`):

```
pub fn caller() {
    let mut transcript = String::new();
    let mut previous: &str = "(start)";
    for word in ["regions", "not", "durations"] {
        let start = transcript.len();
        transcript.push_str(word);
        println!("{previous} -> {word}"); // uses the borrow made last time round
        previous = &transcript[start..];
    }
}
```

```
error[E0502]: cannot borrow `transcript` as mutable because it is also borrowed as immutable
  --> compile_fail/ch01_loop_conflict.rs:11:9
   |
11 |         transcript.push_str(word);
```

```
   |         ^^^^^^^^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
12 |         println!("{previous} -> {word}"); // uses the borrow made last time round
   |                   -------- immutable borrow later used here
13 |         previous = &transcript[start..];
   |                     ---------- immutable borrow occurs here
```

This diagnostic looks strange at first, because its labels are out of source order: the shared borrow "occurs" on line 13, *after* the mutable borrow on line 11 that it conflicts with. Read it with the loop in mind. The shared borrow made on line 13 in one iteration is still live when control comes back around to line 11 in the next iteration, because line 12 uses it after that. Whenever the label that starts a borrow appears below the conflict, look for a loop.

There is one pattern involving branches that NLL, as implemented in stable rustc, still rejects even though it is safe: returning a borrow from a function on one path and mutating the borrowed value on another. Chapter 5 covers it as *NLL problem case #3*, together with Polonius, the borrow checker being developed to accept it.

1.6 Annotations describe; they never extend

So far every region was inferred. Lifetime *names*, such as `'a`, appear where the compiler cannot infer anything because it is looking at one function at a time: in signatures and in type definitions. The classic example is a function that returns one of two string slices (`code/examples/ch01_longest.rs`):

```rust
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() >= y.len() { x } else { y }
}
```

The signature declares one lifetime parameter, `'a`, and uses it three times. Read it as a contract between `longest` and its callers:

- the caller picks a region for `'a`, at each call;
- the caller must pass two references that are both valid throughout `'a`;
- in return, it gets a reference that is valid throughout `'a`, and no longer.

Since both arguments must be valid for all of `'a`, the region that a caller can pick is at most the part of the program where *both* are valid: the result is usable only where both inputs are. Nothing here makes either string live longer. The annotation describes a relationship between the three references, and the compiler checks that every call respects it.

A call that uses the result while both inputs are alive compiles. Here is the first of two such callers in the same file:

```rust
fn use_inside() {
    let outer = String::from("a longer string");
    let result;
    {
        let inner = String::from("short");
        result = longest(outer.as_str(), inner.as_str());
        println!("{result}"); // fine: `inner` is still alive here
    }
}
```

```
a longer string
a longer string
```

(The output has two lines because `main` also runs a second caller, shown below.) Move the `println!` after the inner block, and the compiler rejects the call (`code/compile_fail/ch01_longest_dangling.rs`):

```rust
pub fn caller() {
    let outer = String::from("a longer string");
    let result;
    {
        let inner = String::from("short");
        result = longest(outer.as_str(), inner.as_str());
    } // `inner` is dropped here...
    println!("{result}"); // ...but `result` may point into it
}
```

```
error[E0597]: `inner` does not live long enough
  --> compile_fail/ch01_longest_dangling.rs:15:42
   |
14 |         let inner = String::from("short");
   |             ----- binding `inner` declared here
15 |         result = longest(outer.as_str(), inner.as_str());
   |                                           ^^^^^ borrowed value does not live long enough
16 |     } // `inner` is dropped here...
   |     - `inner` dropped here while still borrowed
17 |     println!("{result}"); // ...but `result` may point into it
   |                ------ borrow later used here
```

Write down the constraints and the contradiction appears:

Constraint	Reason
`'a` contains line 17	`result` has type `&'a str` and is used on line 17
the borrow of `inner` lasts for all of `'a`	`inner.as_str()` is passed as a `&'a str`
`inner` is dropped on line 16	the end of its block

The borrow of `inner` would have to be live on line 17, after `inner` has been dropped. No choice of `'a` satisfies all three rows, so the compiler reports E0597, " `inner` does not live long enough". Notice what the compiler did *not* do: it did not look inside `longest` to see which string is returned (for these inputs it would be `outer` , and the program would happen to be safe). It checked the call against the signature, and the signature says that the result may borrow from either argument.

An annotation can only add requirements

Since annotations describe, writing a longer lifetime cannot make a reference last longer. It can only add a requirement that the program then fails to meet (`code/compile_fail/ch01_annotation_static.rs`):

```rust
pub fn caller() {
    let name = String::from("config.toml");
    let forever: &'static str = &name; // asks for a borrow that never ends
    println!("{forever}");
}
```

```
error[E0597]: `name` does not live long enough
  --> compile_fail/ch01_annotation_static.rs:8:33
   |
 7 |     let name = String::from("config.toml");
```

```
   |         ---- binding `name` declared here
 8 |     let forever: &'static str = &name; // asks for a borrow that never ends
   |                  ------------   ^^^^^ borrowed value does not live long enough
   |                  |
   |                  type annotation requires that `name` is borrowed for `'static`
 9 |     println!("{forever}");
10 | }
   | - `name` dropped here while still borrowed
```

The annotation `&'static str` says that the borrow must be valid for the region `'static`, which contains every point of the program, including every point after `name` is dropped at the end of `caller`. The diagnostic adds a kind of label you will see often, under the annotation: the type annotation "requires that `name` is borrowed for `'static`". Such a label tells you *where a requirement came from*, here the type written after `forever:`. Remove the annotation and the program compiles, with a region that ends at the `println!`. The general lesson: when a lifetime error appears, adding or lengthening an annotation never makes a value live longer. The fix is always to change the program (where values live, where references are used, or who owns the data), or to correct a signature that promises more, or demands more, than it should.

The signature is the contract

The body of a function is checked against its signature, and every caller is checked against the signature *and nothing else*. This is the thread that runs through the whole book, and the two halves of it are worth seeing separately.

Callers see only the signature. This function returns its first argument and merely prints the second (`code/compile_fail/ch01_signature_contract.rs`):

```rust
fn first<'a>(x: &'a str, y: &'a str) -> &'a str {
    println!("ignoring {y:?}");
    x
}
```

The result never borrows from `y`. But the signature, which uses `'a` for `y` as well, says that it may, and the same caller as before is rejected with the same error:

```rust
pub fn caller() {
    let outer = String::from("a longer string");
    let result;
    {
        let inner = String::from("short");
        result = first(outer.as_str(), inner.as_str());
    }
    println!("{result}");
}
```

```
error[E0597]: `inner` does not live long enough
  --> compile_fail/ch01_signature_contract.rs:18:40
   |
17 |         let inner = String::from("short");
   |             ----- binding `inner` declared here
18 |         result = first(outer.as_str(), inner.as_str());
   |                                        ^^^^^ borrowed value does not live long enough
19 |     }
   |     - `inner` dropped here while still borrowed
```

```
20 |     println!("{result}");
   |                ------ borrow later used here
```

The fix is in the signature, not in the caller: give `y` its own lifetime, so that the contract says what the body does (`code/examples/ch01_signature_fixed.rs`):

```
fn first<'a, 'b>(x: &'a str, y: &'b str) -> &'a str {
    println!("ignoring {y:?}");
    x
}
```

```
ignoring "short"
a longer string
```

With the same caller, this version compiles: the result has lifetime `'a`, and nothing ties `'a` to the borrow of `inner`.

The body must keep the promise. The other half: the compiler checks the body against the signature, without looking at any caller. Here `longest` is given two independent lifetimes, and the result is declared to borrow from `x` only (`code/compile_fail/ch01_body_vs_signature.rs`):

```
fn longest<'a, 'b>(x: &'a str, y: &'b str) -> &'a str {
    if x.len() >= y.len() { x } else { y }
}
```

```
error: lifetime may not live long enough
 --> compile_fail/ch01_body_vs_signature.rs:7:40
  |
6 | fn longest<'a, 'b>(x: &'a str, y: &'b str) -> &'a str {
  |            --  -- lifetime `'b` defined here
  |            |
  |            lifetime `'a` defined here
7 |     if x.len() >= y.len() { x } else { y }
  |                                        ^ function was supposed to return data with
lifetime `'a` but it is returning data with lifetime `'b`
  |
  = help: consider adding the following bound: `'b: 'a`
```

This error has no code, only the message "lifetime may not live long enough", which the compiler uses whenever a body breaks the lifetime promises of its signature. The labels name the two lifetime parameters, and the primary label explains the broken promise: the function "was supposed to return data with lifetime `'a` but it is returning data with lifetime `'b`". Inside `longest`, `'a` and `'b` are regions chosen by some caller the function cannot see; nothing says that `'b` contains all of `'a`, so returning `y` as a `&'a str` is not justified. The `help` line proposes to add that relation to the contract, as the bound `'b: 'a` (read "`'b` outlives `'a`"; the notation is explained below). Adding it makes the body correct, and moves the requirement to the callers.

Why design the language this way, when the compiler could look at the body of `first` and see that `y` is never returned? Because checking functions one at a time is what makes the result predictable and stable. A caller does not break when the body of a function it calls changes, as long as the signature stays the same; a library can change its implementation without breaking its users; and you, reading a signature, know everything the borrow checker will assume about the function. Every

default the compiler writes for you, from elided lifetimes (Chapter 3) to the hidden bounds of trait objects (Chapter 8), matters because it becomes part of this contract.

1.7 Reading a borrow-checker diagnostic

The diagnostics in this chapter all have the same anatomy, and learning to read it pays off for the rest of the book. Take the error of the dangling caller again:

```
error[E0597]: `inner` does not live long enough
  --> compile_fail/ch01_longest_dangling.rs:15:42
   |
14 |         let inner = String::from("short");
   |             ----- binding `inner` declared here
15 |         result = longest(outer.as_str(), inner.as_str());
   |                                           ^^^^^ borrowed value does not live long enough
16 |     } // `inner` is dropped here...
   |     - `inner` dropped here while still borrowed
17 |     println!("{result}"); // ...but `result` may point into it
   |                 ------ borrow later used here
```

- **The headline** gives the error code and a one-line summary: E0597, "`inner` does not live long enough". The code is the key to the long explanation that `rustc --explain E0597` prints, and to Appendix B of this book.
- **The location** (`-->`) is the file, line and column of the primary label.
- **The primary label**, underlined with `^^^`, marks the operation that the compiler refuses, here the borrow of `inner`: "borrowed value does not live long enough".
- **The secondary labels**, underlined with `---`, are the evidence. They are the points of the regions involved: where the borrowed value was declared ("binding `inner` declared here"), where it stops existing ("`inner` dropped here while still borrowed"), and the use that requires the borrow to be live at that point ("borrow later used here").
- **Notes and help** (`= note:`, `= help:`), when present, explain where a requirement comes from or suggest a change. Read them, but judge the suggestion against what you mean: a suggested bound such as `'b: 'a` makes the body compile by asking more of every caller.

The secondary labels, read in program order, form a timeline. A good habit is to turn every borrow-checker error into four questions:

1. **Where is the borrow created?** ("borrowed value", "immutable borrow occurs here", "first mutable borrow occurs here")
2. **What ends it, or conflicts with it?** ("dropped here while still borrowed", "mutable borrow occurs here", "assigned to here")
3. **Which later use keeps it alive?** ("borrow later used here", "immutable borrow later used here", "first borrow later used here")
4. **Where does the requirement come from?** ("type annotation requires that…", "argument requires that…", "returning this value requires that…")

The labels used in this chapter, and what each one marks:

Label	Marks
`borrowed value does not live long enough`	a borrow whose region must reach past the end of its value
`binding `x` declared here`	where the borrowed variable was created
`` `x` dropped here while still borrowed ``	the end of the value's scope, inside the region
`... borrow occurs here`	the start of a borrow involved in a conflict
`... later used here`	the use that keeps a borrow live across a conflict
`type annotation requires that ...`	a written type that imposes a region

Every fix removes one of the three ingredients of the timeline. For the dangling caller, all of these work:

- **Use the result earlier**, while the borrowed value still exists: move the `println!` into the inner block (the `use_inside` caller above).
- **Make the borrowed value live longer**: declare `inner` in the outer block, next to `outer`.
- **Stop borrowing**: make the result own its data, so that it no longer needs the inputs. The second caller in `code/examples/ch01_longest.rs` does this:

```rust
fn own_result() {
    let outer = String::from("a longer string");
    let result: String;
    {
        let inner = String::from("short");
        result = longest(outer.as_str(), inner.as_str()).to_owned(); // a copy
    }
    println!("{result}"); // fine: `result` borrows nothing
}
```

- **Correct the contract**, when the signature promises less than the function could: that was the fix for `first`, whose result never borrows from `y`.

Which fix is right depends on what the program means; the diagnostic only tells you that the current combination is contradictory. Chapter 14 discusses the design side of the choice (own or borrow).

1.8 Notation

The rest of the book uses a small vocabulary of lifetime syntax. Here it is in one place; the last column says where each form is explained in depth.

Syntax	Read as	Meaning	More in
`'a`	"lifetime a"	a named region, a parameter	this chapter
`&'a T`, `&'a mut T`	"a reference with lifetime `'a`"	valid throughout `'a`	this chapter
`'a: 'b`	"`'a` outlives `'b`"	`'a` contains every point of `'b`	Chapter 4
`T: 'a`	"`T` outlives `'a`"	every borrow inside `T` outlives `'a`	Chapter 7
`'static`	"static"	the region of the whole program run	Chapter 7
`'_`	"anonymous lifetime"	a lifetime the compiler picks by position	Chapter 6
`for<'a>`	"for all `'a`"	a bound that holds for every lifetime	Chapter 9
`use<'a, T>`	"uses `'a` and `T`"	what an `impl Trait` may capture	Chapter 8

Every form in the table appears in `code/examples/ch01_notation.rs`, which compiles; each function is a minimal use of one form:

```
// `'a` names a region; `&'a T` is a shared reference valid throughout it.
fn first_line<'a>(text: &'a str) -> &'a str {
    text.lines().next().unwrap_or("")
}

// `&'a mut T`: an exclusive reference, valid throughout `'a`.
fn append<'a>(buffer: &'a mut String, word: &str) -> &'a mut String {
    buffer.push_str(word);
    buffer
}

// `'long: 'short` ("'long outlives 'short"): every point of `'short` is in `'long`.
fn either<'short, 'long: 'short>(a: &'short str, b: &'long str) -> &'short str {
    if a.is_empty() { b } else { a }
}

// `T: 'a`: every borrow inside `T` is valid throughout `'a` (Chapters 7 and 8).
fn boxed<'a, T: Display + 'a>(value: T) -> Box<dyn Display + 'a> {
    Box::new(value)
}

// `'static`: the region that covers the whole run of the program (Chapter 7).
const GREETING: &'static str = "hello";

// `'_`: "some lifetime", here the one of `text` (Chapter 6).
fn words(text: &str) -> std::str::SplitWhitespace<'_> {
    text.split_whitespace()
}

// `for<'a>`: `F` must accept a borrow of every lifetime `'a` (Chapter 9).
fn apply<F>(f: F) -> usize
where
    F: for<'a> Fn(&'a str) -> &'a str,
{
    let local = String::from("built inside apply");
    f(&local).len()
}

// `use<'a>`: the lifetimes an `impl Trait` return type may capture (Chapter 8).
fn lengths<'a>(text: &'a str) -> impl Iterator<Item = usize> + use<'a> {
    text.split_whitespace().map(str::len)
}
```

```
regions, not durations
lifetimes are regions
hello
regions
5
18
[8, 3, 9, 6, 4]
```

Two of these deserve a word now. The **outlives** relation `'long: 'short` says that the region `'long` contains every point of `'short`. "Outlives" is meant inclusively: every lifetime outlives itself, so `'a: 'a` always holds. A reference valid for a larger region can always be used where a smaller one is expected, which is why `either` can return `b`, a `&'long str`, as a `&'short str`. Chapter 4 shows that this is a form of subtyping, and what happens to it inside other types.

The bound `T: 'a` applies the same idea to a type: every reference (and every lifetime parameter) inside `T` outlives `'a`. A type that contains no borrows at all, such as `String` or `u64`, outlives every lifetime, including `'static`. That is why `boxed` can be called with a `&str` borrowed from `text`: `T` is

`&'x str` for the region `'x` of that borrow, and the bound `T: 'a` holds for any `'a` inside `'x`. Chapter 7 is about the most common such bound, `T: 'static`, and why it does not mean "lives forever".

Implied bounds

Some outlives relations never need to be written, because the types in a signature already imply them. A reference `&'a T` is only meaningful if what it points to is valid for all of `'a`, so the type `&'a T` is **well formed** only if `T: 'a`. The compiler assumes that the types in a function's signature (its parameters and its return type) are well formed, and therefore that every relation they imply holds. These are called **implied bounds** (`code/examples/ch01_implied_bounds.rs`):

```
// `&'a &'b str` is a valid type only if `'b: 'a`, so the body may rely on it.
fn inner<'a, 'b>(outer: &'a &'b str) -> &'a str {
    *outer // a `&'b str`, returned as a `&'a str`
}

// No parameter type implies `'b: 'a` here, so the signature must say it.
fn pick<'a, 'b>(x: &'a str, y: &'b str, first: bool) -> &'a str
where
    'b: 'a,
{
    if first { x } else { y }
}

// A field `&'a T` implies `T: 'a`: the struct need not write it.
struct Labeled<'a, T> {
    label: &'a str,
    value: &'a T,
}

fn value_of<'a, T>(labeled: &Labeled<'a, T>) -> &'a T {
    labeled.value
}
```

```
config.toml
config.toml
port = 8080
```

- `inner` receives a `&'a &'b str`: a reference, valid for `'a`, to a reference valid for `'b`. That type is well formed only if `&'b str: 'a`, that is, if `'b: 'a`. So the body may return the inner `&'b str` as a `&'a str`, without any `where` clause.
- `pick` has the same body as the rejected `longest` above, but its parameters are two separate references, and nothing in their types relates `'a` and `'b`. The relation must be written, and the `where 'b: 'a` clause is exactly the bound that the `help` line suggested. Now the body compiles, and every caller must pass a `y` that is valid at least as long as `x`'s region.
- `Labeled<'a, T>` has a field of type `&'a T`, which implies `T: 'a`. The struct does not have to say so, and neither does `value_of`, whose parameter type mentions `Labeled<'a, T>`. Older code often writes `struct Labeled<'a, T: 'a>`; the bound has been inferred since Rust 1.31, as the release announcement linked above puts it: "The `: 'a` is inferred."

Implied bounds are why you rarely write `T: 'a` by hand. When you do need it, it is because a type you produce (a boxed trait object, an `impl Trait`, a thread's closure) must carry a borrow that no parameter type already constrains, and the compiler asks for the bound explicitly (Chapters 7 and 8).

1.9 Common confusions

"**A lifetime is how long a value lives.**" A lifetime is the region where a *reference* may be used. The value it points to usually lives much longer. The scope of the value is an upper bound on the region, not the region itself.

"**The borrow lasts until the closing brace.**" Not since NLL. A borrow ends after the last use of the reference or of anything derived from it, path by path. One exception you will meet in Chapter 5: a value whose type implements `Drop` is used once more when it is dropped at the end of its scope, and a borrow it holds is live until then.

"**Adding `'a` or `'static` can fix a 'does not live long enough' error.**" An annotation adds a requirement; it never extends a value's scope. The `&'static str` annotation above *caused* the error. Lifetime errors are fixed by changing where values live, where references are used, who owns the data, or which relationship the signature claims.

"**`'a` in `longest` is the lifetime of `x`.**" It is a region that the caller picks at each call, subject to both arguments being valid throughout it. With arguments of different lengths of validity, `'a` can be at most the part where both are valid.

"**If the body only returns `x`, callers can rely on that.**" Callers see only the signature. To let them rely on it, say it in the signature (`ch01_signature_fixed.rs`).

"**Lifetimes are checked at run time, or cost something at run time.**" They are erased before code generation. A program is either proven safe at compile time or rejected.

"**Two mutable borrows of the same value can never coexist in a function.**" Their *regions* must not overlap. Two mutable borrows one after the other, each ending before the next begins, are fine (`ch01_aliasing.rs`).

1.10 Summary

- For the borrow checker, a lifetime is a **region**: a set of points in the control-flow graph of a function body. It has no run-time representation; it is computed, checked and erased before code generation.
- The scope of a value, the scope of a variable holding a reference, and the region of a borrow are three different things. The region is the smallest, and must fit inside the scope of the borrowed value.
- With **non-lexical lifetimes**, a borrow is live from its creation to the last use of the reference or of anything derived from it, along each path. A borrow can be live on one branch and dead on another, and live across a loop's back edge.
- **Aliasing XOR mutation**: at every point, a value has any number of shared borrows or one mutable borrow. Overlapping regions produce E0502 (shared and mutable), E0499 (two mutable) and E0506 (assignment while borrowed).

- Lifetime annotations **describe relationships**; they never extend anything. In `longest`, one name `'a` means that the result is usable only where both inputs are valid. E0597 says that a borrow would have to outlive its value.
- **The signature is the contract**: callers are checked against the signature only (E0597 at the call), and the body is checked against it ("lifetime may not live long enough" in the body).
- Read a diagnostic as a timeline: where the borrow starts, what conflicts with it or ends its value, which later use keeps it alive, and where a requirement comes from. Fix it by moving a use, moving a value, owning the data, or correcting the contract.
- Notation: `'a`, `&'a T`, `'a: 'b`, `T: 'a`, `'static`, `'_`, `for<'a>`, `use<..>`. Implied bounds: a type such as `&'a T` implies `T: 'a`, so such bounds are rarely written by hand.

1.11 Exercises

1. In `code/examples/ch01_nll.rs`, add the line `let last = &v[2];` before the `push`, and use `last` once before the `push` and once after it. Predict the error code and the three labels before compiling. Then remove the use after the `push` and check that the program compiles again. *Hint:* only the use after the `push` puts the `push` inside the region of the new borrow.
2. Change `describe_or_extend` in `code/examples/ch01_branches.rs` so that the `then` branch prints `first` after its `push`. Explain, in terms of the paths of the control-flow graph, why the error points at the `push` and not at the `if`. Then move the `println!` before the `push` and check that the function compiles again. *Hint:* on the `then` path, the new use comes after the `push`, so the borrow is now live at the `push` on that path.
3. Write a function `fn starts_with_either<'a>(text: &'a str, a: &str, b: &str) -> Option<&'a str>` that returns `text` if it starts with `a` or `b`, and call it with `a` and `b` borrowed from strings that are dropped before the result is used. Then change the signature to use `'a` for all three parameters and explain the new error. *Hint:* in the first version, the elided lifetimes of `a` and `b` are independent of `'a` (Chapter 3 explains why), so the result does not depend on them.
4. Fix `code/compile_fail/ch01_longest_dangling.rs` in three different ways: by moving the use, by moving a declaration, and by owning the result. For each fix, say which of the three labels of the original diagnostic it makes disappear. *Hint:* `code/examples/ch01_longest.rs` shows two of the fixes; the third moves `let inner` next to `let outer`.
5. Write `fn first_of<'a, 'b>(items: &'a [&'b str]) -> &'a str` returning the first element (or `""`). Does it need a `where` clause? Now change the parameter type to two separate parameters, `first: &'b str, _rest: &'a [u8]`, keep the return type `&'a str`, and see what the compiler suggests. *Hint:* `&'a [&'b str]` is well formed only if `'b: 'a`; that is an implied bound. With two separate parameters, nothing implies it.
6. Explain why `std::any::TypeId::of::<T>()` requires `T: 'static`, using what this chapter says about erasure. Then check your explanation against Chapter 7. *Hint:* if `&'a str` and `&'static str` had different `TypeId`s, a run-time value would depend on a lifetime that no longer exists when the program runs.

CHAPTER 2

Whose Borrow Is It? Lifetimes in Types and Methods

In Chapter 1, every reference lived in a local variable, and the compiler could see all of its uses in one function body. Real programs put references into types: a parser that reads a borrowed text, a tokenizer that hands out slices of its input, an iterator over a collection, a view of a configuration file. A type that holds a reference carries the borrow with it, so its values are subject to the same rules as the reference itself, wherever they go.

Types with references raise a question that local variables never did. When a method of such a type returns a reference, *whose borrow is it*? The result can borrow from the value the method was called on, or from the data that value itself borrows. The two answers look almost the same in a signature, differ by a single lifetime name, and have very different consequences: one lets callers keep results while they go on using the parser, the other locks the parser until the last result is gone. This chapter is about telling the two apart, choosing on purpose, and recognizing the annotation that locks a value for the rest of its life.

In this chapter you will learn:

- why a struct that stores a reference needs a lifetime parameter, and what that parameter means for every value of the type;
- the two lifetimes inside every method of `impl<'src> Parser<'src>`, the data's and the receiver's, and how a return type picks one of them;
- why the elided return type pins the parser (E0499), and when it is exactly the right contract;
- why `slice::Iter<'a, T>::next` can return `&'a T`, how to implement `Iterator` for a borrowing struct, and why a lending iterator cannot be an `Iterator`;
- lifetimes in enums, structs with one or two lifetime parameters, and traits with a lifetime parameter;
- the `&'a mut self` trap: how one extra annotation borrows a value for the rest of its life.

2.1 A struct that holds a reference

A parser that works on a borrowed text, without copying it, needs a field that is a reference. The first attempt does not compile (`code/compile_fail/ch02_missing_lifetime.rs`):

```
pub struct Parser {
    input: &str,
    pos: usize,
}
```

```
error[E0106]: missing lifetime specifier
 --> compile_fail/ch02_missing_lifetime.rs:7:12
  |
7 |     input: &str,
  |            ^ expected named lifetime parameter
  |
help: consider introducing a named lifetime parameter
  |
6 ~ pub struct Parser<'a> {
7 ~     input: &'a str,
  |
```

E0106, "missing lifetime specifier": the type of a field must be complete, and `&str` without a lifetime is not. In a function signature the compiler can often fill in a missing lifetime for you (that is lifetime elision, Chapter 3), but in a type definition it never does; Chapter 3 explains why no default would be a good one. The `help` shows the fix: declare a lifetime parameter on the struct and use it in the field. Here is the parser we will use for the rest of the chapter, from `code/examples/ch02_parser.rs`:

```
struct Parser<'src> {
    input: &'src str,
    pos: usize,
}
```

The parameter is named `'src` after what it borrows, the source text; the name is only a convention (Chapter 14 discusses naming), and `'a` would work the same. `Parser<'src>` declares a family of types, exactly like a generic struct `Wrapper<T>`: each use of the type supplies a region for `'src`, just as each use of `Wrapper` supplies a type for `T`. `Parser<'x>` and `Parser<'y>` are different types, in the same way that `Vec<u8>` and `Vec<char>` are, although lifetimes give them a relationship that ordinary type parameters do not have: a `Parser<'long>` can be used where a `Parser<'short>` is expected. That is subtyping, and Chapter 4 explains when it applies.

What the parameter means for a value is the important part. A value of type `Parser<'src>` contains a `&'src str`, so it is only valid where that reference is valid. Chapter 1 stated the rule for references in local variables: a reference must be valid wherever it may still be used. The rule for types is the same, applied to every lifetime in the type: **wherever a value of type `Parser<'x>` may still be used, the region `'x` must contain that point**. A struct with a lifetime parameter carries a borrow, and using the struct is using the borrow. This program tries to use a parser after its text is gone (`code/compile_fail/ch02_parser_outlives_text.rs`):

```
pub fn caller() {
    let parser;
    {
        let text = String::from("lifetimes are regions");
        parser = Parser::new(&text);
    } // `text` is dropped here
    println!("{} at {}", parser.input, parser.pos);
}
```

```
error[E0597]: `text` does not live long enough
  --> compile_fail/ch02_parser_outlives_text.rs:21:30
   |
20 |         let text = String::from("lifetimes are regions");
   |             ---- binding `text` declared here
21 |         parser = Parser::new(&text);
   |                              ^^^^^ borrowed value does not live long enough
22 |     } // `text` is dropped here
   |     - `text` dropped here while still borrowed
23 |     println!("{} at {}", parser.input, parser.pos);
   |                          ------------ borrow later used here
```

The labels are those of Chapter 1's dangling caller, with one difference worth noticing: the "borrow later used here" label points at `parser.input`, a use of the parser. The borrow of `text` is stored inside `parser`, so the parser keeps it live, just as `shortlist` kept the borrow of the vector live in Chapter 1.

2.2 Two lifetimes in every method

Methods are written in an `impl` block, which must declare the lifetime parameter before it can use it, exactly as `impl<T> Wrapper<T>` declares `T`:

```
impl<'src> Parser<'src> {
    fn new(input: &'src str) -> Self {
        Parser { input, pos: 0 }
    }

    /// The result borrows from the text, so earlier results stay usable
    /// across later calls.
    fn next_word(&mut self) -> Option<&'src str> {
        let input = self.input; // a copy of the field: a `&'src str`
        let rest = &input[self.pos..];
        let trimmed = rest.trim_start();
        if trimmed.is_empty() {
            return None;
        }
        let start = self.pos + (rest.len() - trimmed.len());
        let end = trimmed
            .find(char::is_whitespace)
            .map_or(input.len(), |i| start + i);
        self.pos = end;
        Some(&input[start..end])
    }
}
```

In `impl<'src> Parser<'src>`, the first `'src` declares a lifetime parameter for the block and the second one uses it: inside the block, `Self` means `Parser<'src>` for that `'src`. `new` takes a `&'src str` and returns a `Parser<'src>`, so a parser built from a borrowed text has the region of that borrow in its type.

`next_word` is where the question of this chapter comes up. Its body copies the field into a local variable, `input`, of type `&'src str` (shared references are `Copy`), and returns a slice of it. A slice of a `&'src str` is again a `&'src str`: it points into the text, not into the parser. The return type says exactly that, `Option<&'src str>`.

There is a second lifetime in this method, which you do not see because it is elided: the lifetime of the borrow `&mut self`. Every call borrows the parser mutably for some region, and that region is a fresh lifetime, different at each call. Written out in full, the method has two lifetimes:

```
// You write:
fn next_word(&mut self) -> Option<&'src str>
fn next_word(&mut self) -> Option<&str>

// The compiler reads:
fn next_word<'p>(self: &'p mut Parser<'src>) -> Option<&'src str>
fn next_word<'p>(self: &'p mut Parser<'src>) -> Option<&'p str>
```

This listing is not compiled code: it shows how the compiler reads the two signatures, with `'p` standing for the lifetime of the receiver (Chapter 3 states the rules that produce it). The first line is our method: the result borrows from the text. The second line is what you get if you write the return type without a lifetime: the elision rules then give the result the lifetime of `&mut self`, so the result borrows from the *borrow of the parser*. Notice also an implied bound (Chapter 1): the type `&'p mut Parser<'src>` is well formed only if `'src: 'p`. The text always outlives any borrow of the parser.

The first version lets callers do what you would expect from a parser:

```
fn main() {
    let text = String::from("lifetimes are regions");
    let mut parser = Parser::new(&text);
    let first = parser.next_word();
    let second = parser.next_word(); // fine: `first` does not keep `parser` borrowed
    println!("{first:?} {second:?} / {parser}");
    println!("{:?}", all_words(&text));
}
```

```
Some("lifetimes") Some("are") / Parser@13 over "lifetimes are regions"
["lifetimes", "are", "regions"]
```

`first` and `second` are both alive at the `println!`, together with a use of `parser` itself (its `Display` implementation). Each call to `next_word` borrows the parser only for the duration of the call, because nothing that the call returns is tied to that borrow. The results borrow `text`, which is shared and outlives them all. The `Display` implementation, by the way, uses `Parser<'_>`, which means "a `Parser` with any lifetime, which I do not need to name":

```
// `'_`: "the parser's lifetime parameter, whatever it is" (Chapter 6).
impl fmt::Display for Parser<'_> {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "Parser@{} over {:?}", self.pos, self.input)
    }
}
```

Chapter 6 is about `'_`. Here it saves declaring `impl<'src> fmt::Display for Parser<'src>` for a block that never mentions `'src`.

Since the words borrow the text and not the parser, they can even outlive the parser:

```
fn all_words(text: &str) -> Vec<&str> {
    let mut parser = Parser::new(text);
    let mut words = Vec::new();
    while let Some(word) = parser.next_word() {
        words.push(word);
    }
    words // `parser` is dropped here; the words borrow `text`, not `parser`
}
```

The parser is a local variable of `all_words`, dropped when the function returns. The returned vector contains slices of `text`, which belongs to the caller, so returning them is fine. (The signature `fn all_words(text: &str) -> Vec<&str>` ties the result to `text` by elision: there is only one input lifetime.)

The elided version pins the parser

Now write the return type without a lifetime. The body is unchanged, character for character, and it compiles; only the signature differs (`code/compile_fail/ch02_parser_elided.rs`):

```
// Elided: the result has the lifetime of the `&mut self` borrow.
pub fn next_word(&mut self) -> Option<&str> {
```

The same caller is rejected:

```
pub fn caller() {
    let text = String::from("lifetimes are regions");
    let mut parser = Parser::new(&text);
    let first = parser.next_word();
    let second = parser.next_word(); // a second `&mut parser` while `first` is live
    println!("{first:?} {second:?}");
}
```

```
error[E0499]: cannot borrow `parser` as mutable more than once at a time
  --> compile_fail/ch02_parser_elided.rs:39:18
   |
38 |     let first = parser.next_word();
   |                 ------ first mutable borrow occurs here
39 |     let second = parser.next_word(); // a second `&mut parser` while `first` is live
   |                  ^^^^^^ second mutable borrow occurs here
40 |     println!("{first:?} {second:?}");
   |                ----- first borrow later used here
```

statement	borrow of text ('src)	1st &mut parser (returns Option<&str>)	1st &mut parser (returns Option<&'src str>)
let mut parser = Parser::new(&text);	created		
let first = parser.next_word();	live	created	created, ends
let second = parser.next_word();	live	2nd borrow: E0499	2nd borrow: fine
println!("{first:?} {second:?}");	last use	first used	

The same caller with both versions of `next_word`. When the result borrows from `&mut self`, the first mutable borrow of the parser lasts as long as `first` and conflicts with the second call; when it borrows from `'src`, each mutable borrow ends with its call.

Read the labels with the expanded signature in mind:

- `first mutable borrow occurs here`, under `parser` on line 38. A method call that takes `&mut self` borrows its receiver: `parser.next_word()` is `Parser::next_word(&mut parser)`. The compiler labels the receiver.
- `second mutable borrow occurs here`, on line 39: the second call needs another `&mut parser`.

- `first borrow later used here`, under `first` on line 40. With the elided signature, `first` has type `Option<&'p str>`, where `'p` is the region of the first `&mut parser`. It is a reference *derived from* the first mutable borrow, and Chapter 1 showed that a borrow stays live as long as anything derived from it may be used. So the first mutable borrow is live until line 40, across the second call.

The figure above puts the two versions side by side. The body compiled in both cases for the same reason: the slice it returns is a `&'src str`, and since `'src: 'p`, a `&'src str` can always be used as a `&'p str` (a reference valid for a larger region can be used for a smaller one). The elided signature does not make the body do anything different. It *promises less*: it tells callers that the result may borrow from the parser, and the callers, which see only the signature, must act accordingly. This is "the signature is the contract" again, and here the contract is weaker than the body could support.

When the elided form is right

The elided form is not a mistake in general. It is the correct contract whenever the result really does point into the value behind `self`. Consider a reader that normalizes the lines of a text (lowercase, single spaces, no blank lines) into a buffer that it owns and reuses, to avoid allocating a new `String` per line (`code/examples/ch02_lending.rs`):

```
/// Yields the non-blank lines of a text, lowercased, with single spaces.
struct Normalizer<'src> {
    lines: std::str::Lines<'src>, // borrows the text
    buffer: String,               // owned, reused for every line
}

impl<'src> Normalizer<'src> {
    fn new(text: &'src str) -> Self {
        Normalizer {
            lines: text.lines(),
            buffer: String::new(),
        }
    }

    /// The line borrows from `self.buffer`, which the next call overwrites.
    fn next_line(&mut self) -> Option<&str> {
        loop {
            let raw = self.lines.next()?;
            if raw.trim().is_empty() {
                continue;
            }
            self.buffer.clear();
            for (i, word) in raw.split_whitespace().enumerate() {
                if i > 0 {
                    self.buffer.push(' ');
                }
                self.buffer.push_str(&word.to_lowercase());
            }
            return Some(&self.buffer);
        }
    }
}
```

The field `lines` is a `std::str::Lines<'src>`, the standard library's iterator over the lines of a `&'src str`: a borrowing type, with a lifetime parameter, just like our parser. The field `buffer` is owned by the reader. `next_line` returns a reference to `buffer`, so the result really borrows from `self`, and the elided return type says so. Callers use one line at a time:

```
fn main() {
    let text = String::from("  Port =   8080\n\n HOST = Example.ORG \n");
    let mut normalizer = Normalizer::new(&text);
    let mut kept: Vec<String> = Vec::new();
    while let Some(line) = normalizer.next_line() {
        println!("{line:?}");
        kept.push(line.to_owned()); // to keep a line, copy it out of the buffer
    }
    println!("{kept:?}");
}
```

```
"port = 8080"
"host = example.org"
["port = 8080", "host = example.org"]
```

To keep a line beyond the next call, the caller copies it (`to_owned`). Holding two lines at once is rejected (`code/compile_fail/ch02_lending_keep.rs`), with the very error that the elided parser produced:

```
pub fn caller(normalizer: &mut Normalizer<'_>) {
    let first = normalizer.next_line();
    let second = normalizer.next_line(); // would overwrite what `first` shows
    println!("{first:?} {second:?}");
}
```

```
error[E0499]: cannot borrow `*normalizer` as mutable more than once at a time
  --> compile_fail/ch02_lending_keep.rs:22:18
   |
21 |     let first = normalizer.next_line();
   |                 ---------- first mutable borrow occurs here
22 |     let second = normalizer.next_line(); // would overwrite what `first` shows
   |                  ^^^^^^^^^^ second mutable borrow occurs here
23 |     println!("{first:?} {second:?}");
   |                ----- first borrow later used here
```

This time the error protects you. The second call clears the buffer and writes the next line into it; if the compiler accepted the program, `first` would show the second line's text, or point into memory that `push_str` had reallocated. The labels are the same as for the parser because the signatures have the same shape. Only you know whether the data lives in the struct or in something the struct borrows; the compiler checks that your signature tells the truth.

And it does check. Claim `'src` for a line that lives in the buffer, and the body is rejected (`code/compile_fail/ch02_lending_promise.rs`):

```
impl<'src> Normalizer<'src> {
    pub fn next_line(&mut self) -> Option<&'src str> {
        let raw = self.lines.next()?;
        self.buffer.clear();
        self.buffer.push_str(raw.trim());
        Some(&self.buffer) // borrows `self`, not the text
    }
}
```

```
error: lifetime may not live long enough
  --> compile_fail/ch02_lending_promise.rs:16:9
   |
11 | impl<'src> Normalizer<'src> {
   |      ---- lifetime `'src` defined here
12 |     pub fn next_line(&mut self) -> Option<&'src str> {
   |                       - let's call the lifetime of this reference `'1`
...
```

```
16 |         Some(&self.buffer) // borrows `self`, not the text
   |         ^^^^^^^^^^^^^^^^^^ method was supposed to return data with lifetime `'src` but
it is returning data with lifetime `'1`
```

This is the "lifetime may not live long enough" error of Chapter 1, in a method. The first label marks where `'src` is declared. The second gives the elided lifetime of `&mut self` a name so that the message can talk about it: "let's call the lifetime of this reference `'1`". Names such as `'1` and `'2` never appear in your code; they are the compiler's labels for anonymous lifetimes. The primary label says the body returns data with lifetime `'1`, the borrow of the reader, where the signature promised `'src`. The `'src` version of the parser passed this check because its data really does come from the text.

> **TIP**
>
> To choose the return type of a method, ask where the returned data lives. In something the struct *borrows* (a `&'src` field): name the struct's lifetime, `-> &'src str`. In something the struct *owns* (a `String`, a `Vec`, a buffer): let the result borrow `self`, which elision does for you, `-> &str`. Computed fresh by the method: return an owned value, `-> String`.

2.3 Iterators whose items borrow from the data

The standard library's `Iterator` trait settles the question of this chapter once and for all for every iterator. Its central method is `fn next(&mut self) -> Option<Self::Item>`, and `Item` is an associated type: one type, fixed by each implementation, the same for every call of `next`. The lifetime of the `&mut self` borrow is different at every call, so `Item` cannot mention it. Whatever an iterator yields is therefore independent of the borrow of the iterator, and this is exactly what makes `collect`, `zip`, `peekable` and a plain `let a = it.next(); let b = it.next();` possible.

Take the iterator over a slice. The documentation of `std::slice::Iter` lists its implementation as `impl<'a, T> Iterator for Iter<'a, T>`, with `type Item = &'a T` and `fn next(&mut self) -> Option<&'a T>`. Shown as code (not compiled here; the standard library generates it with a macro):

```
impl<'a, T> Iterator for Iter<'a, T> {
    type Item = &'a T;
    fn next(&mut self) -> Option<&'a T> { /* ... */ }
}
```

It is our parser's design: the struct `Iter<'a, T>` borrows a slice for `'a`, and the items borrow from the slice, for `'a`, not from the `&mut self` borrow of the iterator. So two items can be held at once (`code/examples/ch02_words_iterator.rs`):

```
fn two_at_once() {
    let ports = [8080, 8443, 9000];
    let mut iter = ports.iter(); // a `std::slice::Iter<'_, i32>`
    let first = iter.next(); // `Option<&i32>`, borrowing `ports`...
    let second = iter.next(); // ...so `iter` is free for another call
    println!("{first:?} {second:?}");
}
```

Implementing `Iterator` for a borrowing struct of your own follows the same pattern. `Words<'src>` yields the words of a borrowed text:

```
/// The words of a borrowed text.
struct Words<'src> {
    rest: &'src str,
}

impl<'src> Iterator for Words<'src> {
    type Item = &'src str; // one type for every call of `next`

    fn next(&mut self) -> Option<&'src str> {
        let trimmed = self.rest.trim_start();
        if trimmed.is_empty() {
            return None;
        }
        let end = trimmed.find(char::is_whitespace).unwrap_or(trimmed.len());
        let (word, rest) = trimmed.split_at(end);
        self.rest = rest;
        Some(word)
    }
}
```

`type Item = &'src str` names the struct's lifetime, which is available because the `impl` declares it. Because the items borrow the text, everything the standard library offers for iterators works, including keeping the items after the iterator is gone:

```
fn main() {
    two_at_once();
    let text = String::from("lifetimes are regions not durations");
    // The `Words` value is a temporary, gone at the end of the statement;
    // the words it produced borrow `text` and remain usable.
    let long_words: Vec<&str> = Words { rest: &text }.filter(|w| w.len() > 3).collect();
    println!("{long_words:?}");
}
```

```
Some(8080) Some(8443)
["lifetimes", "regions", "durations"]
```

A lending iterator is not an `Iterator`

The `Normalizer` would make a fine iterator, and `for line in normalizer` would read better than `while let`. The attempt fails (`code/compile_fail/ch02_lending_iterator.rs`):

```
impl<'src> Iterator for Normalizer<'src> {
    type Item = &str; // which lifetime? not one of the iterator's parameters

    fn next(&mut self) -> Option<&str> {
        let raw = self.lines.next()?;
        self.buffer.clear();
        self.buffer.push_str(raw.trim());
        Some(&self.buffer)
    }
}
```

```
error: associated type `Iterator::Item` is declared without lifetime parameters, so using a
borrowed type for them requires that lifetime to come from the implemented type
  --> compile_fail/ch02_lending_iterator.rs:11:17
   |
11 |     type Item = &str; // which lifetime? not one of the iterator's parameters
   |                 ^
   |
```

```
note: you can't create an `Iterator` that borrows each `Item` from itself, but you can
instead create a new type that borrows your existing type and implement `Iterator` for that
new type
  --> compile_fail/ch02_lending_iterator.rs:10:25
   |
10 | impl<'src> Iterator for Normalizer<'src> {
   |                         ^^^^^^^^^^^^^^^^
```

The error has no code, and it explains itself. `type Item = &str` needs a lifetime, and the only lifetimes available for `Item` are those of the implementation, here `'src` ("requires that lifetime to come from the implemented type"). The line lives in the buffer, so `'src` would be a lie, as `ch02_lending_promise.rs` showed; the lifetime it would need is the one of each `&mut self` borrow, and `Item` cannot name it. The note says it plainly: "you can't create an `Iterator` that borrows each `Item` from itself".

An iterator whose items borrow from the iterator is called a **lending iterator** (or streaming iterator). `Iterator` cannot express one, and that is not an oversight: the guarantee that items are independent of the iterator is what makes `collect` sound. You have three options:

- **Keep the inherent method**, `fn next_line(&mut self) -> Option<&str>`, and loop with `while let`, as `ch02_lending.rs` does. You give up `for` loops and iterator adapters.
- **Yield owned items**, so that `type Item = String`. This is the choice of the standard library's `std::io::BufRead::lines`, whose iterator yields `Result<String, std::io::Error>`, one new `String` per line.
- **Follow the note's suggestion**: keep the data somewhere else, in a value that the iterator only borrows, so that the items can borrow from that value with the iterator's lifetime parameter. `slice::Iter<'a, T>` is this design: the elements live in the slice, not in the iterator.

A lending iterator *trait*, with a `next` whose items may borrow from `&mut self`, can be written with generic associated types (GATs, stable since Rust 1.65). Chapter 10 builds one, explains the `where Self: 'a` it requires, and shows why `for` loops and adapters still do not apply.

2.4 Enums with references

An enum follows the same rule as a struct: if any variant holds a reference, the enum needs a lifetime parameter. A tokenizer for lines such as `port = 8080` can return tokens that borrow their text from the input (`code/examples/ch02_tokens.rs`):

```
#[derive(Debug)]
enum Token<'src> {
    Key(&'src str), // borrows from the input
    Number(u64),    // owns its data
    Equals,
}

enum LexError<'src> {
    BadWord { word: &'src str, position: usize },
}

fn tokenize<'src>(line: &'src str) -> Result<Vec<Token<'src>>, LexError<'src>> {
    let mut tokens = Vec::new();
    for (position, word) in line.split_whitespace().enumerate() {
        let token = if word == "=" {
            Token::Equals
```

```
        } else if let Ok(n) = word.parse() {
            Token::Number(n)
        } else if word.chars().all(|c| c.is_alphanumeric() || c == '_') {
            Token::Key(word)
        } else {
            return Err(LexError::BadWord { word, position });
        };
        tokens.push(token);
    }
    Ok(tokens)
}
```

```
fn main() {
    let config = String::from("port = 8080\nname = ???");
    for line in config.lines() {
        match tokenize(line) {
            Ok(tokens) => {
                let shown: Vec<String> = tokens
                    .iter()
                    .map(|token| match token {
                        Token::Key(key) => format!("key {key:?}"),
                        Token::Number(n) => format!("number {n}"),
                        Token::Equals => "equals".to_string(),
                    })
                    .collect();
                println!("{}", shown.join(", "));
            }
            Err(LexError::BadWord { word, position }) => {
                println!("bad word {word:?} at position {position}")
            }
        }
    }
    // A variant without a borrow still has the type's lifetime parameter,
    // which can be any lifetime at all, `'static` included.
    let always: Token<'static> = Token::Equals;
    println!("{always:?}");
}
```

```
key "port", equals, number 8080
bad word "???" at position 2
Equals
```

Three things are worth noticing.

The signature ties everything to the input. The return type of `tokenize` mentions `'src` twice, in `Vec<Token<'src>>` and in `LexError<'src>`: both the tokens and the error borrow from `line`, and from nothing else. The key `"port"` and the bad word `"???"` are slices of the input, not copies.

The parameter belongs to the type, not to the variant. `Token::Number(8080)` and `Token::Equals` hold no reference, but their type is still `Token<'src>`, and they are subject to `'src` like any other token: a `Vec<Token<'src>>` is a borrowing collection even if it happens to contain only numbers. The last lines of `main` show the other side of this: since `Token::Equals` borrows nothing, it can be given any lifetime at all, including `Token<'static>`.

An error that borrows is cheap and precise, and limited. `LexError<'src>` points at the offending word without allocating. The price is that the error cannot outlive the input: it cannot be returned from a function that owns the line, or stored after the input is dropped. Even with an implementation of `std::error::Error`, it cannot be converted into a `Box<dyn std::error::Error>` unless the input is

`'static`, because of a hidden `'static` bound that Chapter 8 explains; Chapter 14 discusses errors that borrow versus errors that own.

The standard library's own enums work the same way, through their type parameters: `Option<&'a str>` and `Result<&'a str, E>` are enums containing a reference, and the lifetime travels in the type argument.

2.5 How many lifetime parameters?

A struct with two reference fields can use one lifetime parameter for both, or one for each. One is usually enough, because the compiler can shorten a reference to make it fit: a `&'long str` can be used as a `&'short str`. This struct (`code/examples/ch02_two_lifetimes.rs`) stores a key borrowed from a local `String` and a value borrowed from the caller's text, which lives longer:

```
/// One lifetime for both fields: the compiler shrinks the longer borrow.
struct Entry<'a> {
    key: &'a str,
    value: &'a str,
}

fn describe(text: &str) {
    let key = String::from("host");
    // The value (from `text`, or the literal) outlives `key`; both fields
    // are used as `&'a str` for an `'a` that ends with this function.
    let entry = Entry {
        key: &key,
        value: text.lines().next().unwrap_or("unknown"),
    };
    println!("{} -> {}", entry.key, entry.value);
}
```

The compiler picks one region for `'a`, small enough for the shorter of the two borrows, and views the longer one at that region. Nothing is lost, because both fields are used only inside `describe`.

One parameter stops being enough when a method hands one of the borrows back to the caller, and the caller needs it after the *other* borrow has ended. Here is a `Finder` with one lifetime for a text and a search pattern (`code/compile_fail/ch02_one_lifetime.rs`):

```
pub struct Finder<'a> {
    text: &'a str,
    pattern: &'a str,
}

impl<'a> Finder<'a> {
    pub fn new(pattern: &'a str, text: &'a str) -> Self {
        Finder { text, pattern }
    }

    pub fn first_line_with(&self) -> Option<&'a str> {
        self.text.lines().find(|line| line.contains(self.pattern))
    }
}
```

The result of `first_line_with` is a line of `text`, but its type says `Option<&'a str>`, and `'a` is also the lifetime of `pattern`. A caller with a short-lived pattern is rejected:

```
pub fn caller() {
    let text = String::from("[server]\nport = 8080\nhost = example.org");
    let found;
    {
        let pattern = String::from("port");
        let finder = Finder::new(&pattern, &text);
        found = finder.first_line_with();
    } // `pattern` is dropped here
    println!("{found:?}");
}
```

```
error[E0597]: `pattern` does not live long enough
  --> compile_fail/ch02_one_lifetime.rs:28:34
   |
27 |         let pattern = String::from("port");
   |             ------- binding `pattern` declared here
28 |         let finder = Finder::new(&pattern, &text);
   |                                  ^^^^^^^^ borrowed value does not live long enough
29 |         found = finder.first_line_with();
30 |     } // `pattern` is dropped here
   |     - `pattern` dropped here while still borrowed
31 |     println!("{found:?}");
   |                -----
   |                borrow later used here
```

Follow the constraints. `found` is used on line 31, so `'a` must reach line 31. `&pattern` is passed as a `&'a str` on line 28, so the borrow of `pattern` must last for all of `'a`. But `pattern` is dropped on line 30. The single parameter has welded the two borrows together. Give each borrow its own parameter, and let the result name only the one it comes from:

```
/// Two lifetimes: the result names `'t` only, so the pattern may die first.
struct Finder<'t, 'p> {
    text: &'t str,
    pattern: &'p str,
}

impl<'t, 'p> Finder<'t, 'p> {
    fn new(pattern: &'p str, text: &'t str) -> Self {
        Finder { text, pattern }
    }

    fn first_line_with(&self) -> Option<&'t str> {
        self.text.lines().find(|line| line.contains(self.pattern))
    }
}

fn main() {
    describe("example.org");
    let text = String::from("[server]\nport = 8080\nhost = example.org");
    let found;
    {
        let pattern = String::from("port");
        let finder = Finder::new(&pattern, &text);
        found = finder.first_line_with();
    } // `pattern` is dropped here
    println!("{found:?}"); // fine: `found` borrows `text` only
}
```

```
host -> example.org
Some("port = 8080")
```

`first_line_with` returns `Option<&'t str>`, and nothing ties `'t` to `'p`, so the pattern may die first. This is the same fix as `first<'a, 'b>` in Chapter 1, moved from a function to a type.

The guideline that follows: **start with one lifetime parameter, and add a second when a method returns one of the borrows and callers need it to outlive the other.** Every extra parameter has a cost. Each mention of the type gets longer (`Finder<'t, 'p>`), every `impl` must declare both, and relations between them sometimes have to be written out. Mutable references add a twist that Chapter 4 explains: the compiler cannot shorten a lifetime that appears *behind* a `&mut` (in `&'a mut Vec<&'a str>`, the inner `'a` is fixed), so a single lifetime used both for a mutable reference and for what it points to is far more constraining than it looks. The last section of this chapter shows the extreme case. Chapter 14 comes back to the design question.

2.6 Traits with a lifetime parameter

A trait can have a lifetime parameter too. This is how a trait says that its methods return data borrowed from *something other than* `self`, for a region that is part of the trait. Here is a trait for sources of pieces of text (`code/examples/ch02_source_trait.rs`):

```
/// Something that hands out pieces of text borrowed for `'a`.
trait Source<'a> {
    fn next_piece(&mut self) -> Option<&'a str>;
}
```

`next_piece` borrows the source mutably, for the duration of a call, and returns a `&'a str` whose lifetime comes from the trait, not from `&mut self`. It is the `'src` version of our parser, as a trait. An implementation for a borrowing struct ties the trait's parameter to the struct's:

```
/// The comma-separated fields of a borrowed line.
struct Fields<'src> {
    rest: Option<&'src str>,
}

impl<'src> Source<'src> for Fields<'src> {
    fn next_piece(&mut self) -> Option<&'src str> {
        let rest = self.rest?;
        match rest.split_once(',') {
            Some((head, tail)) => {
                self.rest = Some(tail);
                Some(head.trim())
            }
            None => {
                self.rest = None;
                Some(rest.trim())
            }
        }
    }
}
```

`impl<'src> Source<'src> for Fields<'src>` reads: for every region `'src`, a `Fields<'src>` is a source of `&'src str` pieces. A type that hands out string literals can do better, and implement the trait for every lifetime at once:

```
/// Hands out string literals, so it is a `Source<'a>` for every `'a`.
struct Defaults {
    left: usize,
}

impl<'a> Source<'a> for Defaults {
```

```
    fn next_piece(&mut self) -> Option<&'a str> {
        if self.left == 0 {
            return None;
        }
        self.left -= 1;
        Some("default")
    }
}
```

The literal `"default"` has type `&'static str`, and a reference valid for the whole program can be used as a `&'a str` for any `'a`; so the implementation holds for all `'a` at once. Generic code can now collect pieces while it goes on calling `next_piece`:

```
/// The pieces borrow from the source's data, not from `source` itself.
fn collect_all<'a, S: Source<'a>>(source: &mut S) -> Vec<&'a str> {
    let mut pieces = Vec::new();
    while let Some(piece) = source.next_piece() {
        pieces.push(piece);
    }
    pieces
}

fn main() {
    let line = String::from("name, port, host");
    let mut fields = Fields { rest: Some(&line) };
    let pieces = collect_all(&mut fields);
    let mut defaults = Defaults { left: 2 };
    let more = collect_all(&mut defaults);
    println!("{pieces:?} {more:?}");
}
```

```
["name", "port", "host"] ["default", "default"]
```

`collect_all` is generic over both the lifetime `'a` and the source type `S`. Its result, `Vec<&'a str>`, does not mention the lifetime of `source`: the borrow of the source ends with the function, and the pieces survive it.

Without the lifetime parameter, the trait can only promise pieces that borrow from `&mut self`, and the same function is rejected (`code/compile_fail/ch02_source_elided.rs`):

```
pub trait Source {
    fn next_piece(&mut self) -> Option<&str>; // borrows from `&mut self`
}

pub fn collect_all<S: Source>(source: &mut S) -> Vec<&str> {
    let mut pieces = Vec::new();
    while let Some(piece) = source.next_piece() {
        pieces.push(piece);
    }
    pieces
}
```

```
error[E0499]: cannot borrow `*source` as mutable more than once at a time
  --> compile_fail/ch02_source_elided.rs:12:29
   |
10 | pub fn collect_all<S: Source>(source: &mut S) -> Vec<&str> {
   |                                       - let's call the lifetime of this reference `'1`
11 |     let mut pieces = Vec::new();
12 |     while let Some(piece) = source.next_piece() {
   |                             ^^^^^^ `*source` was mutably borrowed here in the previous
iteration of the loop
...
```

```
15 |     pieces
   |     ------ returning this value requires that `*source` is borrowed for `'1`
```

This diagnostic introduces two new labels. On line 10, the compiler names the elided lifetime of `source` `'1`. On line 15, "returning this value requires that `*source` is borrowed for `'1`": the return type `Vec<&str>` has the elided lifetime `'1` too, so every piece pushed into `pieces` must be valid for `'1`. Each piece comes from a call `source.next_piece()` that borrows `*source` (the value behind the reference); since the piece borrows from that call's borrow, the borrow must last for all of `'1`, which extends past the end of the function. The primary label, on line 12, is where two such borrows meet: "`*source` was mutably borrowed here in the previous iteration of the loop". As in Chapter 1's loop, the borrow made in one iteration is still live when the next iteration borrows again.

Traits with lifetime parameters are common in libraries that hand out borrowed data. The best-known example is serde's `Deserialize<'de>`, where `'de` is the lifetime of the data being deserialized from; it is what lets a deserialized struct borrow string slices from its input, which serde calls zero-copy deserialization (https://serde.rs/lifetimes.html). Chapter 9 shows how generic code requires such a trait "for every lifetime", with `for<'de> Deserialize<'de>`.

2.7 The `&'a mut self` trap

The last pattern of this chapter is the one most worth recognizing on sight. In `impl<'src> Parser<'src>`, the name `'src` is right there, and it is tempting to use it for the receiver as well, perhaps in an attempt to silence a lifetime error by giving everything the same name (`code/compile_fail/ch02_borrowed_forever.rs`):

```
// The same name for the borrow of the parser and for the text it reads.
pub fn next_word(&'src mut self) -> Option<&'src str> {
```

The method compiles, and so does a first call. The second call does not, even when the caller throws the first result away:

```
pub fn caller(text: &str) {
    let mut parser = Parser::new(text);
    parser.next_word(); // the result is thrown away...
    parser.next_word(); // ...and the parser is still borrowed
}
```

```
error[E0499]: cannot borrow `parser` as mutable more than once at a time
  --> compile_fail/ch02_borrowed_forever.rs:38:5
   |
37 |     parser.next_word(); // the result is thrown away...
   |     ------ first mutable borrow occurs here
38 |     parser.next_word(); // ...and the parser is still borrowed
   |     ^^^^^^
   |     |
   |     second mutable borrow occurs here
   |     first borrow later used here
```

Two labels share one span on line 38: the second call is both the "second mutable borrow" and the place where the "first borrow [is] later used". There is no variable holding the first result, so what uses the first borrow? The parser itself. Follow the signature: `&'src mut self` means

`self: &'src mut Parser<'src>`, so the first call borrows `parser` mutably *for the region* `'src`. And `'src` is the lifetime in the type of `parser`, which, by the rule at the start of this chapter, must contain every point where `parser` is still used. So the first mutable borrow covers every later use of the parser, and every later use conflicts with it. After the first call, the parser is borrowed for the rest of its life: it is **borrowed forever**.

Why can the compiler not choose a shorter region for this call? For a shared reference it could, and it does. With `&'src self`, the compiler is allowed to view a `Parser<'long>` as a `Parser<'short>` for the duration of the call, pick `'short` as small as the call needs, and the parser is free again afterwards. This program compiles (`code/examples/ch02_shared_self.rs`):

```
impl<'src> Parser<'src> {
    fn rest(&'src self) -> &'src str {
        &self.input[self.pos..]
    }

    fn advance(&mut self, bytes: usize) {
        self.pos = (self.pos + bytes).min(self.input.len());
    }
}

fn main() {
    let text = String::from("lifetimes are regions");
    let mut parser = Parser::new(&text);
    let before = parser.rest();
    println!("{before}"); // last use of `before`
    parser.advance(10); // fine: the shared borrow has ended
    println!("{}", parser.rest());
}
```

```
lifetimes are regions
are regions
```

Behind a `&mut`, that shortening is forbidden: a mutable reference could be used to store a short-lived reference into a value whose type promises a long-lived one. So the type behind `&'src mut` must be exactly the parser's own type, and the borrow inherits the parser's whole region. This property (`&mut T` is *invariant* in `T`, while `&T` is *covariant*) is the subject of Chapter 4, which explains the "borrowed forever" effect from the type system's point of view.

Even the shared version buys nothing. Its result is limited to the shortened `'src`, which is the region of the shared borrow of the parser, so it behaves exactly like an elided `&self` method: while the result is in use, the parser cannot be mutated (`code/compile_fail/ch02_shared_self_keep.rs`):

```
pub fn caller(text: &str) {
    let mut parser = Parser::new(text);
    let before = parser.rest();
    parser.advance(10);
    println!("{before}"); // `before` keeps the shared borrow of `parser` live
}
```

```
error[E0502]: cannot borrow `parser` as mutable because it is also borrowed as immutable
  --> compile_fail/ch02_shared_self_keep.rs:28:5
   |
27 |     let before = parser.rest();
   |                  ------ immutable borrow occurs here
28 |     parser.advance(10);
```

```
   |     ^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
29 |     println!("{before}"); // `before` keeps the shared borrow of `parser` live
   |                ------ immutable borrow later used here
```

> **WARNING**
>
> In `impl<'a> S<'a>`, never write `&'a mut self`. It ties the borrow of the value to the lifetime of the data inside it, and the value can never be used again after the call. The fix is to leave the receiver alone (`&mut self`) and put `'a` only where the data comes from, typically the return type.

The same shape, `&'a mut T<'a>`, also appears in function parameters and struct fields, and is one of the most common sources of confusing borrow errors in real code. Chapter 3 meets it as the consequence of a tempting but wrong elision rule, and Chapter 14 shows how to refactor it away.

2.8 Common confusions

"**`'src` is the lifetime of the parser.**" It is the region of the borrowed text. A parser can be created and dropped many times within it: `all_words` drops its parser long before the words it returned stop being usable.

"**The elided return type is what the compiler chose, so it must be right.**" Elision applies a fixed rule that looks only at the signature (Chapter 3). For a method with `&self` or `&mut self`, it picks the receiver, which is right when the result points into data the struct owns and wrong for views and cursors that point into borrowed data.

"**To fix E0499 on a method call, add `'a` to `&mut self`.**" That annotation is the trap of the last section. The fix for a method that pins its receiver is to name the struct's lifetime in the *return type* and leave `&mut self` elided.

"**A lending iterator just needs the right annotation.**" With `Iterator`, no annotation works: `Item` cannot depend on the borrow of `self`. Use an inherent method, owned items, or a GAT-based trait (Chapter 10).

"**More lifetime parameters make a struct more flexible, so more is better.**" Each parameter is something every user of the type has to reason about. One is usually enough; add the second when a result must outlive one of the borrows.

"**A struct with a lifetime parameter cannot be returned from a function.**" It can, whenever the data it borrows belongs to the caller: `Parser::new` returns a `Parser<'src>`, and `tokenize` returns tokens borrowing its argument.

"**`Token::Equals` has no lifetime.**" Its *type*, `Token<'src>`, has one. The lifetime is a property of the type, and every value of the type is subject to it.

2.9 Summary

- A struct or enum that stores a reference needs a lifetime parameter (`Parser<'src>`), which is part of its type. Wherever a value of type `Parser<'x>` may be used, `'x` must contain that point: using the struct is using the borrow.
- In `impl<'src> Parser<'src>`, every method has two lifetimes: `'src`, the region of the data the parser borrows, and the anonymous lifetime of the `&self` or `&mut self` borrow. A return type chooses between them.
- `-> Option<&'src str>` borrows from the data: results can be kept, collected, and can outlive the parser. The elided `-> Option<&str>` borrows from the receiver and keeps the parser exclusively borrowed while any result lives (E0499).
- The elided form is correct when the result points into data the struct owns, such as a reused buffer; claiming `'src` there is rejected with "lifetime may not live long enough".
- `Iterator::Item` cannot depend on the borrow of the iterator, which is why `slice::Iter<'a, T>` yields `&'a T` and why items can be collected. A lending iterator cannot implement `Iterator`; use an inherent method, owned items, or the GAT-based design of Chapter 10.
- One lifetime parameter usually suffices for several reference fields; use two when a method returns one borrow that callers need after the other has ended.
- A trait with a lifetime parameter (`trait Source<'a>`) lets methods return data that borrows from something other than `self`.
- `&'a mut self` in `impl<'a> S<'a>` borrows the value for its whole lifetime ("borrowed forever"), because `&mut` cannot shrink the type behind it (Chapter 4). Leave the receiver elided and name `'a` in the return type.

2.10 Exercises

1. Add a method `fn peek(&self) -> Option<&'src str>` to the parser of `code/examples/ch02_parser.rs` that returns the next word without advancing. Show that a peeked word can be kept across a call to `next_word`. Then elide its return type and explain the error. *Hint:* `peek` can run the body of `next_word` on a copy of `self.pos`. With the elided type, the peeked word holds a shared borrow of `parser`, and `next_word` needs a mutable one.
2. Implement `Iterator` for `Parser<'src>` itself, with `type Item = &'src str`, and rewrite `all_words` as `Parser::new(text).collect()`. Why does the implementation compile, when the one for `Normalizer` did not? *Hint:* `next` can simply call `self.next_word()`; ask where each item lives.
3. Change `Normalizer` so that it implements `Iterator<Item = String>`. What changes for the caller, and what does it cost? Compare with `std::io::BufRead::lines`. *Hint:* `kept.push(line.to_owned())` disappears, and so does the reuse of the buffer: there is now one allocation per line, even for lines the caller only prints.
4. A `Replace<'a>` struct holds a text, a pattern and a replacement, all as `&'a str`, and has a method `fn first_match(&self) -> Option<&'a str>` that returns the part of the text that matches. Which fields should get their own lifetime parameter if callers build the pattern and the replacement in

a short-lived scope and keep the match? Write both versions and a caller. *Hint:* only the field that the result borrows from needs to be separate; `Finder<'t, 'p>` is the model.

5. Implement `Source<'a>` for `std::slice::Iter<'a, String>`, so that `collect_all(&mut lines.iter())` works when `lines` is a `Vec<String>`. Which lifetime does each piece have? *Hint:* the body is `self.next().map(String::as_str)`. The pieces borrow the strings of the vector, for the `'a` of the slice iterator.
6. A colleague's lexer has `impl<'a> Lexer<'a> { fn skip(&'a mut self) { ... } }` and complains that "the lexer can only be used once". Reproduce the problem, explain it with the labels of the diagnostic, and fix it. Would `fn skip(&'a self)` have the same problem? *Hint:* compare `ch02_borrowed_forever.rs` and `ch02_shared_self.rs`; a method that skips input needs `&mut self` anyway.

CHAPTER 3

Lifetime Elision

You have written `fn first_word(text: &str) -> &str` many times without ever writing a lifetime in it. Yet that signature says something precise about lifetimes: the result borrows from `text`, so it must not outlive it. The compiler knows this because it fills in the missing lifetimes itself, following a small set of fixed rules called *lifetime elision*. Most of the time the rules produce exactly the contract you meant, so you never think about them. Then you add a second reference parameter and get E0106, "missing lifetime specifier", for a function whose body is obviously fine.

This chapter takes elision apart. We state the rules precisely, the way rustc implements them (which is not quite how the Rust Reference words them), expand every kind of signature by hand, and read the E0106 diagnostics label by label. Then we ask the more interesting question: why does the compiler stop at three rules? Why does it refuse to guess when the body makes the answer obvious? We answer it the way this book answers everything, with programs: we invent three plausible extra rules, apply each one by hand, and let the compiler judge the result.

In this chapter you will learn:

- that elision is a fixed, syntactic rewrite of a signature that never looks at the body;
- the three rules as rustc 1.99 implements them, including the precise form of rule 2 and the `self` rule as extended in Rust 1.81;
- how to expand any elided signature: functions, methods, function pointers, `Fn` sugar, `impl` headers, `static` and `const` items, and associated consts;
- how to read E0106 and choose between the three honest contracts it asks for;
- why a struct field cannot elide its lifetime, and why `'static` would be a bad default;
- why "the first parameter", "the same lifetime everywhere" and "whatever the body returns" would all be worse rules than none.

3.1 Elision is a rewrite of the signature

An *elided* lifetime is a lifetime position where you did not write a name: the lifetime of a reference such as `&str` or `&mut Vec<u8>`, the hidden lifetime parameter of a type such as `std::str::SplitWhitespace` when you write it without `<'_>`, and the anonymous lifetime `'_` itself (Chapter 6). Elision is the step in which the compiler gives each elided lifetime in a signature a meaning, by turning it into a fresh lifetime parameter or by reusing one that is already there. Lifetimes in the parameter types are in *input* positions; lifetimes in the return type are in *output* positions.

The result is an ordinary signature with every lifetime named, and you can check that it is the same contract. The program `code/examples/ch03_same_signature.rs` writes `first_word` both ways:

```
// What you write:
fn first_word(text: &str) -> &str {
    text.split_whitespace().next().unwrap_or("")
}

// What the compiler reads:
fn first_word_explicit<'a>(text: &'a str) -> &'a str {
    text.split_whitespace().next().unwrap_or("")
}
```

and puts both functions into one array, which can only hold values of a single type:

```
// An array holds values of a single type. Both functions coerce to the fn
// pointer type `fn(&str) -> &str`, itself elided (see "Function pointers and
// `Fn` sugar" below).
let versions: [fn(&str) -> &str; 2] = [first_word, first_word_explicit];
let line = String::from("parse the first word");
for f in versions {
    println!("{}", f(&line));
}
```

```
parse
parse
```

Both functions coerce to the same function pointer type. The elided version is not a weaker or "inferred" variant of the explicit one; it is the same signature, written shorter. Three properties of this rewrite matter for everything that follows.

- **It happens in signatures, not in bodies.** Inside a function body, a type annotation such as `let r: &str = ...;` also leaves a lifetime unnamed, but there the borrow checker *infers* it from the code, as Chapter 1 described. Elision is not inference: it is a fixed rule, applied to the signature alone.
- **It is syntactic.** The rules look at the lifetimes *written* in the signature, named or elided, and at nothing else. A lifetime hidden behind a type parameter `T` or behind `Self` is invisible to them, as we will see.
- **It never looks at the body.** The expanded signature is the contract. The body is checked against it, and every caller is checked against it without looking at the body. This is the thread that runs through the whole book: the signature is the contract.

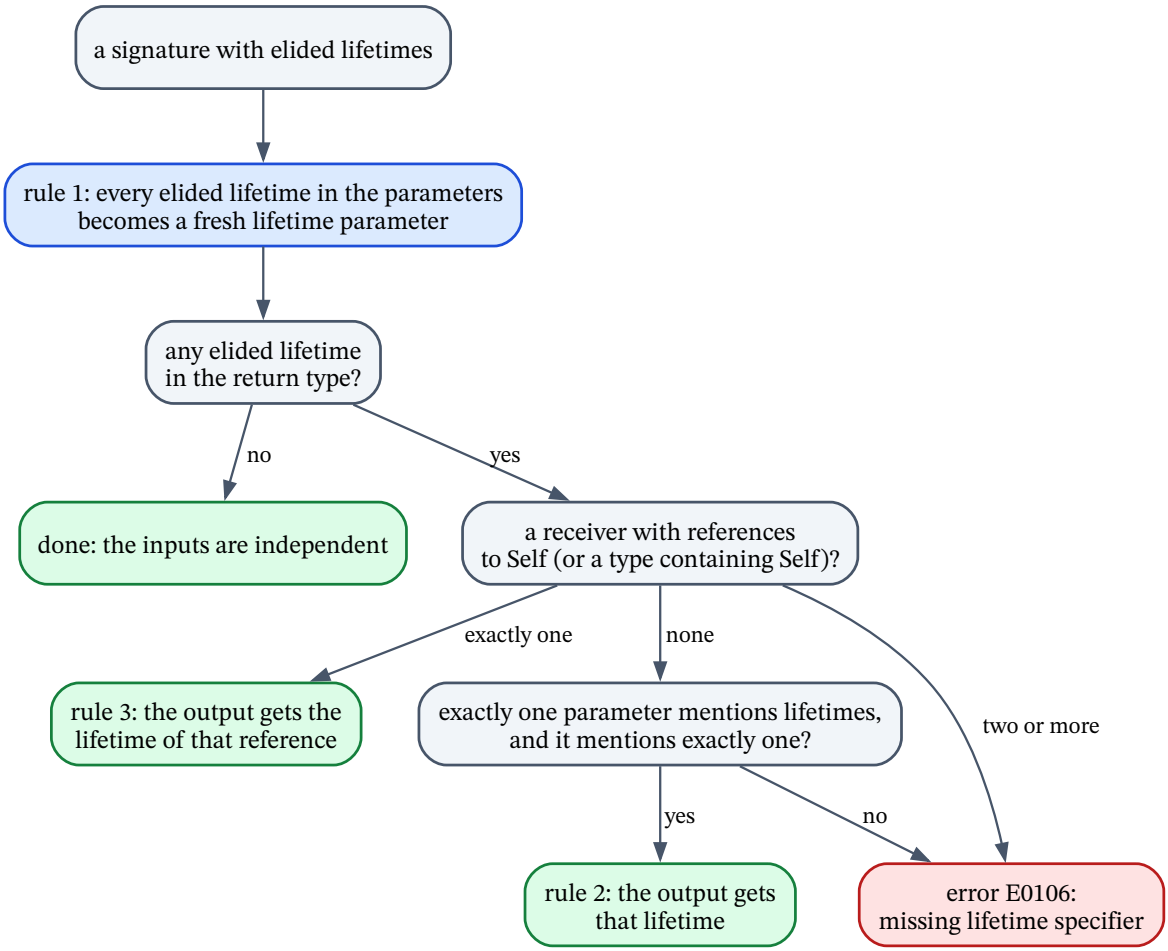
3.2 The three rules

Here are the rules as rustc applies them to a function signature. Rule 1 handles the elided lifetimes in the inputs. If the return type also contains elided lifetimes, rule 3 or rule 2 decides what they become:

1. **Inputs.** Each elided lifetime in the parameters becomes a fresh lifetime parameter, distinct from all the others.
2. **One input.** If exactly one parameter mentions lifetimes, and it mentions exactly one lifetime (elided or named, possibly several times), that lifetime is assigned to every elided lifetime in the output.

3. **Methods.** If the receiver contains exactly one reference to `Self` (or to a type that contains `Self`), as `&self` and `&mut self` do, the lifetime of that reference is assigned to every elided lifetime in the output, whatever the other parameters are.

If the output contains an elided lifetime and neither rule 2 nor rule 3 gives it a value, elision fails with E0106, "missing lifetime specifier". Rule 3 is checked first: when a method's receiver contains a reference to `Self`, the other parameters are not counted at all. The figure shows the order in which rustc takes the decisions.



How rustc fills in the elided lifetimes of a signature: rule 1 for the inputs, then the receiver rule, then the single-parameter rule, otherwise E0106.

The numbering follows the Rust Reference (https://doc.rust-lang.org/reference/lifetime-elision.html), which states the rules like this:

> Each elided lifetime in the parameters becomes a distinct lifetime parameter.
>
> If there is exactly one lifetime used in the parameters (elided or not), that lifetime is assigned to *all* elided output lifetimes.
>
> If the receiver has type `&Self` or `&mut Self`, then the lifetime of that reference to `Self` is assigned to all elided output lifetime parameters.

The first rule matches what rustc does. The second and the third do not quite: as the programs below show, rustc counts *parameters* where the Reference counts lifetimes, and rustc's `self` rule covers more receivers than `&Self` and `&mut Self`. Where the two disagree, this book shows what rustc does.

Rule 1: a fresh lifetime for every elided input

Rule 1 on its own never fails: any number of inputs can get any number of fresh lifetimes. `fn f(x: &mut &str)` becomes `fn f<'a, 'b>(x: &'a mut &'b str)`: two positions, two independent lifetimes. Independence is the right default, because it is the most flexible contract for callers, who may then pass references of any lifetimes at all. But it is also a promise to the callers, and the body has to keep it. Here is a function whose body needs a relation between two of its inputs, from `code/compile_fail/ch03_push_elided.rs`:

```
pub fn push(list: &mut Vec<&str>, item: &str) {
    list.push(item);
}
```

The compiler rejects it:

```
error: lifetime may not live long enough
 --> compile_fail/ch03_push_elided.rs:7:5
  |
6 | pub fn push(list: &mut Vec<&str>, item: &str) {
  |                           -            - let's call the lifetime of this reference `'1`
  |                           |
  |                           let's call the lifetime of this reference `'2`
7 |     list.push(item);
  |     ^^^^^^^^^^^^^^^ argument requires that `'1` must outlive `'2`
  |
  = note: requirement occurs because of a mutable reference to `Vec<&str>`
  = note: mutable references are invariant over their type parameter
  = help: see <https://doc.rust-lang.org/nomicon/subtyping.html> for more information about
variance
help: consider introducing a named lifetime parameter
  |
6 | pub fn push<'a>(list: &mut Vec<&'a str>, item: &'a str) {
  |            ++++                ++              ++
```

Read the diagnostic from the top. The error has no code: "lifetime may not live long enough" is the borrow checker's general message for a relation between two lifetimes that the signature does not guarantee. Since both lifetimes are anonymous, the compiler names them for the occasion: `'2` for the strings already in the vector and `'1` for `item`. The label under `list.push(item)` states the missing relation, "`'1` must outlive `'2`": storing `item` among the vector's elements is only safe if `item` lives at least as long as they must. The two notes about invariance explain why the compiler cannot simply shorten `'2` to make things fit; that is the subject of Chapter 4. The final help proposes exactly the contract the function needs, which is `code/examples/ch03_push_contract.rs`:

```
fn push<'a>(list: &mut Vec<&'a str>, item: &'a str) {
    list.push(item);
}
```

```
fn main() {
    let first = String::from("first");
    let second = String::from("second");
    let mut list: Vec<&str> = Vec::new();
```

```
    push(&mut list, &first);
    push(&mut list, &second); // the first `&mut list` has already ended
    println!("{} items: {list:?}", list.len());
}
```

```
2 items: ["first", "second"]
```

Notice what the fix does *not* do: it does not give the `&mut` borrow of the vector the name `'a`. That borrow stays elided, so it gets its own fresh lifetime, which can be as short as the call. That is why the caller can call `push` twice and then read `list`. Keep this program in mind; it comes back at the end of the chapter, with a rule that names too much.

Rule 2: exactly one parameter, exactly one lifetime

Rule 2 covers the most common signature of all, a function that takes one reference and returns a part of it. What it counts is easy to misremember, so here are three functions it accepts, from `code/examples/ch03_rule2_one_parameter.rs`:

```
// `index` has no lifetime, so `line` is the only parameter that mentions one.
fn field(line: &str, index: usize) -> &str {
    line.split(',').nth(index).unwrap_or("").trim()
}

// The `&str` inside `fn(&str) -> bool` belongs to the fn pointer type itself,
// which elides on its own: `pred` mentions no lifetime of `first_where`.
fn first_where(line: &str, pred: fn(&str) -> bool) -> &str {
    line.split_whitespace()
        .find(|word| pred(word))
        .unwrap_or("")
}

// `S` may well be a type that borrows (`&str` below), but elision only counts
// the lifetimes that are written in the signature.
fn last_field<S: AsRef<str>>(line: &str, separator: S) -> &str {
    line.rsplit(separator.as_ref()).next().unwrap_or("")
}
```

In each function `line` is the only parameter that mentions a lifetime, and it mentions one, so the output gets it: `fn field<'l>(line: &'l str, index: usize) -> &'l str`. The other parameters do not count, for three different reasons. `index` has no lifetime at all. `pred` has a reference in its type, but that `&str` belongs to the function pointer type `fn(&str) -> bool`, which is a small signature with its own elision (see *Function pointers and `Fn` sugar* below); it says nothing about the lifetimes of `first_where`. And `S` is a type parameter: the caller may choose a type that borrows (the program passes a `&'static str`), but elision only counts what is written in the signature, and no lifetime is written in `S`.

Rule 2 counts per *parameter*, not per reference. A single parameter can mention two lifetimes, and then rule 2 does not apply. The classic example is a slice of string slices; the program `code/compile_fail/ch03_rule2_slice_of_refs.rs` returns its longest element:

```
pub fn longest_word(words: &[&str]) -> &str {
    words.iter().copied().max_by_key(|w| w.len()).unwrap_or("")
}
```

```
error[E0106]: missing lifetime specifier
 --> compile_fail/ch03_rule2_slice_of_refs.rs:6:40
  |
6 | pub fn longest_word(words: &[&str]) -> &str {
  |                            -------     ^ expected named lifetime parameter
  |
  = help: this function's return type contains a borrowed value, but the signature does not
say which one of `words`'s 2 lifetimes it is borrowed from
help: consider introducing a named lifetime parameter
  |
6 | pub fn longest_word<'a>(words: &'a [&'a str]) -> &'a str {
  |                     ++++         ++   ++           ++
```

The labels are precise. The dashes under `&[&str]` mark the parameter that supplies lifetimes, the caret under the `&` of the return type marks the elided output lifetime that received no value ("expected named lifetime parameter"), and the help says what is missing: "which one of `words`'s 2 lifetimes it is borrowed from". The borrow of the slice has one lifetime, the words inside it have another, and a returned `&str` could be tied to either: to the borrow of the slice, if it was obtained through the slice, or to the strings themselves, if it is one of the elements copied out.

The compiler suggests naming everything `'a`. That compiles, but it is not the best contract here. `longest_word` returns one of the elements, so the result only needs to borrow from what the elements borrow from. The program `code/examples/ch03_slice_of_refs.rs` says so, and gains a caller in the process:

```rust
fn longest_word<'w>(words: &[&'w str]) -> &'w str {
    words.iter().copied().max_by_key(|w| w.len()).unwrap_or("")
}

// Elided: fn longest_in<'t>(text: &'t str) -> &'t str
fn longest_in(text: &str) -> &str {
    let words: Vec<&str> = text.split_whitespace().collect();
    longest_word(&words) // borrows from `text`, not from the local vector
}
```

```
longest
```

`longest_in` collects the words into a local vector and returns the longest. With the compiler's suggestion instead, the result would be tied to that local vector, and `longest_in` would no longer compile. The program `code/compile_fail/ch03_slice_suggested.rs` shows it; E0515 means "this function returns a reference to something it owns, which is about to be dropped", and Chapter 5 has more on it:

```rust
pub fn longest_word<'a>(words: &'a [&'a str]) -> &'a str {
    words.iter().copied().max_by_key(|w| w.len()).unwrap_or("")
}

pub fn longest_in(text: &str) -> &str {
    let words: Vec<&str> = text.split_whitespace().collect();
    longest_word(&words)
}
```

```
error[E0515]: cannot return value referencing local variable `words`
  --> compile_fail/ch03_slice_suggested.rs:12:5
   |
12 |     longest_word(&words)
   |     ^^^^^^^^^^^^^------^
   |     |            |
```

```
   |     |           `words` is borrowed here
   |     returns a value referencing data owned by the current function
```

A suggestion in a diagnostic is a signature that makes *this* error go away. It is often the most restrictive one that does. Whether it is the contract you want is a design question that only you can answer, and we will meet it again in *When elision fails: E0106* below.

Named lifetimes count exactly like elided ones, including `'static`:

```
// One parameter, one lifetime written twice: rule 2 applies.
#[allow(mismatched_lifetime_syntaxes)] // the lint is discussed below
fn first_of<'a>(pair: (&'a str, &'a str)) -> &str {
    pair.0
}

// A named lifetime counts like an elided one, `'static` included: the result is
// a `&'static str`.
#[allow(mismatched_lifetime_syntaxes)]
fn static_prefix(text: &'static str, len: usize) -> &str {
    &text[..len]
}
```

The whole program prints:

```
field 1:     "name"
first_where: "8080"
last_field:  " extra"
words:       6
first_of:    "left"
prefix:      "local"
```

`first_of` has one parameter that mentions one lifetime, twice, so rule 2 applies. `static_prefix` has one parameter that mentions `'static`, so its output is `'static` as well: `main` stores the result in a variable of type `&'static str`.

Here is where rustc and the Reference part ways. Two parameters that use the *same* named lifetime (`code/compile_fail/ch03_rule2_shared_name.rs`):

```
pub fn first<'a>(x: &'a str, y: &'a str) -> &str {
    x
}
```

```
error[E0106]: missing lifetime specifier
 --> compile_fail/ch03_rule2_shared_name.rs:7:45
  |
7 | pub fn first<'a>(x: &'a str, y: &'a str) -> &str {
  |                     -------     -------     ^ expected named lifetime parameter
  |
  = help: this function's return type contains a borrowed value with an elided lifetime, but
the lifetime cannot be derived from the arguments
help: consider using the `'a` lifetime
  |
7 | pub fn first<'a>(x: &'a str, y: &'a str) -> &'a str {
  |                                              ++
```

The Reference's rule 2 speaks of "exactly one lifetime used in the parameters", and this signature uses exactly one, `'a`. rustc counts the parameters that mention lifetimes, finds two, and does not apply rule 2. (The help is worded differently from the case of two elided inputs below, "the

lifetime cannot be derived from the arguments", but it is the same E0106.) The same happens with `fn f(x: &'static str, y: &'static str) -> &str`: one lifetime, two parameters, E0106.

In practice you rarely notice the difference, because every signature that rule 2 accepts with a named input lifetime and an elided output triggers a warning. The two functions above carry `#[allow(mismatched_lifetime_syntaxes)]`; without it, rustc warns about both: "eliding a lifetime that's named elsewhere is confusing". The `mismatched_lifetime_syntaxes` lint, warn-by-default since Rust 1.89, fires whenever a signature refers to one lifetime in two different ways: `'a` in an input and an elided `&str` in the output, or an elided `&str` in an input and a type with a hidden lifetime parameter in the output, such as `SplitWhitespace` without `<'_>` ("hiding a lifetime that's elided elsewhere is confusing"). The fix is always to write the lifetime the same way everywhere: `-> &'a str`, `-> &'static str`, `-> SplitWhitespace<'_>`. So once you have named a lifetime in the inputs, name it in the output too. Chapter 6 covers this lint and its allow-by-default relative `elided_lifetimes_in_paths` in detail.

Rule 3: methods borrow from `self`

A method that takes `&self` and returns a reference usually returns a part of `self`: a field, an element, a slice of a buffer. Rule 3 makes that the default, even when other parameters have lifetimes too. From `code/examples/ch03_self_receivers.rs`:

```
// Two inputs with lifetimes, but rule 3 picks `&self`:
// fn get<'s, 'k>(&'s self, key: &'k str) -> Option<&'s str>
fn get(&self, key: &str) -> Option<&str> {
    let entry = self.entries.iter().find(|(k, _)| k == key);
    entry.map(|(_, value)| value.as_str())
}
```

Rule 2 would fail here, since two parameters mention lifetimes, but rule 3 comes first and gives the result the lifetime of `&self`. For the caller, this means that the key can be a temporary:

```
let config = Config {
    name: String::from("server"),
    entries: vec![(String::from("mode"), String::from("fast"))],
};
let mode;
{
    let key = String::from("mode"); // a temporary key...
    mode = config.get(&key);
} // ...dropped here: fine, the result borrows from `config` only
println!("mode = {mode:?}");
```

```
mode = Some("fast")
```

This default is a convention, not a deduction, and it is wrong whenever the result really comes from another argument. A method that returns (a part of) its argument must say so, as in `fn pick<'k>(&self, key: &'k str) -> &'k str`. The most important case is a struct that holds a reference and returns data borrowed from it rather than from `self`; Chapter 2 is about exactly that (`Option<&'src str>` instead of the elided `Option<&str>`).

Receivers other than `&self`

A method can take `self` in other forms: `self: Box<Self>`, `self: Rc<Self>`, `self: Pin<&mut Self>`, and references to those, such as `self: &Rc<Self>`. Since Rust 1.81, rustc's rule 3 looks inside the receiver type for references whose referent *contains* `Self`:

```
// The reference points to a type that contains `Self` (Rust 1.81 and later).
fn name_shared(self: &Rc<Self>, _requested_by: &str) -> &str {
    &self.name
}

fn name_boxed(self: &Box<Self>, _requested_by: &str) -> &str {
    &self.name
}

// The reference points directly to `Self`, inside `Pin`.
fn name_pinned(self: Pin<&mut Self>, _requested_by: &str) -> &str {
    &self.get_mut().name
}
```

`&Rc<Self>` and `&Box<Self>` are references to types that contain `Self`, and `Pin<&mut Self>` contains a reference directly to `Self`. In all three methods the result borrows from the receiver, not from `_requested_by`.

If the receiver contains no reference to `Self`, rule 3 does not apply, and rule 2 looks at the other parameters. A `self: Box<Self>` owns the value and has nothing to lend:

```
// A receiver without a reference lends nothing, so rule 3 does not apply and
// rule 2 looks at the other parameters: the result borrows from `line`.
fn strip_name(self: Box<Self>, line: &str) -> &str {
    line.strip_prefix(self.name.as_str()).unwrap_or(line)
}
```

The rest of the program calls the four methods:

```
shared
boxed
pinned
: ready
```

If the receiver contains *two or more* references to `Self`, rule 3 is ambiguous, and rustc gives up rather than fall back to the other parameters (`code/compile_fail/ch03_self_two_refs.rs`):

```
pub struct Config {
    name: String,
}

impl Config {
    pub fn name(self: &&Self, _caller: &str) -> &str {
        &self.name
    }
}
```

```
error[E0106]: missing lifetime specifier
  --> compile_fail/ch03_self_two_refs.rs:11:49
   |
11 |     pub fn name(self: &&Self, _caller: &str) -> &str {
   |                       ------          ----     ^ expected named lifetime parameter
   |
   = help: this function's return type contains a borrowed value, but the signature does not
say whether it is borrowed from one of `self`'s 2 lifetimes or `_caller`
help: consider introducing a named lifetime parameter
```

```
   |
11 |     pub fn name<'a>(self: &'a &'a Self, _caller: &'a str) -> &'a str {
   |                ++++        ++  ++                 ++          ++
```

The help counts the candidates for you: "one of `self`'s 2 lifetimes", one for each `&` in `&&Self`, "or `_caller`".

> **NOTE**
>
> The receiver rule changed in Rust 1.81 (https://github.com/rust-lang/rust/pull/117967). Before, only a reference *directly* to `Self` counted, as in `&self`, `&mut self` and `Pin<&mut Self>`. A receiver such as `self: &Box<Self>` contributed no lifetime, so `fn f(self: &Box<Self>) -> &u32` was rejected, and in `fn e(self: &mut Box<Self>, arg: &usize) -> &usize` the result borrowed from `arg`; since 1.81 it borrows from `self`. Some receivers with two references, such as `self: &Box<&Self>`, were accepted before and are E0106 now. The Reference's wording still mentions only `&Self` and `&mut Self`.

Elision does not see through `Self`

The rules count the lifetimes that are *written*. In an `impl` block for a type with a lifetime parameter, `self` has the type `Self`, which is `Wrapper<'a>`, but no lifetime is written in the receiver (`code/compile_fail/ch03_self_by_value.rs`):

```rust
pub struct Wrapper<'a> {
    text: &'a str,
}

impl<'a> Wrapper<'a> {
    pub fn into_text(self) -> &str {
        self.text
    }
}
```

```
error[E0106]: missing lifetime specifier
  --> compile_fail/ch03_self_by_value.rs:11:31
   |
11 |     pub fn into_text(self) -> &str {
   |                               ^ expected named lifetime parameter
   |
   = help: this function's return type contains a borrowed value, but there is no value for
it to be borrowed from
help: consider using the `'a` lifetime
   |
11 |     pub fn into_text(self) -> &'a str {
   |                                ++
```

"There is no value for it to be borrowed from": as far as elision is concerned, `into_text` has no input lifetime at all. The help shows the fix, and here it is also the right contract: `into_text` consumes the wrapper and returns the text it was holding, which stays valid for `'a`, however long the wrapper itself lived.

3.3 You write, the compiler reads

With the three rules in hand, every signature can be expanded by hand. The table collects the common shapes; the rows below the rule-based ones are the places where elision works differently, each explained in its own section after the table.

You write	The compiler reads	Rule
`fn f(x: &str) -> &str`	`fn f<'a>(x: &'a str) -> &'a str`	1, 2
`fn f(x: &str, n: usize) -> &str`	`fn f<'a>(x: &'a str, n: usize) -> &'a str`	1, 2
`fn f(x: &mut &str)`	`fn f<'a, 'b>(x: &'a mut &'b str)`	1
`fn f(x: &str, y: &str) -> &str`	E0106	none applies
`fn f<'a>(x: &'a str, y: &str) -> &str`	E0106	none applies
`fn f(x: &[&str]) -> &str`	E0106	none applies
`fn m(&self, k: &str) -> &str`	`fn m<'s, 'k>(&'s self, k: &'k str) -> &'s str`	1, 3
`fn m(self: &Rc<Self>) -> &str`	`fn m<'s>(self: &'s Rc<Self>) -> &'s str`	1, 3 (1.81)
`fn f(s: &str) -> SplitWhitespace<'_>`	`fn f<'a>(s: &'a str) -> SplitWhitespace<'a>`	1, 2
`fn(&str) -> &str`	`for<'a> fn(&'a str) -> &'a str`	1, 2 inside the type
`dyn Fn(&str) -> Vec<&str>`	`dyn for<'a> Fn(&'a str) -> Vec<&'a str>`	1, 2 inside the bound
`impl Trait for &str`	`impl<'a> Trait for &'a str`	like 1
`static S: &str`	`static S: &'static str`	`'static`
`const C: &[&str]`	`const C: &'static [&'static str]`	`'static`
`const C: &str` in an `impl`	`const C: &'static str`	`'static` (1.99)
`struct S { r: &str }`	E0106	no elision

Function pointers and `Fn` sugar

The Reference's chapter on elision opens with "lifetime arguments can be *elided* in function item, function pointer, and closure trait signatures". A function pointer type such as `fn(&str) -> &str`, and the `Fn(&str) -> &str` sugar of the closure traits, are signatures too, and the rules apply to them on their own, inside the type. The lifetimes they produce belong to the type itself: `fn(&str) -> &str` is `for<'a> fn(&'a str) -> &'a str`, which reads "for every lifetime `'a`, a function from `&'a str` to `&'a str`". Such a type is called *higher-ranked*, and Chapter 9 is about it; here the point is only where the lifetimes come from.

Because their lifetimes are bound inside the type, these types can appear where elision is otherwise not allowed, such as struct fields (`code/examples/ch03_fn_types.rs`):

```
// Each fn type below is a small signature with its own elision:
//   fn(&str) -> &str             is  for<'a> fn(&'a str) -> &'a str
//   dyn Fn(&str) -> Vec<&str>    is  dyn for<'a> Fn(&'a str) -> Vec<&'a str>
struct Pipeline {
    normalize: fn(&str) -> &str,
    split: Box<dyn Fn(&str) -> Vec<&str>>,
}

impl Pipeline {
```

```
    fn run<'t>(&self, text: &'t str) -> Vec<&'t str> {
        (self.split)((self.normalize)(text))
    }
}
```

```
["red", "green", "blue"]
```

`Pipeline` has no lifetime parameter and needs none: each field holds a function that works for strings of *any* lifetime, and the explicit `'t` of `run` decides, call by call, which lifetime the results have. (A `Box<dyn ...>` has one more elided lifetime, the bound of the trait object itself, which follows different defaults: here it is `'static`. Chapter 8 has the rules.)

The program `code/compile_fail/ch03_fn_pointer_two_inputs.rs` shows that elision inside a function type fails exactly like elision in a function:

```
pub type Choose = fn(&str, &str) -> &str;
```

```
error[E0106]: missing lifetime specifier
 --> compile_fail/ch03_fn_pointer_two_inputs.rs:5:37
  |
5 | pub type Choose = fn(&str, &str) -> &str;
  |                       ----  ----     ^ expected named lifetime parameter
  |
  = help: this function's return type contains a borrowed value, but the signature does not
say whether it is borrowed from argument 1 or argument 2
  = note: for more information on higher-ranked polymorphism, visit https://doc.rust-lang.
org/nomicon/hrtb.html
help: consider making the type lifetime-generic with a new `'a` lifetime
  |
5 | pub type Choose = for<'a> fn(&'a str, &'a str) -> &'a str;
  |                   +++++++     ++       ++          ++
help: consider introducing a named lifetime parameter
  |
5 | pub type Choose<'a> = fn(&'a str, &'a str) -> &'a str;
  |                ++++       ++       ++          ++
```

The parameters of a function type have no names, so the help speaks of "argument 1 or argument 2". The two suggestions are different contracts. With `for<'a>`, the type stays higher-ranked: one function that works for every lifetime. With a lifetime parameter on the type alias, `Choose<'a>` becomes a family of types, one for each particular `'a`. Chapter 9 explains when you want which.

`impl` headers

```
trait Describe {
    fn describe(&self) -> String;
}

// The compiler reads: impl<'a> Describe for &'a str
impl Describe for &str {
    fn describe(&self) -> String {
        format!("{} bytes of text", self.len())
    }
}
```

```
18 bytes of text
```

Every elided lifetime in an `impl` header becomes a fresh lifetime parameter of the `impl`, as in rule 1. A header has no output, so nothing can fail. The methods inside then follow their own elision; `describe` returns a `String`, so it has nothing to elide. For a type with a lifetime parameter, the header is written with `'_`, as in `impl fmt::Display for Wrapper<'_>`; Chapter 6 discusses `'_` in headers and when you need a real name instead.

`static` and `const` items, and associated consts

An item has neither inputs nor outputs, so the three rules have nothing to work with. Its value must stay valid for as long as the item can be used, which is the whole program, so an elided lifetime can only mean one thing. In the words of the Reference: "Both constant and static declarations of reference types have *implicit* `'static` lifetimes unless an explicit lifetime is specified." From `code/examples/ch03_static_items.rs`:

```
// The compiler reads: static GREETING: &'static str
static GREETING: &str = "hello";
// The compiler reads: const STOP_WORDS: &'static [&'static str]
const STOP_WORDS: &[&str] = &["a", "an", "the"];
```

Since Rust 1.99, the same holds for associated consts, the constants declared in an `impl` block or a trait, including those in an `impl` with lifetime parameters of its own:

```
struct Tokenizer<'src> {
    text: &'src str,
}

impl<'src> Tokenizer<'src> {
    // The compiler reads: const DELIMITERS: &'static [char]
    const DELIMITERS: &[char] = &[' ', ',', ';'];

    fn words(&self) -> Vec<&'src str> {
        let pieces = self.text.split(Self::DELIMITERS);
        pieces
            .filter(|w| !w.is_empty() && !STOP_WORDS.contains(w))
            .collect()
    }

    // Compiles only because DELIMITERS is `'static`, not `'src`.
    fn delimiters() -> &'static [char] {
        Self::DELIMITERS
    }
}
```

```
hello: ["parser", "engine", "tokenizer"]
delimiters: [' ', ',', ';']
```

`delimiters` is the proof: inside `impl<'src>`, it returns `Self::DELIMITERS` as a `&'static [char]`, which compiles only because the constant does not borrow for `'src`.

> **NOTE**
>
> Before Rust 1.99, an elided lifetime in an associated const meant `'static` only when no other lifetime was in scope. rustc 1.98 rejects the `DELIMITERS` constant above, inside `impl<'src>`, with the deny-by-default future-compatibility lint `elided_lifetimes_in_associated_constant`, and reports E0106 for `const C: &str;` in a trait that has a lifetime parameter. The 1.99 release notes list the change as "Infer anonymous lifetimes in the types of associated consts as `'static`".

3.4 When elision fails: E0106

Now the two signatures that every Rust programmer meets sooner or later. Take each one and apply the rules mechanically.

Case A: two references in, one out

The first is in `code/compile_fail/ch03_two_inputs.rs`:

```
pub fn longer(x: &str, y: &str) -> &str {
    if x.len() >= y.len() { x } else { y }
}
```

Rule 1 gives each elided input its own lifetime. Calling them `'1` and `'2`, and the output lifetime that we are looking for `'?` (a notation, not Rust syntax), the signature so far is:

```
fn longer<'1, '2>(x: &'1 str, y: &'2 str) -> &'? str
```

There is no receiver, so rule 3 does not apply. Two parameters mention lifetimes, so rule 2 does not apply either. Nothing gives `'?` a value, and the compiler reports E0106:

```
error[E0106]: missing lifetime specifier
 --> compile_fail/ch03_two_inputs.rs:6:36
  |
6 | pub fn longer(x: &str, y: &str) -> &str {
  |                  ----     ----     ^ expected named lifetime parameter
  |
  = help: this function's return type contains a borrowed value, but the signature does not
say whether it is borrowed from `x` or `y`
help: consider introducing a named lifetime parameter
  |
6 | pub fn longer<'a>(x: &'a str, y: &'a str) -> &'a str {
  |              ++++     ++          ++          ++
```

Read it label by label. The dashes under the two input references mark the candidates. The caret under the `&` of the return type marks the lifetime that has no value: "expected named lifetime parameter". The help line asks the question the compiler cannot answer by itself: "the signature does not say whether it is borrowed from `x` or `y`". The suggestion below it, with `++` under each piece of inserted text, is one possible answer, not the only one. `rustc --explain E0106` prints a longer description of the error with more examples.

Case B: naming one input is not enough

The second is in `code/compile_fail/ch03_named_and_elided.rs`:

```
pub fn first<'a>(x: &'a str, y: &str) -> &str {
    x
}
```

```
error[E0106]: missing lifetime specifier
 --> compile_fail/ch03_named_and_elided.rs:6:42
  |
6 | pub fn first<'a>(x: &'a str, y: &str) -> &str {
  |                     -------     ----     ^ expected named lifetime parameter
  |
  = help: this function's return type contains a borrowed value, but the signature does not
say whether it is borrowed from `x` or `y`
help: consider using the `'a` lifetime
  |
6 | pub fn first<'a>(x: &'a str, y: &str) -> &'a str {
  |                                           ++
```

Rule 2 counts every parameter that mentions a lifetime, named or elided, and there are two. Naming `x` does not make it "the" input; the compiler does not assume that the named parameter, or the elided one, is the intended source. The help suggests `'a` for the output because `'a` is the only name available to suggest. It is a suggestion, not an assumption: the error stands until you write it.

Why the compiler refuses to guess

In both cases the body leaves no doubt: `longer` returns `x` or `y`, and `first` returns `x`. Why doesn't the compiler use that? Because the signature is the contract that callers are checked against, and callers are checked *without* looking at the body. If the body determined the contract, then editing the body, say to return `y` instead of `x`, would silently change the signature that callers in other modules and other crates rely on, and they would break without any visible change to the API. Requiring the relationship to be written keeps the signature authoritative: a change of contract is a change of the signature, visible in code review and in the documentation (rustdoc shows signatures, not bodies). The last section of this chapter tests this argument with a program.

Three honest contracts

When elision fails, the compiler is asking you to decide what the result may borrow from. For two string inputs there are three honest answers, which `code/examples/ch03_three_contracts.rs` puts side by side:

```
// 1. The result may come from EITHER input: tie all three together.
fn longer<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() >= y.len() { x } else { y }
}

// 2. The result comes only from `x`: say so, and leave `y` independent.
fn first_only<'a>(x: &'a str, _y: &str) -> &'a str {
    x
}

// 3. The result borrows from neither: return an owned value.
fn joined(x: &str, y: &str) -> String {
    format!("{x}{y}")
}
```

```
fn main() {
    let long_lived = String::from("long-lived");
```

```
    let kept;
    {
        let short_lived = String::from("short");
        kept = first_only(&long_lived, &short_lived); // does not borrow `short_lived`
        println!("{}", longer(&long_lived, &short_lived)); // used before the `}`
        println!("{}", joined(&long_lived, &short_lived));
    }
    println!("{kept}");
}
```

```
long-lived
long-livedshort
long-lived
```

The caller shows what each contract allows. The result of `first_only` is stored in `kept` and outlives `short_lived`, because the contract says it does not borrow from `y`. The result of `longer` must be used before the closing brace, because it may borrow from either argument. The result of `joined` borrows from nothing.

Signature	The result may borrow from	For the caller
`fn f<'a>(x: &'a str, y: &'a str) -> &'a str`	either input	the result must not outlive either argument
`fn f<'a>(x: &'a str, y: &str) -> &'a str`	only `x`	`y` may die while the result lives; the body may not return `y`
`fn f(x: &str, y: &str) -> String`	neither	no borrow at all

The choice is a real design decision, not a syntax chore. The first contract is the one the compiler suggests and the one people reach for, but it is the most restrictive for callers; prefer the second whenever the result can only come from one input.

3.5 Why a struct field cannot be elided

Strip the problem to its minimum: one struct, one field, one reference. If elision ever worked for a type definition, it would work here (`code/compile_fail/ch03_struct_field_elided.rs`):

```
pub struct Holder {
    text: &str,
}
```

```
error[E0106]: missing lifetime specifier
 --> compile_fail/ch03_struct_field_elided.rs:6:11
  |
6 |     text: &str,
  |           ^ expected named lifetime parameter
  |
help: consider introducing a named lifetime parameter
  |
5 ~ pub struct Holder<'a> {
6 ~     text: &'a str,
  |
```

The error is E0106 although there is only one reference and nothing to be ambiguous about. The suggestion changes two lines (marked with `~`): it declares a lifetime parameter on the struct and

uses it in the field. Neither placeholder helps (`code/compile_fail/ch03_struct_field_placeholders.rs`, which contains two errors):

```
pub struct Anonymous {
    text: &'_ str,
}

pub struct Undeclared {
    text: &'a str,
}
```

```
error[E0106]: missing lifetime specifier
 --> compile_fail/ch03_struct_field_placeholders.rs:7:12
  |
7 |     text: &'_ str,
  |            ^^ expected named lifetime parameter
  |
help: consider introducing a named lifetime parameter
  |
6 ~ pub struct Anonymous<'a> {
7 ~     text: &'a str,
  |

error[E0261]: use of undeclared lifetime name `'a`
  --> compile_fail/ch03_struct_field_placeholders.rs:11:12
   |
11 |     text: &'a str,
   |            ^^ undeclared lifetime
   |
help: consider introducing lifetime `'a` here
   |
10 | pub struct Undeclared<'a> {
   |                       ++++
```

`'_` in a field is E0106 again, and a name that was never declared is E0261, "use of undeclared lifetime name". The reasons are structural, not a matter of missing rules:

1. **Elision is defined for signatures.** Its rules choose the output lifetimes from the input lifetimes. A struct definition has no inputs and no outputs; a field is not a parameter, so "exactly one parameter" has nothing to count. That there is a single reference is irrelevant.
2. **In a struct, the lifetime is part of the type.** The fresh lifetimes of rule 1 are local to one signature and change nothing outside it. A field's lifetime would have to become a parameter of the struct itself: `Holder` would secretly be `Holder<'x>`. Every use of the type, every `impl`, every struct that contains a `Holder`, and the variance of the type (Chapter 4) depend on that parameter, yet the declaration `struct Holder` would not show it. Rust requires the parameter to be written where the type is declared.
3. **The only plausible default would be `'static`, and it is a trap.** `static` and `const` items default to `'static` because there the answer is forced. For a field it would quietly restrict the type to literals and leaked data, which is rarely what the author of a struct with a reference wants.

The program `code/compile_fail/ch03_static_field_local.rs` shows what the third point would cost, written out with an explicit `'static`:

```
pub struct Literal {
    text: &'static str,
}
```

```
pub fn caller() {
    let name = String::from("hello");
    // A borrow of a local variable is not 'static:
    let lit = Literal { text: &name };
    println!("{}", lit.text);
}
```

```
error[E0597]: `name` does not live long enough
  --> compile_fail/ch03_static_field_local.rs:13:31
   |
11 |     let name = String::from("hello");
   |         ---- binding `name` declared here
12 |     // A borrow of a local variable is not 'static:
13 |     let lit = Literal { text: &name };
   |                               ^^^^^
   |                               |
   |                               borrowed value does not live long enough
   |                               this usage requires that `name` is borrowed for `'static`
14 |     println!("{}", lit.text);
15 | }
   | - `name` dropped here while still borrowed
```

The label "this usage requires that `name` is borrowed for `'static`" points at the field initializer: a `&'static str` field accepts no borrow of a local variable, however briefly the struct is used. Rust makes you choose instead, and each choice means something different for everyone who uses the struct:

Field	Meaning	Consequence
`text: &'a str` in `struct S<'a>`	borrows from some string	`S` cannot outlive that string; `'a` appears in every use of the type
`text: String`	owns its data	no lifetime anywhere; costs an allocation or a clone
`text: &'static str`	literals or leaked data only	no lifetime parameter, but only `'static` data fits

The program `code/examples/ch03_struct_field_choices.rs` shows the three side by side:

```
// 1. Borrow: the lifetime becomes a parameter of the TYPE.
struct Borrowed<'a> {
    text: &'a str,
}

// 2. Own the data: no lifetime anywhere.
struct Owned {
    text: String,
}

// 3. Literals (or leaked data) only: an explicit `'static`.
struct Literal {
    text: &'static str,
}
```

```
hello hello literal
```

Once the parameter is declared, the *uses* of the type elide like any other lifetime. In a signature, `Borrowed<'_>` takes part in rules 1 and 2:

```
// Once the parameter is declared, signatures that USE the type elide as usual:
// the compiler reads fn wrap<'a>(text: &'a str) -> Borrowed<'a>
fn wrap(text: &str) -> Borrowed<'_> {
    Borrowed { text }
}
```

You may even write `Borrowed` without `<'_>` in a signature; the hidden lifetime is still elided, but the lint `mismatched_lifetime_syntaxes` warns that it hides a borrow (Chapter 6). Only the *definition* must be explicit. And remember `Pipeline` above: fields of function pointer and `dyn Fn` types are fine, because their lifetimes are bound inside the field's own type, not by the struct. Chapter 14 returns to the choice between borrowing and owning in a struct.

3.6 Candidate rules that would not work

In `longer` the right answer feels obvious because *we* have read the body. Elision may not read the body, so any extra rule would have to choose from the signature alone. To see why that is not enough, take a function whose correct contract is *not* the obvious one, and test the three most natural candidate rules on it. These rules are hypothetical: Rust has none of them. For each one, the signature it would produce is written out by hand, and the compiler judges it.

The function, in `code/examples/ch03_lookup_explicit.rs`, looks a key up in a table. The result borrows from the *table*, not from the key, and the key may be a temporary that is dropped first, which is perfectly valid:

```
struct Table {
    rows: Vec<(String, String)>,
}

fn lookup<'t>(key: &str, table: &'t Table) -> Option<&'t str> {
    for (name, value) in &table.rows {
        if name == key {
            return Some(value.as_str());
        }
    }
    None
}
```

```
fn main() {
    let table = Table {
        rows: vec![("a".to_string(), "alpha".to_string())],
    };
    let hit;
    {
        let key = String::from("a"); // a temporary key...
        hit = lookup(&key, &table);
    } // ...dropped here, but the result comes from `table`
    println!("{hit:?}");
}
```

```
Some("alpha")
```

Candidate A: the first reference parameter

This is the rule most people propose first, and for `longer`-like code it looks harmless. For `lookup` it produces (`code/compile_fail/ch03_rule_first.rs`):

```
pub fn lookup<'k, 't>(key: &'k str, table: &'t Table) -> Option<&'k str> {
    for (name, value) in &table.rows {
        if name == key {
            return Some(value.as_str());
        }
    }
    None
}
```

with the same caller as before, written as a function that receives the table:

```
pub fn caller(table: &Table) {
    let hit;
    {
        let key = String::from("a"); // a temporary key...
        hit = lookup(&key, table);
    } // ...dropped here, but the result comes from `table`
    println!("{hit:?}");
}
```

```
error: lifetime may not live long enough
  --> compile_fail/ch03_rule_first.rs:13:20
   |
10 | pub fn lookup<'k, 't>(key: &'k str, table: &'t Table) -> Option<&'k str> {
   |               --  -- lifetime `'t` defined here
   |               |
   |               lifetime `'k` defined here
...
13 |             return Some(value.as_str());
   |                    ^^^^^^^^^^^^^^^^^^^^ function was supposed to return data with
lifetime `'k` but it is returning data with lifetime `'t`
   |
   = help: consider adding the following bound: `'t: 'k`

error[E0597]: `key` does not live long enough
  --> compile_fail/ch03_rule_first.rs:25:22
   |
24 |         let key = String::from("a"); // a temporary key...
   |             --- binding `key` declared here
25 |         hit = lookup(&key, table);
   |                      ^^^^ borrowed value does not live long enough
26 |     } // ...dropped here, but the result comes from `table`
   |     - `key` dropped here while still borrowed
27 |     println!("{hit:?}");
   |                --- borrow later used here
```

It fails twice. The correct body is rejected: it returns data with lifetime `'t`, and the contract promised `'k` ("function was supposed to return data with lifetime `'k` but it is returning data with lifetime `'t`"). The help, "consider adding the following bound: `'t: 'k`", would make the body compile by demanding that the table outlive the key, but it would leave the wrong contract in place. And because that contract ties the result to `key`, the perfectly valid caller is rejected as well, with E0597: `key` is dropped while the result still borrows from it. Both errors point at code that is correct, while the real cause, an invisible rule, appears nowhere in the source. The only escape would be to reorder the parameters, so that argument order would carry lifetime meaning. The mirror rule, "the last parameter", fails in the same way for `first_only`.

Rule 3 is a "first parameter" rule of sorts, and it works, because the first parameter of a method is special: it is the receiver, and returning a part of the receiver is what methods overwhelmingly do.

Candidate B: the same lifetime everywhere

This is the most defensible candidate: give every input and the output one lifetime. For `lookup` it produces (`code/compile_fail/ch03_rule_unify.rs`):

```
pub fn lookup<'a>(key: &'a str, table: &'a Table) -> Option<&'a str> {
    for (name, value) in &table.rows {
        if name == key {
            return Some(value.as_str());
        }
    }
    None
}
```

```
error[E0597]: `key` does not live long enough
  --> compile_fail/ch03_rule_unify.rs:25:22
   |
24 |         let key = String::from("a"); // a temporary key...
   |             --- binding `key` declared here
25 |         hit = lookup(&key, table);
   |                      ^^^^ borrowed value does not live long enough
26 |     } // ...dropped here, but the result comes from `table`
   |     - `key` dropped here while still borrowed
27 |     println!("{hit:?}");
   |               --- borrow later used here
```

The body compiles, but the same caller is rejected: the contract forces the result to die with `key` . The error points at the caller's code, in a program in which no lifetime is written anywhere. Only a reader who knows the hidden expansion could understand it, and to fix it they would have to write the explicit signature anyway.

It is tempting to defend rule B because it "never accepts an unsound program". That is true, but it is true of every candidate rule, A and C included: whatever contract a rule produces, the body is checked against it and every caller is checked against it, so a bad rule can only reject programs, never let a dangling reference through. What distinguishes B is *where* its rejections land. With all lifetimes equal, every relation between them holds, so B never rejects a body because of how the signature relates its lifetimes. The whole cost falls on the callers.

Behind `&mut` , that cost becomes severe. Here is rule B applied to `push` from the beginning of the chapter (`code/compile_fail/ch03_rule_unify_push.rs`):

```
pub fn push<'a>(list: &'a mut Vec<&'a str>, item: &'a str) {
    list.push(item);
}
```

```
pub fn caller() {
    let first = String::from("first");
    let mut list: Vec<&str> = Vec::new();
    push(&mut list, &first);
    println!("{} items", list.len()); // any later use of `list` is rejected
}
```

```
error[E0502]: cannot borrow `list` as immutable because it is also borrowed as mutable
  --> compile_fail/ch03_rule_unify_push.rs:16:26
   |
15 |     push(&mut list, &first);
   |          --------- mutable borrow occurs here
16 |     println!("{} items", list.len()); // any later use of `list` is rejected
```

```
   |                         ^^^^
   |                         |
   |                         immutable borrow occurs here
   |                         mutable borrow later used here
```

Read the labels on line 16. `list.len()` is an "immutable borrow", and the same spot is where the "mutable borrow later used here": the `&mut list` taken for the call on line 15 is still alive, although the call returned long ago. The reason is the single lifetime. The strings in the vector have the lifetime `'a`, which is part of the type of `list` (`Vec<&'a str>`), so `'a` must last as long as `list` is used. The `&mut` borrow has the same `'a`, so it lasts just as long. And the compiler cannot pick a shorter `'a` for this one call, because the type behind a `&mut` cannot be adjusted: `&mut T` is *invariant* in `T`, as Chapter 4 explains. So the vector is mutably borrowed for the rest of its life, and any later use, even reading its length, conflicts with that borrow. The value is "borrowed forever". Compare `ch03_push_contract.rs` above, where the borrow of the vector stays elided and short. This shape, a mutable borrow annotated with the lifetime of the value's own contents (`&'a mut T<'a>`), is a well-known trap: Chapter 4 explains the mechanism, and Chapter 14 shows how to design around it.

Candidate C: infer the contract from the body

With this rule the signature would no longer be the contract; the body would. Consider two versions of one library function whose *only* difference is which argument the body returns, with identical caller code. Version 1 (`code/examples/ch03_contract_v1.rs`):

```
fn pick<'a>(x: &'a str, _y: &str) -> &'a str {
    x
}
```

```
fn main() {
    let long_lived = String::from("long-lived");
    let result;
    {
        let short_lived = String::from("short");
        result = pick(&long_lived, &short_lived); // relies on: not borrowed from `y`
    }
    println!("{result}");
}
```

```
long-lived
```

Version 2 returns `y`. Rule C would infer the signature below from the new body, and the caller, unchanged, no longer compiles (`code/compile_fail/ch03_contract_v2.rs`):

```
pub fn pick<'a>(_x: &str, y: &'a str) -> &'a str {
    y
}
```

```
error[E0597]: `short_lived` does not live long enough
  --> compile_fail/ch03_contract_v2.rs:19:36
   |
17 |         let short_lived = String::from("short");
   |             ----------- binding `short_lived` declared here
18 |         // Identical caller code:
19 |         result = pick(&long_lived, &short_lived);
   |                                    ^^^^^^^^^^^^ borrowed value does not live long
```

```
enough
20 |     }
   |     - `short_lived` dropped here while still borrowed
21 |     println!("{result}");
   |                 ------ borrow later used here
```

If the contract were inferred from the body, an innocent edit inside a function, with no change to the text of its signature, would break callers, possibly in other crates that were never recompiled against the new version during review. With explicit signatures, the author who wants to return `y` must change the signature, and the change is visible for what it is: a breaking change of the API. The rule also has nothing to work with wherever there is no body (`code/compile_fail/ch03_trait_no_body.rs`):

```
pub trait Pick {
    fn pick(x: &str, y: &str) -> &str;
}
```

```
error[E0106]: missing lifetime specifier
 --> compile_fail/ch03_trait_no_body.rs:7:34
  |
7 |     fn pick(x: &str, y: &str) -> &str;
  |                ----     ----     ^ expected named lifetime parameter
  |
  = help: this function's return type contains a borrowed value, but the signature does not
say whether it is borrowed from argument 1 or argument 2
help: consider introducing a named lifetime parameter
  |
7 |     fn pick<'a>(x: &'a str, y: &'a str) -> &'a str;
  |            ++++     ++          ++          ++
```

A trait method declaration has no body, and every implementation, in any crate, must follow one contract. The same holds for function pointer types such as `Choose` above: there is no body behind a type.

What the candidates teach

Candidate rule	What it gives `lookup`	Outcome
A: first reference	result tied to `key`	body and valid caller rejected
B: unify everything	one lifetime for all four	body accepted, valid caller rejected; `&mut` borrowed forever
C: infer from the body	whatever the body returns	body edits break callers; nothing to infer in traits
Rust: elide only when forced or by `&self`	E0106	you write `'t` once

A lifetime in a signature records *which input the output is derived from*. That is a fact about the function's intent, and unlike a type it cannot be recovered from parameter names or positions. The three elision rules cover the situations where the answer is forced (there is a single input lifetime) or settled by a strong convention (a method's `&self`). Anywhere else, every mechanical guess encodes a claim that is false for some perfectly reasonable function, so the language asks the author.

3.7 Common confusions

"**Elision means the compiler infers the lifetimes.**" In a signature nothing is inferred: the rules are fixed and look only at the written types. Inference happens in function bodies, where the borrow checker computes regions from the code (Chapter 1). That is why a body that obviously returns `x` does not help a signature that fails to elide.

"**Writing `'_` in the return type asks the compiler to work it out.**" In a return type, `'_` requests elision, exactly like writing nothing. It succeeds where elision succeeds and fails with the same E0106 where it fails (Chapter 6).

"**Rule 2 counts references.**" It counts parameters, then the lifetimes in that one parameter. `&[&str]` is one parameter with two lifetimes (E0106), and `(&'a str, &'a str)` is one parameter with one lifetime (accepted). Two parameters that share one named lifetime are still two (E0106, where the Reference's wording would accept them).

"**A method's result always borrows from `self`.**" Only when the output lifetime is elided. If the result comes from another argument, or from data the struct merely points to, write the lifetime (Chapter 2).

"**The compiler's suggestion is the fix.**" It is a signature that makes the error go away, and often the most restrictive one: `&'a [&'a str]` where `&[&'w str]` was enough, `'a` on both inputs where only one is returned.

"**An elided lifetime is a short one, or `'static`.**" An elided input lifetime is an ordinary lifetime parameter, chosen anew by every caller. Only in `static`, `const` and associated const items does an elided lifetime mean `'static`.

3.8 Summary

- Elision is a fixed, syntactic rewrite of a signature. It never looks at the body; the expanded signature is the contract for the body and for every caller.
- Rule 1: every elided input lifetime becomes a fresh, independent lifetime parameter. Rule 2: if exactly one parameter mentions lifetimes, and it mentions exactly one, the output gets it. Rule 3, checked first: if the receiver contains exactly one reference to (a type containing) `Self`, the output gets its lifetime. Otherwise an elided output lifetime is E0106.
- rustc's rule 2 counts parameters, unlike the Reference's wording: two parameters sharing `'a` are E0106. rustc's rule 3 accepts receivers such as `&Rc<Self>` and `&Box<Self>` since Rust 1.81. Elision does not see lifetimes hidden in `Self` or in type parameters.
- Function pointer types and `Fn` sugar elide inside the type (higher-ranked, Chapter 9); `impl` headers get fresh lifetimes; `static`, `const` and, since 1.99, all associated consts get `'static`.
- E0106 asks you to choose a contract: the result borrows from one input, from either, or from none. The suggestion in the diagnostic is one choice, not necessarily the best.
- Struct fields never elide: the lifetime is a parameter of the type and must be declared; `'static` would be a trap. Uses of the type elide as usual.

- Every candidate extra rule is sound but worse: "first parameter" rejects correct bodies and callers, "unify everything" rejects callers and turns `&mut` parameters into `&'a mut T<'a>` (borrowed forever), and "infer from the body" lets body edits break callers and cannot work without a body.
- The `mismatched_lifetime_syntaxes` lint (1.89) asks you to write a lifetime the same way everywhere in a signature.

3.9 Exercises

1. Expand each signature by hand, say which rule applies, and predict which ones compile. Then check your answers with the compiler.

```
fn split_key(line: &str, sep: char) -> (&str, &str)
fn first(words: &Vec<String>) -> &str
fn merge(target: &mut String, extra: &str) -> &str
type Callback = Box<dyn Fn(&str) -> Option<&str>>;
fn get(&self, index: usize, default: &str) -> &str // in an impl block
```

   Hint: look for a receiver first; then count the parameters that mention a lifetime, and the lifetimes in each.

2. Add a method `get_or` to the `Config` of `ch03_self_receivers.rs`, which returns `default` when the key is missing:

```
fn get_or(&self, key: &str, default: &str) -> &str
```

   Read the error, then give the method the least restrictive contract that compiles, and write a caller in which `key` is a temporary. *Hint:* rule 3 ties the result to `&self`, but the result may also be `default`; which two inputs need one name, and which can stay elided?

3. Someone proposes a fourth candidate rule: "tie the output to the input that lives the longest". Explain why the compiler could not apply it when it checks the signature, and what it would mean for two different callers of `longer`. *Hint:* the lifetimes of a signature are chosen by each caller, call by call; at the signature, no lifetime parameter is longer than another unless a bound says so.

4. In `ch03_rule_unify_push.rs`, delete the last line of `caller`. Does it compile? Now call `push` twice instead. Explain both results with the single lifetime `'a`, then fix `push` without touching `caller`. *Hint:* the mutable borrow lasts as long as the strings in the vector must stay valid, which is as long as the vector is used.

5. Rewrite `Holder` from `ch03_struct_field_elided.rs` in the three ways of the struct table, and for each one write a function `make(text: &str)` that builds a `Holder` from its argument. Which of the three can you write without cloning, and what does the caller of each one have to guarantee? *Hint:* for the borrowing version, the return type is `Holder<'_>`; for the `'static` one, try it and read the error.

CHAPTER 4

Subtyping and Variance

You pass a string literal, of type `&'static str`, to a function that expects a `&'a str` for some short `'a`, and you never give it a thought. Yet something non-trivial happens there. Rust has no class hierarchy, no `i32` that is also an `i64`; in general a value of one type cannot be used where another type is expected. Here it can: a reference that is valid for the whole program is accepted where a reference valid for a shorter region is needed. That is *subtyping*, and in Rust it comes from lifetimes.

As soon as lifetimes have subtypes, every type that contains a lifetime raises a question. If a `&'static str` can be used as a `&'a str`, can a `Vec<&'static str>` be used as a `Vec<&'a str>`? Can a `fn(&'static str)` be used as a `fn(&'a str)`? Can a `&mut &'static str` be used as a `&mut &'a str`? The answers are yes, no, and absolutely not, and the rules that give them are called *variance*. They are not an academic curiosity. The "absolutely not" for `&mut` is what keeps a dangling reference out of safe code; variance notes appear in borrow-checker diagnostics ("mutable references are invariant over their type parameter"); and variance is the reason a method taking `&'a mut self` on a `Thing<'a>` can lock a value for the rest of its life.

In this chapter you will learn:

- how the outlives relation orders regions, and how that order makes `&'long T` a subtype of `&'short T`;
- what covariant, contravariant and invariant mean, and the variance of the standard types, from `&'a T` and `Box<T>` to `Cell<T>`, `fn(T) -> U`, `dyn Trait + 'a` and `PhantomData<T>`;
- how to make the compiler confirm a variance with a *witness* function;
- why `&mut T` and `Cell<T>` must be invariant in `T`, through the dangling reference that covariance would allow;
- why function arguments are contravariant, with a callback that accepts only `'static` strings;
- how the compiler computes the variance of your own types, why `&'a mut T<'a>` locks a value, and how to read variance in diagnostics.

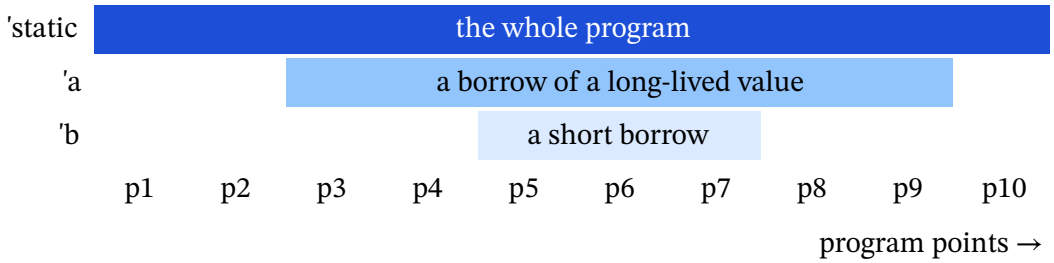
4.1 The outlives order

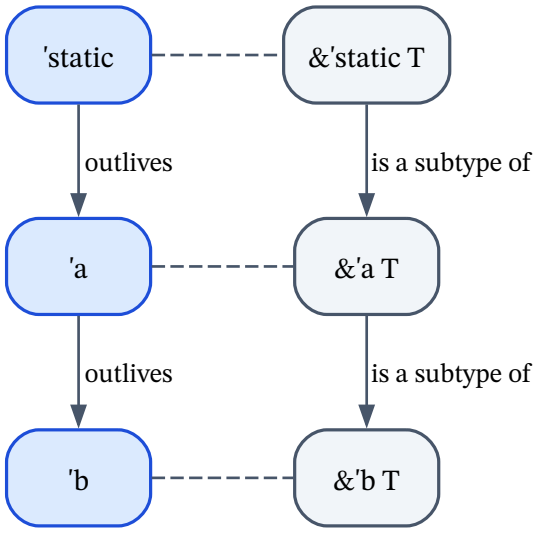
Chapter 1 described a lifetime as a *region*: a set of points in the control flow of a function body (a simplification of what rustc computes, but a faithful one for everything in this chapter). Regions can be compared. We say that `'a` *outlives* `'b`, written `'a: 'b`, when `'a` contains every point of `'b`:

a reference that is valid throughout `'a` is in particular valid throughout `'b`. The relation has the properties you would expect of "is at least as long as":

- every region outlives itself (`'a: 'a`);
- it is transitive: if `'a: 'b` and `'b: 'c`, then `'a: 'c`;
- `'static`, the region of the whole program, outlives every other region, so it is the largest one.

It is not a total order. Two borrows in different branches of an `if`, or in two sibling blocks, have regions that do not contain each other, and then neither outlives the other. Inside a function body the borrow checker computes the regions and checks the outlives relations that the code needs. In a signature, you can demand one with a bound such as `'long: 'short`, which every caller must then satisfy.





The outlives order on regions: `'static` contains `'a`, which contains `'b`, and the order carries over to subtyping: `&'static T` is a subtype of `&'a T`, which is a subtype of `&'b T`.

4.2 Subtyping: a longer borrow is a shorter one

A type `S` is a *subtype* of a type `T` when a value of type `S` can be used wherever a value of type `T` is expected. The usual notation is `S <: T`; it is not Rust syntax, but it is convenient in prose. Rust has exactly one source of subtyping, the outlives order:

If `'long: 'short`, then `&'long T <: &'short T`.

A reference that is valid over a large region is valid over any part of it. Read the direction carefully, because it is easy to get backwards: the *longer* lifetime gives the *subtype*. `&'static T` is a subtype of every `&'a T`, because it carries the strongest guarantee. A subtype is the type with more guarantees, which is why it can stand in for the type with fewer.

Here is the everyday case, from `code/examples/ch04_covariance.rs`:

```
fn pick<'a>(preferred: &'a str, fallback: &'a str) -> &'a str {
    if preferred.is_empty() {
        fallback
    } else {
        preferred
    }
}
```

```
let fallback: &'static str = "anonymous";
let typed = String::from("ferris");
// `'a` becomes (at most) the region of the borrow of `typed`; the
// `&'static str` is used as a `&'a str`.
println!("{}", pick(&typed, fallback));
```

```
ferris
```

`pick` asks for two arguments of one type, `&'a str`. The call passes a borrow of `typed`, which is valid only while `typed` lives, and `fallback`, which is valid for the whole program. The compiler picks for `'a` a region no larger than the borrow of `typed`, and uses subtyping to accept `fallback` as a `&'a str`. This is sometimes described as the compiler "shrinking" the lifetime of `fallback`. Nothing happens at run time: it is the same pointer, and lifetimes do not exist in the compiled program. Subtyping is purely a fact the type checker uses to accept the program.

Where can a subtype be used in place of its supertype? Anywhere a value flows into a place of a known type: a function argument, a `let` with a type annotation, an assignment, a return value, a struct field. Rust's other kind of subtyping, between higher-ranked function types such as `for<'a> fn(&'a str)` and `fn(&'x str)`, follows the same idea and belongs to Chapter 9.

4.3 Variance: how subtyping passes through types

Subtyping between two references raises the question from the introduction: what about types that *contain* references? The answer depends on the type, and it is called its *variance*. Take a generic type `F<X>` and two types with `S <: T`. Then `F` is, in its parameter `X`:

- **covariant** if `F<S> <: F<T>`: the subtyping passes through in the same direction;
- **contravariant** if `F<T> <: F<S>`: the subtyping passes through reversed;
- **invariant** if neither holds (unless `S` and `T` are the same type): no subtyping passes through.

A type with several parameters has a variance in each, and lifetime parameters count: `&'a T` has a variance in `'a` and another in `T`. These are the variances of the standard types:

Type	Variance in `'a`	Variance in `T` (and `U`)
`&'a T`	covariant	covariant
`&'a mut T`	covariant	**invariant**
`Box<T>`, `Vec<T>`, `Option<T>`, `Rc<T>`, `Arc<T>`		covariant
`Cell<T>`, `RefCell<T>`, `UnsafeCell<T>`, `Mutex<T>`		**invariant**
`*const T` / `*mut T`		covariant / **invariant**
`fn(T) -> U`		**contravariant** in `T`, covariant in `U`
`dyn Trait<T> + 'a`	covariant	**invariant**
`PhantomData<T>`		covariant

The Reference (https://doc.rust-lang.org/reference/subtyping.html) has the same table, with the two halves of the function row as separate entries, `fn() -> T` covariant and `fn(T) -> ()` contravariant. The Rustonomicon's chapter on subtyping and variance (https://doc.rust-lang.org/nomicon/subtyping.html) is the page the compiler itself points to in its variance notes, as we will see.

Making the compiler prove it

You do not have to take the table on trust. A function whose body is just its argument, and whose signature converts one type into another, compiles only if the argument type is a subtype of the return type. Such a function is a *witness*: if it compiles, the subtyping holds. Here are witnesses for the first rows, from `code/examples/ch04_variance_witnesses.rs`:

```
// In every witness, 'long: 'short ("'long outlives 'short").

// `&'a T` is covariant in 'a...
fn shrink_ref<'long: 'short, 'short>(r: &'long str) -> &'short str {
    r
}

// ...and in T (here T = &'long str, behind a reference of some lifetime 'r).
fn shrink_inner<'long: 'short, 'short, 'r>(r: &'r &'long str) -> &'r &'short str {
    r
}

// Owning containers are covariant in their element type.
fn shrink_vec<'long: 'short, 'short>(v: Vec<&'long str>) -> Vec<&'short str> {
    v
}

// `&'a mut T` is covariant in 'a: the loan can be shortened...
fn shrink_loan<'long: 'short, 'short, 'v>(
    r: &'long mut &'v str,
) -> &'short mut &'v str {
    r
}
// ...but its T (here `&'v str`) must stay exactly as it is.
```

Read each signature as a claim. `shrink_vec` says: "given that `'long` outlives `'short`, a `Vec<&'long str>` is a `Vec<&'short str>`." The body adds nothing, so the compiler accepts the function only if `Vec` is covariant in its element type. `shrink_inner` makes the same claim for the `T` of a shared reference, and `shrink_loan` for the *lifetime* of a mutable reference, not its `T`. The same file has a witness for each

of the other covariant rows (`Box`, `Option`, `Rc`, `Arc`, `*const T`, the result of `fn() -> T`, the bound of `dyn Display + 'a`, and `PhantomData`), and runs them all:

```
shrink_ref:       literal
shrink_inner:     inner
shrink_vec:       ["a", "b"]
shrink_loan:      slot
shrink_box:       boxed
shrink_option:    Some("some")
shrink_rc:        shared
shrink_arc:       atomic
shrink_const_ptr: false
shrink_fn_result: hello
shrink_dyn_bound: 42
grow_fn_arg:      4
shrink_twice:     a literal
```

For the rows marked invariant, the opposite happens: the witness does not compile. We will meet those errors in a moment, because they are the important part of the table.

4.4 Covariance in action

Covariance is what makes ordinary code work without a second thought. The rest of `ch04_covariance.rs` moves values with `'static` contents into variables whose types have shorter lifetimes:

```
let defaults: Vec<&'static str> = vec!["red", "green"];
let custom = String::from("teal");
let mut palette: Vec<&str> = defaults; // Vec<&'static str> used as Vec<&'p str>
palette.push(&custom);
println!("{palette:?}");

let boxed: Box<&'static str> = Box::new("static");
let mut current: Box<&str> = boxed; // Box<&'static str> used as Box<&'c str>
println!("{current}");
current = Box::new(&custom);
println!("{current}");
```

```
["red", "green", "teal"]
static
teal
```

`defaults` is a `Vec<&'static str>`. Moving it into `palette`, a `Vec<&'p str>` with a shorter `'p`, is allowed because `Vec` is covariant; then `palette` can receive a borrow of the local `custom`. The same happens with `Box<&'static str>` and `current`.

Why is this safe, when a `Vec` can be written to? Because after the move nobody holds the vector at the old type any more. The only one who could still expect `'static` strings in it would be someone who can look at it as a `Vec<&'static str>`, and the move took that view away. Owning containers are covariant for this reason: the owner is the only one who can read or write, so it can choose the type it sees the contents at. Writing into a container that someone *else* owns takes a `&mut`, and that is where invariance comes in.

4.5 Invariance: the safety net for writes

Suppose, for a moment, that `&mut T` were covariant in `T`. Here is what would then compile, from `code/compile_fail/ch04_mut_invariance.rs`:

```
fn overwrite<'a>(slot: &mut &'a str, value: &'a str) {
    *slot = value;
}

pub fn caller() {
    let mut name: &'static str = "static";
    {
        let local = String::from("local");
        overwrite(&mut name, &local); // would leave `name` dangling
    }
    println!("{name}");
}
```

`name` has the type `&'static str`, so `&mut name` is a `&mut &'static str`. With covariance, the compiler could view it as a `&mut &'short str`, where `'short` is the region of the borrow of `local`, and the call `overwrite(&mut name, &local)` would type-check with `'a = 'short`. `overwrite` stores the borrow of `local` in `name`. At the closing brace `local` is dropped, and `println!("{name}")` reads freed memory, through a variable whose type still promises a string that lives forever. Every step is locally reasonable; only the covariance assumption is wrong. Rust's actual rule rejects the program:

```
error[E0597]: `local` does not live long enough
  --> compile_fail/ch04_mut_invariance.rs:14:30
   |
11 |     let mut name: &'static str = "static";
   |                   ------------ type annotation requires that `local` is borrowed for
`'static`
12 |     {
13 |         let local = String::from("local");
   |             ----- binding `local` declared here
14 |         overwrite(&mut name, &local); // would leave `name` dangling
   |                              ^^^^^^ borrowed value does not live long enough
15 |     }
   |     - `local` dropped here while still borrowed
```

Because `&mut T` is invariant in `T`, the type `&mut &'static str` of the first argument fixes `T = &'static str` exactly, with no room to shorten it, so `'a` must be `'static`. Then the second argument, `&local`, must be a `&'static str`, which it cannot be. The diagnostic is the familiar E0597, and its first label names the root of the requirement: the "type annotation requires that `local` is borrowed for `'static`". The annotation `&'static str` on `name` fixed one end, and invariance carried it, through `overwrite`'s signature, to `local`.

The invariant rows, confirmed

The program `code/compile_fail/ch04_invariant_witnesses.rs` has witnesses for four of the invariant entries of the table, and every one of them fails:

```
pub fn shrink_target<'long: 'short, 'short, 'v>(
    r: &'v mut &'long str,
) -> &'v mut &'short str {
    r
}
```

```
pub fn shrink_cell<'long: 'short, 'short>(c: Cell<&'long str>) -> Cell<&'short str> {
    c
}

pub fn shrink_mut_ptr<'long: 'short, 'short>(p: *mut &'long str) -> *mut &'short str {
    p
}

pub fn shrink_dyn_fn<'long: 'short, 'short>(
    f: Box<dyn Fn(&'long str)>,
) -> Box<dyn Fn(&'short str)> {
    f
}
```

Here is the first error in full:

```
error: lifetime may not live long enough
  --> compile_fail/ch04_invariant_witnesses.rs:11:5
   |
 8 | pub fn shrink_target<'long: 'short, 'short, 'v>(
   |                      -----          ------ lifetime `'short` defined here
   |                      |
   |                      lifetime `'long` defined here
...
11 |     r
   |     ^ function was supposed to return data with lifetime `'long` but it is returning
data with lifetime `'short`
   |
   = help: consider adding the following bound: `'short: 'long`
   = note: requirement occurs because of a mutable reference to `&str`
   = note: mutable references are invariant over their type parameter
   = help: see <https://doc.rust-lang.org/nomicon/subtyping.html> for more information about
variance
```

The errors for `Cell` and for the raw pointer have the same shape, and differ only in their notes. For `Cell`:

```
= note: requirement occurs because of the type `Cell<&str>`, which makes the generic
argument `&str` invariant
= note: the struct `Cell<T>` is invariant over the parameter `T`
```

and for `*mut`:

```
= note: requirement occurs because of a mutable pointer to `&str`
= note: mutable pointers are invariant over their type parameter
```

(The fourth error, for `dyn Fn`, comes up in the section on contravariance.) In all of them, the primary message is "lifetime may not live long enough", and the label under the returned value says "function was supposed to return data with lifetime `'long` but it is returning data with lifetime `'short`", which is a roundabout way of saying that the conversion needs the two lifetimes to be *equal*. The bound `'long: 'short` gives one direction; the help asks for the other, `'short: 'long`, which together would make them the same region. (Adding that bound is almost never the fix you want: it removes the difference between the two lifetimes, and with it the point of the function.) Then the notes name the reason: "a mutable reference to `&str`" with "mutable references are invariant over their type parameter"; "the type `Cell<&str>`, which makes the generic argument `&str` invariant" with "the struct `Cell<T>` is invariant over the parameter `T`"; and the same for "mutable pointers". The last help links to

the Rustonomicon's chapter on variance. The file `code/compile_fail/ch04_invariant_cells.rs` checks `RefCell<T>`, `UnsafeCell<T>` and `Mutex<T>` the same way, with the same notes.

`Cell` writes through a shared reference

`Cell<T>` is the clearest case of why invariance is about *writing* rather than about `mut`. A `Cell` can be written through a shared reference (Chapter 12), so the overwrite attack needs no `&mut` at all. The program `code/compile_fail/ch04_cell_invariance.rs` tries:

```
pub fn caller() {
    let label: Cell<&'static str> = Cell::new("static");
    let local = String::from("local");
    let view: &Cell<&str> = &label; // Cell<&'static str> is NOT a Cell<&'short str>
    view.set(&local);
}
```

```
error[E0597]: `local` does not live long enough
  --> compile_fail/ch04_cell_invariance.rs:12:14
   |
 9 |     let label: Cell<&'static str> = Cell::new("static");
   |                ------------------ type annotation requires that `local` is borrowed for
`'static`
10 |     let local = String::from("local");
   |         ----- binding `local` declared here
11 |     let view: &Cell<&str> = &label; // Cell<&'static str> is NOT a Cell<&'short str>
12 |     view.set(&local);
   |              ^^^^^^ borrowed value does not live long enough
13 | }
   | - `local` dropped here while still borrowed
```

If `Cell` were covariant, `view` would be a `&Cell<&'short str>` looking at the same cell as `label`, `view.set(&local)` would store a short-lived borrow in it, and `label.get()` would later return it as a `&'static str`. Invariance makes the annotation on `label` fix the type of the cell's contents for everyone, so `set` demands a `&'static str`. Interior mutability is a write through a shared path, and every type built on `UnsafeCell<T>` is invariant in `T` for this reason.

Reading, writing, and both

The principle behind these rules fits in one line: **reading is covariant, writing is contravariant, and both is invariant.** A place you can only read from is a *source* of `T` values; if it produces `&'static str`s, you can certainly treat what it produces as `&'short str`s. A place you can write to is a *sink*; if it accepts `&'short str`s, it certainly accepts `&'static str`s, so the subtyping goes the other way. A `&mut T`, a `Cell<T>` or a `*mut T` is both a source and a sink of `T`, and the only way for both directions to hold is for the types to be equal.

The same principle explains the one covariant entry of `&'a mut T`: its lifetime `'a`. Shortening a loan only gives the exclusive access back sooner, which is always safe; that is `shrink_loan` above. What may not change is the type of what the loan points to.

4.6 Contravariance: function arguments

A function is a sink for its arguments and a source of its result, so the reading and writing principle predicts its variance: covariant in the result, contravariant in the arguments. The contravariant

direction is easiest to see with callbacks. Suppose some code will call a callback with `&'static str` arguments only. A callback that accepts *any* short-lived `&'a str` is perfectly acceptable there: whatever it receives, it can handle. So `fn(&'a str) <: fn(&'static str)`, even though `&'static str <: &'a str`: the function that accepts more is the subtype. The witness from `ch04_variance_witnesses.rs`:

```
// The direction is reversed: a callback for SHORT-lived strings is usable as a
// callback for LONG-lived ones.
fn grow_fn_arg<'long: 'short, 'short>(
    f: fn(&'short str) -> usize,
) -> fn(&'long str) -> usize {
    f
}
```

The opposite substitution, a callback that can only handle `'static` strings offered to code that will call it with a short-lived one, would be a disaster. The compiler rejects it, as `code/compile_fail/ch04_static_callback.rs` shows:

```
fn apply<'a>(callback: fn(&'a str) -> usize, text: &'a str) -> usize {
    callback(text)
}

fn only_static(text: &'static str) -> usize {
    text.len()
}

pub fn caller() -> usize {
    let local = String::from("local");
    apply(only_static, &local)
}
```

```
error[E0597]: `local` does not live long enough
  --> compile_fail/ch04_static_callback.rs:16:24
   |
15 |     let local = String::from("local");
   |         ----- binding `local` declared here
16 |     apply(only_static, &local)
   |           -----------  ^^^^^^ borrowed value does not live long enough
   |           |
   |           coercion requires that `local` is borrowed for `'static`
17 | }
   | - `local` dropped here while still borrowed
```

`apply` takes a callback of type `fn(&'a str) -> usize` and a text of type `&'a str`, with one `'a`. The function `only_static` has type `fn(&'static str) -> usize`. For it to be a `fn(&'a str) -> usize`, contravariance requires `&'a str <: &'static str`, which holds only if `'a` is `'static`. So `'a` becomes `'static`, and `&local` would have to be a `&'static str`. The label "coercion requires that `local` is borrowed for `'static`" points at `only_static`: the function item is converted (coerced) to a function pointer there, and that is where the subtyping requirement arose. Imagine that `only_static` stored its argument in a global list of names; with a short-lived string, the list would end up holding a dangling reference.

You will meet contravariance less often than the other two variances, for a reason that Chapter 3 already showed: a function pointer type you write, such as `fn(&str) -> usize`, is higher-ranked, so it accepts every lifetime and needs no subtyping at all. Contravariance matters when the argument's

lifetime is fixed by a surrounding lifetime parameter, as in `apply`, or in a struct like `Handler<'a>` in the next section.

Trait objects behave differently. A `Box<dyn Fn(&'a str)>` looks like a function taking `&'a str`, but `'a` there is a parameter of the trait `Fn`, and the parameters of a trait object are invariant. That is the fourth witness of `ch04_invariant_witnesses.rs`:

```
error: lifetime may not live long enough
  --> compile_fail/ch04_invariant_witnesses.rs:25:5
   |
22 | pub fn shrink_dyn_fn<'long: 'short, 'short>(
   |                      -----          ------ lifetime `'short` defined here
   |                      |
   |                      lifetime `'long` defined here
...
25 |     f
   |     ^ function was supposed to return data with lifetime `'long` but it is returning
data with lifetime `'short`
   |
   = help: consider adding the following bound: `'short: 'long`
```

Note what is missing: no note explains the invariance this time. When a "lifetime may not live long enough" error involves a `dyn Trait<...>`, remember that only the `+ 'a` bound of a trait object is covariant; everything inside the trait's own parameters is invariant.

4.7 The variance of your own types

You never declare the variance of a struct or an enum; the compiler computes it from the fields. For each parameter, it looks at every position in which the parameter occurs, and combines their variances:

- if every occurrence is covariant, the type is covariant in the parameter;
- if every occurrence is contravariant, it is contravariant;
- if the occurrences disagree, or any of them is invariant, it is invariant.

The Reference puts it like this: "The variance of other `struct`, `enum`, and `union` types is decided by looking at the variance of the types of their fields. If the parameter is used in positions with different variances then the parameter is invariant." Nesting composes: a parameter inside a covariant type keeps the variance it has there, a parameter anywhere inside an invariant type is invariant, and a parameter inside a contravariant position is flipped. Three structs from `code/examples/ch04_struct_variance.rs` show the three outcomes:

```rust
// Only read through: covariant in 'src.
struct Tokens<'src> {
    text: &'src str,
    position: usize,
}

// 'a appears only as an argument of a fn pointer: contravariant in 'a.
struct Handler<'a> {
    on_line: fn(&'a str) -> usize,
}

// Written through a shared reference: invariant in 'a.
struct Slot<'a> {
    current: Cell<&'a str>,
}
```

`Tokens` uses `'src` once, in `&'src str`, which is covariant in its lifetime; `position` does not mention `'src`. So `Tokens` is covariant in `'src`. `Handler` uses `'a` only in the argument of a function pointer, a contravariant position, so it is contravariant. `Slot` uses `'a` inside a `Cell`, which is invariant, so it is invariant. The witnesses confirm the first two:

```
// Covariant: Tokens<'long> can be used as Tokens<'short>.
fn shrink_tokens<'long: 'short, 'short>(t: Tokens<'long>) -> Tokens<'short> {
    t
}

// Contravariant: a handler for any short-lived line can handle 'static lines.
fn handler_for_static<'a>(h: Handler<'a>) -> Handler<'static> {
    h
}

// Invariant: the only conversion is to the very same type.
fn same_slot<'a>(s: Slot<'a>) -> Slot<'a> {
    s
}
```

```
rest: "x = 1;"
length: 14
slot: new
```

`handler_for_static` is contravariance in a struct: a handler that accepts lines of any lifetime `'a` can be stored where a handler for `'static` lines is expected. `same_slot` is there for contrast: the identity is the only conversion an invariant type allows. The proof that `Slot` is invariant is a witness that fails, in `code/compile_fail/ch04_struct_invariant.rs`:

```
pub struct Slot<'a> {
    current: Cell<&'a str>,
}

pub fn shrink_slot<'long: 'short, 'short>(s: Slot<'long>) -> Slot<'short> {
    s
}
```

```
error: lifetime may not live long enough
  --> compile_fail/ch04_struct_invariant.rs:13:5
   |
12 | pub fn shrink_slot<'long: 'short, 'short>(s: Slot<'long>) -> Slot<'short> {
   |                    -----          ------ lifetime `'short` defined here
   |                    |
   |                    lifetime `'long` defined here
13 |     s
   |     ^ function was supposed to return data with lifetime `'long` but it is returning
data with lifetime `'short`
   |
   = help: consider adding the following bound: `'short: 'long`
   = note: requirement occurs because of the type `Slot<'_>`, which makes the generic
argument `'_` invariant
   = note: the struct `Slot<'a>` is invariant over the parameter `'a`
   = help: see <https://doc.rust-lang.org/nomicon/subtyping.html> for more information about
variance
```

The notes report the variance the compiler computed: "the type `Slot<'_>`, which makes the generic argument `'_` invariant", and "the struct `Slot<'a>` is invariant over the parameter `'a`". When you see this note about a type of your own, look for the field that makes it invariant: a `&mut`, a `Cell` or

`RefCell`, a `*mut`, a trait object with the parameter inside, or a parameter used both as an input and as an output of a function pointer.

Two consequences are worth remembering.

- **Variance is part of a type's public interface.** Changing a private field from `&'a str` to `Cell<&'a str>` changes no signature, but it makes the struct invariant, and code elsewhere that relied on shortening its lifetime stops compiling.
- **A parameter that no field uses has no variance at all**, and Rust rejects the struct (E0392) instead of guessing. This happens with types built on raw pointers, where the lifetime the type stands for is not visible in any field. The fix is a field of type `PhantomData<X>`, a zero-sized marker that makes the compiler compute the variance as if the struct contained an `X`: with `PhantomData<&'a T>` the struct behaves like a `&'a T`, covariant in both. Chapter 13 shows how to choose the marker for each variance.

4.8 Invariance in practice: `&'a mut T<'a>`

Invariance has one consequence that every Rust programmer meets sooner or later. Chapter 2 showed the symptom: a method with a receiver of type `&'a mut self` on a type with lifetime parameter `'a` borrows the value for the rest of its life. Here is the mechanism. The program `code/compile_fail/ch04_mut_self_lock.rs` has a collector of words borrowed from some text, with the problematic signature:

```
pub struct Collector<'src> {
    words: Vec<&'src str>,
}

impl<'src> Collector<'src> {
    pub fn add(&'src mut self, line: &'src str) {
        self.words.extend(line.split_whitespace());
    }
}
```

```
pub fn caller() {
    let text = String::from("alpha beta");
    let more = String::from("gamma");
    let mut collector = Collector { words: Vec::new() };
    collector.add(&text);
    collector.add(&more);
}
```

```
error[E0499]: cannot borrow `collector` as mutable more than once at a time
  --> compile_fail/ch04_mut_self_lock.rs:23:5
   |
22 |     collector.add(&text);
   |     --------- first mutable borrow occurs here
23 |     collector.add(&more);
   |     ^^^^^^^^^
   |     |
   |     second mutable borrow occurs here
   |     first borrow later used here
```

Follow the lifetimes. Say `collector` has the type `Collector<'c>`. Because the value's type mentions `'c`, the region `'c` must include every point where `collector` is still going to be used; that is how

the borrow checker keeps the words in the collector valid. The call `collector.add(&text)` creates a mutable borrow `&'m mut Collector<'c>` and passes it where a `&'src mut Collector<'src>` is expected:

1. `&'m mut X` is covariant in `'m`, so the borrow is acceptable if `'m: 'src`: the loan lasts at least as long as `'src`.
2. `&'m mut X` is invariant in `X`, so `Collector<'c>` must *equal* `Collector<'src>`: `'src` is `'c`, with no room to shorten it.
3. Together: `'m: 'c`. The mutable borrow lasts as long as the collector is used.

So the first mutable borrow is still alive at the second call, and the compiler reports E0499, "cannot borrow `collector` as mutable more than once at a time", with the first borrow "later used here" at the second call. Any other use of `collector` after the first `add`, even reading `collector.words.len()`, would be rejected too (as E0502, a shared borrow while a mutable one is alive). Chapter 3 met the same shape when the hypothetical rule "the same lifetime everywhere" turned `push` into `fn push<'a>(list: &'a mut Vec<&'a str>, item: &'a str)`.

The shared version of the same signature is harmless, and the difference is variance again. Here it is in `code/examples/ch04_shared_self_borrow.rs`:

```
struct Collector<'src> {
    words: Vec<&'src str>,
}

impl<'src> Collector<'src> {
    // The right signature: a short `&mut` borrow, and lines that live for 'src.
    fn add(&mut self, line: &'src str) {
        self.words.extend(line.split_whitespace());
    }

    // Over-constrained (`&'src self`), but harmless thanks to covariance.
    fn first(&'src self) -> Option<&'src str> {
        self.words.first().copied()
    }
}
```

```
fn main() {
    let text = String::from("alpha beta");
    let more = String::from("gamma");
    let mut collector = Collector { words: Vec::new() };
    collector.add(&text);
    println!("{:?}", collector.first()); // a shared borrow...
    collector.add(&more); // ...that has already ended: `add` may borrow mutably
    println!("{:?} of {}", collector.first(), collector.words.len());
}
```

```
Some("alpha")
Some("alpha") of 3
```

`first` takes `&'src self`, the same suspicious shape with `&` instead of `&mut`. A `&'m Collector<'c>` is passed where a `&'src Collector<'src>` is expected, and this time `Collector<'c>` only has to be a *subtype* of `Collector<'src>`, which covariance grants for any `'src` shorter than `'c`. The compiler picks a short `'src` for this one call, the shared borrow ends after the `println!`, and `add` can borrow the collector mutably again. The fix for the mutable case is the signature `add` has here: `&mut self` with

its own elided, short lifetime, and `'src` only on the data that really lives that long. Chapter 14 returns to this pattern and the ways to design around it.

4.9 Variance in diagnostics

Variance rarely appears in the primary message of an error. It shows up in the notes, and in a few labels that are easier to read once you know what to look for. All of these come from the programs of this chapter and of Chapter 3:

The diagnostic says	What it means
"requirement occurs because of a mutable reference to `X`" and "mutable references are invariant over their type parameter"	a lifetime inside `X` cannot change, because `X` is behind a `&mut`
"requirement occurs because of a mutable pointer to `X`" and "mutable pointers are invariant over their type parameter"	the same, behind a `*mut`
"requirement occurs because of the type `S<A>`, which makes the generic argument `A` invariant" and "the struct `S<T>` is invariant over the parameter `T`"	a field of `S` (yours, or `Cell`, `RefCell`, `Mutex` in the standard library) makes `S` invariant
"function was supposed to return data with lifetime `'x` but it is returning data with lifetime `'y`" with "consider adding the following bound: `'y: 'x`"	an invariant position needs the two lifetimes to be equal
"type annotation requires that `v` is borrowed for `'static`"	an annotation fixed a lifetime, and invariance carried the requirement to `v`
"coercion requires that `v` is borrowed for `'static`"	a function item became a function pointer whose argument lifetime is fixed

Two habits help. First, when a note mentions invariance, find the `&mut`, the cell or the struct it names, and ask which lifetime inside it is being forced to match another; that is usually the lifetime you should separate. Second, when an error involves a trait object, remember that its trait parameters are invariant even though no note says so.

4.10 For the type-theoretically inclined

The regions of a program, ordered by outlives, form a partial order with `'static` as its greatest element. A type constructor maps regions (or types) to types, and variance says how it treats the order: a covariant constructor preserves it (a monotone map), a contravariant one reverses it, and an invariant one relates only equal arguments. Variances compose like signs: a contravariant position inside another contravariant position is covariant, which is why a function that takes a callback is covariant in the callback's argument:

```
// An argument of an argument: two reversals, so covariant again.
fn shrink_twice<'long: 'short, 'short>(f: fn(fn(&'long str))) -> fn(fn(&'short str)) {
```

```
    f
}
```

`&mut T` puts `T` in a read position (covariant) and a write position (contravariant) at once, and the only variance compatible with both is invariance. The same calculation, done field by field, is how the compiler infers the variance of your structs.

4.11 Common confusions

"**Rust has no subtyping.**" It has no subtyping between structs or traits, but it has subtyping between lifetimes (and between higher-ranked function types, Chapter 9), and through variance that subtyping reaches every type that contains a lifetime. You use it every time you pass a `&'static str` where a shorter `&str` is expected.

"**A longer lifetime is a supertype.**" The other way round: `&'static T` is a subtype of every `&'a T`. The subtype is the type with the stronger guarantee.

"**`&mut T` is invariant.**" In `T`, yes. In its own lifetime it is covariant: shortening a loan is always safe; changing what it points to is not.

"**Invariant means `'static`.**" Invariance only means that a lifetime cannot be adjusted. The `'static` in many errors comes from somewhere else, an annotation or a `'static` value, and invariance carries it to a place where it does not fit.

"**`Box<dyn Fn(&'a str)>` is contravariant, like `fn(&'a str)`.**" The lifetime is a parameter of the trait, and trait parameters of a trait object are invariant. Only the object's `+ 'a` bound is covariant.

"**A coercion is subtyping.**" Converting `&String` to `&str` is a deref coercion: it converts one value, at one place (an argument, a `let` with a type), and it does not pass through other types. Subtyping does, according to variance. The program `code/compile_fail/ch04_no_coercion_in_vec.rs` shows the difference:

```
fn total_len(names: Vec<&str>) -> usize {
    names.iter().map(|name| name.len()).sum()
}

pub fn caller() -> usize {
    let owned = String::from("ferris");
    let one: &str = &owned; // fine: a deref coercion from &String to &str
    let names: Vec<&String> = vec![&owned];
    total_len(names) + one.len()
}
```

```
error[E0308]: mismatched types
  --> compile_fail/ch04_no_coercion_in_vec.rs:14:15
   |
14 |     total_len(names) + one.len()
   |     --------- ^^^^^ expected `Vec<&str>`, found `Vec<&String>`
   |     |
   |     arguments to this function are incorrect
   |
   = note: expected struct `Vec<&str>`
              found struct `Vec<&String>`
note: function defined here
```

```
  --> compile_fail/ch04_no_coercion_in_vec.rs:6:4
   |
6  | fn total_len(names: Vec<&str>) -> usize {
   |    ^^^^^^^^^ -----------------
```

A `Vec<&'static str>` can be used as a `Vec<&'a str>` because `Vec` is covariant, but a `Vec<&String>` cannot become a `Vec<&str>`: that would require converting every element, and building a new vector.

"**I can declare the variance I want.**" In Rust the variance of a type is always computed from its fields. To influence it, you change the fields, or add a `PhantomData` marker (Chapter 13).

4.12 Summary

- Regions are ordered by outlives: `'a: 'b` means `'a` contains `'b`. The order is reflexive and transitive, not total, and `'static` is the largest region.
- If `'long: 'short`, then `&'long T` is a subtype of `&'short T`. This is Rust's only subtyping (besides higher-ranked function types), it costs nothing at run time, and it is not a coercion.
- Variance says how subtyping passes through a type: covariant (same direction), contravariant (reversed), invariant (not at all). `&'a T`, `Box`, `Vec`, `Option`, `Rc`, `Arc`, `*const T` and `PhantomData<T>` are covariant; `&'a mut T` is covariant in `'a` and invariant in `T`; `Cell`, `RefCell`, `UnsafeCell`, `Mutex` and `*mut T` are invariant; `fn(T) -> U` is contravariant in `T` and covariant in `U`; `dyn Trait<T> + 'a` is covariant in `'a` and invariant in `T`.
- A witness function whose body is its argument proves a subtyping by compiling.
- Invariance is the safety net for writes: with a covariant `&mut T` or `Cell<T>`, safe code could store a short-lived borrow where a `'static` one is promised. Reading is covariant, writing contravariant, both invariant.
- Function arguments are contravariant: a callback that accepts more lifetimes is the subtype; a callback that needs `'static` cannot be used for short-lived data.
- The variance of your own types is computed from their fields, and it is part of their interface. An unused parameter is rejected (E0392); `PhantomData` chooses the variance (Chapter 13).
- `&'a mut T<'a>` locks the value: invariance pins `'a` to the value's own lifetime, and the `&mut` borrow must last that long. The shared `&'a T<'a>` is harmless thanks to covariance.

4.13 Exercises

1. Determine the variance in each lifetime parameter of these two structs:

   ```
   struct Transform<'a> { apply: fn(&'a str) -> &'a str }
   struct Lookup<'a, 'b> { keys: &'a [&'b str], hits: Cell<usize> }
   ```

   Confirm each answer with a witness that compiles and one that does not. *Hint:* in `Transform`, `'a` occurs in an argument and in a result. In `Lookup`, ask whether `Cell<usize>` mentions either lifetime.

2. Rewrite `ch04_cell_invariance.rs` with a `RefCell<&'static str>` and `borrow_mut`, and explain, as in the overwrite story, how a covariant `RefCell` would let `main` read a dangling reference. *Hint:* `*view.borrow_mut() = &local;` plays the role of `set`.
3. Given the struct below, can a `Registry<'static>` be passed to a function that expects a `Registry<'short>`?

   ```
   struct Registry<'a> { handlers: Vec<fn(&'a str) -> usize> }
   ```

   Can a `Registry<'short>` be passed where a `Registry<'static>` is expected? Write both witnesses and predict which compiles before you try. *Hint:* `Vec` keeps the variance of its element, and the element is contravariant in `'a`.
4. Write a witness proving that `Option<&'a mut Vec<&'b str>>` is covariant in `'a`, and one showing that it is not covariant in `'b`. Which note does the compiler attach to the second? *Hint:* `Option` is covariant, so it passes on the variances of `&'a mut Vec<&'b str>` unchanged.
5. In `ch04_mut_self_lock.rs`, keep the signature of `add` and remove the second call. Does the program compile? Add a line that only reads `collector.words.len()` after the call, and explain the error with the three steps of *Invariance in practice*. *Hint:* the mutable borrow lasts as long as `'c`, and `'c` covers every later use of `collector`.

CHAPTER 5

Borrows in Practice: Reborrows, Temporaries and the Limits of NLL

The first four chapters built a model: a lifetime is a region of the program (Chapter 1), a type that holds a borrow carries that region as a parameter (Chapter 2), elision writes the regions of a signature for you (Chapter 3), and the outlives order makes regions a subtyping relation (Chapter 4). That model explains why a program is accepted or rejected. It does not yet explain some things you see every day.

You pass the same `&mut Vec<String>` to a function twice in a row, although a `&mut` reference cannot be copied. You write `v.push(v.len())`, which borrows `v` mutably and immutably at once, and the compiler says nothing. You add one method call to a line, `String::from("x").as_str()`, and get an error about a "temporary value". You give a struct a `Drop` impl, and code that compiled a minute ago is rejected with a note about drop order. And now and then the compiler rejects a function that is plainly correct.

This chapter is about the borrow checker as you meet it in practice: the mechanics it applies without telling you (reborrows, two-phase borrows), the values whose lifetime you do not see in the source (temporaries, drops), and the known cases where it is more conservative than it needs to be.

In this chapter you will learn:

- what a reborrow is, when the compiler inserts one, and why `let r2 = r;` moves a `&mut` reference while `f(r)` does not;
- how two-phase borrows make `v.push(v.len())` compile, and where they stop;
- how to hold two mutable borrows into one value at once (fields, `split_at_mut`, `get_disjoint_mut`), and why accessor methods cannot do it;
- what E0505, E0515 and E0716 mean, and their standard fixes;
- when temporaries are dropped, what lifetime extension does, and what edition 2024 changed for tail expressions and `if let`;
- how drop order and the drop check decide whether a value with a `Drop` impl may hold a borrow;
- which correct programs the current borrow checker still rejects, and what Polonius changes.

5.1 Reborrowing

A `&mut` reference is not `Copy`

A shared reference `&T` is `Copy`: duplicating it creates a second reader, which is harmless. A mutable reference `&mut T` is not, because its whole point is to be the *only* path to the data while it is live. Two copies would be two writers. So assigning a `&mut` to another variable moves it, as with any non-`Copy` value. The full program is in `code/compile_fail/ch05_move_mut_ref.rs`, and the compiler rejects it:

```
pub fn collect(words: &mut Vec<String>) {
    let other = words; // moves the reference out of `words`
    add_word(other, "alpha");
    add_word(words, "beta");
}
```

```
error[E0382]: borrow of moved value: `words`
  --> compile_fail/ch05_move_mut_ref.rs:14:14
   |
11 | pub fn collect(words: &mut Vec<String>) {
   |                ----- move occurs because `words` has type `&mut Vec<String>`, which
does not implement the `Copy` trait
12 |     let other = words; // moves the reference out of `words`
   |                 ----- value moved here
13 |     add_word(other, "alpha");
14 |     add_word(words, "beta");
   |              ^^^^^ value borrowed here after move
```

The three labels tell the whole story. The first explains why there is a move at all: `words` has type `&mut Vec<String>`, "which does not implement the `Copy` trait". The second marks the move (`let other = words`), and the third the use after it. Nothing here is about lifetimes; it is ordinary ownership, applied to a reference.

Implicit reborrows

Now look at a function that uses its `&mut` parameter twice. It compiles (the program is `code/examples/ch05_reborrow.rs`):

```
fn add_word(words: &mut Vec<String>, word: &str) {
    words.push(word.to_owned());
}

fn collect(words: &mut Vec<String>) {
    add_word(words, "alpha"); // really `add_word(&mut *words, "alpha")`
    add_word(words, "beta"); // `words` is usable again: only a reborrow was passed
}
```

If `add_word(words, "alpha")` moved `words` into the call, the second call would be the same error as above. It is not, because the compiler does not pass `words` itself: it passes a **reborrow**, a new mutable reference to the same place, written `&mut *words`. You dereference the reference (`*words` is the `Vec<String>` it points to) and borrow that place again. The new reference has its own region, which may be shorter than the region of `words` and lasts here only for the call. While the reborrow is live, `words` is frozen; when it ends, `words` is usable again.

The compiler inserts a reborrow wherever it knows that the expected type is a `&mut` reference: an argument for a parameter declared `&mut T`, a `let` with a written `&mut` type, the receiver of a method

that takes `&mut self`. The same `let` that moved `words` above reborrows it as soon as you write the type:

```
fn collect_more(words: &mut Vec<String>) {
    let other: &mut Vec<String> = words; // the type is written: a reborrow
    add_word(other, "gamma");
    add_word(words, "delta"); // fine: `other` is no longer used

    let explicit = &mut *words; // the same reborrow, spelled out
    explicit.push(String::from("epsilon"));
    words.sort(); // a method call reborrows its receiver too
}
```

The three forms (`let other: &mut Vec<String> = words`, `&mut *words` written by hand, and the implicit `&mut *words` of `words.sort()`) are the same operation.

Where no reborrow happens

The rule has a consequence that surprises everyone once: when the expected type is a type parameter, the compiler does not know that it is a reference, so it does not reborrow. It moves. From `code/compile_fail/ch05_generic_moves.rs`:

```
fn passthrough<T>(value: T) -> T {
    value
}

pub fn generic_call(words: &mut Vec<String>) {
    passthrough(words).push(String::from("zeta"));
    passthrough(words).push(String::from("eta"));
}
```

```
error[E0382]: use of moved value: `words`
  --> compile_fail/ch05_generic_moves.rs:12:17
   |
10 | pub fn generic_call(words: &mut Vec<String>) {
   |                     ----- move occurs because `words` has type `&mut Vec<String>`,
which does not implement the `Copy` trait
11 |     passthrough(words).push(String::from("zeta"));
   |                 ----- value moved here
12 |     passthrough(words).push(String::from("eta"));
   |                 ^^^^^ value used here after move
   |
help: consider creating a fresh reborrow of `words` here
   |
11 |     passthrough(&mut *words).push(String::from("zeta"));
   |                 ++++++
```

`T` is inferred as `&mut Vec<String>`, and the reference is moved into the first call. This time the compiler suggests the fix itself: "consider creating a fresh reborrow of `words` here", that is, `passthrough(&mut *words)`, which is what `ch05_reborrow.rs` does:

```
fn passthrough<T>(value: T) -> T {
    value
}

fn generic_call(words: &mut Vec<String>) {
    passthrough(&mut *words).push(String::from("zeta")); // explicit reborrow
    passthrough(&mut *words).push(String::from("eta"));
}
```

The same happens when the `&mut` is wrapped in another type. An `Option<&mut u32>` is not a reference, so passing it moves it (`code/compile_fail/ch05_option_mut_moves.rs`):

```
fn bump(counter: Option<&mut u32>) {
    if let Some(count) = counter {
        *count += 1;
    }
}

pub fn bump_twice(counter: Option<&mut u32>) {
    bump(counter);
    bump(counter);
}
```

```
error[E0382]: use of moved value: `counter`
  --> compile_fail/ch05_option_mut_moves.rs:14:10
   |
12 | pub fn bump_twice(counter: Option<&mut u32>) {
   |                   ------- move occurs because `counter` has type `Option<&mut u32>`,
which does not implement the `Copy` trait
13 |     bump(counter);
   |          ------- value moved here
14 |     bump(counter);
   |          ^^^^^^^ value used here after move
   |
note: consider changing this parameter type in function `bump` to borrow instead if owning
the value isn't necessary
  --> compile_fail/ch05_option_mut_moves.rs:6:18
   |
 6 | fn bump(counter: Option<&mut u32>) {
   |    ----          ^^^^^^^^^^^^^^^^ this parameter takes ownership of the value
   |    |
   |    in this function
```

Here the note points in an unhelpful direction: changing `bump` to take a reference to the `Option` would work, but it changes the function's contract for every caller. The local fix is to reborrow what is inside the `Option` : `counter.as_deref_mut()` borrows `counter` for a moment and returns a new `Option<&mut u32>` that reborrows the same `u32` :

```
fn bump(counter: Option<&mut u32>) {
    if let Some(count) = counter {
        *count += 1;
    }
}

fn bump_twice(mut counter: Option<&mut u32>) {
    bump(counter.as_deref_mut()); // reborrow what is inside the `Option`
    bump(counter.as_deref_mut());
}
```

A reborrow freezes the original

While a reborrow is live, the reference it came from cannot be used, not even to read: that would be a second path to the same data (`code/compile_fail/ch05_reborrow_frozen.rs`):

```
pub fn collect(words: &mut Vec<String>) {
    let other = &mut *words;
    words.push(String::from("alpha"));
    other.push(String::from("beta"));
}
```

```
error[E0499]: cannot borrow `*words` as mutable more than once at a time
 --> compile_fail/ch05_reborrow_frozen.rs:8:5
  |
7 |     let other = &mut *words;
  |                 ----------- first mutable borrow occurs here
8 |     words.push(String::from("alpha"));
  |     ^^^^^ second mutable borrow occurs here
9 |     other.push(String::from("beta"));
  |     ----- first borrow later used here
```

Read the first line carefully: the conflict is on `*words`, the vector, not on the variable `words`. Borrows are tracked per *place*. `other` borrows `*words` from line 7 until its last use on line 9, and `words.push` (which reborrows `*words` again for the call) falls inside that region. Swap lines 8 and 9 and the program compiles, because `other` is dead by the time `words` is used.

This is also how a function that returns a `&mut` works. The result of

```
/// The result is a reborrow of `*words`: it may live as long as the caller's
/// borrow, but no longer.
fn longest_mut(words: &mut [String]) -> Option<&mut String> {
    words.iter_mut().max_by_key(|w| w.len())
}
```

is a reborrow of `*words` whose region the caller decides, within the region of the caller's own borrow. While the caller holds the returned `&mut String`, its vector is frozen. You met this in Chapter 2 as "the result pins the parser": it is the same mechanism.

The output of the example confirms that every call went through:

```
["alpha", "beta", "delta", "epsilon", "gamma", "zeta", "eta"]
["alpha", "beta", "delta", "EPSILON", "gamma", "zeta", "eta"]
count = 2
```

5.2 Two-phase borrows

Method calls evaluate the receiver before the arguments. So `lengths.push(lengths.len())` means: borrow `lengths` mutably for `push`, then evaluate `lengths.len()`, which borrows it immutably, then call `push`. By the rules of Chapter 1 that is a shared borrow inside a mutable one, error E0502. Yet this compiles (`code/examples/ch05_two_phase.rs`):

```
let mut lengths = vec![3, 1];
lengths.push(lengths.len()); // `&mut lengths` reserved, `lengths.len()` reads

let mut ids: HashMap<String, usize> = HashMap::new();
ids.insert(String::from("alpha"), ids.len());
ids.insert(String::from("beta"), ids.len());

let mut text = String::from("ab");
text.push_str(&text.to_uppercase()); // the shared borrow ends before the call

let first = &lengths[0];
lengths.push(*first); // `*first` is copied out before the push is activated
```

```
lengths = [3, 1, 2, 3, 4]
ids = [("alpha", 0), ("beta", 1)]
text = abAB
total = Total(10)
```

The rule that accepts it is called **two-phase borrows**. Some mutable borrows are created in two steps. When the receiver is evaluated, the `&mut` is only *reserved*; it is *activated* when the call actually starts. Between the two, the borrow behaves like a shared borrow, so other code may read the value but not modify it. In `lengths.push(lengths.len())`, the read happens during the reservation, finishes, and only then is the borrow activated.

The last two lines of the listing show what "finishes" means. In `text.push_str(&text.to_uppercase())`, the shared borrow `&text` is used by `to_uppercase`, which returns a new `String`; the argument passed to `push_str` borrows that new string, not `text`. In `lengths.push(*first)`, the expression `*first` copies the number out of the vector during the reservation, and `first` is never used again. In both cases no shared borrow of the receiver is live when the mutable one is activated.

The rustc development guide lists the three places where two-phase borrows are created: https://rustc-dev-guide.rust-lang.org/borrow-check/two-phase-borrows.html. The method-call receiver is the first. The second is the implicit reborrow of a `&mut` argument, from the previous section:

```
fn record(lengths: &mut Vec<usize>, value: usize) {
    lengths.push(value);
}

fn record_count(lengths: &mut Vec<usize>) {
    record(lengths, lengths.len()); // the implicit reborrow is two-phase too
}
```

The third is the `&mut` borrow of the left operand of an overloaded compound assignment operator, such as `+=` on a type that implements `AddAssign`:

```
#[derive(Debug)]
struct Total(usize);

impl std::ops::AddAssign<usize> for Total {
    fn add_assign(&mut self, amount: usize) {
        self.0 += amount;
    }
}
```

```
let mut total = Total(5);
total += total.0; // `AddAssign::add_assign(&mut total, total.0)`, two-phase
```

Where two-phase borrows stop

A `&mut` that you write yourself is an ordinary borrow, active immediately. Spell the same call in function syntax and it is rejected (`code/compile_fail/ch05_two_phase_explicit.rs`):

```
pub fn record_length(mut lengths: Vec<usize>) -> Vec<usize> {
    Vec::push(&mut lengths, lengths.len());
    lengths
}
```

```
error[E0502]: cannot borrow `lengths` as immutable because it is also borrowed as mutable
 --> compile_fail/ch05_two_phase_explicit.rs:7:29
  |
7 |     Vec::push(&mut lengths, lengths.len());
  |     --------- ------------  ^^^^^^^ immutable borrow occurs here
  |     |         |
```

```
  |     |         mutable borrow occurs here
  |     mutable borrow later used by call
```

The labels lay out the order of evaluation: the mutable borrow `&mut lengths` first, then the immutable borrow in `lengths.len()` while it is active, and "mutable borrow later used by call" to show why the first borrow is still live.

The reservation also does not excuse a shared borrow that is still *in use* when the call starts. `text.push_str(&text)` would append a string to itself; the argument is a borrow of `text` that `push_str` reads while it writes to `text` (`code/compile_fail/ch05_two_phase_shared.rs`):

```rust
pub fn double(mut text: String) -> String {
    text.push_str(&text);
    text
}
```

```
error[E0502]: cannot borrow `text` as mutable because it is also borrowed as immutable
 --> compile_fail/ch05_two_phase_shared.rs:7:5
  |
7 |     text.push_str(&text);
  |     ^^^^^--------^-----^
  |     |    |        |
  |     |    |        immutable borrow occurs here
  |     |    immutable borrow later used by call
  |     mutable borrow occurs here
```

"immutable borrow later used by call" is the key label: the shared borrow is passed *into* the call, so it overlaps the activated mutable borrow. `push_str` may reallocate the buffer that `&text` points to while reading from it, so this is a real bug, not a limitation. The fix is to make the argument independent of `text` , as `to_uppercase()` did above (or `text.clone()` , or `text.repeat(2)` instead of the whole call).

5.3 Splitting borrows

Aliasing XOR mutation is a rule about *places*. Two mutable borrows may coexist when they point to places that cannot overlap. The question is always whether the compiler can *see* that they do not overlap.

Disjoint fields

Fields of a struct are distinct places, and the borrow checker tracks them separately. This compiles (`code/examples/ch05_split_borrows.rs`):

```rust
struct Document {
    title: String,
    words: Vec<String>,
}

fn tag_as_draft(doc: &mut Document) {
    let title = &mut doc.title; // borrows only `doc.title`
    let words = &mut doc.words; // borrows only `doc.words`
    title.push_str(" (draft)");
    words.push(String::from("draft"));
}
```

`&mut doc.title` borrows the place `(*doc).title`; `&mut doc.words` borrows `(*doc).words`. Neither is a prefix of the other, so there is no conflict.

Methods hide the split

Wrap the same two borrows in accessor methods and the program is rejected (`code/compile_fail/ch05_split_methods.rs`):

```
impl Document {
    pub fn title_mut(&mut self) -> &mut String {
        &mut self.title
    }

    pub fn words_mut(&mut self) -> &mut Vec<String> {
        &mut self.words
    }
}

pub fn tag_as_draft(doc: &mut Document) {
    let title = doc.title_mut();
    let words = doc.words_mut();
    title.push_str(" (draft)");
    words.push(String::from("draft"));
}
```

```
error[E0499]: cannot borrow `*doc` as mutable more than once at a time
  --> compile_fail/ch05_split_methods.rs:23:17
   |
22 |     let title = doc.title_mut();
   |                 --- first mutable borrow occurs here
23 |     let words = doc.words_mut();
   |                 ^^^ second mutable borrow occurs here
24 |     title.push_str(" (draft)");
   |     ----- first borrow later used here
```

The first mutable borrow is of `*doc`, the whole document. This is the signature-is-the-contract principle at work. The signature `fn title_mut(&mut self) -> &mut String` says: "give me exclusive access to all of `*self`, and I return a reference that may keep that access for as long as you use it". The caller is checked against the signature only, so it cannot know that the body touches only `title`. There is no syntax in stable Rust for "this method borrows only some fields of `self`".

There are three good ways out:

- **Borrow the fields directly** where they are visible (in the same module, or if they are public), as `tag_as_draft` did above.
- **One method, several results.** Inside the body the fields are visible, so a single method can split the borrow and return all the pieces at once. The caller still sees one borrow of `*doc`, and the two results share it:

```
impl Document {
    /// One method, one borrow of `self`, two disjoint results.
    fn parts_mut(&mut self) -> (&mut String, &mut Vec<String>) {
        (&mut self.title, &mut self.words)
    }
}

fn tag_as_final(doc: &mut Document) {
    let (title, words) = doc.parts_mut();
    title.push_str(" (final)");
```

```
    words.push(String::from("final"));
}
```

- **Destructure** the value, which binds one `&mut` reference per field:

```
fn tag_as_checked(doc: &mut Document) {
    let Document { title, words } = doc; // two `&mut` bindings, one per field
    title.push_str(" (checked)");
    words.push(String::from("checked"));
}
```

Closures get the same precision for free since edition 2021: a closure captures the individual places it uses, not the whole variable, so two closures that touch different fields can both be alive:

```
fn tag_with_closures(doc: &mut Document) {
    // Each closure captures only the field it uses (edition 2021 and later).
    let mut retitle = |suffix: &str| doc.title.push_str(suffix);
    let mut add_word = |word: &str| doc.words.push(word.to_owned());
    retitle(" (reviewed)");
    add_word("reviewed");
}
```

Slices and indices

Indexing is where the compiler stops being able to see disjointness (`code/compile_fail/ch05_split_index.rs`):

```
pub fn swap_first_two(words: &mut [String]) {
    let a = &mut words[0];
    let b = &mut words[1];
    std::mem::swap(a, b);
}
```

```
error[E0499]: cannot borrow `words[_]` as mutable more than once at a time
 --> compile_fail/ch05_split_index.rs:8:13
  |
7 |     let a = &mut words[0];
  |             -------------- first mutable borrow occurs here
8 |     let b = &mut words[1];
  |             ^^^^^^^^^^^^^^ second mutable borrow occurs here
9 |     std::mem::swap(a, b);
  |                    - first borrow later used here
  |
  = help: use `.split_at_mut(position)` to obtain two mutable non-overlapping sub-slices
```

The error names the place `words[_]`. The `_` is the borrow checker being honest: it does not track index values, which are run-time data, so every element of a slice is the same place to it. `words[0]` and `words[1]` might as well be `words[i]` and `words[j]`, which can be equal.

The standard library offers safe APIs that prove disjointness, either by construction or by a check at run time:

```
fn swap_ends(words: &mut [String]) {
    let last = words.len() - 1;
    let (head, tail) = words.split_at_mut(1); // [0, 1) and [1, len)
    std::mem::swap(&mut head[0], &mut tail[last - 1]);
}

fn swap_first_two(words: &mut [String]) {
    // Checks at run time that the indices are distinct and in bounds.
```

```
    if let Ok([a, b]) = words.get_disjoint_mut([0, 1]) {
        std::mem::swap(a, b);
    }
}
```

`split_at_mut(1)` returns two slices, `[0, 1)` and `[1, len)`, that cannot overlap by construction. (It is implemented with `unsafe` code, which is the subject of Chapter 13; its signature promises two independent borrows, and the promise is what you rely on.) `get_disjoint_mut`, stable since Rust 1.86, takes an array of indices, checks at run time that they are in bounds and pairwise distinct, and returns an array of `&mut` references or an error. Iterators such as `iter_mut()` and `chunks_mut()` give out disjoint `&mut` references one at a time. The output of the example:

```
Notes (draft) (final) (reviewed) (checked): ["draft", "final", "reviewed", "checked"]
["checked", "final", "reviewed", "draft"]
["final", "checked", "reviewed", "draft"]
```

5.4 Moves while borrowed, and borrows that cannot leave

Moving a borrowed value (E0505)

A move is a conflicting access, like a write: after the move, the old owner no longer owns the data, and the new owner may modify or free it. So you cannot move a value while a borrow of it is still going to be used (`code/compile_fail/ch05_move_while_borrowed.rs`):

```
pub fn parse_setting() -> Vec<String> {
    let line = String::from("timeout=30");
    let key = line.split('=').next().unwrap_or("");
    let mut lines = Vec::new();
    lines.push(line); // moves `line` into the vector
    println!("key: {key}");
    lines
}
```

```
error[E0505]: cannot move out of `line` because it is borrowed
  --> compile_fail/ch05_move_while_borrowed.rs:10:16
   |
 7 |     let line = String::from("timeout=30");
   |         ---- binding `line` declared here
 8 |     let key = line.split('=').next().unwrap_or("");
   |               ---- borrow of `line` occurs here
 9 |     let mut lines = Vec::new();
10 |     lines.push(line); // moves `line` into the vector
   |                ^^^^ move out of `line` occurs here
11 |     println!("key: {key}");
   |                     --- borrow later used here
```

The four labels form a timeline: where `line` was declared, where it was borrowed (`line.split('=')` returns slices of `line`, so `key` borrows it), where it was moved (into the vector), and where the borrow is used later (the `println!`). The error exists only because of the last label: delete the `println!` and the program compiles, because `key` is dead before the move.

The fixes follow from the timeline. Use the borrow before the move, or move after the last use. Or stop borrowing: give `key` its own copy, which is what `code/examples/ch05_ownership_fixes.rs` does:

```rust
fn parse_setting() -> (String, Vec<String>) {
    let line = String::from("timeout=30");
    let key = line.split('=').next().unwrap_or("").to_owned(); // an owned copy
    let mut lines = Vec::new();
    lines.push(line); // fine: `key` does not borrow `line`
    (key, lines)
}
```

A third option is to borrow from the new owner (`lines[0].split('=')`) after the move.

Returning a reference to a local (E0515)

A function cannot return a borrow of data it owns, because that data is dropped when the function returns. The compiler has two wordings for this error, depending on whether you return the reference directly or a value that contains one; `code/compile_fail/ch05_return_local.rs` shows both (two functions, one error each):

```rust
pub fn first_word(path: &str) -> &str {
    let text = format!("contents of {path}");
    text.split_whitespace().next().unwrap_or("")
}

pub fn whole_text(path: &str) -> &String {
    let text = format!("contents of {path}");
    &text
}
```

```
error[E0515]: cannot return value referencing local variable `text`
 --> compile_fail/ch05_return_local.rs:8:5
  |
8 |     text.split_whitespace().next().unwrap_or("")
  |     ----^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |     |
  |     returns a value referencing data owned by the current function
  |     `text` is borrowed here

error[E0515]: cannot return reference to local variable `text`
  --> compile_fail/ch05_return_local.rs:13:5
   |
13 |     &text
   |     ^^^^^ returns a reference to data owned by the current function
```

"returns a value referencing data owned by the current function" is the essence of E0515. Notice what the signature says: by elision rule 2 (Chapter 3), the result of `first_word(path: &str) -> &str` borrows from `path`. The body tries to return a borrow of `text` instead. No lifetime annotation can fix that, because annotations describe what a reference may point to; they never make anything live longer (Chapter 1). The fix is to change what is returned: an owned value,

```rust
fn first_word(path: &str) -> String {
    let text = format!("contents of {path}");
    text.split_whitespace().next().unwrap_or("").to_owned()
}
```

or a borrow of something the caller owns (take the text as a parameter), or a `Cow<'_, str>` that is sometimes borrowed and sometimes owned (Chapter 14).

A borrow of a temporary (E0716)

The third error of this family has a twist: the value that dies is not a variable at all. From `code/compile_fail/ch05_temporary_dropped.rs`:

```
pub fn config_name() -> usize {
    let name: &str = String::from("config.toml").as_str();
    name.len()
}
```

```
error[E0716]: temporary value dropped while borrowed
 --> compile_fail/ch05_temporary_dropped.rs:8:22
  |
8 |     let name: &str = String::from("config.toml").as_str();
  |                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^         - temporary value is freed at
the end of this statement
  |                      |
  |                      creates a temporary value which is freed while still in use
9 |     name.len()
  |     ---- borrow later used here
  |
help: consider using a `let` binding to create a longer lived value
  |
8 ~     let binding = String::from("config.toml");
9 ~     let name: &str = binding.as_str();
  |
```

"creates a temporary value which is freed while still in use": the `String` built by `String::from` has no name, and it is freed at the `;` (the label "temporary value is freed at the end of this statement" points at it), while `name` still points into it. The help is the standard fix: give the value a name, so that it lives until the end of the block. To understand why the compiler created an unnamed value, and how long such values live, we need a section of their own.

5.5 Temporaries

What a temporary is

Many expressions need a *place* to work on: `as_str` takes `&self`, so in `String::from("config.toml").as_str()` the new `String` must be stored somewhere before it can be borrowed. When the value is not already in a variable, the compiler stores it in a **temporary**: an unnamed local that it creates for you. Every temporary is dropped at the end of its **temporary scope**, which is usually the end of the enclosing statement.

The rules are easiest to see with a value that announces when it is created and dropped. `code/examples/ch05_temporaries.rs` defines one:

```
/// A value that announces its creation and its destruction.
struct Noisy(&'static str);

fn make(name: &'static str) -> Noisy {
    println!("  make {name}");
    Noisy(name)
}

impl Noisy {
    fn len(&self) -> usize {
        self.0.len()
```

```
    }

    fn find(&self, c: char) -> Option<usize> {
        self.0.find(c)
    }
}

impl Drop for Noisy {
    fn drop(&mut self) {
        println!("  drop {}", self.0);
    }
}
```

The first experiment is the default rule:

```
println!("1. end of statement");
let len = make("a").len();
println!("  after the let, len = {len}");
```

```
1. end of statement
  make a
  drop a
  after the let, len = 1
```

The `Noisy` is created, `len` reads it, and it is dropped at the `;` of the `let`, before the next statement. `len` is a `usize`, so nothing refers to the dropped value, and all is well. In `ch05_temporary_dropped.rs`, the result of `as_str()` was a reference into the temporary: same timing, but now a dangling reference, and the compiler rejects it.

Temporary lifetime extension

There is one exception that you use all the time without noticing. When the initializer of a `let` borrows a temporary directly, the temporary lives until the end of the enclosing block, exactly as if it had a name:

```
println!("2. lifetime extension");
{
    let held = &make("b");
    println!("  after the let, still holding {}", held.0);
}
println!("  after the block");
```

```
2. lifetime extension
  make b
  after the let, still holding b
  drop b
  after the block
```

This is **temporary lifetime extension**. It is a *syntactic* rule: it applies when the borrow of the temporary appears in an "extending" position of the initializer. `code/examples/ch05_lifetime_extension.rs` collects the positions that rustc 1.99 extends:

```
let direct = &load("direct"); // operand of `&`
let entry = Entry { key: &load("lit") }; // field of a struct literal
let pair = (&load("tuple"), 1); // element of a tuple
let list = [&load("array")]; // element of an array
let block = { &load("block") }; // tail of a block
let branch = if quiet { &load("then") } else { &load("else") }; // branches
```

```
let arm = match quiet {
    true => &load("yes"),
    false => &load("no"), // arms of a `match`
};
let wrapped = Some(&load("variant")); // tuple-variant constructor
```

```
<direct> <lit> <tuple> <array>
<block> <else> <no> Some("<variant>")
```

Extension goes through the operand of `&`, the fields of struct, tuple and array expressions, the tail of a block, the branches of `if` and `match`, and (in rustc 1.99) the arguments of a tuple-struct or enum-variant constructor such as `Some`. It does not go through a method call or a function call, because the compiler cannot know what the function does with the reference. That is why `String::from("config.toml").as_str()` was an error, and why this second function in `ch05_temporary_dropped.rs` is one too:

```
pub fn config_path() -> usize {
    let path = std::convert::identity(&String::from("/etc/app"));
    path.len()
}
```

```
error[E0716]: temporary value dropped while borrowed
  --> compile_fail/ch05_temporary_dropped.rs:15:40
   |
15 |     let path = std::convert::identity(&String::from("/etc/app"));
   |                                        ^^^^^^^^^^^^^^^^^^^^^^^^ - temporary value is
freed at the end of this statement
   |                                        |
   |                                        creates a temporary value which is freed while
still in use
16 |     path.len()
   |     ---- borrow later used here
   |
help: consider using a `let` binding to create a longer lived value
   |
15 ~     let binding = String::from("/etc/app");
16 ~     let path = std::convert::identity(&binding);
   |
```

The fix for both is the help's suggestion: a named variable, which `ch05_ownership_fixes.rs` shows next to an extended temporary:

```
fn config_name() -> usize {
    let owned = String::from("config.toml"); // a named value lives to the block end
    let name: &str = owned.as_str();
    let extended: &String = &String::from("config.toml"); // lifetime-extended
    name.len() + extended.len()
}
```

Temporaries in `match` and `if let`

A temporary created in the *scrutinee* of a `match` (the expression being matched) lives until the end of the whole `match`, in every edition:

```
println!("3. match scrutinee");
match make("c").len() {
    n => println!("  in the arm, n = {n}"),
```

```
}
println!("  after the match");
```

```
3. match scrutinee
  make c
  in the arm, n = 1
  drop c
  after the match
```

The arm runs while `c` is still alive, although the value being matched, a `usize`, has nothing to do with it. The reason is that patterns can bind references into the scrutinee, so the compiler keeps its temporaries alive for all the arms; it does not check whether a particular `match` needs that.

For values without a destructor this timing is invisible. It becomes visible when the temporary holds a borrow *and* has a `Drop` impl, because dropping it then counts as a use of the borrow (more on this in the next section). Here is such a type, a view of a list of lines (`code/compile_fail/ch05_if_let_2021.rs`):

```
impl<'a> Lines<'a> {
    pub fn new(lines: &'a Vec<String>) -> Self {
        Lines { lines }
    }

    pub fn first(&self) -> Option<&'a str> {
        self.lines.first().map(|line| line.as_str())
    }
}

impl Drop for Lines<'_> {
    fn drop(&mut self) {}
}
```

and a function that matches on its first line, and adds a line when there is none. The compiler rejects it, in every edition (`code/compile_fail/ch05_match_scrutinee.rs`):

```
pub fn show_first(lines: &mut Vec<String>) {
    match Lines::new(lines).first() {
        Some(first) => println!("first line: {first}"),
        None => lines.push(String::from("(empty)")),
    }
}
```

```
error[E0502]: cannot borrow `*lines` as mutable because it is also borrowed as immutable
  --> compile_fail/ch05_match_scrutinee.rs:27:17
   |
25 |     match Lines::new(lines).first() {
   |           ------------------
   |           |          |
   |           |          immutable borrow occurs here
   |           a temporary with access to the immutable borrow is created here ...
26 |         Some(first) => println!("first line: {first}"),
27 |         None => lines.push(String::from("(empty)")),
   |                 ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
28 |     }
   |     - ... and the immutable borrow might be used here, when that temporary is dropped
and runs the `Drop` code for type `Lines`
```

The diagnostic now has a two-part story, linked by "…". First: "a temporary with access to the immutable borrow is created here ...", pointing at `Lines::new(lines)` . Then, at the closing brace of the `match` : "… and the immutable borrow might be used here, when that temporary is dropped

and runs the `Drop` code for type `Lines`". In between, the `None` arm wants to push to the vector. The `Option<&str>` returned by `first()` is not the problem (in the `None` arm there is no `&str`); the temporary `Lines` is. The fix is to end the temporary's life before the `match`, with a `let` (`code/examples/ch05_temporaries_2024.rs`):

```
fn show_first_match(lines: &mut Vec<String>) {
    let first = Lines::new(lines).first(); // the temporary dies at this `;`
    match first {
        Some(first) => println!("first line: {first}"),
        None => lines.push(String::from("(empty)")),
    }
}
```

`if let` used to follow the same rule as `match`, and edition 2024 changed it. Here is the experiment with `Noisy`, once with a match that succeeds and once with one that fails:

```
println!("4. if let scrutinee");
if let Some(i) = make("dog").find('o') {
    println!("  in the then-branch, i = {i}");
}
if let Some(i) = make("cat").find('o') {
    println!("  in the then-branch, i = {i}");
} else {
    println!("  in the else-branch");
}
```

```
4. if let scrutinee
  make dog
  in the then-branch, i = 1
  drop dog
  make cat
  drop cat
  in the else-branch
```

The scrutinee's temporaries live through the then-branch (`dog` is dropped after it) but are dropped *before* the else-branch runs (`cat` is dropped first). In edition 2021 they lived until the end of the whole `if let`, `else` included. The `Lines` version of `show_first` written with `if let` shows the difference. Compiled with edition 2021, it is rejected with the same story as the `match`:

```
pub fn show_first(lines: &mut Vec<String>) {
    if let Some(first) = Lines::new(lines).first() {
        println!("first line: {first}");
    } else {
        lines.push(String::from("(empty)"));
    }
}
```

```
error[E0502]: cannot borrow `*lines` as mutable because it is also borrowed as immutable
  --> compile_fail/ch05_if_let_2021.rs:31:9
   |
28 |     if let Some(first) = Lines::new(lines).first() {
   |                          -----------------
   |                          |          |
   |                          |          immutable borrow occurs here
   |                          a temporary with access to the immutable borrow is created
here ...
...
31 |         lines.push(String::from("(empty)"));
   |         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
32 |     }
```

```
33 | }
   | - ... and the immutable borrow might be used here, when that temporary is dropped and
runs the `Drop` code for type `Lines`
```

The same function, unchanged, compiles in edition 2024; it is part of `ch05_temporaries_2024.rs`.

> **NOTE**
>
> Edition 2024 (Rust 1.85 and later) drops the temporaries of an `if let` scrutinee before the `else` branch. The edition guide describes the change at https://doc.rust-lang.org/edition-guide/rust-2024/temporary-if-let-scope.html. The lint `if_let_rescope` (in the `rust_2024_compatibility` group) flags the `if let` expressions whose behavior changes when you migrate a crate. A `match` keeps the old scope in every edition, and so does `while let` (Chapter 12 shows what that means for lock guards).

Temporaries in tail expressions

The second change of edition 2024 concerns the *tail expression* of a block, the final expression without a `;` that gives the block its value. In edition 2021, temporaries created there were dropped *after* the block's local variables, so a temporary that borrowed a local outlived it. Edition 2024 drops them *before* the locals:

```
fn tail_expression() -> usize {
    let _local = make("local");
    make("tail").len() // edition 2024: `tail` is dropped before `_local`
}
```

```
5. tail expression
  make local
  make tail
  drop tail
  drop local
  returned 4
```

`tail` is dropped before `_local`, although `_local` was created first. Edition 2021 printed the two drops in the opposite order. When the temporary borrows the local and has a destructor, the old order is a compile error. Here is a reading view with a `Drop` impl (`code/compile_fail/ch05_tail_expr_2021.rs`):

```
/// A view of a text. It has a destructor, so dropping it counts as a use of
/// the borrow it holds.
pub struct Reader<'a> {
    text: &'a str,
}

impl<'a> Reader<'a> {
    pub fn new(text: &'a str) -> Self {
        Reader { text }
    }

    pub fn word_count(&self) -> usize {
        self.text.split_whitespace().count()
    }
}

impl Drop for Reader<'_> {
    fn drop(&mut self) {}
}
```

and a function whose tail expression creates one. Compiled with edition 2021, it is rejected:

```
pub fn count_words() -> usize {
    let text = String::from("one two three");
    Reader::new(&text).word_count()
}
```

```
error[E0597]: `text` does not live long enough
  --> compile_fail/ch05_tail_expr_2021.rs:32:17
   |
31 |     let text = String::from("one two three");
   |         ---- binding `text` declared here
32 |     Reader::new(&text).word_count()
   |     ------------^^^^^-
   |     |           |
   |     |           borrowed value does not live long enough
   |     a temporary with access to the borrow is created here ...
33 | }
   | -
   | |
   | `text` dropped here while still borrowed
   | ... and the borrow might be used here, when that temporary is dropped and runs the
`Drop` code for type `Reader`
   |
   = note: the temporary is part of an expression at the end of a block;
           consider forcing this temporary to be dropped sooner, before the block's local
variables are dropped
help: for example, you could save the expression's value in a new local variable `x` and
then make `x` be the expression at the end of the block
   |
32 |     let x = Reader::new(&text).word_count(); x
   |     +++++++                                +++
```

The labels at the closing brace show the problem in edition 2021's order: first "`text` dropped here while still borrowed", then "… and the borrow might be used here, when that temporary is dropped". The note and the help describe the workaround that edition 2021 code is full of: store the result in a local and make that local the tail expression. In edition 2024 the function compiles as written (it is in `ch05_temporaries_2024.rs`), because the `Reader` is dropped first. The change is described at https://doc.rust-lang.org/edition-guide/rust-2024/temporary-tail-expr-scope.html, and the migration lint is `tail_expr_drop_order`.

Here are the rules of this section in one table:

A temporary created in…	is dropped (edition 2024)	in edition 2021
a statement, or a `let` without extension	at the end of the statement	same
a `let` initializer, extended	at the end of the block	same
a `match` scrutinee	at the end of the `match`	same
a `while let` scrutinee	at the end of each iteration	same
an `if let` scrutinee	before the `else` branch	at the end of the `if let`
the tail expression of a block	before the block's locals	after the block's locals

5.6 Drop order and the drop check

The order of drops

Rust drops values at precise, predictable points. Local variables are dropped at the end of their scope, in the *reverse* order of their declaration. The fields of a struct are dropped in the order they are *declared*, after the struct's own `Drop` impl (if any) has run. A `let _ = expr;` does not bind anything: `_` is a pattern that matches without moving, so the value of `expr` stays a temporary and is dropped at the end of the statement. `code/examples/ch05_drop_order.rs` shows all three:

```
struct Pair {
    first: Noisy,
    second: Noisy,
}
```

```
println!("locals and fields:");
{
    let _a = Noisy("a");
    let _b = Noisy("b");
    let pair = Pair {
        first: Noisy("pair.first"),
        second: Noisy("pair.second"),
    };
    let _ = Noisy("ignored"); // `_` is not a binding: dropped right away
    println!("  end of block ({} and {})", pair.first.0, pair.second.0);
}
```

```
locals and fields:
  drop ignored
  end of block (pair.first and pair.second)
  drop pair.first
  drop pair.second
  drop b
  drop a
```

Dropping is a use

A destructor is code that runs at the end of a scope, and it can read the value's fields, including the references inside them. So when a value of a type with a `Drop` impl is dropped, the borrow checker treats the drop as a *use* of every borrow the value holds. This is the **drop check**. Here is a report that prints a summary when it goes away:

```
pub struct Report<'a> {
    lines: Vec<&'a str>,
}

impl Drop for Report<'_> {
    fn drop(&mut self) {
        println!("report: {} lines", self.lines.len());
    }
}
```

Declare the report first and the text second, and the program is rejected (`code/compile_fail/ch05_drop_check.rs`):

```
pub fn summarize() {
    let mut report = Report { lines: Vec::new() };
    let text = String::from("first line\nsecond line");
```

```
    report.lines.extend(text.lines());
}
```

```
error[E0597]: `text` does not live long enough
  --> compile_fail/ch05_drop_check.rs:21:25
   |
20 |     let text = String::from("first line\nsecond line");
   |         ---- binding `text` declared here
21 |     report.lines.extend(text.lines());
   |                         ^^^^ borrowed value does not live long enough
22 | }
   | -
   | |
   | `text` dropped here while still borrowed
   | borrow might be used here, when `report` is dropped and runs the `Drop` code for type
`Report`
   |
   = note: values in a scope are dropped in the opposite order they are defined
```

All the action is at the closing brace. Variables are dropped in reverse order, so `text` is dropped first ("`text` dropped here while still borrowed"); then `report` is dropped, and "borrow might be used here, when `report` is dropped and runs the `Drop` code for type `Report`". The note states the rule that decides it: "values in a scope are dropped in the opposite order they are defined". If the compiler accepted this program, `Report::drop` could read the `&str`s in `self.lines`, which point into a freed `String`. This destructor happens to look only at the length, but the borrow checker does not read destructors: it goes by the existence of the `impl Drop`, as it goes by signatures everywhere else.

The fix is the order of declaration. Declare the data before the value that borrows it, so that the borrower is dropped first:

```
println!("a report declared after its text:");
{
    let text = String::from("first line\nsecond line");
    let mut report = Report { lines: Vec::new() };
    report.lines.extend(text.lines());
} // `report` is dropped first, while `text` is still alive
```

```
a report declared after its text:
  report: 2 lines, first "first line"
```

Why `Vec<&str>` is more relaxed

The same code with a plain `Vec<&str>` instead of a `Report` compiles in the "wrong" order:

```
println!("a plain Vec<&str> declared before its text:");
{
    let mut lines: Vec<&str> = Vec::new();
    let text = String::from("first line\nsecond line");
    lines.extend(text.lines());
    println!("  {} lines", lines.len());
} // `text` is dropped first; `Vec`'s destructor never reads the `&str`s
```

```
a plain Vec<&str> declared before its text:
  2 lines
```

`Vec` has a destructor too (it frees its buffer), so why is its drop not a use of the `&str`s? Because the standard library tells the compiler that `Vec`'s destructor does not read the borrowed data of its

elements. It only drops the elements, and dropping a `&str` does nothing. The promise is made with an unstable attribute, `#[may_dangle]`, that only the standard library can use on stable Rust; Chapter 13 explains it at the level a user of `Vec` needs. `Box` makes the same promise, and `Option` has no destructor of its own.

A struct of your own is just as relaxed as long as it has no `Drop` impl, because then its drop glue only drops its fields one by one:

```
/// The same fields, but no `Drop` impl.
struct Notes<'a> {
    lines: Vec<&'a str>,
}
```

```
println!("a struct without Drop declared before its text:");
{
    let mut notes = Notes { lines: Vec::new() };
    let text = String::from("first line\nsecond line");
    notes.lines.extend(text.lines());
    println!("  {} lines", notes.lines.len());
} // no `impl Drop for Notes`: dropping `notes` reads nothing
```

```
a struct without Drop declared before its text:
  2 lines
```

So adding an `impl Drop` to a type that holds references is a change to its public contract: it makes every user's declaration order matter. If the destructor does not need the borrowed data, consider whether the type needs a destructor at all, or whether the destructor can live in a field that holds only owned data.

5.7 The limits of NLL

The borrow checker that you use is called NLL, for non-lexical lifetimes (Chapter 1). It computes, for each borrow, a region: the set of program points where the borrow may still be used. It is precise enough that almost every correct program you write is accepted. Almost.

NLL problem case #3

Here is a function that looks up a word in a cache, and computes and inserts the value when it is missing (`code/compile_fail/ch05_nll_case3.rs`):

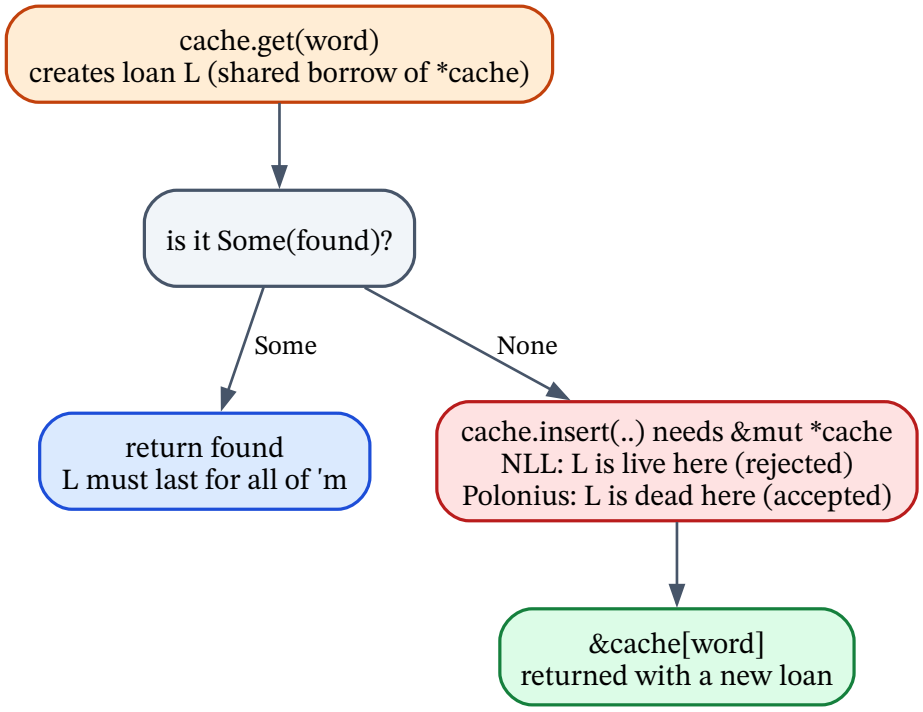
```
pub fn cached<'m>(cache: &'m mut HashMap<String, String>, word: &str) -> &'m String {
    if let Some(found) = cache.get(word) {
        return found;
    }
    cache.insert(word.to_owned(), word.to_uppercase());
    &cache[word]
}
```

The function is correct. If the lookup succeeds, it returns the borrowed value and never touches the map again. If it fails, there is no `found`, so nothing refers into the map when `insert` modifies it. Stable Rust rejects it anyway:

```
error[E0502]: cannot borrow `*cache` as mutable because it is also borrowed as immutable
  --> compile_fail/ch05_nll_case3.rs:13:5
   |
 9 | pub fn cached<'m>(cache: &'m mut HashMap<String, String>, word: &str) -> &'m String {
   |               -- lifetime `'m` defined here
10 |     if let Some(found) = cache.get(word) {
   |                          ----- immutable borrow occurs here
11 |         return found;
   |                ----- returning this value requires that `*cache` is borrowed for `'m`
12 |     }
13 |     cache.insert(word.to_owned(), word.to_uppercase());
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
```

Read the labels in order. "lifetime `'m` defined here": the caller picks `'m`, and it covers the whole call, from the caller's point of view. "immutable borrow occurs here": `cache.get(word)` borrows the map. "returning this value requires that `*cache` is borrowed for `'m`": because `found` is returned, the borrow must last for all of `'m`. And then "mutable borrow occurs here" at the `insert`.

The problem is in the third label. NLL represents the borrow by one region, and the constraint "this region outlives `'m`" does not depend on the path taken: a region that must outlive a lifetime parameter of the function contains every point of the function body. So the borrow is considered live at the `insert` too, although `insert` can only be reached on the path where nothing was returned:



NLL problem case #3. The loan created by `cache.get` must last for all of `'m` because it is returned on the `Some` path; NLL therefore considers it live on the `None` path too, at the `insert`. Polonius sees that the loan cannot reach the `insert`.

This example has a name. The RFC that introduced NLL listed it as "Problem case #3: conditional control flow across functions", one of the cases that NLL would *not* solve: https://rust-lang.github.io/rfcs/2094-nll.html.

Polonius

Polonius is a reformulation of the borrow checker designed to accept such programs. Roughly, instead of asking at each point "is this region live here?", it asks "which loans can reach this point, along the paths that actually lead here?". At the `insert`, the only path that arrives there is the `None` path, on which the loan from `cache.get` is not held by any live reference, so there is no conflict.

> **NOTE**
>
> As of this writing Polonius is not part of stable Rust. Its first version, called Polonius Alpha, is enabled by default on nightly compilers since August 2026 (announced at https://blog.rust-lang.org/2026/08/04/enabling-polonius-alpha-on-nightly/, which says the team is "aiming to stabilize prior to the end of the year"). The nightly of 2026-10-08 (`rustc 1.101.0-nightly`) compiles `ch05_nll_case3.rs` without errors, and the flag `-Zpolonius=off` brings the E0502 back. Check the current status before you rely on it.

You can try it yourself, from the `code/` directory:

```
rustup toolchain install nightly
rustc +nightly --edition 2024 --crate-type lib --emit=metadata \
    --out-dir target compile_fail/ch05_nll_case3.rs
```

Workarounds on stable Rust

Until then, you rewrite the function so that no borrow is held on the path that mutates. The entry API does the lookup and the insertion with one mutable borrow (`code/examples/ch05_nll_workarounds.rs`):

```
fn cached_entry<'m>(cache: &'m mut HashMap<String, String>, word: &str) -> &'m String {
    cache
        .entry(word.to_owned()) // allocates the key even on a hit
        .or_insert_with(|| word.to_uppercase())
}
```

It costs a `String` allocation for the key even when the word is already there. The other classic rewrite asks a question whose answer is a `bool`, so that no borrow survives the test, and looks up the value a second time:

```
fn cached_twice<'m>(cache: &'m mut HashMap<String, String>, word: &str) -> &'m String {
    if !cache.contains_key(word) {
        // `contains_key` returned a `bool`: no borrow survives the test.
        cache.insert(word.to_owned(), word.to_uppercase());
    }
    &cache[word] // a second lookup
}
```

```
ALPHA
ALPHA
BETA
BETA
2 entries
```

The recognizable shape of the problem is a function that receives a `&mut`, returns a borrow obtained from it on one path, and uses it mutably on another path. If an error carries the label "returning this value requires that ... is borrowed for `'a`", and the conflicting access is on a path where that value was

never returned, you are looking at case #3. The usual tools are the ones above: an API that does both steps at once (`entry`), a test that returns no borrow (`contains_key`, a `bool`, an index), or returning an index or an owned value instead of a reference.

5.8 Common confusions

"**A `&mut` parameter can be used only once.**" It can be used as often as you like, as long as each use is a reborrow: calls to functions that take `&mut T`, method calls, and `let` with a written type all reborrow. Only a plain move (`let r2 = r;`, a generic parameter, a wrapper such as `Option<&mut T>`) gives the reference away.

"**Two-phase borrows change the order of evaluation.**" They do not. The receiver is still evaluated first; only the *activation* of its mutable borrow waits for the call. That is why the arguments may read the receiver but not keep a borrow of it into the call.

"**Lifetime extension keeps temporaries alive when I need it.**" Only in the syntactic positions listed above. A temporary passed through a function or a method call is never extended, whatever the function returns.

"**I'll call `drop` explicitly to fix the drop order.**" For the drop check, the order of *declaration* matters, and an explicit `drop` does not change it. This program (`code/compile_fail/ch05_drop_check_explicit.rs`) is still rejected:

```rust
pub fn summarize() {
    let mut report = Report { lines: Vec::new() };
    let text = String::from("first line\nsecond line");
    report.lines.extend(text.lines()); // may panic (allocation)
    drop(report);
}
```

```
error[E0597]: `text` does not live long enough
  --> compile_fail/ch05_drop_check_explicit.rs:19:25
   |
18 |     let text = String::from("first line\nsecond line");
   |         ---- binding `text` declared here
19 |     report.lines.extend(text.lines()); // may panic (allocation)
   |                         ^^^^ borrowed value does not live long enough
20 |     drop(report);
21 | }
   | -
   | |
   | `text` dropped here while still borrowed
   | borrow might be used here, when `report` is dropped and runs the `Drop` code for type
`Report`
   |
   = note: values in a scope are dropped in the opposite order they are defined
```

And rightly so: if `extend` panicked, unwinding would drop the variables in reverse order of declaration, `text` and then `report`, whose destructor could read the freed lines. Declare `text` first.

"**The compiler rejected it, so it must be unsafe.**" Usually, yes. But the borrow checker is conservative by design, and case #3 is a correct program that stable Rust rejects. When you are sure a program is correct, look for the shape of case #3 before you look for a bug.

5.9 Summary

- A `&mut T` is not `Copy`. Where the expected type is known to be `&mut` (function arguments, `let` with a type, method receivers), the compiler *reborrows* it (`&mut *r`) instead of moving it; with generics and wrappers such as `Option<&mut T>` it moves, and you reborrow by hand (`&mut *r`, `as_deref_mut()`).
- While a reborrow is live, the original reference is frozen. Borrows are tracked per place (`*words`, `doc.title`, `words[_]`).
- Two-phase borrows reserve the `&mut` of a method receiver, an implicit argument reborrow, or an overloaded compound assignment, and activate it when the call starts: reads in between are allowed, a shared borrow passed into the call is not. A `&mut` you write yourself is never two-phase.
- Disjoint fields can be borrowed mutably at the same time; accessor methods cannot express that, because the signature borrows all of `self`. Return several borrows from one method, or destructure. For slices, use `split_at_mut`, `get_disjoint_mut` (1.86) or `iter_mut`.
- E0505: a value is moved while a borrow of it is still used. E0515: a function returns a borrow of its own local. E0716: a borrow outlives the temporary it points to.
- Temporaries die at the end of their statement unless lifetime extension applies (a borrow in an extending position of a `let`). `match` and `while let` scrutinees keep their temporaries to the end; in edition 2024, `if let` drops them before `else`, and tail expressions drop them before the block's locals.
- Locals are dropped in reverse order of declaration, fields in declaration order. The drop check treats dropping a value whose type has a `Drop` impl as a use of its borrows; `Vec<&T>` and types without a `Drop` impl are exempt.
- NLL rejects some correct programs, most famously a borrow returned on one path and a mutation on another (problem case #3). Polonius, on by default on nightly since August 2026, accepts it; on stable, use the entry API or a test that holds no borrow.

5.10 Exercises

1. Without compiling, predict which of these calls compile when `buf` has type `&mut String`: `fill(buf); fill(buf);` with `fn fill(b: &mut String)`; `show(buf); show(buf);` with `fn show<T: std::fmt::Debug>(x: T)`; `let b2 = buf; fill(b2); fill(buf);`. Then check your answers and fix the ones that fail with the smallest change. *Hint:* ask, at each call, whether the compiler knows that the expected type is a `&mut` reference.
2. Give `Document` from `ch05_split_borrows.rs` a third field, `errors: Vec<String>`, and write a method that returns mutable references to `words` and `errors` at the same time. Then write a function that moves every word that contains a digit from `words` to `errors`. *Hint:* the method's body can borrow the two fields separately; `retain` and `push` on the two results do the rest.
3. Copy `ch05_temporaries.rs` to a scratch directory and compile it with `rustc --edition 2021`. Which lines of the output change, and why? Then delete the `impl Drop for Lines` from `ch05_match_scrutinee.rs` and compile it again: what happens, and what does that tell you about

when temporary scopes matter? *Hint:* without a destructor, dropping the temporary is not a use of the borrow it holds.

4. Make `ch05_drop_check.rs` compile in three different ways: by changing the order of declarations, by changing the field type so that `Report` owns its lines, and by moving the printing out of the `Drop` impl. Which one would you choose for a type in a library, and why? *Hint:* think about what each change costs the *users* of `Report`.
5. Write `fn longest_or_insert<'v>(words: &'v mut Vec<String>) -> &'v String`, which returns the longest word if it has more than five letters and otherwise pushes `"(none)"` and returns a reference to it. Write it first in the natural way, observe the error, then write a version that compiles on stable Rust. *Hint:* find the *index* of the longest word first; an index holds no borrow.

PART II

Lifetimes in Signatures

Part II turns to the lifetimes that appear in the signatures of real APIs, and to the many places where the compiler writes a lifetime for you. The anonymous lifetime `'_` makes hidden borrows visible; `'static` turns out to mean two different things, one of which is far more common than the other; trait objects and `impl Trait` carry lifetime bounds and captures that are easy to miss and changed in edition 2024; and higher-ranked trait bounds, written `for<'a>`, describe callbacks and traits that must work for every lifetime, including lifetimes that exist only inside the function that receives them.

CHAPTER 6

The Anonymous Lifetime `'_`

Chapter 3 showed that the compiler writes most lifetimes for you: elision fills in the lifetimes of references in a signature, by a fixed set of rules, without looking at the body. That works well as long as every lifetime is the lifetime of an `&`. It works less well for the many types that carry a lifetime parameter of their own, such as `SplitWhitespace<'a>`, `fmt::Formatter<'a>` or your own `Parser<'src>`, and it does not reach the hidden lifetime bound of a trait object at all.

The anonymous lifetime `'_` is how you say "there is a lifetime here, and I do not need to name it". It looks like a wildcard, and that impression is the source of most confusion about it: `'_` does not mean "any lifetime" and it does not mean "figure it out from the body". It means "do what elision would do", and what elision does depends on where you are. This chapter goes through its meaning in each position, its three everyday jobs, and the things it cannot do.

In this chapter you will learn:

- what `'_` means in each position where it is allowed, and why two occurrences of `'_` are never the same lifetime;
- how to make the hidden lifetime parameters of types such as `SplitWhitespace<'a>` and `Formatter<'a>` visible, and what the lints `mismatched_lifetime_syntaxes` and `elided_lifetimes_in_paths` report;
- when `'_` in an `impl` header is enough, and when the `impl` needs a name;
- how `+ '_` lets a boxed trait object borrow, in a return type, in a parameter and in an `impl dyn Trait` block;
- what `'_` means inside a function body;
- what `'_` cannot do, and a checklist for choosing between `'a`, `'_` and writing nothing.

6.1 `'_` is not a name

A named lifetime such as `'a` is a parameter: it is declared once (`fn longest<'a>`) and every occurrence refers to the same lifetime. That is the whole point of a name. `fn longest<'a>(x: &'a str, y: &'a str) -> &'a str` uses `'a` three times to say that the three references share one lifetime.

`'_` is not declared anywhere and cannot be referred to. Each occurrence is a placeholder that the compiler fills in, by the same rules it uses when you leave a lifetime out. In a signature, writing `&'_ str`

is exactly the same as writing `&str`; the difference is only that `'_` can also go where there is no `&` to leave the lifetime off, such as the lifetime argument of a type (`SplitWhitespace<'_>`) or the bound of a trait object (`dyn Fn() + '_`).

Meaning by position

What the compiler puts in place of `'_` depends on where it appears:

Position	`'_` means	Example
Parameter type	a fresh lifetime parameter	`it: SplitWhitespace<'_>`
Return type	the lifetime elision picks; E0106 if none	`-> SplitWhitespace<'_>`
`impl` header	a fresh, unnameable parameter of the `impl`	`impl Display for Words<'_>`
`dyn` bound, return type	the lifetime elision picks	`-> Box<dyn Fn() + '_>`
`dyn` bound, parameter	a fresh lifetime parameter	`Vec<Box<dyn FnMut() + '_>>`
`use<'_>` in a return type	the lifetime elision picks	`impl Sized + use<'_>`
A type in a function body	a lifetime to infer	`MutexGuard<'_, T>`

"A fresh lifetime parameter" is rule 1 of elision (Chapter 3): every elided lifetime in the parameters becomes a distinct lifetime parameter of the function, chosen by the caller. "The lifetime elision picks" is rules 2 and 3: if exactly one parameter carries a lifetime, or if there is a `&self` or `&mut self`, that lifetime is used for every elided lifetime in the output. A `'_` in the output takes part in those rules exactly like the missing lifetime of an `&`. When the rules have no answer, `'_` has none either, and the error is E0106, as we will see.

The table also has holes. `'_` is not allowed in struct fields, in bounds and `where` clauses, in the associated types of an `impl`, or inside an `impl Trait` in argument position. The section *What `'_` cannot do* below shows the errors.

Two `'_` are never the same lifetime

Because every `'_` in the parameters is a fresh lifetime, writing it twice does not tie two positions together. Here is a function that remembers words in a list (`code/compile_fail/ch06_never_equal.rs`):

```
pub fn remember(seen: &mut Vec<&'_ str>, word: &'_ str) {
    seen.push(word);
}
```

The compiler rejects it:

```
error: lifetime may not live long enough
 --> compile_fail/ch06_never_equal.rs:7:5
  |
6 | pub fn remember(seen: &mut Vec<&'_ str>, word: &'_ str) {
  |                               -              - let's call the lifetime of this
reference `'1`
  |                               |
  |                               let's call the lifetime of this reference `'2`
7 |     seen.push(word);
  |     ^^^^^^^^^^^^^^^ argument requires that `'1` must outlive `'2`
  |
  = note: requirement occurs because of a mutable reference to `Vec<&str>`
```

```
  = note: mutable references are invariant over their type parameter
  = help: see <https://doc.rust-lang.org/nomicon/subtyping.html> for more information about
variance
help: consider introducing a named lifetime parameter
  |
6 - pub fn remember(seen: &mut Vec<&'_ str>, word: &'_ str) {
6 + pub fn remember<'a>(seen: &mut Vec<&'a str>, word: &'a str) {
  |
```

The error has no code; its message is "lifetime may not live long enough". The compiler names the two anonymous lifetimes for us: `'2` is the lifetime of the `&str` items inside the vector, and `'1` is the lifetime of `word`. With explicit names, the signature is `fn remember<'v, 'w>(seen: &mut Vec<&'v str>, word: &'w str)`: two unrelated lifetimes, each chosen by the caller. Pushing `word` into the vector stores a `&'w str` where a `&'v str` is expected, which requires `'w` to outlive `'v` ("argument requires that `'1` must outlive `'2`"), and nothing in the signature promises that. The two notes explain why the compiler cannot simply shrink the vector's lifetime to fit: `seen` is a mutable reference, and `&mut T` is invariant in `T` (Chapter 4).

The help is the fix. To say "these two are the same", you need a name, used twice: `fn remember<'a>(seen: &mut Vec<&'a str>, word: &'a str)`. Read `'_` as "a lifetime exists here, and nothing else in the signature refers to it".

6.2 Job 1: making hidden lifetime parameters visible

Many types in the standard library carry a lifetime parameter, because they borrow something: `SplitWhitespace<'a>` and `Lines<'a>` borrow the string they split, `slice::Iter<'a, T>` borrows the slice, `MutexGuard<'a, T>` borrows the mutex, and `fmt::Formatter<'a>` borrows the output buffer it writes to. Rust lets you write these types in a signature without their lifetime argument, as `SplitWhitespace` instead of `SplitWhitespace<'_>`. The compiler then elides the lifetime as it would for an `&`. The lifetime is still there; it is only *hidden*.

The trouble is the reader. `fn words(text: &str) -> SplitWhitespace` looks as if it returned an owned value, but it returns a borrow of `text`, and the caller finds out only when the borrow checker objects to dropping `text` too early. The fix is to write the lifetime, and since nothing else needs to refer to it, `'_` is the way to write it.

The `mismatched_lifetime_syntaxes` lint

Since Rust 1.89, the compiler warns about the most misleading case by default. The program `code/compile_fail/ch06_hidden_return.rs` turns the warnings into errors with `#![deny(mismatched_lifetime_syntaxes)]`, so that they can be shown here. Its first function is the one above:

```
pub fn words(text: &str) -> SplitWhitespace {
    text.split_whitespace()
}
```

```
error: hiding a lifetime that's elided elsewhere is confusing
  --> compile_fail/ch06_hidden_return.rs:10:20
```

```
   |
10 | pub fn words(text: &str) -> SplitWhitespace {
   |                    ^^^^     ^^^^^^^^^^^^^^^^ the same lifetime is hidden here
   |                    |
   |                    the lifetime is elided here
   |
   = help: the same lifetime is referred to in inconsistent ways, making the signature
confusing
note: the lint level is defined here
  --> compile_fail/ch06_hidden_return.rs:5:9
   |
 5 | #![deny(mismatched_lifetime_syntaxes)]
   |         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
help: use `'_` for type paths
   |
10 | pub fn words(text: &str) -> SplitWhitespace<'_> {
   |                                             ++++
```

The lint sorts the ways of writing a lifetime into three kinds: *named* (`&'a str`, `Iter<'a, T>`), *elided* (`&str`, or `'_` anywhere), and *hidden* (a path such as `SplitWhitespace` whose lifetime argument is missing). It fires when one lifetime, connecting an input to the output, is written in two different kinds. Here the labels show both ends of the connection: "the lifetime is elided here" under `&str`, and "the same lifetime is hidden here" under `SplitWhitespace`. The help adds the missing `<'_>`. Without the `deny`, this is a warning and the program compiles; the contract is fine, only its spelling is "confusing".

The second function of the file mixes the named and the elided kinds:

```
pub fn trimmed<'a>(text: &'a str) -> &str {
    text.trim()
}
```

```
error: eliding a lifetime that's named elsewhere is confusing
  --> compile_fail/ch06_hidden_return.rs:16:38
   |
16 | pub fn trimmed<'a>(text: &'a str) -> &str {
   |                           --         ^^^^ the same lifetime is elided here
   |                           |
   |                           the lifetime is named here
   |
   = help: the same lifetime is referred to in inconsistent ways, making the signature
confusing
help: consistently use `'a`
   |
16 | pub fn trimmed<'a>(text: &'a str) -> &'a str {
   |                                       ++
```

By rule 2 of elision, the output is the lifetime of `text`, which the signature calls `'a`; leaving it out of the return type hides that fact one line away from where it was named. The help writes `&'a str`. The corrected signatures are in `code/examples/ch06_hidden_lifetimes.rs`:

```
// In the return type, `'_` is the lifetime elision picks: the one of `text`.
fn words(text: &str) -> SplitWhitespace<'_> {
    text.split_whitespace()
}

// In a parameter, `'_` is a fresh lifetime parameter, as if it were elided.
fn count_long(words: SplitWhitespace<'_>, min_len: usize) -> usize {
    words.filter(|w| w.len() >= min_len).count()
}
```

In `words`, `'_` sits in the return type and means the lifetime of `text`, by rule 2. In `count_long`, it sits in a parameter and means a fresh lifetime parameter, so `count_long` accepts a `SplitWhitespace` borrowing any string at all. Since `&'_ str` and `&str` are both of the elided kind, mixing them is consistent, and the lint stays silent:

```
// `&'_ str` and `&str` are both "elided": using one in the parameter and the
// other in the return type is consistent, and the lint stays silent.
fn trimmed(text: &'_ str) -> &str {
    text.trim()
}
```

> **NOTE**
>
> `mismatched_lifetime_syntaxes` was added in Rust 1.89 (August 2025), as a warn-by-default lint that replaced the earlier `elided_named_lifetimes`. Code that compiled without warnings on older compilers may warn now. The field guide this book grew out of showed `fn words_hidden(s: &str) -> SplitWhitespace` as a warning you had to opt into with `elided_lifetimes_in_paths`; since 1.89 it warns by default. The warning never changes what a program means.

The `elided_lifetimes_in_paths` lint

`mismatched_lifetime_syntaxes` only compares lifetimes that connect inputs with outputs. A hidden lifetime in a parameter, with no output to connect to, passes it. The most common case is the one every `Display` implementation contains, the `Formatter`. A second lint, `elided_lifetimes_in_paths`, catches every hidden lifetime parameter. It is *allow-by-default* (silent unless you ask for it) and belongs to the lint group `rust_2018_idioms`. The program `code/compile_fail/ch06_hidden_param.rs` denies it, and it reports two errors:

```
impl fmt::Display for Port {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "port {}", self.0)
    }
}

pub fn record(log: &Mutex<Vec<String>>, line: &str) -> usize {
    let mut guard: MutexGuard<Vec<String>> = log.lock().unwrap();
    guard.push(line.to_string());
    guard.len()
}
```

```
error: hidden lifetime parameters in types are deprecated
  --> compile_fail/ch06_hidden_param.rs:16:32
   |
16 |     fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
   |                           -----^^^^^^^^^
   |                           |
   |                           expected lifetime parameter
   |
note: the lint level is defined here
  --> compile_fail/ch06_hidden_param.rs:7:9
   |
 7 | #![deny(elided_lifetimes_in_paths)]
   |         ^^^^^^^^^^^^^^^^^^^^^^^^^
help: indicate the anonymous lifetime
   |
```

```
16 |     fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
   |                                         ++++

error: hidden lifetime parameters in types are deprecated
  --> compile_fail/ch06_hidden_param.rs:24:30
   |
24 |     let mut guard: MutexGuard<Vec<String>> = log.lock().unwrap();
   |                    ----------^------------ expected lifetime parameter
   |
help: indicate the anonymous lifetime
   |
24 |     let mut guard: MutexGuard<'_, Vec<String>> = log.lock().unwrap();
   |                               +++
```

The message is "hidden lifetime parameters in types are deprecated", and the label "expected lifetime parameter" points between `Formatter` and its missing `<'_>`. The help, "indicate the anonymous lifetime", writes it. The second error shows that the lint also looks inside function bodies: the guard's type in the `let` hides the lifetime of its borrow of `log`.

To enable the lint for a whole crate, add `#![warn(elided_lifetimes_in_paths)]` (or `#![warn(rust_2018_idioms)]`) at the top of `lib.rs` or `main.rs`, or put it in `Cargo.toml`:

```
[lints.rust]
elided_lifetimes_in_paths = "warn"
```

The example `code/examples/ch06_hidden_lifetimes.rs` denies the lint, and it compiles because every lifetime is visible. Its `Display` implementation is the one you should write out of habit:

```
struct Port(u16);

impl fmt::Display for Port {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "port {}", self.0)
    }
}
```

Running the example prints:

```
3 long words
2 entries: ["listen on port 8080 for incoming requests", "port 8080"]
```

Writing `<'_>` costs four characters and tells every reader of the signature that a borrow is involved. In a crate's public API, it is worth enabling the lint.

6.3 Job 2: `impl` headers

An `impl` block for a type with a lifetime parameter must declare that parameter, as `impl<'src> Parser<'src>` declares `'src` and then uses it. When nothing inside the `impl` mentions the lifetime, the declaration is noise, and `'_` replaces it: in an `impl` header, `'_` declares a fresh lifetime parameter of the `impl` that has no name. `impl fmt::Display for Words<'_>` means exactly `impl<'a> fmt::Display for Words<'a>`. The example `code/examples/ch06_impl_headers.rs` uses a `Words` type that hands out the words of a borrowed text:

```
/// The words of a borrowed text.
struct Words<'src> {
    rest: &'src str,
}
```

The `Display` implementation and the helper method `is_done` never mention the text's lifetime, so `'_` is enough:

```
// Nothing here mentions the lifetime: `'_` is enough.
impl fmt::Display for Words<'_> {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "Words({:?})", self.rest.trim())
    }
}

impl Words<'_> {
    fn is_done(&self) -> bool {
        self.rest.trim().is_empty()
    }
}
```

In `is_done`, the only lifetime in the signature is that of `&self`, which is elided. The `Formatter<'_>` of the previous section is in there too.

When the `impl` needs a name

The `Iterator` implementation is different. Its associated type `Item` is a `&str` borrowed from the text, not from the iterator, and `rest` returns what is left of the text, also as a borrow of the text. Both need to say "the lifetime of the text", and that requires a name:

```
// The associated type and `rest` mention the lifetime: it needs a name.
impl<'src> Iterator for Words<'src> {
    type Item = &'src str;

    fn next(&mut self) -> Option<Self::Item> {
        let trimmed = self.rest.trim_start();
        let end = trimmed.find(' ').unwrap_or(trimmed.len());
        let (word, rest) = trimmed.split_at(end);
        self.rest = rest;
        (!word.is_empty()).then_some(word)
    }
}

impl<'src> Words<'src> {
    /// What is left of the text: a borrow of the text, not of `self`.
    fn rest(&self) -> &'src str {
        self.rest
    }
}
```

Since `rest` returns a `&'src str` rather than a borrow of `self`, its result remains usable after the `Words` value is gone:

```
fn main() {
    let text = String::from("parse the first two words only");
    let mut words = Words { rest: &text };
    let first = words.next();
    let second = words.next();
    let rest = words.rest(); // borrows `text`, so `words` can be dropped
    drop(words);
    println!("{first:?} {second:?} then {rest:?}");
}
```

```
    let done = Words { rest: "   " };
    println!("{done} is done: {}", done.is_done());
}
```

```
Some("parse") Some("the") then " first two words only"
Words("") is done: true
```

What happens if you keep the `'_` in the header and try to refer to the lifetime anyway? The program `code/compile_fail/ch06_impl_header_name.rs` has a `Parser` like the one of Chapter 2:

```
impl Parser<'_> {
    pub fn rest(&self) -> &'src str {
        &self.input[self.pos..]
    }
}
```

```
error[E0261]: use of undeclared lifetime name `'src`
  --> compile_fail/ch06_impl_header_name.rs:12:28
   |
12 |     pub fn rest(&self) -> &'src str {
   |                            ^^^^ undeclared lifetime
   |
help: consider introducing lifetime `'src` here
   |
12 |     pub fn rest<'src>(&self) -> &'src str {
   |                ++++++
help: consider introducing lifetime `'src` here
   |
11 | impl<'src> Parser<'_> {
   |     ++++++
```

E0261, "use of undeclared lifetime name", says that `'src` is a name nobody declared: the struct's definition declares a `'src`, but that name is local to the struct definition, and the `impl` declared its own lifetime anonymously. Both suggestions make the error go away, and both are traps. Each introduces a *new* lifetime called `'src`, on the method or on the `impl`, that has nothing to do with the parser's lifetime. Following the second suggestion literally gives `code/compile_fail/ch06_impl_unrelated.rs`:

```
impl<'src> Parser<'_> {
    pub fn rest(&self) -> &'src str {
        &self.input[self.pos..]
    }
}
```

```
error: lifetime may not live long enough
  --> compile_fail/ch06_impl_unrelated.rs:13:9
   |
11 | impl<'src> Parser<'_> {
   |      ---- lifetime `'src` defined here
12 |     pub fn rest(&self) -> &'src str {
   |                 ----- has type `&Parser<'1>`
13 |         &self.input[self.pos..]
   |         ^^^^^^^^^^^^^^^^^^^^^^^ method was supposed to return data with lifetime `'src`
but it is returning data with lifetime `'1`
```

Now the signature is valid and the body is wrong. The label "has type `&Parser<'1>`" shows that the compiler calls the anonymous lifetime of the header `'1`, and the message says that the method promised data with lifetime `'src` while returning data with lifetime `'1`. Nothing relates the two. (The first suggestion fails in the same way: a `'src` declared on the method is chosen by each caller of `rest`.)

The fix is to give the lifetime its name where it is introduced, in the header: `impl<'src> Parser<'src>`, as `Words` does above.

Associated types follow the same rule. An associated type is not a function signature, so there is no elision for it, and `'_` is rejected outright (`code/compile_fail/ch06_assoc_type.rs`):

```
impl Iterator for Words<'_> {
    type Item = &'_ str;

    fn next(&mut self) -> Option<Self::Item> {
        let trimmed = self.rest.trim_start();
        let end = trimmed.find(' ').unwrap_or(trimmed.len());
        let (word, rest) = trimmed.split_at(end);
        self.rest = rest;
        (!word.is_empty()).then_some(word)
    }
}
```

```
error[E0637]: `'_` cannot be used here
  --> compile_fail/ch06_assoc_type.rs:11:18
   |
11 |     type Item = &'_ str;
   |                  ^^ `'_` is a reserved lifetime name
```

E0637, "`'_` cannot be used here", with the label "`'_` is a reserved lifetime name": the compiler will not invent a lifetime for `Item`. The rule of thumb for `impl` headers: write `'_` when the `impl` holds for every lifetime and nothing inside refers to it; declare a name as soon as a method signature, an associated type or a `where` clause must mention it.

6.4 Job 3: letting trait objects borrow

A trait object type always carries a lifetime bound. `dyn Fn(&str) -> String + 'a` means "a value of some type that implements `Fn(&str) -> String` and contains no borrow shorter than `'a`". The bound is needed because the concrete type is erased: the compiler can no longer look inside the value to see what it borrows, so the type must say how long the value may be used. When you do not write a bound, the compiler uses a default, and inside a `Box` the default is `'static`. Chapter 7 met this default in a registry of callbacks; Chapter 8 gives all the rules.

The default is the wrong contract for a function that builds a closure from a borrowed argument (`code/compile_fail/ch06_dyn_static.rs`):

```
pub fn make_prefixer(prefix: &str) -> Box<dyn Fn(&str) -> String> {
    Box::new(move |line| format!("{prefix}{line}"))
}
```

```
error: lifetime may not live long enough
 --> compile_fail/ch06_dyn_static.rs:7:5
  |
6 | pub fn make_prefixer(prefix: &str) -> Box<dyn Fn(&str) -> String> {
  |                              - let's call the lifetime of this reference `'1`
7 |     Box::new(move |line| format!("{prefix}{line}"))
  |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ returning this value requires that
`'1` must outlive `'static`
  |
help: to declare that the trait object captures data from argument `prefix`, you can add an
```

```
explicit `'_` lifetime bound
  |
6 | pub fn make_prefixer(prefix: &str) -> Box<dyn Fn(&str) -> String + '_> {
  |                                                                  ++++
```

Read the labels from the top. The compiler calls the lifetime of `prefix` `'1`. The closure captures `prefix` (a `&'1 str`), so the boxed closure contains a borrow with lifetime `'1`. "Returning this value requires that `'1` must outlive `'static`" is the hidden default at work: the return type is really `Box<dyn Fn(&str) -> String + 'static>`. The help gives the fix and names the job: "to declare that the trait object captures data from argument `prefix`, you can add an explicit `'_` lifetime bound". In a return type, `+ '_` is the lifetime that elision picks, here the lifetime of `prefix`. The working version is in `code/examples/ch06_dyn_borrow.rs`:

```
// The closure borrows `prefix`, and `+ '_` says so.
fn make_prefixer(prefix: &str) -> Box<dyn Fn(&str) -> String + '_> {
    Box::new(move |line| format!("{prefix}{line}"))
}
```

`+ '_` in a parameter

In a parameter, `+ '_` follows rule 1 instead: it is a fresh lifetime parameter, not "the same lifetime as something else". A function that runs a list of tasks can accept tasks that borrow anything at all, for the duration of the call:

```
// In a parameter, `+ '_` is a fresh lifetime: tasks that borrow anything at all
// are accepted, for the duration of the call.
fn run_all(tasks: Vec<Box<dyn FnMut() + '_>>) {
    for mut task in tasks {
        task();
    }
}
```

Without `+ '_`, the parameter would be `Vec<Box<dyn FnMut() + 'static>>`, and a task that borrows a local variable would be rejected, just as Chapter 7's `register` rejected a closure that borrows `prefix`. With it, `main` can pass tasks that borrow two local variables mutably:

```
fn main() {
    let prefix = String::from(">> ");
    let quote = make_prefixer(&prefix);
    println!("{}", quote("hello"));

    let mut log = Vec::new();
    let mut runs = 0;
    let tasks: Vec<Box<dyn FnMut() + '_>> =
        vec![Box::new(|| log.push("indexed")), Box::new(|| runs += 1)];
    run_all(tasks);
    println!("{log:?} after {runs} counted run(s)");

    println!("{}", report(&Square(1.5)));
}
```

The `'_` in the `let` of `main` is in a function body, where it means "infer this lifetime" (see below).

`impl dyn Trait + '_`

The default also applies to the type in an `impl` header. An inherent `impl` for a trait object type adds methods that every trait object of that trait can use,

but `impl dyn Shape` means `impl dyn Shape + 'static`, and that is rarely what you want (`code/compile_fail/ch06_impl_dyn_static.rs`):

```
impl dyn Shape {
    pub fn describe(&self) -> String {
        format!("a shape of area {:.2}", self.area())
    }
}

pub fn report(shape: &dyn Shape) -> String {
    shape.describe()
}
```

```
error[E0521]: borrowed data escapes outside of function
  --> compile_fail/ch06_impl_dyn_static.rs:17:5
   |
16 | pub fn report(shape: &dyn Shape) -> String {
   |               -----  - let's call the lifetime of this reference `'1`
   |               |
   |               `shape` is a reference that is only valid in the function body
17 |     shape.describe()
   |     ^^^^^^^^^^^^^^^^
   |     |
   |     `shape` escapes the function body here
   |     argument requires that `'1` must outlive `'static`
   |
note: the used `impl` has a `'static` requirement
  --> compile_fail/ch06_impl_dyn_static.rs:10:10
   |
10 | impl dyn Shape {
   |          ^^^^^ this has an implicit `'static` lifetime requirement
11 |     pub fn describe(&self) -> String {
   |            -------- calling this method introduces the `impl`'s `'static` requirement
help: consider relaxing the implicit `'static` requirement
   |
10 | impl dyn Shape + '_ {
   |                ++++
```

E0521 is "borrowed data escapes outside of function", which sounds more dramatic than it is: nothing is stored anywhere. In `report`, the parameter `&dyn Shape` means `&'1 (dyn Shape + '1)` (Chapter 8 explains why), so the object behind it may borrow data that lives only for `'1`. Calling `describe` requires a `dyn Shape + 'static`, so it requires `'1` to outlive `'static`, and in the compiler's words, the borrow "escapes" into a context that demands more. The note names the culprit: "the used `impl` has a `'static` requirement", "this has an implicit `'static` lifetime requirement". The help relaxes it with `'_`:

```
trait Shape {
    fn area(&self) -> f64;
}

// `impl dyn Shape + '_` is `impl<'a> dyn Shape + 'a`: every trait object.
impl dyn Shape + '_ {
    fn describe(&self) -> String {
        format!("a shape of area {:.2}", self.area())
    }
}

fn report(shape: &dyn Shape) -> String {
    shape.describe()
}
```

In an `impl` header, `'_` is a fresh parameter of the `impl`, so `impl dyn Shape + '_` is `impl<'a> dyn Shape + 'a`: the methods are available on trait objects of every lifetime. The output of the example:

```
>> hello
["indexed"] after 1 counted run(s)
a shape of area 2.25
```

Return-position `impl Trait` used to need `+ '_` for the same reason, and code written for edition 2021 is full of `-> impl Iterator<Item = char> + '_`. In edition 2024 it is rarely needed, because the rules for what an `impl Trait` captures changed; Chapter 8 explains them, together with the `use<..>` syntax that replaced most uses of `+ '_` on `impl Trait`.

6.5 `'_` inside function bodies

Inside a function body, nothing is elided: every lifetime that is not written is *inferred* by the borrow checker from how the values are used (Chapter 1). A type annotation on a `let`, in a turbofish or in a cast can therefore leave all its lifetimes out, and `'_` there means "infer this one". Its use is the same as in signatures, to make a hidden lifetime parameter visible:

```rust
fn record(log: &Mutex<Vec<String>>, line: &str) -> usize {
    // In a body, `'_` stands for "infer this lifetime": the guard borrows `log`.
    let mut guard: MutexGuard<'_, Vec<String>> = log.lock().unwrap();
    guard.push(trimmed(line).to_string());
    guard.len()
}
```

The guard borrows `log`, and `MutexGuard<'_, Vec<String>>` says so. As the second error of `ch06_hidden_param.rs` showed, `elided_lifetimes_in_paths` asks for the `'_` here too. For plain references, `&str` and `&'_ str` mean the same in a body as they do in a signature, so there is no reason to write `'_` after an `&`.

6.6 What `'_` cannot do

It does not resolve ambiguity

When elision has no answer, `'_` has none either. Here is a function with two reference parameters (`code/compile_fail/ch06_ambiguous.rs`):

```rust
// The body only ever returns a piece of `text`, but the signature is read
// without the body: two inputs, so elision has nothing to choose.
pub fn first_word(text: &str, separators: &str) -> &'_ str {
    text.split(|c| separators.contains(c)).next().unwrap_or("")
}
```

```
error[E0106]: missing lifetime specifier
 --> compile_fail/ch06_ambiguous.rs:8:53
  |
8 | pub fn first_word(text: &str, separators: &str) -> &'_ str {
  |                         ----              ----      ^^ expected named lifetime parameter
  |
  = help: this function's return type contains a borrowed value, but the signature does not
```

```
say whether it is borrowed from `text` or `separators`
help: consider introducing a named lifetime parameter
  |
8 - pub fn first_word(text: &str, separators: &str) -> &'_ str {
8 + pub fn first_word<'a>(text: &'a str, separators: &'a str) -> &'a str {
  |
```

E0106 is "missing lifetime specifier". The `'_` in the return type asks elision for a lifetime, and with two input lifetimes and no `self`, rule 2 does not apply. The help explains the problem in plain words: "the signature does not say whether it is borrowed from `text` or `separators`". Writing `&str` instead of `&'_ str` gives the same error, because the two spellings mean the same.

It does not look at the body

The body of `first_word` returns only pieces of `text`, so a human can see which input the result borrows from. The compiler does not look. Signatures are contracts, checked on their own: the body is checked against the signature, and callers are checked against the signature and nothing else. If the signature depended on the body, a change inside the function could break its callers, and the compiler could not check callers without reading every function they call.

So the fix must go in the signature, and the help's suggestion deserves a second look: it gives all three references the lifetime `'a`, which compiles but promises too much. Callers would have to keep `separators` alive for as long as they use the result, although the result never borrows it. The honest contract names only the lifetime that the result shares with an input:

```
// 2. Two positions that must be the same lifetime: a name. The result borrows
//    `text` only, so `separators` keeps an elided lifetime of its own.
fn first_word<'t>(text: &'t str, separators: &str) -> &'t str {
    text.split(|c| separators.contains(c)).next().unwrap_or("")
}
```

It cannot go in fields, bounds and `where` clauses

`'_` exists to stand for a lifetime that some rule can fill in. Where no rule applies, it is an error:

- In a struct field, there is nothing to elide from: a struct must declare its lifetime parameters, and a field such as `text: &'_ str` is E0106 (see `code/compile_fail/ch03_struct_field_placeholders.rs` in Chapter 3).
- In an associated type of an `impl`, it is E0637, as shown above.
- Inside an `impl Trait` in argument position, such as `words: impl Iterator<Item = &'_ str>`, it is E0658, "anonymous lifetimes in `impl Trait` are unstable". Chapter 8 shows the error and the fix.
- In a bound or a `where` clause, it is E0637 again (`code/compile_fail/ch06_where_clause.rs`):

```
pub fn longest_line<I>(lines: I) -> usize
where
    I: Iterator<Item = &'_ str>,
{
    lines.map(str::len).max().unwrap_or(0)
}
```

```
error[E0637]: `'_` cannot be used here
 --> compile_fail/ch06_where_clause.rs:8:25
```

```
  |
8 |     I: Iterator<Item = &'_ str>,
  |                         ^^ `'_` is a reserved lifetime name
```

The fix is a named lifetime parameter: declare `'a` next to `I` and write `I: Iterator<Item = &'a str>`. The iterator yields references of one lifetime, picked by the caller.

6.7 Checklist: `'a`, `'_`, or nothing?

The rules above reduce to a short checklist:

1. **A lifetime that appears once and is never referred to again:** write nothing for a reference, and `'_` for a type path.
2. **Two positions that must be the same lifetime** (two inputs, or an input and the output when elision cannot tell which): a name, as in `first_word` above.
3. **A result that borrows from `self`:** elide it. Rule 3 ties it to `&self`.
4. **A result that borrows from the data a struct borrows**, rather than from `self`: name the struct's lifetime parameter in the `impl` header and use it.
5. **A boxed trait object that borrows an argument:** `+ '_` when elision picks the right input, `+ 'a` otherwise. In an `impl` for a trait object type, `impl dyn Trait + '_`.
6. **In a function body:** nothing, or `'_` where a type path would otherwise hide its lifetime.

The example `code/examples/ch06_checklist.rs` has one function for each of the first five cases (the second is `first_word`, shown above):

```rust
// 1. A lifetime that appears once and is never referred to again: write nothing
//    for a reference, and `'_` for a type path.
fn words(text: &str) -> SplitWhitespace<'_> {
    text.split_whitespace()
}

// 3. The result borrows from `self`: elide (rule 3 of elision).
struct Config {
    name: String,
}

impl Config {
    fn name(&self) -> &str {
        &self.name
    }
}

// 4. The result borrows from the data the struct borrows, not from `self`:
//    name the struct's parameter in the `impl` header.
struct Cursor<'src> {
    rest: &'src str,
}

impl<'src> Cursor<'src> {
    fn take(&mut self, n: usize) -> &'src str {
        let (head, tail) = self.rest.split_at(n.min(self.rest.len()));
        self.rest = tail;
        head
    }
}

// 5. A boxed trait object that borrows an argument: `+ '_`.
```

```
fn matcher(needle: &str) -> Box<dyn Fn(&str) -> bool + '_> {
    Box::new(move |line| line.contains(needle))
}
```

The example's `main` checks the contracts: `key` outlives `separators`, and the two results of `take` are used together, which works only because they borrow the text and not the cursor:

```
fn main() {
    let text = String::from("key=value; other=thing");
    let key;
    {
        let separators = String::from("=;");
        key = first_word(&text, &separators);
    } // `separators` is gone; `key` borrows only `text`
    println!("{key} / {} words", words(&text).count());

    let config = Config {
        name: String::from("parser"),
    };
    println!("{}", config.name());

    let mut cursor = Cursor { rest: &text };
    let (a, b) = (cursor.take(3), cursor.take(1)); // both borrow `text`
    println!("{a:?} {b:?}");

    let has_value = matcher("value");
    println!("{}", has_value(&text));
}
```

```
key / 2 words
parser
"key" "="
true
```

6.8 Common confusions

"**`'_` means 'any lifetime'" or "`'_` means 'infer it'.**" In a signature, `'_` means "elide": a fresh lifetime in a parameter, the lifetime chosen by the elision rules in the output, a fresh parameter in an `impl` header. Only in a function body does it mean "infer".

"**Writing `'_` twice makes the two lifetimes equal.**" Never. Every `'_` is a different lifetime, as `remember` showed. Equality needs a name.

"**`'_` is a short lifetime.**" It is whatever the rules make it. If `make_prefixer` is called with a string literal, the `'_` of its result is `'static`.

"**`Box<dyn Trait + '_>` in a parameter is the same as `Box<dyn Trait>`.**" No: without `+ '_` the parameter accepts only trait objects that borrow nothing shorter than `'static`; with it, any borrow is accepted for the duration of the call.

"**`impl Display for Words<'_>` is a different `impl` from `impl<'a> Display for Words<'a>`.**" They are the same `impl`, written two ways. Writing both is an error, "conflicting implementations" (E0119).

"**The lints are about style.**" They are about style in that the meaning of the program does not change. But a hidden lifetime is a borrow that the signature does not show, and showing it is the purpose of a signature.

6.9 Summary

- '_ is a placeholder, not a name: each occurrence stands for its own lifetime, filled in by the rules of elision for its position. To make two positions share a lifetime, use a name.
- In parameters it is a fresh lifetime; in return types and in + '_ or use<'_> on a return type, it is the lifetime elision picks (E0106 when there is none); in an impl header, a fresh unnameable parameter of the impl; in a body, a lifetime to infer.
- Job 1: make hidden lifetime parameters visible, SplitWhitespace<'_>, Formatter<'_>, MutexGuard<'_, T>. mismatched_lifetime_syntaxes (warn-by-default since 1.89) flags a lifetime written in two different ways between inputs and output; elided_lifetimes_in_paths (allow-by-default, in rust_2018_idioms) flags every hidden one.
- Job 2: in impl headers, '_ replaces a lifetime that nothing inside refers to. A method that returns data with the struct's lifetime, or an associated type that mentions it, needs a named parameter on the impl; the compiler's suggestion to declare a new 'src elsewhere creates an unrelated lifetime.
- Job 3: trait objects default to + 'static in a Box; + '_ lets them borrow. In a parameter it accepts any borrow; impl dyn Trait + '_ makes an inherent impl apply to trait objects of every lifetime.
- '_ never resolves an ambiguous signature and never consults the body. It is not allowed in fields, bounds, where clauses, the associated types of an impl, or argument-position impl Trait.

6.10 Exercises

1. Which of these signatures does mismatched_lifetime_syntaxes flag, and which does elided_lifetimes_in_paths flag? Write each with the hidden lifetimes visible. fn lines_of(s: &str) -> std::str::Lines, fn peek(it: &mut Peekable<Chars>) -> Option<char>, fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result. *Hint:* the first lint needs a lifetime that connects an input to the output; only one of the three has one.
2. The function fn peek_ref(it: &mut Peekable<Chars>) -> Option<&char> is rejected with E0106, although it has a single reference parameter. Explain the error, then write the signature with every lifetime visible and decide which one the result should borrow from. *Hint:* write Chars<'_> and count the lifetimes in the parameter; rule 2 needs exactly one.
3. Expand fn tag(word: &'_ str, out: &mut Vec<&'_ str>) -> Box<dyn Fn() + '_> into named lifetimes. Does it compile? If not, give a signature that does and that lets out be used after the closure is gone. *Hint:* there are three elided input lifetimes. The compiler's suggestion gives all of them one name, &'a mut Vec<&'a str>; Chapter 3 explains why that locks out.
4. Remove the + '_ from impl dyn Shape + '_ in ch06_dyn_borrow.rs. Which calls still compile: describe on a Box<dyn Shape>, on a &dyn Shape parameter, on a &Square coerced in main? Predict, then check. *Hint:* ask, for each receiver, what the object's lifetime bound is; a Box<dyn Shape> written in a signature has the default 'static.

5. In `ch06_impl_headers.rs`, change `rest` to `fn rest(&self) -> &str`, keeping `impl<'src>`. What does the function now promise, and which line of `main` stops compiling? *Hint:* rule 3 ties the result to `&self`; look at the `drop(words)`.

CHAPTER 7

The `'static` Lifetime

Sooner or later the compiler tells you that something "must outlive `'static`". It happens when you hand a closure to `thread::spawn`, when you push a boxed callback into a registry, when you return a `Box<dyn Error>`, or when a generic function you wrote calls one of those. You never wrote `'static` anywhere, and the value in question lives for a few microseconds. The quickest way to make the message go away is to write `&'static` on a parameter, and that is usually the wrong fix: it moves the error to every caller.

The confusion has a simple cause. `'static` appears in two different positions, and they say two different things. `&'static T` is a *reference* that stays valid until the program ends. `T: 'static` is a *bound* on a type that says the type holds no borrow that can expire. A `String` created and dropped inside one function satisfies the second and has nothing to do with the first. This chapter separates the two meanings, shows where each one comes from, explains why some APIs insist on `'static`, and ends with a ladder of fixes for when they do.

In this chapter you will learn:

- the difference between `&'static T`, a reference to data that stays valid until the program ends, and `T: 'static`, a bound that says a type holds no borrow that can expire;
- where `&'static` references come from: literals, `static` and `const` items, constant promotion, leaking, and lazily initialized globals (`OnceLock`, `LazyLock`);
- why `T: 'static` does not mean "lives forever", and the one question that tells you whether a type satisfies it;
- why `thread::spawn`, `Any`, stored callbacks and generic code ask for `'static`, and how to read E0373, E0521, E0597 and E0310 when they do;
- the hidden `'static` in `Box<dyn Trait>`;
- the common myths, and a decision ladder for "the compiler wants `'static`": own, scope, mix, leak.

7.1 Two meanings of `'static`

Chapter 4 placed lifetimes in an order: `'a: 'b` ("`'a` outlives `'b`") when the region `'a` contains the region `'b`. `'static` is the largest region of all. It covers the whole run of the program, so `'static: 'a` holds for every lifetime `'a`. That is the only `'static` there is. What changes between the two uses is what the lifetime is attached to:

You write	What it is	What it says
`&'static T`	a reference type	the referent stays valid until the program ends
`T: 'static`	a bound on a type	every borrow inside `T` is valid until the program ends

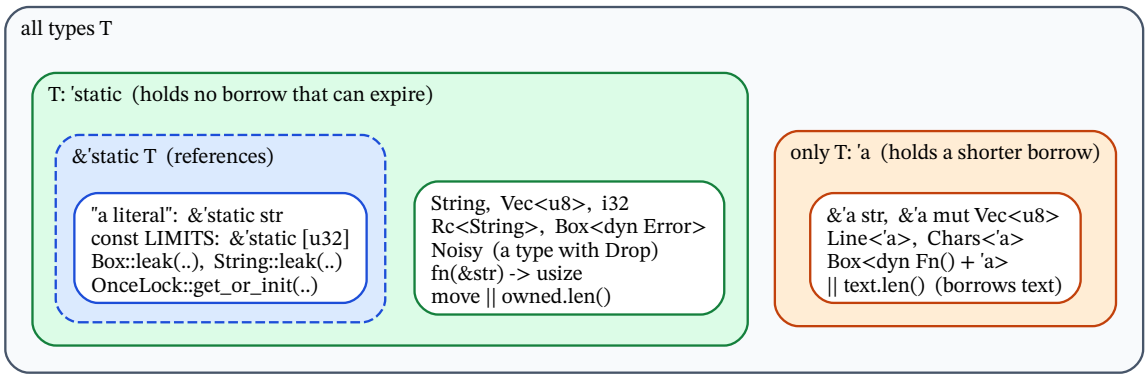
The first is a type: a pointer with a guarantee about its target. A string literal has type `&'static str`, because its bytes are part of the executable. And since `'static` outlives everything, a `&'static T` can be used wherever a `&'a T` is expected, for any `'a` (that is covariance, Chapter 4).

The second is a property of a type, written with the outlives notation of Chapter 1. `T: 'a` means "every lifetime that appears in `T` outlives `'a`", or more simply "every borrow that `T` holds is valid for at least `'a`". With `'static` in place of `'a`, it says that every borrow inside `T` is valid until the program ends. Concretely:

- A type that contains no lifetime at all, such as `String`, `Vec<u8>`, `i32` or `Box<[f64]>`, satisfies `T: 'static` unconditionally. It holds no borrow, so it holds no borrow that can expire.
- A reference type `&'a U` satisfies it only if `'a` is `'static` (and `U` does too). A borrow of a local variable never does.
- A struct with a lifetime parameter, such as `Line<'a>`, satisfies it only if `'a` is `'static`.
- A generic type such as `Vec<T>` or `Rc<T>` satisfies it exactly when `T` does.

So `T: 'static` is not about how long a *value* lives. It is about what the *type* is allowed to contain. A good way to read it is "`T` could, if someone wanted to, be kept until the end of the program without any of its borrows dangling". Whether anyone actually keeps it that long is a different question.

Drawn as sets of types, the two meanings look like this:



The types that satisfy `T: 'static` hold no borrow that can expire; the `&'static T` reference types are a small part of them, and types that hold a shorter borrow satisfy only `T: 'a`.

The diagram makes two points. First, every `&'static T` (with `T: 'static`) satisfies the bound, but most types that satisfy the bound are not references at all: they are owning types. Second, the types outside the green set are not defective. `&'a str` or `Line<'a>` are perfectly good types; they hold a borrow, so they can be used only while that borrow is valid, and an API that needs to keep a value for an unknown amount of time cannot accept them.

7.2 Where &'static references come from

A &'static T must point to something that is never freed, or at least not before the program ends. There are five sources of such references: literals, static and const items, constant promotion, leaking, and lazily initialized globals. The program code/examples/ch07_static_sources.rs uses the first four.

Literals, static items and const items

```
// The type of a string literal is `&'static str`: the bytes are part of the binary.
static BANNER: &str = "lifetimes"; // elided: `&'static str`
const LIMITS: &[u32] = &[1, 2, 3]; // elided: `&'static [u32]`

// A `static` item lives at one address for the whole run, so `&GREETING` is
// `&'static String`, even though `String` has a destructor (it never runs).
static GREETING: String = String::new();
```

A string literal is stored in the read-only data of the executable, so it exists before main starts and after it returns: its type is &'static str. A static item is a single value at a fixed address for the whole run of the program, and it is never dropped, so &GREETING is a &'static String even though String has a destructor (the destructor simply never runs). A const item is different: it has no address of its own, and its value is copied into every place that uses it. A const of reference type, such as LIMITS, holds a reference into static memory, so it is &'static too.

Notice that BANNER and LIMITS are written &str and &[u32] without a lifetime. In the type of a static or const item, an elided lifetime means 'static (Chapter 3 lists this among the elision rules). It is the only sensible choice: a global has no shorter region to borrow from.

Constant promotion

The second source is less visible. When you write & in front of a *constant expression* inside a function, the compiler can store the value in static memory instead of in a temporary on the stack, and give you a &'static reference to it. This is called *constant promotion*:

```
fn promoted_number() -> &'static i32 {
    &42 // a constant expression: promoted to an anonymous static
}

fn promoted_table() -> &'static [&'static str] {
    &["fn", "let", "match"] // an array of literals is a constant too
}
```

Without promotion, &42 would be a reference to a temporary that dies at the end of the function, and returning it would be an error. With promotion, it is a reference to an anonymous static that holds 42. The same goes for an array of literals.

Promotion has limits, though. The value must be a constant, and it must not have a destructor or interior mutability: a promoted value is shared by every call of the function, so it must be something that can neither be dropped nor changed. The program code/compile_fail/ch07_not_promoted.rs tries both:

```
pub fn empty_name() -> &'static String {
    &String::new() // `String` has a destructor: not promoted
}

pub fn counter() -> &'static Cell<u32> {
    &Cell::new(0) // interior mutability: not promoted
}
```

The compiler rejects both functions with the same error:

```
error[E0515]: cannot return reference to temporary value
 --> compile_fail/ch07_not_promoted.rs:9:5
  |
9 |     &String::new() // `String` has a destructor: not promoted
  |     ^-------------
  |     ||
  |     |temporary value created here
  |     returns a reference to data owned by the current function

error[E0515]: cannot return reference to temporary value
  --> compile_fail/ch07_not_promoted.rs:13:5
   |
13 |     &Cell::new(0) // interior mutability: not promoted
   |     ^------------
   |     ||
   |     |temporary value created here
   |     returns a reference to data owned by the current function
```

Read the first diagnostic label by label. "Temporary value created here" points at `String::new()`: the value is an ordinary temporary, not a promoted constant. "Returns a reference to data owned by the current function" points at the `&`: the reference would outlive the temporary, which is dropped when the function returns. Error E0515 is the general "returning a reference to a local" error, which Chapter 5 covers in detail; here it tells you that promotion did not happen. If you really need a `&'static String`, declare a `static` item, as `GREETING` does above. More often, the right answer is to return an owned `String`.

Leaking

The third source turns heap data into static data, by giving up on ever freeing it. `Box::leak`, `String::leak` and `Vec::leak` consume an owning value and return a reference to its contents:

```
fn leaked(name: String) -> &'static str {
    // The heap buffer is never freed: the reference stays valid until exit.
    name.leak()
}
```

The signature of `String::leak` in the standard library is `pub fn leak<'a>(self) -> &'a mut str` (stable since Rust 1.72; `Box::leak` has the same shape). The lifetime `'a` appears only in the output, so the caller picks it, and `'static` is a valid choice: nothing will ever free the buffer, so the reference can be valid for as long as anyone wants. Note that the result is a *mutable* reference: `&'static mut str` is a perfectly good type.

Leaking is safe (Chapter 13 explains why Rust treats it as safe), but it is a memory leak by definition. It is the right tool for data that must live until the program exits anyway, such as a configuration loaded once at startup or a table of interned names. It is a bug when it happens once per request or once per loop iteration, because the memory use then grows without bound.

Here are the first four sources at work:

```
let literal: &'static str = "a literal";
let empty: &'static String = &GREETING;
let label: &'static str = leaked(format!("{}-{}", "worker", 7));
println!("literal:  {literal}");
println!("static:   {BANNER} and {empty:?}");
println!("const:    {LIMITS:?}");
println!("promoted: {} and {:?}", promoted_number(), promoted_table());
println!("leaked:   {label}");
```

```
literal:  a literal
static:   lifetimes and ""
const:    [1, 2, 3]
promoted: 42 and ["fn", "let", "match"]
leaked:   worker-7
```

Lazily initialized globals: `OnceLock` and `LazyLock`

A `static` item must be initialized with a constant expression, which rules out anything that allocates, reads a file or parses text. The standard library has two types that fill this gap: `OnceLock<T>` (stable since Rust 1.70) is a cell that can be written once, and `LazyLock<T>` (since Rust 1.80) is a value computed by a closure the first time it is used. Both are safe to use from several threads. Put one in a `static`, and you get `&'static` references to data built at run time. The program is `code/examples/ch07_globals.rs`:

```
static CONFIG: OnceLock<Config> = OnceLock::new();

/// Initializes the configuration once; later calls return the same value.
fn config() -> &'static Config {
    CONFIG.get_or_init(|| {
        println!("(loading config)");
        Config {
            name: String::from("indexer"),
            workers: 4,
        }
    })
}
```

The signature of `OnceLock::get_or_init` is `pub fn get_or_init<F>(&self, f: F) -> &T`, with the output lifetime elided: the result borrows from `&self` (elision rule 3, Chapter 3). Since `CONFIG` is a `static`, `&CONFIG` is `&'static OnceLock<Config>`, so the result is a `&'static Config`. No leak and no `unsafe` are involved; the global was always going to live until the end of the program.

`LazyLock` puts the initializer in the declaration. It dereferences to the value, so you can call methods on it directly:

```
static KEYWORDS: LazyLock<BTreeMap<&'static str, usize>> = LazyLock::new(|| {
    println!("(building keyword table)");
    ["fn", "let", "match", "static"]
        .into_iter()
        .enumerate()
        .map(|(id, word)| (word, id))
        .collect()
});

fn keyword_id(word: &str) -> Option<usize> {
    KEYWORDS.get(word).copied()
}
```

```
let first: &'static Config = config();
let second: &'static Config = config(); // no second "(loading config)"
assert!(std::ptr::eq(first, second));
println!("{} with {} workers", first.name, second.workers);

println!("static -> {:?}", keyword_id("static"));
println!("struct -> {:?}", keyword_id("struct"));
```

```
(loading config)
indexer with 4 workers
(building keyword table)
static -> Some(3)
struct -> None
```

Each initializer runs once, the first time the global is used, and both calls to `config()` return the same reference (the `assert!` checks the address). Use `OnceLock` when the code that initializes the global is somewhere else (for example `main` sets it from the command line); use `LazyLock` when the initializer is self-contained. Chapter 12 shows `OnceCell`, the single-threaded version, whose `get_or_init` returns a reference tied to the cell rather than a `&'static` one.

7.3 `T: 'static` does not mean "lives forever"

Now the bound. The most common misreading is that `T: 'static` means "the value lives until the end of the program". The program `code/examples/ch07_static_type_dropped.rs` shows that it does not. `Noisy` is a type whose only field is a `&'static str`, so it satisfies `T: 'static`. It prints a line when it is dropped:

```
/// Prints a line when it is dropped, so we can see when that happens.
struct Noisy(&'static str);

impl Drop for Noisy {
    fn drop(&mut self) {
        println!("  drop {}", self.0);
    }
}

/// Accepts any value whose type holds no borrow that can expire.
fn requires_static<T: 'static>(_value: T) {
    println!("  inside requires_static");
} // `_value` is dropped here, like any parameter
```

```
println!("call 1");
requires_static(Noisy("first")); // `Noisy: 'static`, and it dies right away
println!("call 2");
let local = String::from("owned by main");
requires_static(local); // a local `String` is fine: it is moved in
println!("call 3");
requires_static(7_u8);
requires_static("a literal"); // `&'static str: 'static`
println!("done");
```

```
call 1
  inside requires_static
  drop first
call 2
  inside requires_static
call 3
  inside requires_static
```

```
  inside requires_static
done
```

The `Noisy("first")` value is created in `main`, moved into `requires_static`, and dropped at the end of that call, before "call 2" is printed. Its type is `'static`; the value lived for a few lines. The same is true of the `String` created by `main`: it is a local, and it is moved into the function and dropped there. `7_u8` and a string literal are accepted too.

What fails the bound is a *borrow* of a local variable. In `code/compile_fail/ch07_static_bound_borrow.rs`, the caller passes `&local` instead of `local`:

```
pub fn caller() {
    let local = String::from("owned by caller");
    requires_static(&local); // `&'a String: 'static` only if `'a` is `'static`
}
```

The compiler rejects it:

```
error[E0597]: `local` does not live long enough
  --> compile_fail/ch07_static_bound_borrow.rs:10:21
   |
 9 |     let local = String::from("owned by caller");
   |         ----- binding `local` declared here
10 |     requires_static(&local); // `&'a String: 'static` only if `'a` is `'static`
   |     ----------------^^^^^^_
   |     |               |
   |     |               borrowed value does not live long enough
   |     argument requires that `local` is borrowed for `'static`
11 | }
   | - `local` dropped here while still borrowed
   |
note: requirement that the value outlives `'static` introduced here
  --> compile_fail/ch07_static_bound_borrow.rs:5:23
   |
 5 | fn requires_static<T: 'static>(_value: T) {}
   |                       ^^^^^^^
```

The labels tell the whole story. `T` is inferred as `&'a String` for some region `'a`. The bound `T: 'static` requires `'a` to be `'static` ("argument requires that `local` is borrowed for `'static`"), but `local` is dropped at the closing brace ("`local` dropped here while still borrowed"), so no borrow of it can last that long ("borrowed value does not live long enough"). The note at the bottom is the most useful line: it points at the place where the requirement comes from, the `T: 'static` bound of `requires_static`. When you meet a "borrowed for `'static`" error in real code, look for that note: it tells you which API asked for `'static`.

The test: does it hold a borrow that can expire?

So the practical question is not "how long does the value live?" but "does the type hold a borrow that can expire?". Here is the answer for common types:

Type	T: 'static ?	Why
String , Vec<String> , i32	yes	owns all its data
&'static str	yes	the only borrow inside is 'static
Rc<String> , Arc<String>	yes	Rc<T>: 'static exactly when T: 'static
Box<dyn Error>	yes	means Box<dyn Error + 'static>
fn(&str) -> usize	yes	a function pointer holds no data
a move closure owning a String	yes	captures by value
&'a str (borrowing a local)	no	holds a short borrow
Line<'a> (borrowing a local)	no	its lifetime parameter is not 'static
a closure that uses a local by reference	no	captures a borrow

Every "yes" row is checked by code/examples/ch07_static_table.rs , which passes one value of each type to a function with a T: 'static bound. To make the run visible, the function prints the name of the type it received:

```
is_static(&String::from("owned"));
is_static(&vec![String::from("a"), String::from("b")]);
is_static(&42_i32);
is_static(&"a literal"); // `&'static str`
is_static(&Rc::new(String::from("shared"))); // `Rc<T>: 'static` iff `T: 'static`
let error: Box<dyn Error> = "boom".into(); // `Box<dyn Error + 'static>`
is_static(&error);
is_static(&(word_count as fn(&str) -> usize)); // a function pointer
let owned = String::from("captured by move");
is_static(&move || owned.len()); // a `move` closure that owns a `String`
```

```
alloc::string::String
alloc::vec::Vec<alloc::string::String>
i32
&str
alloc::rc::Rc<alloc::string::String>
alloc::boxed::Box<dyn core::error::Error>
fn(&'_ str) -> usize
ch07_static_table::main::{{closure}}
```

The output contains a small lesson of its own. std::any::type_name prints &str for a &'static str and Box<dyn core::error::Error> without the + 'static : lifetimes are erased before the program runs (Chapter 1), so the running program cannot even name them. Keep this in mind; it explains the Any rule below. (The exact text of type_name is not guaranteed and is meant for diagnostics only.)

Every "no" row is in code/compile_fail/ch07_static_table_no.rs :

```
pub fn caller() {
    let text = String::from("local text");
    is_static(&text.as_str()); // `&'a str` with a local `'a`
    is_static(&Line(&text)); // `Line<'a>` with a local `'a`
    is_static(&|| text.len()); // a closure that borrows `text`
}
```

The compiler rejects all three lines: the first two with the E0597 we just read, and the closure with E0373, which we will read in the next section, because it is the error you get from thread::spawn . The full diagnostics are in code/compile_fail/ch07_static_table_no.stderr .

7.4 Why APIs ask for 'static

When you call a function, the signature is the contract (Chapter 1). If the function only uses your value during the call, it can accept borrows of any lifetime: the borrow outlives the call, and that is all it needs. An API asks for `T: 'static` when it keeps your value for a span of time that it cannot describe with a lifetime parameter: a thread that may still run after your function has returned, a value whose type has been erased, a callback stored in a registry that lives as long as the program. In each case the callee has no shorter region to name, so it asks for the only one that is guaranteed to be long enough.

Threads: spawn and thread::scope

Here is the signature of `thread::spawn` in Rust 1.99, without its attributes and documentation:

```
pub fn spawn<F, T>(f: F) -> JoinHandle<T>
where
    F: FnOnce() -> T,
    F: Send + 'static,
    T: Send + 'static,
```

The closure `F` and its result `T` must both be `Send` (they cross to another thread) and `'static` (they may stay there for an unknown time). Why "unknown"? A `JoinHandle` can be dropped without joining; the thread then keeps running, detached, possibly after the function that spawned it has returned and its locals have been freed. The signature of `spawn` cannot know whether you will join, so it must assume you will not.

The program `code/compile_fail/ch07_spawn_borrow.rs` spawns a closure that uses a local vector:

```
pub fn caller() {
    let words = vec![String::from("alpha"), String::from("beta")];
    let handle = thread::spawn(|| words.len()); // borrows `words`
    println!("{}", handle.join().unwrap());
}
```

The compiler rejects it:

```
error[E0373]: closure may outlive the current function, but it borrows `words`, which is
owned by the current function
  --> compile_fail/ch07_spawn_borrow.rs:10:32
   |
10 |     let handle = thread::spawn(|| words.len()); // borrows `words`
   |                                ^^ ----- `words` is borrowed here
   |                                |
   |                                may outlive borrowed value `words`
   |
note: function requires argument type to outlive `'static`
  --> compile_fail/ch07_spawn_borrow.rs:10:18
   |
10 |     let handle = thread::spawn(|| words.len()); // borrows `words`
   |                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
help: to force the closure to take ownership of `words` (and any other referenced
variables), use the `move` keyword
   |
10 |     let handle = thread::spawn(move || words.len()); // borrows `words`
   |                                ++++
```

Error E0373 is reported for a closure (or an `async` block) that captures a local variable by reference and must outlive the function. The closure uses `words` only through a `&self` method (`len`), so it

captures `words` by reference, and its type holds a borrow of a local: it is not `'static`. The first label, "may outlive borrowed value `words`", says why that is a problem; the note "function requires argument type to outlive `'static`" says who demands it (`spawn`). The help is right this time: with `move`, the closure takes ownership of the vector, its type holds no borrow, and the program compiles. Notice also that the `join` on the next line changes nothing. The compiler checks the call to `spawn` against the signature of `spawn`, not against what you do afterward.

The help is not always right, because `move` only moves what the closure captures, and sometimes that is already a reference. The program `code/compile_fail/ch07_spawn_param.rs` receives the data by reference:

```
pub fn count_in_background(words: &[String]) -> usize {
    let handle = thread::spawn(move || words.len()); // moves the *reference* in
    handle.join().unwrap()
}
```

The closure is `move`, so it owns its capture, but its capture is `words`, a `&[String]`. Moving a reference moves the borrow, not the data. The compiler reports a different error:

```
error[E0521]: borrowed data escapes outside of function
 --> compile_fail/ch07_spawn_param.rs:9:18
  |
8 | pub fn count_in_background(words: &[String]) -> usize {
  |                            -----  - let's call the lifetime of this reference `'1`
  |                            |
  |                            `words` is a reference that is only valid in the function
body
9 |     let handle = thread::spawn(move || words.len()); // moves the *reference* in
  |                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |                  |
  |                  `words` escapes the function body here
  |                  argument requires that `'1` must outlive `'static`
```

E0521, "borrowed data escapes outside of function", appears when a borrow that came in through a parameter flows into something that must outlive the function. Read it label by label:

- "`words` is a reference that is only valid in the function body": from the function's point of view, the lifetime of `words` is chosen by the caller and covers at least the call, but nothing more is known about it.
- "let's call the lifetime of this reference `'1`": the parameter's lifetime is elided, so it has no name; rustc invents one (`'1`) to talk about it.
- "`words` escapes the function body here": the call to `spawn`, which wants to keep the closure, and therefore the borrow, for an unknown time.
- "argument requires that `'1` must outlive `'static`": the requirement that cannot be proved. The caller might pass a borrow of a local of its own.

There are two honest fixes. Either the thread owns the data (take a `Vec<String>` or an `Arc<[String]>` parameter, or clone inside the function), or the thread does not outlive the function, which is exactly what scoped threads guarantee. The program `code/examples/ch07_threads.rs` shows the owning fixes first:

```
// 1. Own: the closure owns a clone, so its type is `'static`.
let owned = words.clone();
let handle = thread::spawn(move || owned.join(" "));
println!("moved:  {}", handle.join().unwrap());
```

```
// 2. Share: `Arc<Vec<String>>` owns its data, so it is `'static` too.
let shared = Arc::new(words.clone());
let handles: Vec<_> = (0..shared.len())
    .map(|i| {
        let shared = Arc::clone(&shared);
        thread::spawn(move || shared[i].len())
    })
    .collect();
let lengths: Vec<usize> = handles.into_iter().map(|h| h.join().unwrap()).collect();
println!("shared: {lengths:?}");
```

The first closure owns a clone of the vector. The others each own an `Arc<Vec<String>>`, a shared handle that keeps the vector alive as long as any thread holds one: `Arc<T>` owns its contents, so it is `'static` when `T` is. Cloning an `Arc` copies a pointer and increments a counter; it does not copy the strings.

The third way borrows. `thread::scope` (stable since Rust 1.63) calls your closure with a `Scope` value, and every thread spawned through that `Scope` is joined before `thread::scope` returns. Since the threads cannot outlive the call, they may borrow anything that outlives the call:

```
// 3. Borrow: every thread spawned in the scope is joined before `scope`
//    returns, so the closures may borrow `words`.
let (total, longest) = thread::scope(|s| {
    let total = s.spawn(|| words.iter().map(|w| w.len()).sum::<usize>());
    let longest = s.spawn(|| words.iter().max_by_key(|w| w.len()).unwrap());
    (total.join().unwrap(), longest.join().unwrap())
});
println!("scoped: total {total}, longest {longest}"); // `longest: &String`
println!("still mine: {words:?}");
```

```
moved:  alpha beta gamma
shared: [5, 4, 5]
scoped: total 14, longest gamma
still mine: ["alpha", "beta", "gamma"]
```

The signature says the same thing in lifetimes. The bounds of `Scope::spawn` are `F: FnOnce() -> T + Send + 'scope` and `T: Send + 'scope`, where the lifetime `'scope` is the region of the call to `thread::scope`. The data the closures borrow lives in a longer region (the standard library calls it `'env`), so the borrows satisfy `'scope`. A scoped thread can even *return* a borrow: `longest` is a `&String` pointing into `words`, and it is still usable after the scope ends. And `words` is still owned by `main` at the end. Why `thread::scope` takes a closure instead of returning a guard that joins on drop is a story about leaks, told in Chapter 13; why async runtimes cannot easily offer the same thing is told in Chapter 11.

`Any` and `TypeId`

`std::any::Any` lets you erase the type of a value and recover it later by checking at run time. Its definition starts like this:

```
pub trait Any: 'static {
    fn type_id(&self) -> TypeId;
```

```
}

impl<T: 'static + ?Sized> Any for T { /* ... */ }
```

and `TypeId::of` is declared as `pub const fn of<T: ?Sized + 'static>() -> TypeId`. `'static` everywhere. The program `code/examples/ch07_any.rs` uses them to build a type map, a collection that holds at most one value per type, like the "extensions" that web frameworks attach to a request:

```
/// At most one value per type, like the "extensions" of an HTTP request.
#[derive(Default)]
struct Extensions {
    values: HashMap<TypeId, Box<dyn Any>>, // `Box<dyn Any + 'static>`
}

impl Extensions {
    fn insert<T: 'static>(&mut self, value: T) {
        self.values.insert(TypeId::of::<T>(), Box::new(value));
    }

    fn get<T: 'static>(&self) -> Option<&T> {
        self.values.get(&TypeId::of::<T>())?.downcast_ref::<T>()
    }
}
```

```
let mut extensions = Extensions::default();
extensions.insert(RequestId(42));
extensions.insert(UserName(String::from("ada")));
extensions.insert("a literal"); // `&'static str` is a `'static` type

let id: Option<&RequestId> = extensions.get::<RequestId>();
let user: Option<&UserName> = extensions.get::<UserName>();
println!("request id: {:?}", id.map(|id| id.0));
println!("user name:  {:?}", user.map(|user| user.0.as_str()));
println!("literal:    {:?}", extensions.get::<&'static str>());
println!("String:     {:?}", extensions.get::<String>()); // never inserted
```

```
request id: Some(42)
user name:  Some("ada")
literal:    Some("a literal")
String:     None
```

`insert` stores each value under the `TypeId` of its type; `get` looks up the `TypeId` of the requested type and asks `downcast_ref` to turn the `&dyn Any` back into a `&T`. `downcast_ref` succeeds only if the stored value's `TypeId` is the `TypeId` of `T`. Removing `T: 'static` from `insert` or `get` would be E0310, which we meet below.

Why must the types be `'static`? Because a `TypeId` is computed by the compiler after lifetimes have been erased, and the running program has no lifetimes to compare. `Token<'a>` and `Token<'static>` would get the same `TypeId`. If `Any` accepted borrowed types, downcasting could turn a short borrow into a `'static` one. In this pseudo-code, which does *not* compile, `forge` would hand out a reference to data that is about to be freed:

```
// Not real Rust: imagine `Any` without its `'static` bound.
fn forge<'a>(text: &'a str) -> &'static str {
    let erased: Box<dyn Any + 'a> = Box::new(text); // TypeId of `&str`...
    *erased.downcast::<&'static str>().unwrap() // ...equals TypeId of `&'static str`
}
```

The 'static bound on Any rules this out: only types whose lifetimes are all 'static can be erased, and for those, forgetting the lifetimes loses nothing.

So a type that borrows cannot become a dyn Any. Here is an attempt, in code/compile_fail/ch07_any_borrowed.rs:

```
/// A token that borrows its text from the source it was read from.
pub struct Token<'src> {
    pub text: &'src str,
}

pub fn erase<'src>(token: Token<'src>) -> Box<dyn Any> {
    Box::new(token)
}
```

```
error: lifetime may not live long enough
  --> compile_fail/ch07_any_borrowed.rs:14:5
   |
13 | pub fn erase<'src>(token: Token<'src>) -> Box<dyn Any> {
   |              ---- lifetime `'src` defined here
14 |     Box::new(token)
   |     ^^^^^^^^^^^^^^^ coercion requires that `'src` must outlive `'static`
   |
help: to declare that the trait object captures data from argument `token`, you can add an
explicit `'src` lifetime bound
   |
13 | pub fn erase<'src>(token: Token<'src>) -> Box<dyn Any + 'src> {
   |                                                       ++++++
```

This error has no code; its message is "lifetime may not live long enough". The label "coercion requires that 'src must outlive 'static" refers to the *unsizing coercion* from Box<Token<'src>> to Box<dyn Any> (Chapter 8), and the requirement comes from two places: Box<dyn Any> means Box<dyn Any + 'static>, and Any itself is only implemented for 'static types.

The help suggests writing Box<dyn Any + 'src>. That is a generic suggestion, made for every trait object, and here it does not fix anything. The program code/compile_fail/ch07_any_help_moves_error.rs follows it:

```
pub fn erase<'src>(token: Token<'src>) -> Box<dyn Any + 'src> {
    Box::new(token) // compiles: the signature now implies `'src: 'static`
}

pub fn caller() {
    let source = String::from("let x = 1;");
    let erased = erase(Token { text: &source[..3] });
    println!("{}", erased.is::<Token<'static>>());
}
```

Now erase compiles, but the caller does not:

```
error[E0597]: `source` does not live long enough
  --> compile_fail/ch07_any_help_moves_error.rs:19:39
   |
18 |     let source = String::from("let x = 1;");
   |         ------ binding `source` declared here
19 |     let erased = erase(Token { text: &source[..3] });
   |                  ----------------------^^^^^^--------
   |                  |                     |
   |                  |                     borrowed value does not live long enough
   |                  argument requires that `source` is borrowed for `'static`
20 |     println!("{}", erased.is::<Token<'static>>());
```

```
21 | }
   | - `source` dropped here while still borrowed
```

Because `Any: 'static`, the type `dyn Any + 'src` is only well-formed when `'src: 'static`. The compiler assumes that the types in a signature are well-formed, so the signature of `erase` now carries an *implied bound* `'src: 'static` (implied bounds were introduced in Chapter 1). `erase` compiles, but it accepts only tokens that borrow `'static` data, and the error reappears at every call site that has a real borrow. The fix is to erase only owned data: convert the token into an owned version first (Chapter 14 shows a borrowed token type with an owned twin), or do not use `Any` for borrowed data at all; an `enum` of the possible types, or a trait with the methods you need, works with any lifetime.

Stored callbacks and the hidden `'static` of `Box<dyn Trait>`

A registry of callbacks keeps closures for as long as the registry lives. Here is one from `code/examples/ch07_callbacks.rs`:

```
#[derive(Default)]
struct Registry {
    handlers: Vec<Box<dyn Fn(&str) -> String>>, // `+ 'static` is implied
}

impl Registry {
    fn register(&mut self, handler: Box<dyn Fn(&str) -> String>) {
        self.handlers.push(handler);
    }

    fn run(&self, line: &str) {
        for handler in &self.handlers {
            println!("{}", handler(line));
        }
    }
}
```

The field type is written `Box<dyn Fn(&str) -> String>`, with no lifetime. A trait object always has a lifetime bound, the `+ 'a` in `dyn Trait + 'a`, which says how long the hidden value may borrow. When you leave it out, the compiler picks a default, and inside a `Box` the default is `'static`. So the field is really `Box<dyn Fn(&str) -> String + 'static>`, and only closures that hold no expiring borrow fit. The program registers two that do:

```
let mut registry = Registry::default();
let prefix = String::from(">> ");
registry.register(Box::new(move |line| format!("{prefix}{line}"))); // owns `prefix`
registry.register(Box::new(|line| line.to_uppercase())); // captures nothing
registry.run("hello");
```

```
>> hello
HELLO
```

The first closure is `move`, so it owns `prefix`; the second captures nothing. Remove the `move`, and the closure borrows `prefix`. The program `code/compile_fail/ch07_callback_borrow.rs` does that:

```
pub struct Registry {
    handlers: Vec<Box<dyn Fn(&str) -> String>>,
}

impl Registry {
```

```
    pub fn register(&mut self, handler: Box<dyn Fn(&str) -> String>) {
        self.handlers.push(handler);
    }
}

pub fn caller(registry: &mut Registry) {
    let prefix = String::from(">> ");
    registry.register(Box::new(|line| format!("{prefix}{line}"))); // borrows `prefix`
}
```

```
error[E0597]: `prefix` does not live long enough
  --> compile_fail/ch07_callback_borrow.rs:18:49
   |
17 |     let prefix = String::from(">> ");
   |         ------ binding `prefix` declared here
18 |     registry.register(Box::new(|line| format!("{prefix}{line}"))); // borrows `prefix`
   |                       --------------------------^^^^^^----------
   |                       |        |                |
   |                       |        |                borrowed value does not live long
enough
   |                       |        value captured here
   |                       coercion requires that `prefix` is borrowed for `'static`
19 | }
   | - `prefix` dropped here while still borrowed
   |
   = note: due to object lifetime defaults, `Box<dyn for<'a> Fn(&'a str) -> String>`
actually means `Box<(dyn for<'a> Fn(&'a str) -> String + 'static)>`
```

"Value captured here" points at the closure, which captures `prefix` by reference. "Coercion requires that `prefix` is borrowed for `'static`" points at `Box::new(...)`, where the boxed closure becomes a `Box<dyn Fn ...>`. And the note spells out the hidden default: `Box<dyn for<'a> Fn(&'a str) -> String>` "actually means" `Box<(dyn for<'a> Fn(&'a str) -> String + 'static)>`. (The `for<'a>` is how the compiler writes the elided lifetime of the `&str` argument; Chapter 9 explains it.)

There are two fixes, and they are design decisions. Either the callbacks own what they use (`move`, as in the working program), which keeps the registry free of lifetime parameters and lets it live anywhere. Or the registry admits that its callbacks borrow: `struct Registry<'a>` with a field of type `Vec<Box<dyn Fn(&str) -> String + 'a>>`, which accepts borrowing closures but can then live only as long as what they borrow. Chapter 8 gives the complete rules for the default bounds of trait objects (they differ for `&'a dyn Trait`, for example), and Chapter 14 builds a registry of the second kind (a `Dispatcher<'a>`).

The same hidden `'static` is behind the trouble with `Box<dyn Error>`: an error type that borrows from its input cannot become a `Box<dyn Error>`. Chapter 8 works through that case.

Generic code: E0310

The `'static` requirements of `spawn`, `Any` and boxed trait objects do not stay inside those APIs. If your generic function passes its `T` to one of them, the requirement must appear in your signature too. The program `code/compile_fail/ch07_generic_spawn.rs` moves a value to another thread to drop it there, a trick sometimes used to keep an expensive destructor off a latency-sensitive thread:

```
pub fn drop_in_background<T: Send>(value: T) -> thread::JoinHandle<()> {
    thread::spawn(move || drop(value))
}
```

```
error[E0310]: the parameter type `T` may not live long enough
 --> compile_fail/ch07_generic_spawn.rs:9:5
  |
9 |     thread::spawn(move || drop(value))
  |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |     |
  |     the parameter type `T` must be valid for the static lifetime...
  |     ...so that the type `T` will meet its required lifetime bounds
  |
help: consider adding an explicit lifetime bound
  |
8 | pub fn drop_in_background<T: Send + 'static>(value: T) -> thread::JoinHandle<()> {
  |                                   +++++++++
```

E0310 is "the parameter type `T` may not live long enough". The first label says what `spawn` needs ("must be valid for the static lifetime"), the second why ("so that the type `T` will meet its required lifetime bounds": the closure owns a `T`, and the closure must be `'static`). Inside `drop_in_background`, `T` is unknown: the caller could choose `T = &'a String` with a short `'a`. The body is checked against the signature, and the signature promises nothing about `T`'s borrows. The help gives the fix, and this time it is the right one: add the bound to the signature, so that callers are checked against it.

`code/examples/ch07_generic_spawn.rs`:

```rust
fn drop_in_background<T: Send + 'static>(value: T) -> thread::JoinHandle<()> {
    thread::spawn(move || drop(value))
}
```

```rust
let table = Table((0..1000).map(|i| format!("row {i}")).collect());
let handle = drop_in_background(table); // `Table: Send + 'static`
handle.join().unwrap();
println!("joined");
```

```
dropping a table of 1000 rows
joined
```

The bound is part of the contract now. A caller with an owned `Table` is fine; a caller that tries to pass `&table` gets the E0597 we read earlier, at the call site, where the borrow is visible. This is how `'static` propagates through generic code: up the call chain, one signature at a time, until it reaches code that knows the concrete type.

7.5 Myths and anti-patterns

With the two meanings separated, most of the folklore around `'static` falls apart:

Myth or habit	Reality
"`T: 'static` means the value lives for the whole program."	It restricts the type. A `String` local to one function is `'static`, and it is dropped as usual (`ch07_static_type_dropped`).
"`&'static T` and `T: 'static` are the same thing."	The first is a reference to data that lives until the program ends; the second says that a type holds no borrow that can expire.
"`'static` data is in the executable."	Only literals and constant data are. Leaked buffers and `OnceLock` contents live on the heap.
"The compiler wants `'static`, so I'll write `&'static` on my parameter."	That forces every caller to supply a literal or leaked data. Find the API that asked for `'static` and give it owned data.
"Leak it; it's only a small `String`."	Leaking per call is an unbounded memory leak. Leak only data that lives until exit anyway.
"`struct Config { name: &'static str }` is flexible."	It accepts only literals and leaked strings. Use `String`, `Arc<str>`, `Cow<'static, str>` or a lifetime parameter (Chapter 14).

The fourth row deserves a program, because it is the most common wrong fix. Suppose you started with the E0521 above, in a function that starts a logger thread with a prefix of type `&str`. Writing `&'static str` makes the function compile. Here it is, with a caller, in `code/compile_fail/ch07_static_param.rs`:

```
fn start_logger(prefix: &'static str) -> thread::JoinHandle<()> {
    thread::spawn(move || println!("{prefix}: started"))
}

pub fn caller(id: u32) {
    let prefix = format!("worker-{id}");
    start_logger(&prefix).join().unwrap();
}
```

```
error[E0597]: `prefix` does not live long enough
  --> compile_fail/ch07_static_param.rs:14:18
   |
13 |     let prefix = format!("worker-{id}");
   |         ------ binding `prefix` declared here
14 |     start_logger(&prefix).join().unwrap();
   |     -------------^^^^^^^_
   |     |            |
   |     |            borrowed value does not live long enough
   |     argument requires that `prefix` is borrowed for `'static`
15 | }
   | - `prefix` dropped here while still borrowed
```

The error did not go away; it moved from the function to its caller, which builds the prefix at run time and cannot produce a `'static` borrow of it. Every caller with a `String` will meet the same error. The real fix is to ask for ownership, because the thread really does need to own its prefix. Taking `impl Into<String>` keeps the call sites short (`code/examples/ch07_logger.rs`):

```rust
fn start_logger(prefix: impl Into<String>) -> thread::JoinHandle<()> {
    let prefix: String = prefix.into(); // owned, so the closure is `'static`
    thread::spawn(move || println!("{prefix}: started"))
}
```

```rust
start_logger("main").join().unwrap(); // a literal
for id in 1..=2 {
    start_logger(format!("worker-{id}")).join().unwrap(); // a `String`
}
```

```
main: started
worker-1: started
worker-2: started
```

A literal is converted into a `String` (one small allocation), and a `String` is moved in without copying. The signature now says the truth: this function keeps its argument.

7.6 A decision ladder

When the compiler asks for `'static`, first find out who asks: the note "requirement ... introduced here" or "function requires argument type to outlive `'static`" names the API. Then climb this ladder from the top, and stop at the first rung that fits:

1. **Own it.** Give the API data it owns: move the value in, clone it, convert `&str` to `String`, or share ownership with `Arc` (or `Rc` on one thread). This is the answer most of the time, and it is cheap more often than you think: a clone of a configuration value at startup costs nothing that matters.
2. **Scope it.** If the API has a scoped version, use it: `thread::scope` instead of `thread::spawn`, a `Registry<'a>` with `dyn Fn() + 'a` instead of a `'static` one, a function that takes a callback and calls it before returning instead of storing it. Scoped APIs accept borrows because they promise to be done with them before the borrowed data goes away.
3. **Mix.** If most values are literals and a few are built at run time, `Cow<'static, str>` holds either a `&'static str` or a `String`, and is `'static` in both cases (Chapter 14).
4. **Leak or make it global.** `Box::leak`, `String::leak`, `OnceLock` and `LazyLock`, only for data that genuinely lives until the program exits: configuration, interned names, lookup tables.

What is not on the ladder: writing `&'static` in a signature because an error mentioned `'static`, and `unsafe` code that "extends" a lifetime. The first moves the error to the callers; the second turns a compile error into undefined behavior.

7.7 Common confusions

"The error says `local` must be borrowed for `'static`, so `local` should be a `static`." The error says that a borrow of `local` reached an API that needs `'static`. Making the data global is rung 4 of the ladder; usually rung 1 (give the API an owned value) is the fix.

"`move` makes a closure `'static`." `move` makes the closure own its captures. If a capture is itself a reference, the closure owns a reference and still holds a borrow (E0521 in `ch07_spawn_param`).

"**`T: 'static` and `Send` are the same requirement.**" They are independent. `Rc<String>` is `'static` but not `Send`; `&'a i32` is `Send` but not `'static` unless `'a` is `'static`. `thread::spawn` asks for both.

"**A `&'static` reference is always immutable.**" `Box::leak` and `String::leak` return `&'static mut` references; what `'static` promises is validity, not immutability.

"**`T: 'static` excludes references.**" It excludes references that can expire. `&'static str` satisfies it, which is why the type map above accepts a literal.

"**Following the compiler's help always fixes the error.**" The help for a `'static` requirement is often right (add `move`, add `T: 'static`), but for `dyn Any` it only moves the error to the callers, and adding `&'static` to a parameter does the same. Check the fix from the caller's side too.

7.8 Summary

- `'static` is the largest region: it covers the whole run of the program and outlives every other lifetime. It appears in two positions with different meanings.
- `&'static T` is a reference whose referent stays valid until the program ends. It comes from literals, `static` and `const` items, constant promotion (constant values with no destructor and no interior mutability), leaking (`Box::leak`, `String::leak`), and globals initialized at run time (`OnceLock`, `LazyLock`).
- `T: 'static` is a bound on a type: every borrow inside `T` is valid until the program ends. Owning types satisfy it, whatever the lifetime of the value; a `'static` value can be dropped a line after it is created. The test is "does the type hold a borrow that can expire?".
- APIs ask for `T: 'static` when they keep a value for a time they cannot name: `thread::spawn` (E0373 for a closure that borrows a local, E0521 for a borrow that came in as a parameter), `Any` and `TypeId` (lifetimes are erased, so downcasting could forge them), stored trait objects (`Box<dyn Trait>` means `Box<dyn Trait + 'static>`), and generic code that calls any of these (E0310: write `T: 'static` on your parameter).
- `thread::scope` accepts borrows because every thread it spawns is joined before it returns.
- When the compiler asks for `'static`, find the API that asks, then own, scope, mix or, for process-lifetime data only, leak. Writing `&'static` on a parameter moves the error to the callers.

7.9 Exercises

1. Which of these calls compile, given `fn requires_static<T: 'static>(_value: T) {}`? Check your answers by adding them to a copy of `ch07_static_type_dropped.rs`.

```
requires_static(&42);
requires_static(&[1, 2, 3]);
let n = 42;
requires_static(&n);
requires_static(&String::new());
```

Hint: two of them are references to promoted constants. A local variable is never promoted, and neither is a value with a destructor (the last call is E0716, "temporary value dropped while borrowed", which Chapter 5 explains).

2. Rewrite `count_in_background` from `ch07_spawn_param.rs` so that it compiles without changing its signature and without cloning the strings. *Hint:* the thread only needs to run while the function runs; `thread::scope` expresses exactly that, and its result can be returned directly.
3. Change the `Registry` of `ch07_callbacks.rs` so that it accepts closures that borrow local variables, then register a closure that borrows `prefix` without `move`. What can a `Registry<'a>` no longer do that the original could? *Hint:* the field becomes `Vec<Box<dyn Fn(&str) -> String + 'a>>`. Try to return the registry from the function that owns `prefix`, or to store it in a `static`.
4. Write `fn spawn_all<T: Send>(items: Vec<T>) -> Vec<thread::JoinHandle<()>>`, which spawns one thread per item and drops the item there. Fix the error, then decide whether `spawn_all(vec!["a", "b"])` and `spawn_all(vec![name.as_str()])` (with a local `name: String`) compile. *Hint:* E0310 again. After the fix, `Vec<&'static str>` satisfies the bound; a vector of borrows of a local does not.
5. Write an interner `fn intern(name: &str) -> &'static str` that leaks each distinct name at most once, so that the memory it uses is bounded by the number of distinct names, not by the number of calls. *Hint:* keep the leaked names in a `static NAMES: OnceLock<Mutex<HashSet<&'static str>>>`; look the name up first, and leak (`name.to_owned().leak()`) only when it is missing.

CHAPTER 8

Trait Objects, `impl Trait` and Captured Lifetimes

Rust has two ways to say "some type that implements this trait" without naming the type. A trait object, `dyn Trait` behind a pointer, erases the type at run time: a `Box<dyn Shape>` may hold a circle or a square, and calls go through a table of function pointers. An `impl Trait` in return position hides the type from the caller but not from the compiler: `fn chars_of(s: &str) -> impl Iterator<Item = char>` returns one specific type, which the caller may use only through `Iterator`.

Both hide a type, and a hidden type may contain borrows. The caller cannot look inside to see what the value borrows, yet the borrow checker must know how long the value may be used. So both kinds of type carry lifetime information that you usually do not write: trait objects have a *lifetime bound* with default rules, and opaque `impl Trait` types *capture* lifetimes by rules that changed in edition 2024. Most of the time the defaults are right. When they are not, the messages are some of the most puzzling the compiler produces: "returning this value requires that `'1` must outlive `'static`" for code that never mentions `'static`, or a borrow that lives too long because "Rust 2024 has adjusted the `impl Trait` lifetime capture rules". This chapter writes out every default.

In this chapter you will learn:

- what the lifetime bound of a trait object means, and the complete rules for its default, as rustc 1.99 applies them (the Reference describes them inaccurately);
- why `Box<dyn Error>` and `?` reject errors that borrow their input, and the two ways out;
- where the bound is checked (unsizing coercions) and how to store borrowing trait objects in structs;
- what a return-position `impl Trait` captures in edition 2024, the over-capturing problem, and precise capturing with `use<..>`;
- why `impl Trait + 'a` is not the same as `use<'a>`;
- the edition 2021 rules and E0700, for the code you will still read;
- how `impl Trait` in traits, `async fn` and `impl Trait` in argument position capture lifetimes.

8.1 The lifetime bound of a trait object

What `dyn Trait + 'a` says

A value of type `dyn Shape` is a value of some concrete type that implements `Shape`, with the concrete type forgotten. Once it is forgotten, the compiler can no longer see whether the value contains references, or how long they are valid. The *lifetime bound* of the trait object, the `'a` in `dyn Shape + 'a`, is the part of the type that keeps that information: `dyn Shape + 'a` stands for "some type `T` with `T: Shape` and `T: 'a`". Recall from Chapter 1 that `T: 'a` means every lifetime inside `T` outlives `'a`. A `dyn Shape + 'static` contains no borrow that can expire; a `dyn Shape + 'a` may contain borrows, all of them valid for at least `'a`, and so it may be used only within `'a`.

Every trait object type has such a bound. A longer bound can stand in for a shorter one (`dyn Shape + 'static` can be used where `dyn Shape + 'a` is expected; Chapter 4 lists `dyn Trait + 'a` as covariant in `'a`), but never the other way around. When you write `dyn Shape` with no bound, the compiler picks one: the *default object lifetime bound*. Like elision, the default is a rule about how the compiler reads a type; it never looks at how the value is used.

The defaults at work

The program `code/examples/ch08_object_defaults.rs` has a trait and two shapes, one that borrows its data and one that owns it:

```
trait Shape {
    fn area(&self) -> f64;
}

/// A circle that borrows its radius, from a configuration for example.
struct Circle<'a> {
    radius: &'a f64,
}

impl Shape for Circle<'_> {
    fn area(&self) -> f64 {
        std::f64::consts::PI * self.radius * self.radius
    }
}

/// A square that owns its data.
struct Square(f64);

impl Shape for Square {
    fn area(&self) -> f64 {
        self.0 * self.0
    }
}
```

Boxing a circle needs a bound in the return type, and boxing a square does not:

```
// The default would be `Box<dyn Shape + 'static>`: the bound must be written.
fn boxed_circle<'a>(radius: &'a f64) -> Box<dyn Shape + 'a> {
    Box::new(Circle { radius })
}

// A square borrows nothing, so the `'static` default is the right contract.
fn boxed_square(side: f64) -> Box<dyn Shape> {
    Box::new(Square(side))
}
```

`Box<dyn Shape>` means `Box<dyn Shape + 'static>`. That is the right contract for `boxed_square`, whose result owns everything, and the wrong one for `boxed_circle`, whose result borrows `radius`. Without `+ 'a`, the compiler reports "returning this value requires that `'a` must outlive `'static`", the error Chapter 6 read label by label for a boxed closure. References are different:

```
// The defaults are already right here: `&'x [&'y (dyn Shape + 'y)]`.
fn total(shapes: &[&dyn Shape]) -> f64 {
    shapes.iter().map(|shape| shape.area()).sum()
}
```

The parameter `&[&dyn Shape]` means `&'x [&'y (dyn Shape + 'y)]`: the default for a trait object behind a reference is the lifetime of that reference. That is almost always what you want, since the object cannot be used after the reference expires anyway. So `total` accepts the borrowing circle, a box's contents and a square alike:

```
fn main() {
    let radius = 2.0;
    let circle = boxed_circle(&radius);
    let square = boxed_square(3.0);
    let local = Circle { radius: &radius };
    println!("{:.2} {:.2}", circle.area(), square.area());
    println!("{:.2}", total(&[&local, &*circle, &*square]));
}
```

```
12.57 9.00
34.13
```

Here are the defaults for the types you will meet most often:

You write	It means	Why
`Box<dyn Shape>`	`Box<dyn Shape + 'static>`	`Box<T>` gives no bound
`Rc<dyn Shape>`, `Arc<dyn Shape>`	`... + 'static`	the same
`&'a dyn Shape`	`&'a (dyn Shape + 'a)`	`&'a T` requires `T: 'a`
`&'a mut dyn Shape`	`&'a mut (dyn Shape + 'a)`	the same
`Ref<'a, dyn Shape>`	`Ref<'a, dyn Shape + 'a>`	`Ref` declares `T: 'a`
`&'a Box<dyn Shape>`	`&'a Box<dyn Shape + 'static>`	the innermost type decides
`dyn Shape` in a function body	inferred	from the uses

The program `code/examples/ch08_default_table.rs` checks every row. Each function converts a `&mut` to the elided form into a `&mut` to the explicit form. Because `&mut T` is invariant in `T` (Chapter 4), and each `T` here is a sized type (a reference, a `Box`, a `Ref`) that no coercion can change behind the `&mut`, the function compiles only if the two types are identical, that is, only if the row is right:

```
// No lifetime on the container: `'static`.
fn in_box<'s>(x: &'s mut Box<dyn Shape>) -> &'s mut Box<dyn Shape + 'static> {
    x
}
fn in_rc<'s>(x: &'s mut Rc<dyn Shape>) -> &'s mut Rc<dyn Shape + 'static> {
    x
}
fn in_arc<'s>(x: &'s mut Arc<dyn Shape>) -> &'s mut Arc<dyn Shape + 'static> {
```

```
    x
}

// A reference or a `Ref` to the object: the lifetime of the borrow.
fn in_ref<'s, 'a>(x: &'s mut &'a dyn Shape) -> &'s mut &'a (dyn Shape + 'a) {
    x
}
fn in_mut<'s, 'a>(x: &'s mut &'a mut dyn Shape) -> &'s mut &'a mut (dyn Shape + 'a) {
    x
}
fn in_cell_ref<'s, 'a>(
    x: &'s mut Ref<'a, dyn Shape>,
) -> &'s mut Ref<'a, dyn Shape + 'a> {
    x
}

// The innermost container decides: a `Box` behind a reference is still `'static`.
fn box_behind_ref<'s, 'a>(
    x: &'s mut &'a Box<dyn Shape>,
) -> &'s mut &'a Box<dyn Shape + 'static> {
    x
}
```

The last row is the one that surprises people. In `&'a Box<dyn Shape>`, the trait object is an argument of `Box`, not of `&'a`, and `Box` gives no bound, so the object is `'static` even though it sits behind a short reference. The same program checks the body row, a `Box<dyn Display>` that borrows a local variable:

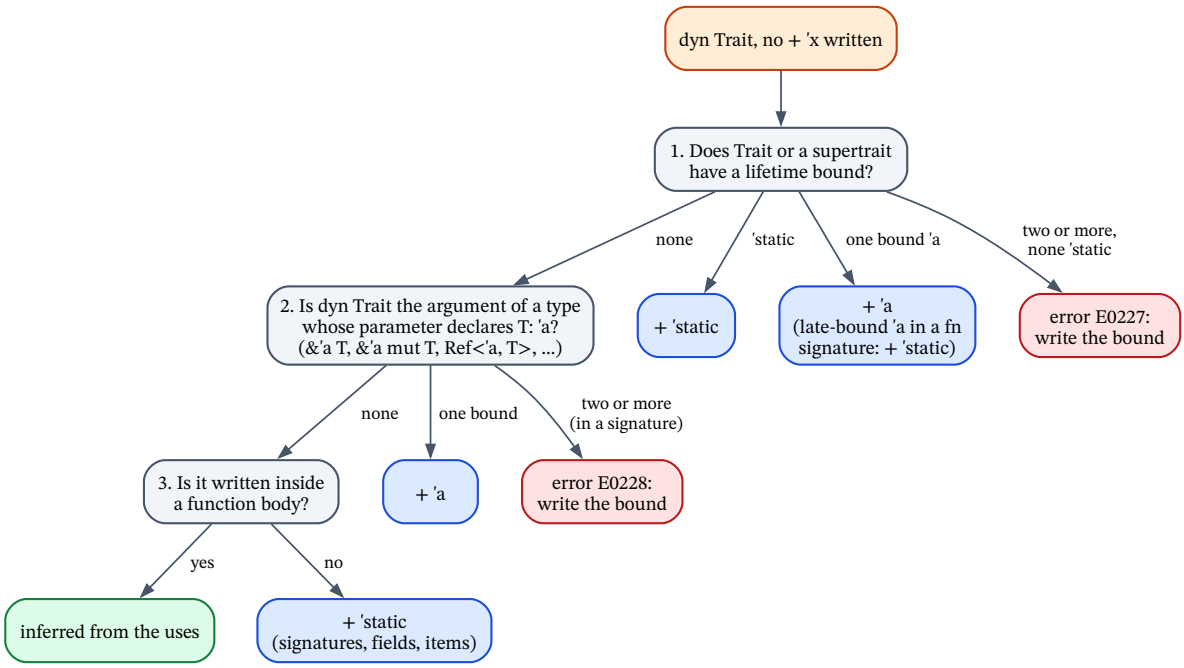
```
// In a function body an elided bound is inferred: this box may borrow `name`.
let name = String::from("local");
let shown: Box<dyn Display> = Box::new(&name);
println!("{shown}");
```

Inside a function body there are no signatures to agree on, and an elided bound becomes one more lifetime for the borrow checker to infer.

The rules

Behind the table are three rules, which rustc 1.99 applies in this order:

1. **The trait's own bounds.** If the trait, or one of its supertraits, declares a lifetime bound on `Self` (`trait Plugin: 'static`, `trait Source<'a>: 'a`), that bound is the default: `'static` if it is among them, otherwise the bound if there is exactly one, and error E0227 if there are several.
2. **The containing type.** Otherwise, if the trait object is the argument of a type whose parameter declares a lifetime bound, that bound is the default: `'a` for `&'a T` and `&'a mut T`, `'b` for `Ref<'b, T: ?Sized + 'b>`, and the same for your own types. Two different bounds give error E0228.
3. **Otherwise**, the bound is inferred inside a function body, and it is `'static` everywhere else: in signatures, struct fields, type aliases and `impl` headers (Chapter 6 showed `impl dyn Shape`).



The default lifetime bound of a trait object, as rustc 1.99 chooses it: the trait's bounds first, then the containing type, then inference in bodies and 'static elsewhere.

> **NOTE**
>
> The Rust Reference describes these defaults in its page on lifetime elision (https://doc.rust-lang.org/reference/lifetime-elision.html#default-trait-object-lifetimes), but lists the rules in the opposite order, containing type first. That section is known to be inaccurate: issue rust-lang/reference#1407 (https://github.com/rust-lang/reference/issues/1407) collects the differences, and the open pull request rust-lang/reference#2281 reorders the rules to match rustc, which "looks at the outlives-bounds of the principal trait (and its supertraits) *first*". This chapter describes what rustc 1.99 does, checked by the programs shown. Rust 1.98 also fixed the defaults inside trait references and associated-type paths (rust-lang/rust#129543), a rare case that this chapter does not cover.

Traits with a lifetime bound

Rule 1 is easy to miss because few traits have a lifetime bound. Here is one that does, from `code/examples/ch08_trait_bounds.rs`: plugins are kept in global registries, so the trait requires `'static`.

```
/// Plugins are stored in global registries, so the trait requires `'static`.
trait Plugin: 'static {
    fn name(&self) -> String;
}

struct Upper;

impl Plugin for Upper {
    fn name(&self) -> String {
        "upper".to_string()
    }
}
```

```
// `&dyn Plugin` is `&'_ (dyn Plugin + 'static)`: the trait's bound, not the
// reference's lifetime, is the default. The reference itself may be short.
fn describe(plugin: &dyn Plugin) -> String {
    format!("plugin {}", plugin.name())
}
```

The parameter `&dyn Plugin` means `&'_ (dyn Plugin + 'static)`, by rule 1, and not `&'a (dyn Plugin + 'a)` as rule 2 would have it. A bound inherited from a supertrait counts too:

```
// A bound inherited from a supertrait counts too: `&dyn Loud` is
// `&'_ (dyn Loud + 'static)`.
trait Loud: Plugin {
    fn shout(&self) -> String {
        self.name().to_uppercase()
    }
}

impl Loud for Upper {}

fn announce(plugin: &dyn Loud) -> String {
    plugin.shout()
}
```

`main` passes a short borrow of a local variable to both functions:

```
fn main() {
    let local = Upper; // a local variable: the borrow below is short
    println!("{} / {}", describe(&local), announce(&local));

    let config = String::from("port = 8080");
    println!("{}", boxed_source(&config).text());
}
```

```
plugin upper / UPPER
port = 8080
```

The order of the rules matters here, and the reason it is the right order shows when you write rule 2's answer by hand (`code/compile_fail/ch08_trait_bounds_explicit.rs`):

```
pub fn describe<'a>(plugin: &'a (dyn Plugin + 'a)) -> String {
    format!("plugin {}", plugin.name())
}

pub fn caller() -> String {
    let local = Upper;
    describe(&local)
}
```

```
error[E0597]: `local` does not live long enough
  --> compile_fail/ch08_trait_bounds_explicit.rs:25:14
   |
24 |     let local = Upper;
   |         ----- binding `local` declared here
25 |     describe(&local)
   |     ---------^^^^^^-
   |     |        |
   |     |        borrowed value does not live long enough
   |     argument requires that `local` is borrowed for `'static`
26 | }
   | - `local` dropped here while still borrowed
```

"Argument requires that `local` is borrowed for `'static`": a reference to a local variable is no longer accepted. Because `Plugin: 'static`, the type `dyn Plugin + 'a` is well-formed only if `'a` outlives `'static`, so the signature of `describe` carries the implied bound `'a: 'static` (Chapter 7 met the same effect with `dyn Any + 'src`), and every caller must provide a `'static` borrow. With the trait's bound as the default, the object is `'static` and the reference can be as short as you like.

For a trait with a lifetime parameter, such as `trait Source<'a>: 'a`, rule 1 gives `'a`, and in a type alias that is what you get:

```
/// A source of text that lives at least as long as `'a`.
trait Source<'a>: 'a {
    fn text(&self) -> &'a str;
}

impl<'a> Source<'a> for &'a str {
    fn text(&self) -> &'a str {
        self
    }
}

// In a type alias the trait's bound applies: this is `Box<dyn Source<'a> + 'a>`.
type BoxedSource<'a> = Box<dyn Source<'a>>;

fn boxed_source<'a>(text: &'a str) -> BoxedSource<'a> {
    Box::new(text)
}
```

The same type written directly in a function signature is a different story (`code/compile_fail/ch08_late_bound_default.rs`):

```
pub fn boxed_source<'a>(text: &'a str) -> Box<dyn Source<'a>> {
    Box::new(text)
}
```

```
error: lifetime may not live long enough
  --> compile_fail/ch08_late_bound_default.rs:18:5
   |
17 | pub fn boxed_source<'a>(text: &'a str) -> Box<dyn Source<'a>> {
   |                       -- lifetime `'a` defined here
18 |     Box::new(text)
   |     ^^^^^^^^^^^^^^ returning this value requires that `'a` must outlive `'static`
   |
help: to declare that the trait object captures data from argument `text`, you can add an
explicit `'a` lifetime bound
   |
17 | pub fn boxed_source<'a>(text: &'a str) -> Box<dyn Source<'a> + 'a> {
   |                                                              ++++
```

The Reference says that `Box<dyn Source<'a>>` means `Box<dyn Source<'a> + 'a>`, and rustc 1.99 reads it as `Box<dyn Source<'a> + 'static>`. The difference is that the `'a` of `boxed_source` is a *late-bound* lifetime, a technical distinction that Chapter 9 explains; rule 1 applies only when the trait's lifetime argument is an ordinary, *early-bound* parameter, as in the type alias. Issue rust-lang/reference#1407 lists this case. The practical advice is simple: when you rely on a bound other than `'static`, write it, as the help suggests.

When there is no unique default

Rules 1 and 2 can each find more than one bound, and then you must write it. A struct whose type parameter must outlive two lifetimes (`code/compile_fail/ch08_two_bounds.rs`):

```
/// Pairs a shape with the labels of two different documents.
pub struct Labeled<'a, 'b, T: ?Sized + 'a + 'b> {
    pub title: &'a str,
    pub note: &'b str,
    pub shape: Box<T>,
}

pub fn area_of(item: &Labeled<'_, '_, dyn Shape>) -> f64 {
    item.shape.area()
}
```

```
error[E0228]: cannot deduce the lifetime bound for this trait object type from context
  --> compile_fail/ch08_two_bounds.rs:17:39
   |
17 | pub fn area_of(item: &Labeled<'_, '_, dyn Shape>) -> f64 {
   |                                       ^^^^^^^^^
   |
help: please supply an explicit bound
   |
17 | pub fn area_of(item: &Labeled<'_, '_, dyn Shape + /* 'a */>) -> f64 {
   |                                                 ++++++++++
```

E0228, "cannot deduce the lifetime bound for this trait object type from context". The help leaves a placeholder, `dyn Shape + /* 'a */` , for you to fill in. A trait with two lifetime bounds, neither of them `'static` , is E0227 (`code/compile_fail/ch08_two_trait_bounds.rs`):

```
/// A table whose keys live for `'k` and whose values live for `'v`.
pub trait Lookup<'k, 'v>: 'k + 'v {
    fn get(&self, key: &str) -> Option<&'v str>;
}

pub type BoxedLookup<'k, 'v> = Box<dyn Lookup<'k, 'v>>;
```

```
error[E0227]: ambiguous lifetime bound, explicit lifetime bound required
  --> compile_fail/ch08_two_trait_bounds.rs:11:36
   |
11 | pub type BoxedLookup<'k, 'v> = Box<dyn Lookup<'k, 'v>>;
   |                                    ^^^^^^^^^^^^^^^^^^
```

8.2 `Box<dyn Error>` and `?`

`Box<dyn Error>` is the usual "any error" type of applications and examples, and `?` converts errors into it. The conversion is the standard library's `impl<'a, E: Error + 'a> From<E> for Box<dyn Error + 'a>` : any error type can become a boxed trait object whose bound it satisfies. With the default bound, `Box<dyn Error>` is `Box<dyn Error + 'static>` , and an error type that borrows its input does not satisfy it (`code/compile_fail/ch08_error_box_static.rs`):

```
pub fn parse_port(s: &str) -> Result<u16, Box<dyn Error>> {
    let port: u16 = s.trim().parse().map_err(|_| BadNumber { input: s })?;
    Ok(port)
}
```

```
error: lifetime may not live long enough
  --> compile_fail/ch08_error_box_static.rs:23:21
   |
22 | pub fn parse_port(s: &str) -> Result<u16, Box<dyn Error>> {
   |                       - let's call the lifetime of this reference `'1`
23 |     let port: u16 = s.trim().parse().map_err(|_| BadNumber { input: s })?;
   |                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ returning
this value requires that `'1` must outlive `'static`
   |
help: to declare that the trait object captures data from argument `s`, you can add an
explicit `'_` lifetime bound
   |
22 | pub fn parse_port(s: &str) -> Result<u16, Box<dyn Error + '_>> {
   |                                                         ++++
```

The label "returning this value" points at the whole `...?` expression, because on an error `?` returns early, converting `BadNumber { input: s }` into the function's error type. A `BadNumber<'1>` in a `Box<dyn Error + 'static>` requires `'1` to outlive `'static`. There are two ways out, and they are a design choice.

The first is the help's: let the box borrow, with `+ '_`. It is in `code/examples/ch08_error_box.rs`:

```rust
/// An error that points into the input: cheap, but tied to the input's lifetime.
#[derive(Debug)]
struct BadNumber<'a> {
    input: &'a str,
}

impl fmt::Display for BadNumber<'_> {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "not a number: {:?}", self.input)
    }
}

impl Error for BadNumber<'_> {}

// `?` converts `BadNumber<'_>` into `Box<dyn Error + '_>`: the box may borrow `s`.
fn parse_port(s: &str) -> Result<u16, Box<dyn Error + '_>> {
    let port: u16 = s.trim().parse().map_err(|_| BadNumber { input: s })?;
    Ok(port)
}
```

This is cheap (nothing is copied) and precise (the error points into the input), but the error now lives no longer than the input. A function that owns the input cannot pass the error on to its caller (`code/compile_fail/ch08_error_escapes.rs`):

```rust
pub fn load_port() -> Result<u16, Box<dyn Error>> {
    let line = String::from(" 80x ");
    let port = parse_port(&line)?;
    Ok(port)
}
```

```
error[E0515]: cannot return value referencing local variable `line`
  --> compile_fail/ch08_error_escapes.rs:29:16
   |
29 |     let port = parse_port(&line)?;
   |                ^^^^^^^^^^^^^^^^^^
   |                |          |
   |                |          `line` is borrowed here
   |                returns a value referencing data owned by the current function
```

E0515, "cannot return value referencing local variable `line`": the error returned by `?` borrows `line`, which is dropped when `load_port` returns. The second way out avoids this: the error owns a copy of the input, satisfies `'static`, and fits the default `Box<dyn Error>`:

```
/// The same error, owning a copy of the input: it satisfies `'static`.
#[derive(Debug)]
struct BadPort {
    input: String,
}

impl fmt::Display for BadPort {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "not a port: {:?}", self.input)
    }
}

impl Error for BadPort {}

fn parse_port_owned(s: &str) -> Result<u16, Box<dyn Error>> {
    let port: u16 = s.trim().parse().map_err(|_| BadPort { input: s.into() })?;
    Ok(port)
}

// The owned error can leave the function that owns the text.
fn load_port() -> Result<u16, Box<dyn Error>> {
    let line = String::from(" 80x ");
    let port = parse_port_owned(&line)?;
    Ok(port)
}
```

```
fn main() {
    let lines = [String::from("8080"), String::from("80x")];
    for line in &lines {
        match parse_port(line) {
            Ok(port) => println!("port {port}"),
            Err(e) => println!("error: {e}"),
        }
    }
    if let Err(e) = load_port() {
        println!("error: {e}");
    }
}
```

```
port 8080
error: not a number: "80x"
error: not a port: " 80x "
```

Borrowed errors suit code that parses and reports on the spot. Errors that travel up a call stack, cross threads or end up in a log should own their data; copying a short input on the error path costs little. Chapter 14 returns to this choice for library APIs.

8.3 Unsizing coercions: where the bound is checked

A trait object is never created directly. It is made by an *unsizing coercion*, which turns a pointer to a concrete type into a pointer to a trait object: `&Circle<'_>` into `&dyn Shape`, `Box<Circle<'_>>` into `Box<dyn Shape>`, and the same for `&mut`, `Rc` and `Arc`. The compiler inserts the coercion where the expected type is known: an annotated `let`, a function argument, a return value, a field in a struct literal, an element of an array. The coercion from `T` to `dyn Trait + 'x` requires `T: Trait` and

`T: 'x`, and this is where the object bound is checked. When the check fails, the borrow checker's label says "coercion requires that ... is borrowed for `'static`", as in Chapter 7's registry. From `code/examples/ch08_coercions.rs`:

```
let (r1, r2) = (1.0, 2.0);
let small = Circle { radius: &r1 };
let big = Circle { radius: &r2 };
// `&Circle<'_>` to `&dyn Shape`: the array's type asks for the coercion.
let shapes: [&dyn Shape; 2] = [&small, &big];
println!("largest: {:.2}", largest(&shapes).area());

// `Box<Circle<'_>>` to `Box<dyn Shape + '_>`: the bound is inferred here.
let boxed: Box<dyn Shape + '_> = Box::new(Circle { radius: &r2 });
println!("boxed: {:.2}", boxed.area());
```

```
fn largest<'a>(shapes: &[&'a dyn Shape]) -> &'a dyn Shape {
    let mut best = shapes[0];
    for &shape in shapes {
        if shape.area() > best.area() {
            best = shape;
        }
    }
    best
}
```

In the array, each `&Circle<'_>` becomes a `&dyn Shape`, whose bound is the lifetime of the reference. In `largest`, `&'a dyn Shape` is `&'a (dyn Shape + 'a)`, so the result can be returned with the lifetime of the input references. In the `let` with `Box<dyn Shape + '_>`, the `'_` is in a function body, so the bound is inferred.

A trait object with a long bound can be used where a shorter bound is expected, in a box and even behind `&mut`:

```
// A `'static` object is accepted where a shorter bound is expected...
fn shorten<'a>(shape: Box<dyn Shape>) -> Box<dyn Shape + 'a> {
    shape
}

// ...even behind `&mut`: the coercion builds a new pointer to the same object.
fn shorten_mut<'a>(shape: &'a mut (dyn Shape + 'static)) -> &'a mut (dyn Shape + 'a) {
    shape
}
```

```
largest: 12.57
boxed: 12.57
2.25
9.00
```

`shorten_mut` is allowed although `&mut T` is invariant in `T`, because it is not subtyping: it is a coercion that builds a new pointer, a `&'a mut (dyn Shape + 'a)`, to the same object. Through that pointer you can call methods on the object, but you cannot replace it with a different value, because a `dyn Shape` has no size known at compile time and cannot be assigned. The same is not true one level down (`code/compile_fail/ch08_mut_box_invariant.rs`):

```
pub fn shorten_slot<'a>(slot: &'a mut Box<dyn Shape>) -> &'a mut Box<dyn Shape + 'a> {
    slot
}
```

```
error: lifetime may not live long enough
  --> compile_fail/ch08_mut_box_invariant.rs:12:5
   |
11 | pub fn shorten_slot<'a>(slot: &'a mut Box<dyn Shape>) -> &'a mut Box<dyn Shape + 'a> {
   |                         -- lifetime `'a` defined here
12 |     slot
   |     ^^^^ returning this value requires that `'a` must outlive `'static`
   |
   = note: requirement occurs because of a mutable reference to `Box<dyn Shape>`
   = note: mutable references are invariant over their type parameter
   = help: see <https://doc.rust-lang.org/nomicon/subtyping.html> for more information about
variance
```

Through a `&mut Box<dyn Shape + 'a>` you could store a new box holding a circle that borrows a short-lived radius, and the original owner would later find it in a slot that promises `'static`. The notes give the reason in general terms: "mutable references are invariant over their type parameter" (Chapter 4).

8.4 Trait objects in structs

A struct that stores trait objects decides, in its field types, how long they may borrow. Chapter 7's `Registry` stored `Box<dyn Fn(&str) -> String>`, which means `+ 'static`, so every callback had to own its data. Writing the bound as a lifetime parameter of the struct admits borrowing callbacks. The filter in `code/examples/ch08_struct_fields.rs` keeps rules that may borrow from the code that builds it:

```
/// A word filter whose rules may borrow from the code that builds it.
struct Filter<'a> {
    rules: Vec<Box<dyn Fn(&str) -> bool + 'a>>,
}

impl<'a> Filter<'a> {
    fn new() -> Self {
        Filter { rules: Vec::new() }
    }

    fn rule(mut self, rule: impl Fn(&str) -> bool + 'a) -> Self {
        self.rules.push(Box::new(rule));
        self
    }

    fn accepts(&self, word: &str) -> bool {
        self.rules.iter().all(|rule| rule(word))
    }
}
```

The `+ 'a` appears twice: in the field, where it is the bound of the stored objects, and on the parameter of `rule`, where `impl Fn(&str) -> bool + 'a` requires the closure's type to outlive `'a` so that the coercion to `Box<dyn Fn(&str) -> bool + 'a>` succeeds. Both rules in `main` borrow local variables:

```
let stop_words = vec!["the".to_string(), "a".to_string()];
let min_len = 3;
let filter = Filter::new()
    .rule(|w| !stop_words.iter().any(|s| s == w)) // borrows `stop_words`
    .rule(|w| w.len() >= min_len); // borrows `min_len`
let text = "the parser reads a token stream";
let kept: Vec<&str> = text.split(' ').filter(|w| filter.accepts(w)).collect();
println!("{kept:?}");
```

```
["parser", "reads", "token", "stream"]
```

The price is the usual price of a lifetime parameter: a `Filter<'a>` cannot outlive `stop_words` and `min_len`, so it cannot be returned from the function that owns them or stored in a `static`. Choose between the two designs as Chapter 7 chose for the registry; Chapter 14 builds a larger example.

Generic containers and rule 2

The second half of the example is a generic struct that holds a reference to any value, including an unsized one:

```
trait Shape {
    fn area(&self) -> f64;
}

struct Circle<'a> {
    radius: &'a f64,
}

impl Shape for Circle<'_> {
    fn area(&self) -> f64 {
        std::f64::consts::PI * self.radius * self.radius
    }
}

/// A labeled reference. `T: ?Sized + 'a` is declared, so in
/// `Labeled<'a, dyn Shape>` the object's default bound is `'a`.
struct Labeled<'a, T: ?Sized + 'a> {
    label: &'a str,
    value: &'a T,
}

fn describe(item: &Labeled<'_, dyn Shape>) -> String {
    format!("{} has area {:.2}", item.label, item.value.area())
}
```

The declaration `T: ?Sized + 'a` is what makes `Labeled<'_, dyn Shape>` mean `Labeled<'_, dyn Shape + '_>`, by rule 2, exactly as `Ref` does. So `describe` accepts a label for a circle that borrows a local variable:

```
let radius = 1.0;
let circle = Circle { radius: &radius };
// The struct literal is built with `T = dyn Shape`: `&circle` is coerced.
let item: Labeled<'_, dyn Shape> = Labeled {
    label: "unit circle",
    value: &circle,
};
println!("{}", describe(&item));
```

```
unit circle has area 3.14
```

Without the declared bound, the containing type gives no default and rule 3 applies, even though the field is a `&'a T` and the compiler knows `T: 'a` must hold (`code/compile_fail/ch08_unbounded_param.rs`):

```
pub struct Labeled<'a, T: ?Sized> {
    pub label: &'a str,
    pub value: &'a T,
}

pub fn describe(item: &Labeled<'_, dyn Shape>) -> String {
    format!("{} has area {:.2}", item.label, item.value.area())
}
```

```
pub fn caller() -> String {
    let radius = 1.0;
    let circle = Circle { radius: &radius };
    describe(&Labeled {
        label: "unit circle",
        value: &circle,
    })
}
```

```
error[E0597]: `radius` does not live long enough
  --> compile_fail/ch08_unbounded_param.rs:32:35
   |
31 |         let radius = 1.0;
   |             ------ binding `radius` declared here
32 |         let circle = Circle { radius: &radius };
   |                                       ^^^^^^^ borrowed value does not live long enough
33 | /       describe(&Labeled {
34 | |           label: "unit circle",
35 | |           value: &circle,
36 | |       })
   | |_______- argument requires that `radius` is borrowed for `'static`
37 |     }
   |     - `radius` dropped here while still borrowed
```

The circle borrows `radius`, and "argument requires that `radius` is borrowed for `'static`" reveals that `describe` now wants a `dyn Shape + 'static`. The bound that `&'a T` implies is not used for the default; only bounds written on the type's parameters count. If you write a container meant to hold trait objects, declare `T: ?Sized + 'a`.

8.5 Return-position `impl Trait`: what an opaque type captures

A function that returns `impl Iterator<Item = char>` returns an *opaque type*: a type that the compiler knows (the *hidden type*, chosen by the body) but that callers may use only through the listed traits. Here is the simplest example, from `code/examples/ch08_rpit_captures.rs`:

```
fn chars_of(s: &str) -> impl Iterator<Item = char> {
    s.chars() // the hidden type is `Chars<'_>`, which borrows `s`
}
```

The hidden type `Chars<'_>` borrows `s`. For callers to use the result safely, the opaque type must reveal at least *which* lifetimes the hidden type may depend on, without revealing the type itself. It does so by *capturing* generic parameters: the opaque type behaves like a type with parameters, say `CharsOf<'s>`, and a value of it keeps every captured lifetime alive as long as the value is used. A lifetime that is not captured cannot appear in the hidden type at all.

In edition 2024, the rule is the simplest one possible. The edition guide puts it in one sentence: "In Rust 2024, all in-scope generic parameters, including lifetime parameters, are implicitly captured when the `use<..>` bound is not present." The elided lifetime of `s` is an anonymous lifetime parameter of `chars_of`, so it is captured, and `chars_of` compiles. Written out, with the precise capturing syntax `use<..>` that lists the captures:

```
// The same contract with the captures written out.
fn chars_of_explicit<'s>(s: &'s str) -> impl Iterator<Item = char> + use<'s> {
```

```
    s.chars()
}
```

A result that borrows from two inputs captures both lifetimes, with no annotation:

```
// The hidden type borrows from both inputs, and both lifetimes are captured.
fn both<'a, 'b>(x: &'a str, y: &'b str) -> impl Iterator<Item = char> {
    x.chars().chain(y.chars())
}
```

Type parameters have always been captured, in every edition:

```
// Type parameters have always been captured, in every edition.
fn repeated<T: Clone>(item: T, times: usize) -> impl Iterator<Item = T> {
    std::iter::repeat_n(item, times)
}
```

```
ABC 3 ["ok", "ok"] abc!
```

The signature is still the contract: a caller of `chars_of` knows only that the result may borrow from `s`, and it is checked against that, whether or not the body actually borrows.

8.6 Over-capturing and precise capturing with `use<..>`

Capturing everything is safe and usually right. It is wrong when the hidden type does not borrow from every input, because callers then keep inputs borrowed for nothing. Here is a function that skips a prefix (`code/compile_fail/ch08_overcapture.rs`):

```
pub fn after_prefix<'s>(s: &'s str, prefix: &str) -> impl Iterator<Item = char> {
    s.strip_prefix(prefix).unwrap_or(s).chars()
}
```

The iterator borrows `s` only; `prefix` is used during the call and never again. But the opaque type captures both lifetimes, so a caller that drops the prefix early is rejected:

```
pub fn caller() -> String {
    let text = String::from("> hello");
    let chars;
    {
        let prefix = String::from("> ");
        chars = after_prefix(&text, &prefix);
    } // `prefix` is dropped here
    chars.collect()
}
```

```
error[E0597]: `prefix` does not live long enough
  --> compile_fail/ch08_overcapture.rs:18:37
   |
17 |         let prefix = String::from("> ");
   |             ------ binding `prefix` declared here
18 |         chars = after_prefix(&text, &prefix);
   |                                     ^^^^^^^ borrowed value does not live long enough
19 |     } // `prefix` is dropped here
   |     - `prefix` dropped here while still borrowed
20 |     chars.collect()
   |     ----- borrow later used here
   |
note: this call may capture more lifetimes than intended, because Rust 2024 has adjusted the
```

```
`impl Trait` lifetime capture rules
  --> compile_fail/ch08_overcapture.rs:18:17
   |
18 |         chars = after_prefix(&text, &prefix);
   |                 ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
help: use the precise capturing `use<...>` syntax to make the captures explicit
   |
 7 | pub fn after_prefix<'s>(s: &'s str, prefix: &str) -> impl Iterator<Item = char> + use<>
{
   |                                                                                +++++++
```

E0597, "`prefix` does not live long enough", with the three usual labels: the borrow, the drop at the end of the block, and the later use of `chars`. The note is new: "this call may capture more lifetimes than intended, because Rust 2024 has adjusted the `impl Trait` lifetime capture rules". The help suggests `use<>`, an empty capture list. Applying it literally (`code/compile_fail/ch08_use_nothing.rs`):

```
fn after_prefix<'s>(s: &'s str, prefix: &str) -> impl Iterator<Item = char> + use<> {
    s.strip_prefix(prefix).unwrap_or(s).chars()
}
```

```
error[E0700]: hidden type for `impl Iterator<Item = char>` captures lifetime that does not
appear in bounds
 --> compile_fail/ch08_use_nothing.rs:7:5
  |
6 | fn after_prefix<'s>(s: &'s str, prefix: &str) -> impl Iterator<Item = char> + use<> {
  |                 --                               ----------------------------------
opaque type defined here
  |                 |
  |                 hidden type `Chars<'s>` captures the lifetime `'s` as defined here
7 |     s.strip_prefix(prefix).unwrap_or(s).chars()
  |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |
help: add `'s` to the `use<...>` bound to explicitly capture it
  |
6 | fn after_prefix<'s>(s: &'s str, prefix: &str) -> impl Iterator<Item = char> + use<'s> {
  |                                                                               ++
```

E0700, "hidden type for `impl Iterator<Item = char>` captures lifetime that does not appear in bounds": the hidden type `Chars<'s>` borrows `s`, and `use<>` just forbade that. The suggestion at the call site is a generic one, made without knowing what the hidden type needs. The second help is the right answer, and it is in `code/examples/ch08_precise_capture.rs`:

```
// The iterator borrows `s` only; `prefix` is used during the call and forgotten.
fn after_prefix<'s>(s: &'s str, prefix: &str) -> impl Iterator<Item = char> + use<'s> {
    s.strip_prefix(prefix).unwrap_or(s).chars()
}
```

`use<'s>` captures `'s` and nothing else. When the result borrows nothing at all, the empty list is the right one:

```
// A result that borrows nothing: capture nothing.
fn char_count(s: &str) -> impl Display + use<> {
    s.chars().count()
}
```

`use<..>` follows a few rules, all checked by rustc 1.99:

- It lists the generic parameters that the hidden type may use: lifetimes, type parameters and const parameters. `use<'_>` names the lifetime that elision picks for the output (Chapter 6).

- It can leave out lifetimes only. Every type parameter in scope must be listed (`code/compile_fail/ch08_use_all_types.rs`):

```rust
pub fn tagged<T: Display>(tag: T, text: &str) -> impl Display + use<'_> {
    format!("[{tag}] {text}")
}
```

```
error: `impl Trait` must mention all type parameters in scope in `use<...>`
 --> compile_fail/ch08_use_all_types.rs:8:50
  |
8 | pub fn tagged<T: Display>(tag: T, text: &str) -> impl Display + use<'_> {
  |               -                                  ^^^^^^^^^^^^^^^^^^^^^
  |               |
  |               type parameter is implicitly captured by this `impl Trait`
  |
  = note: currently, all type parameters are required to be mentioned in the precise
captures list
```

- In a trait, it must list `Self` as well (see *`impl Trait` in traits and `async fn`* below), and it cannot name an `impl Trait` argument (see *`impl Trait` in argument position*).

> **NOTE**
>
> Precise capturing with `use<..>` is stable since Rust 1.82 (October 2024) in all editions, and in `impl Trait` inside traits since Rust 1.87. Edition 2024, with its capture-everything default, is stable since Rust 1.85.

8.7 `impl Trait + 'a` is not `use<'a>`

Before `use<..>` existed, code wrote `impl Trait + 'a`, and you will see both. They answer different questions. `use<'a>` is a *capture list*: the hidden type may mention `'a` (and the other listed parameters) and no other lifetime. `+ 'a` is an *outlives bound*: the hidden type, whatever it captures, outlives `'a`.

Sometimes the bound has the effect you want. `after_prefix_outlives` keeps capturing both lifetimes but promises that the hidden type outlives `'s`:

```rust
// An outlives bound instead of a capture list. It captures `prefix`'s lifetime
// too, but promises that the hidden type outlives `'s`.
fn after_prefix_outlives<'s>(
    s: &'s str,
    prefix: &str,
) -> impl Iterator<Item = char> + 's {
    s.strip_prefix(prefix).unwrap_or(s).chars()
}
```

The caller in the same example drops `prefix` before using the iterator, and compiles:

```rust
fn main() {
    let text = String::from("> hello");
    let (chars, outlives, count);
    {
        let prefix = String::from("> ");
        chars = after_prefix(&text, &prefix);
        outlives = after_prefix_outlives(&text, &prefix);
        count = char_count(&prefix);
    } // `prefix` is dropped here; nothing returned borrows it
    println!("{} {} {count}", chars.collect::<String>(), outlives.count());
```

```
    let name = String::from("parser");
    println!("{}", labeled("name", &name)); // `T` is a short borrow: fine
}
```

```
hello 5 2
name: parser
```

The borrow checker uses the bound: since the hidden type outlives `'s`, a value of the opaque type stays valid as long as `'s` does, whatever else it captured.

But a bound is a requirement on the hidden type, and it can cost more than you meant. It constrains type parameters too, so a hidden type that contains a `T` requires `T: 'a` (`code/compile_fail/ch08_outlives_type_param.rs`):

```
pub fn labeled<'a, T: Display>(label: &'a str, value: T) -> impl Display + 'a {
    Labeled(label, value)
}
```

```
error[E0309]: the parameter type `T` may not live long enough
  --> compile_fail/ch08_outlives_type_param.rs:17:5
   |
16 | pub fn labeled<'a, T: Display>(label: &'a str, value: T) -> impl Display + 'a {
   |                -- the parameter type `T` must be valid for the lifetime `'a` as defined
here...
17 |     Labeled(label, value)
   |     ^^^^^^^^^^^^^^^^^^^^^ ...so that the type `T` will meet its required lifetime
bounds
   |
help: consider adding an explicit lifetime bound
   |
16 | pub fn labeled<'a, T: Display + 'a>(label: &'a str, value: T) -> impl Display + 'a {
   |                               ++++
```

E0309, "the parameter type `T` may not live long enough", with a help that adds `T: Display + 'a`. That fixes the function and moves the cost to callers, who may then pass only values that outlive the label. The edition guide calls this pattern the "outlives trick" and says it "created undue and surprising restrictions on callers". The 2024 default has no such requirement:

```
struct Labeled<'a, T>(&'a str, T);

impl<T: Display> Display for Labeled<'_, T> {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        write!(f, "{}: {}", self.0, self.1)
    }
}

// The default captures `'a` and `T`, and asks nothing about how long `T` lives.
fn labeled<'a, T: Display>(label: &'a str, value: T) -> impl Display {
    Labeled(label, value)
}
```

And when the hidden type borrows from two inputs, `+ 'a` demands that both borrows outlive `'a` (`code/compile_fail/ch08_outlives_two.rs`):

```
pub fn both<'a, 'b>(x: &'a str, y: &'b str) -> impl Iterator<Item = char> + 'a {
    x.chars().chain(y.chars())
}
```

```
error: lifetime may not live long enough
 --> compile_fail/ch08_outlives_two.rs:7:5
  |
6 | pub fn both<'a, 'b>(x: &'a str, y: &'b str) -> impl Iterator<Item = char> + 'a {
  |             --  -- lifetime `'b` defined here
  |             |
  |             lifetime `'a` defined here
7 |     x.chars().chain(y.chars())
  |     ^^^^^^^^^^^^^^^^^^^^^^^^^^ function was supposed to return data with lifetime `'a`
but it is returning data with lifetime `'b`
  |
  = help: consider adding the following bound: `'b: 'a`
```

"Function was supposed to return data with lifetime `'a` but it is returning data with lifetime `'b`", and the help asks for `'b: 'a`. Without the `+ 'a`, the 2024 default captures both lifetimes and the function compiles as it is: it is `both` in `ch08_rpit_captures.rs`.

So in edition 2024: write nothing when the result may borrow from every input, `use<..>` to leave some lifetimes out, and `+ 'a` only when you mean an outlives bound, for example `+ 'static` when callers need a value that borrows nothing.

8.8 Edition 2021: E0700 and the outlives trick

Code written for edition 2021 and earlier follows a different capture rule: type parameters are captured, but lifetime parameters only when they appear in the bounds of the `impl Trait`. The `chars_of` that compiles in 2024 is rejected there. The program `code/compile_fail/ch08_rpit_2021.rs` is compiled with `edition=2021`:

```
pub fn chars_of(s: &str) -> impl Iterator<Item = char> {
    s.chars()
}
```

```
error[E0700]: hidden type for `impl Iterator<Item = char>` captures lifetime that does not
appear in bounds
 --> compile_fail/ch08_rpit_2021.rs:8:5
  |
7 | pub fn chars_of(s: &str) -> impl Iterator<Item = char> {
  |                    ----     -------------------------- opaque type defined here
  |                    |
  |                    hidden type `Chars<'_>` captures the anonymous lifetime defined here
8 |     s.chars()
  |     ^^^^^^^^^
  |
help: add a `use<...>` bound to explicitly capture `'_`
  |
7 | pub fn chars_of(s: &str) -> impl Iterator<Item = char> + use<'_> {
  |                                                          +++++++++
```

E0700 says that the hidden type uses a lifetime the opaque type does not capture: "hidden type `Chars<'_>` captures the anonymous lifetime defined here", pointing at `&str`. rustc 1.99 suggests `use<'_>`. Older compilers suggested `+ '_`, and that is the fix you will find in 2021 code: mentioning `'_` in a bound made it captured. The same file contains both fixes, and the only error reported is the one above, so edition 2021 accepts both:

```
// The outlives trick: mentioning `'_` in a bound made it captured.
pub fn chars_outlives(s: &str) -> impl Iterator<Item = char> + '_ {
```

```
    s.chars()
}

// Precise capturing (Rust 1.82), which works in every edition.
pub fn chars_use(s: &str) -> impl Iterator<Item = char> + use<'_> {
    s.chars()
}
```

In edition 2024, both forms still compile, and `+ '_` now means only its outlives bound. When `cargo fix --edition` migrates a crate to 2024, it keeps the old meaning by adding `use<..>` where the new rule would capture more; the lint behind it, `impl_trait_overcaptures`, is part of the `rust-2024-compatibility` group. You may also meet a helper trait named `Captures` in older code, a workaround for the same 2021 rule; the edition guide notes that its uses "can be replaced by `use<..>` bounds (in all editions) or removed entirely (in Rust 2024)".

8.9 `impl Trait` in traits and `async fn`

A trait method can return `impl Trait` (stable since Rust 1.75, often called RPITIT, "return-position `impl Trait` in traits"). Such an opaque type captures every generic parameter in scope, `Self` and the elided lifetime of `&self` included, in every edition. A generic caller knows only the trait's signature, so it must assume that the result borrows the receiver (`code/compile_fail/ch08_rpitit_captures.rs`):

```
pub trait Inventory {
    fn ids(&self) -> impl Iterator<Item = u32>;
}

pub fn restock<I: Inventory>(inventory: &mut I, add: impl FnOnce(&mut I)) -> u32 {
    let before = inventory.ids();
    add(inventory);
    before.sum()
}
```

```
error[E0502]: cannot borrow `*inventory` as mutable because it is also borrowed as immutable
  --> compile_fail/ch08_rpitit_captures.rs:12:5
   |
11 |     let before = inventory.ids();
   |                  --------- immutable borrow occurs here
12 |     add(inventory);
   |     ^^^^^^^^^^^^^^ mutable borrow occurs here
13 |     before.sum()
   |     ------ immutable borrow later used here
```

E0502: `before` keeps `*inventory` borrowed, because the opaque type of `ids` captures the lifetime of `&self`, so `add` cannot borrow it mutably. Since Rust 1.87, a trait can say otherwise with `use<..>`, which must list `Self` (`code/compile_fail/ch08_rpitit_use_self.rs` shows the error for `use<>`):

```
error: `impl Trait` must mention the `Self` type of the trait in `use<...>`
 --> compile_fail/ch08_rpitit_use_self.rs:7:22
  |
6 | pub trait Inventory {
  | ------------------- `Self` type parameter is implicitly captured by this `impl Trait`
7 |     fn ids(&self) -> impl Iterator<Item = u32> + use<>;
  |                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |
  = note: currently, all type parameters are required to be mentioned in the precise
captures list
```

With `use<Self>`, the result may depend on the implementing type but not on the borrow of `self` (`code/examples/ch08_rpitit.rs`):

```
trait Inventory {
    // The ids are copied out: the iterator does not borrow `self`.
    fn ids(&self) -> impl Iterator<Item = u32> + use<Self>;
}

struct Shelf {
    items: Vec<u32>,
}

impl Inventory for Shelf {
    // `Shelf` has no generic parameters, so the impl captures nothing.
    fn ids(&self) -> impl Iterator<Item = u32> + use<> {
        self.items.clone().into_iter()
    }
}

fn restock<I: Inventory>(inventory: &mut I, add: impl FnOnce(&mut I)) -> u32 {
    let before = inventory.ids(); // does not keep `inventory` borrowed...
    add(inventory); // ...so it can be borrowed mutably here
    before.sum()
}
```

The implementation writes `use<>`, because `Shelf` has no generic parameters and `Self` is not a parameter in an `impl`; for `Shelf`, `use<Self>` and `use<>` capture the same nothing.

`async fn` in traits (also stable since 1.75) is the same feature in disguise: an `async fn` returns an opaque `impl Future`, and that future captures every input lifetime, because the body runs only when the future is polled and may use any argument until then. Chapter 11 covers what this means for async code; the capture is visible without running anything (`code/compile_fail/ch08_async_captures.rs`):

```
pub trait Store {
    async fn fetch(&self, key: &str) -> Option<String>;
}

pub fn caller<S: Store>(store: &S) {
    let future;
    {
        let key = String::from("port");
        future = store.fetch(&key);
    } // `key` is dropped here
    drop(future);
}
```

```
error[E0597]: `key` does not live long enough
  --> compile_fail/ch08_async_captures.rs:14:30
   |
13 |         let key = String::from("port");
   |             --- binding `key` declared here
14 |         future = store.fetch(&key);
   |                              ^^^^ borrowed value does not live long enough
15 |     } // `key` is dropped here
   |     - `key` dropped here while still borrowed
16 |     drop(future);
   |          ------ borrow later used here
```

The future returned by `store.fetch(&key)` borrows `key`, and "borrow later used here" points at `drop(future)`: even a future that is never polled keeps its borrows until it is dropped. The note about edition 2024 does not appear, because this capture rule is not new: `async fn` has always captured

every input lifetime. When a future should not borrow an argument, write the method without `async`, copy what the future needs, and narrow the captures:

```
trait Store {
    // The future borrows `self` and `key` until it completes.
    async fn fetch(&self, key: &str) -> Option<String>;

    // The future borrows `self` only: `'_` is the lifetime of `&self`.
    fn fetch_owned(
        &self,
        key: &str,
    ) -> impl Future<Output = Option<String>> + use<'_, Self>;
}

struct Memory {
    map: BTreeMap<String, String>,
}

impl Store for Memory {
    async fn fetch(&self, key: &str) -> Option<String> {
        self.map.get(key).cloned()
    }

    fn fetch_owned(&self, key: &str) -> impl Future<Output = Option<String>> + use<'_> {
        let key = key.to_owned(); // copied before the future is created
        async move { self.map.get(&key).cloned() }
    }
}
```

`use<'_, Self>` captures `Self` and the lifetime elision picks for the output, the one of `&self`, but not the lifetime of `key`. The example polls the futures once with a no-op waker, enough for futures that never wait:

```
fn main() {
    let mut shelf = Shelf {
        items: vec![1, 2, 3],
    };
    let sum = restock(&mut shelf, |s| s.items.push(4));
    println!("sum before restocking: {sum}, items now: {:?}", shelf.items);

    let store = Memory {
        map: BTreeMap::from([("port".into(), "8080".into())]),
    };
    let future;
    {
        let key = String::from("port");
        future = store.fetch_owned(&key);
    } // `key` is dropped here: the future does not borrow it
    println!("{:?}", poll_once(future));
    println!("{:?}", poll_once(store.fetch("host")));
}
```

```
sum before restocking: 6, items now: [1, 2, 3, 4]
Some(Some("8080"))
Some(None)
```

8.10 `impl Trait` in argument position

In argument position, `impl Trait` means something else entirely: it is an anonymous type parameter, chosen by the caller. `fn longest(words: impl Iterator<Item = ...>)` is a generic function, like

`fn longest<I: Iterator<Item = ...>>(words: I)`, except that callers cannot name `I` with a turbofish. The lifetimes inside it have a restriction of their own (`code/compile_fail/ch08_apit_anonymous.rs`):

```
pub fn longest(words: impl Iterator<Item = &'_ str>) -> usize {
    words.map(str::len).max().unwrap_or(0)
}
```

```
error[E0658]: anonymous lifetimes in `impl Trait` are unstable
 --> compile_fail/ch08_apit_anonymous.rs:6:45
  |
6 | pub fn longest(words: impl Iterator<Item = &'_ str>) -> usize {
  |                                             ^^ expected named lifetime parameter
  |
help: consider introducing a named lifetime parameter
  |
6 - pub fn longest(words: impl Iterator<Item = &'_ str>) -> usize {
6 + pub fn longest<'a>(words: impl Iterator<Item = &'a str>) -> usize {
  |
```

E0658, "anonymous lifetimes in `impl Trait` are unstable": neither `'_` nor an elided `&` is accepted inside an argument-position `impl Trait` on stable Rust 1.99 (nightly 2026-10-08 accepts it behind the feature `anonymous_lifetime_in_impl_trait`; check its current status). The help names the lifetime, and the result can then be tied to it (`code/examples/ch08_apit.rs`):

```
// The same as `fn longest<'a, I: Iterator<Item = &'a str>>(words: I) -> ...`.
fn longest<'a>(words: impl Iterator<Item = &'a str>) -> Option<&'a str> {
    words.max_by_key(|word| word.len())
}
```

The `Fn` sugar is the exception: in `impl Fn(&str) -> usize`, the elided lifetime becomes a higher-ranked one (Chapter 9), so the closure accepts a borrow of any lifetime, including one created inside the function:

```
// `Fn(&str)` sugar makes the elided lifetime higher-ranked (Chapter 9).
fn measure(f: impl Fn(&str) -> usize) -> usize {
    let local = String::from("created here");
    f(&local)
}
```

An argument-position `impl Trait` is a type parameter in scope, so a returned `impl Trait` captures it, together with every lifetime (`code/compile_fail/ch08_apit_overcapture.rs`):

```
pub fn shout(value: &impl Display) -> impl Display {
    format!("{value}!")
}

pub fn caller() -> String {
    let loud;
    {
        let word = String::from("hello");
        loud = shout(&word);
    }
    loud.to_string()
}
```

```
error[E0597]: `word` does not live long enough
  --> compile_fail/ch08_apit_overcapture.rs:17:22
   |
16 |         let word = String::from("hello");
```

```
   |             ---- binding `word` declared here
17 |         loud = shout(&word);
   |                      ^^^^^ borrowed value does not live long enough
18 |     }
   |     - `word` dropped here while still borrowed
19 |     loud.to_string()
   |     ---- borrow later used here
   |
note: this call may capture more lifetimes than intended, because Rust 2024 has adjusted the
`impl Trait` lifetime capture rules
  --> compile_fail/ch08_apit_overcapture.rs:17:16
   |
17 |         loud = shout(&word);
   |                ^^^^^^^^^^^
note: you could use a `use<...>` bound to explicitly specify captures, but argument-position
`impl Trait`s are not nameable
  --> compile_fail/ch08_apit_overcapture.rs:9:22
   |
 9 | pub fn shout(value: &impl Display) -> impl Display {
   |                      ^^^^^^^^^^^^
help: use the precise capturing `use<...>` syntax to make the captures explicit
   |
 9 - pub fn shout(value: &impl Display) -> impl Display {
 9 + pub fn shout<T: Display>(value: &T) -> impl Display + use<T> {
   |
```

The result is a `String` that borrows nothing, but the opaque type captures the lifetime of `value`, so `word` must outlive `loud`. The second note explains why `use<..>` cannot be added as it is: "argument-position `impl Trait`s are not nameable", and every type parameter must be listed. The help converts the anonymous parameter into a named one, which can then be listed without the lifetime:

```
// To leave the reference's lifetime out of the captures, `T` needs a name.
fn shout<T: Display>(value: &T) -> impl Display + use<T> {
    format!("{value}!")
}
```

```
fn main() {
    let text = String::from("the tokenizer reads words");
    println!("{:?}", longest(text.split(' ')));
    println!("{}", measure(|s| s.len()));
    let loud;
    {
        let word = String::from("hello");
        loud = shout(&word);
    }
    println!("{loud}");
}
```

```
Some("tokenizer")
12
hello!
```

8.11 Common confusions

"**`Box<dyn Trait>` accepts values of any lifetime.**" It accepts only values whose type is `'static`, because the default bound inside a `Box` is `'static`. Write `Box<dyn Trait + 'a>` (or `+ '_`) to accept borrowing values.

"**`&dyn Trait` is `&(dyn Trait + 'static)`.**" Behind a reference, the default is the reference's lifetime, unless the trait itself requires `'static`.

"**A `'static` trait object lives forever.**" `dyn Trait + 'static` says the value borrows nothing that can expire; the value itself is dropped like any other (Chapter 7).

"**`impl Trait + 'a` means 'captures `'a`'.**" That was the 2021 outlives trick. It is an outlives bound, with costs for type parameters and other lifetimes. To say which lifetimes are captured, use `use<..>`.

"**In edition 2024 I never need annotations on `impl Trait`.**" You need `use<..>` when the result does not borrow from every input, or callers will keep inputs borrowed for nothing.

"**The compiler's suggestion `use<>` is always safe.**" It is made at the call site, without looking at the hidden type. Check what the result really borrows.

"**My implementation returns an owned iterator, so callers of the trait method are free.**" Generic callers see the trait's signature. If the trait's `impl Trait` captures `&self`, they must assume the result borrows it.

8.12 Summary

- `dyn Trait + 'a` is "some type implementing `Trait` that outlives `'a`". Every trait object type has such a bound; a longer one can stand in for a shorter one.
- Default bounds, as rustc 1.99 applies them: the trait's own lifetime bounds first (`'static` if among them, the single bound otherwise, E0227 for several); then the containing type's declared bound (`&'a`, `&'a mut`, `Ref<'a, _>`, your types with `T: ?Sized + 'a`; E0228 for several); otherwise inferred in bodies and `'static` elsewhere. `Box`, `Rc` and `Arc` give `'static`. The Reference lists the first two rules in the wrong order, and in a function signature a late-bound trait lifetime falls back to `'static`.
- `Box<dyn Error>` is `+ 'static`. Errors that borrow need `Box<dyn Error + '_>` and cannot outlive their input (E0515 when propagated); errors that own their data travel freely.
- Unsizing coercions create trait objects and check the bound ("coercion requires…"). A `'static` object coerces to a shorter bound even behind `&mut`, but `&mut Box<dyn Trait>` is invariant.
- Return-position `impl Trait` in edition 2024 captures every generic parameter in scope. Over-capturing keeps inputs borrowed; `use<..>` (1.82) lists the captures and may leave out lifetimes, not type parameters. rustc's `use<>` suggestion can be wrong.
- `impl Trait + 'a` is an outlives bound, not a capture list: it constrains type parameters (E0309) and other borrows.
- Edition 2021 captured only lifetimes that appear in the bounds (E0700, fixed by `+ '_` or `use<'_>`).
- `impl Trait` in traits and `async fn` capture everything, `Self` and `&self` included; `use<..>` in traits (1.87) must list `Self`. In argument position, `impl Trait` is an anonymous type parameter: elided lifetimes inside it are unstable (E0658), and it cannot appear in `use<..>`.

8.13 Exercises

1. Write the default bound of each trait object, then check your answers with an identity function between `&mut` types, as in `ch08_default_table.rs`: `Box<dyn Fn() + Send>`, `&'a mut Vec<Box<dyn Display>>`, `Arc<Mutex<dyn Write>>`, `RefMut<'a, dyn Write>`. *Hint:* only the innermost type around the trait object counts, and only if its parameter declares a lifetime bound; `Mutex<T>` does not.
2. Make `load_port` in `ch08_error_escapes.rs` compile without changing `parse_port` or the return type of `load_port`. *Hint:* turn the borrowed error into an owned value before `?` sees it; `Box<dyn Error>` implements `From<String>`.
3. Add `use<'s>` to `fn tokens<'s>(src: &'s str, seps: &[char]) -> impl Iterator<Item = &'s str>`, whose body is `src.split(seps).filter(|t| !t.is_empty())`. Why is it rejected, and how can you change the body so that `use<'s>` is accepted? *Hint:* read the hidden type in the E0700 message: the pattern is part of `Split`. A closure that owns a copy of the separators borrows nothing.
4. In `ch08_rpitit.rs`, remove `+ use<Self>` from the trait but keep `use<>` in the implementation. What does the compiler warn about, which function fails to compile, and why does a function that calls `ids` on a `&mut Shelf` directly still compile? *Hint:* a caller that knows the concrete type sees the implementation's signature; a generic caller sees only the trait's.
5. Take a function from your own code that returns `impl Iterator<...> + '_` and was written for edition 2021. What does the `+ '_` mean in 2024? Can you remove it, and when would you replace it with `use<..>`? *Hint:* compare what the 2024 default captures with what the body borrows; the `+ '_` is now only an outlives bound.

CHAPTER 9

Higher-Ranked Trait Bounds

Every lifetime you have met so far was a *parameter*. In the signature of `longest` from Chapter 1,

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str
```

the caller decides what `'a` is: at each call site the compiler picks the region that makes that call work, and the body of `longest` has to be correct for whichever region the caller picked. That arrangement is perfect for functions that receive borrowed data. It breaks down the moment a function needs to *hand out* a borrow of its own: a buffer it allocated, a value it owns, a line it is reading. No caller can name a lifetime that exists only inside someone else's body.

Callbacks are where this shows up first. A function that reads lines and passes each one to a closure needs the closure to accept a borrow that the function creates, uses and drops. The closure's type must therefore promise more than "I accept a `&str` of some lifetime you agree on with my caller". It must promise "I accept a `&str` of *any* lifetime, including ones that do not exist yet". Rust writes that promise `for<'a>`, and a bound that contains it is a *higher-ranked trait bound* (HRTB).

You have been using higher-ranked bounds for as long as you have written `F: Fn(&str) -> usize`: the compiler inserts `for<'a>` for you. This chapter makes the quantifier visible, shows when you must write it yourself, and teaches you to read the small family of error messages that come from it, messages that otherwise look like they were written in a different language: "implementation of `Fn` is not general enough", "one type is more general than the other".

In this chapter you will learn:

- why a lifetime parameter can never be the lifetime of a borrow of a local variable, and how `for<'a>` moves the choice of lifetime from the caller to the callee
- where the compiler already writes `for<'a>` for you
- the three situations in which you must write it yourself: relationships that `Fn` sugar cannot express, bounds on borrowed types, and traits with a lifetime parameter (the `Deserialize<'de>` pattern)
- why higher-ranked types are "more general", and how to read the errors that say so
- why closures that return references misbehave, and three ways to fix them
- how `AsyncFn` (Rust 1.85) solves async callbacks that borrow their argument
- what early-bound and late-bound lifetimes are, and why a redundant bound can break a call
- the current limits of `for<..>` on stable Rust

9.1 Who picks the lifetime?

Here is a function that creates a string and lends it to a callback. The bound on `F` uses a lifetime parameter of `with_local`, the most natural thing to write if you have only ever seen lifetimes as parameters (`code/compile_fail/ch09_caller_picks.rs`):

```
fn with_local<'a, F>(f: F) -> usize
where
    F: Fn(&'a str) -> usize,
{
    let local = String::from("created inside");
    f(&local)
}
```

The compiler rejects it. The diagnostic continues with a note that points into the standard library's definition of `Fn` (it only says that the requirement comes from the `Fn` bound); the first part is the one that matters:

```
error[E0597]: `local` does not live long enough
  --> compile_fail/ch09_caller_picks.rs:11:7
   |
 6 | fn with_local<'a, F>(f: F) -> usize
   |               -- lifetime `'a` defined here
...
10 |     let local = String::from("created inside");
   |         ----- binding `local` declared here
11 |     f(&local)
   |     --^^^^^^-
   |     | |
   |     | borrowed value does not live long enough
   |     argument requires that `local` is borrowed for `'a`
12 | }
   | - `local` dropped here while still borrowed
```

Read the labels from top to bottom. "lifetime `'a` defined here" points at the parameter list of `with_local`: `'a` belongs to the function's signature, so it is chosen by whoever calls `with_local`. "argument requires that `local` is borrowed for `'a`" points at `f(&local)`: because `F` only accepts `&'a str`, the borrow `&local` must last for the whole of `'a`. Finally, "`local` dropped here while still borrowed" points at the closing brace: `local` dies at the end of the body, but `'a` is still going.

Why is `'a` "still going" at the end of the body? Think about what the caller knows. When it calls `with_local::<'a, F>(f)`, it has to pick `'a` *before* the call starts, from regions it can see: regions of its own body. Whatever it picks, the callee must be correct for it, and the callee cannot rule out that the caller picked a region that includes the whole call and more. From inside `with_local`, then, `'a` is a region that contains every point of the body, from the first line to the closing brace. The borrow checker treats every lifetime parameter of the function it is checking this way (rustc calls them *universal regions*): they outlive the body. A borrow of a local variable, which ends before the body does, can never last that long.

This is the key fact of the chapter:

> **IMPORTANT**
>
> Every lifetime parameter of a function outlives the function's body. No borrow of a value that the function creates or owns can have the lifetime of one of its parameters.

What `with_local` actually needs is a callback that accepts a borrow of *any* lifetime, so that at the call `f(&local)` the lifetime can be the short region of that one borrow. That is exactly what `for<'a>` says (`code/examples/ch09_callee_picks.rs`):

```
fn with_local<F>(f: F) -> usize
where
    F: for<'a> Fn(&'a str) -> usize, // the same as `F: Fn(&str) -> usize`
{
    let local = String::from("created inside");
    f(&local) + f("a literal works too")
}
```

This version compiles, and the second call shows that "any lifetime" includes `'static` too:

```
33
```

program point	local is alive	F: Fn(&'a str) 'a picked by the caller	F: for<'a> Fn(&'a str) 'a picked at each call of f
caller: with_local(...) begins		'a	
let local = String::from(...)	local	'a	
f(&local)	local	'a: &local must last to the end	'a: just this call
end of body: local is dropped	dropped	'a still live: error E0597	
caller: with_local(...) returns		'a	

Who picks the lifetime. With a lifetime parameter, the caller picks `'a` before the call, so it covers the whole call and the borrow of `local` would have to outlive `local`. With `for<'a>`, a fresh `'a` is picked at each call of `f`, as short as that call.

Reading `for<'a>`

Read `for<'a>` as "for all `'a`". The two signatures say:

- `fn with_local<'a, F>(f: F) where F: Fn(&'a str) -> usize` — "for some `'a` that my caller picks, `f` accepts `&'a str`".
- `fn with_local<F>(f: F) where F: for<'a> Fn(&'a str) -> usize` — "`f` accepts `&'a str` for every `'a`; I will pick a suitable one each time I call it".

The difference is *where the quantifier sits*. In the first signature, `'a` is quantified at the level of the function: the function is generic over `'a`, and the caller instantiates it. In the second, the quantifier sits *inside the bound*: the function is generic over `F`, and `F` itself must be generic over `'a`. A function whose argument must itself be polymorphic is called *higher-rank* (rank 2) in type theory, and that is

where the name of the feature comes from. Only lifetimes can be quantified this way on stable Rust (see *The limits of* `for<..>` below).

Nothing about `for<'a>` makes anything live longer, and it does not mean `'static`. It is a stronger requirement on the *callback*: a closure that accepts only `&'static str` does not satisfy it, while `|s| s.len()` does, because it works for any lifetime.

9.2 Where the compiler writes `for<'a>` for you

You rarely see `for<'a>` because the elided forms of `Fn` bounds, function pointers and trait objects already mean it. The compiler applies the same elision rules as for function signatures (Chapter 3): every elided lifetime in the arguments becomes a fresh lifetime, but bound by the `for<..>` of that very trait or pointer rather than by the enclosing function, and an elided lifetime in the output takes the lifetime of the single input.

You write	The compiler reads
`F: Fn(&str) -> usize`	`F: for<'a> Fn(&'a str) -> usize`
`F: Fn(&str) -> &str`	`F: for<'a> Fn(&'a str) -> &'a str`
`impl Fn(&str) -> usize` (argument)	`impl for<'a> Fn(&'a str) -> usize`
`Box<dyn Fn(&str) -> usize>`	`Box<dyn for<'a> Fn(&'a str) -> usize + 'static>`
`fn(&str) -> &str` (pointer type)	`for<'a> fn(&'a str) -> &'a str`
`F: AsyncFn(&str) -> usize`	`F: for<'a> AsyncFn(&'a str) -> usize`

(The `+ 'static` in the trait-object row is the default object lifetime bound of Chapter 8; it is unrelated to the `for<'a>`.) The following lines from `code/examples/ch09_hr_subtyping.rs` check two of these equivalences: each elided type converts to its explicit higher-ranked form without any coercion.

```
// The elided forms are the same higher-ranked types.
let p: fn(&str) -> &str = str::trim;
let q: for<'a> fn(&'a str) -> &'a str = p;
let b: Box<dyn Fn(&str) -> usize> = Box::new(|s: &str| s.len());
let c: Box<dyn for<'a> Fn(&'a str) -> usize> = b;
println!("{:?} {}", q("  padded  "), c("four"));
```

Two more places produce higher-ranked types without any syntax at all. A *function item* such as `str::trim` or a `fn` you wrote with elided lifetimes is generic over its lifetime at each call, so it converts to the higher-ranked pointer type, as `str::trim` does above. (There is one exception, explained in *Early-bound and late-bound lifetimes* below.) And a *closure written directly as the argument* of a function whose parameter has an `Fn` bound gets its signature from that bound: in `with_local(|s| s.len())`, the closure is checked against `for<'a> Fn(&'a str) -> usize` from the start.

Where to put the binder

`for<..>` can go in two places, and the Rust Reference describes the difference precisely: "Higher-ranked lifetimes may also be specified just before the trait: the only difference is the scope of the lifetime parameter, which extends only to the end of the following trait instead of the whole bound."

```
where F: for<'a> Fn(&'a str) -> usize   // binder before the trait
where for<'a> F: Fn(&'a str) -> usize   // binder before the whole bound
```

For a bound on a type parameter such as `F`, the two are equivalent. The second form is the only one that works when the *bounded type itself* mentions the lifetime, as in `for<'a> &'a C: IntoIterator`, which you will meet shortly.

9.3 When you have to write it yourself

The sugar covers callbacks that take one reference, or that return nothing borrowed. You need to write `for<..>` yourself in three situations.

When the sugar cannot say which input the result comes from

Elision inside `Fn(..)` is elision, so it fails in the same place as for functions: with two reference inputs and a reference output, there is no rule that picks the output's lifetime (`code/compile_fail/ch09_sugar_two_inputs.rs`):

```
pub fn split_head<F>(f: F) -> String
where
    F: Fn(&str, &str) -> &str,
{
    f("key=value", "=").to_string()
}
```

```
error[E0106]: missing lifetime specifier
 --> compile_fail/ch09_sugar_two_inputs.rs:8:26
  |
8 |     F: Fn(&str, &str) -> &str,
  |           ----  ----     ^ expected named lifetime parameter
  |
  = help: this function's return type contains a borrowed value, but the signature does not
say whether it is borrowed from argument 1 or argument 2
  = note: for more information on higher-ranked polymorphism, visit https://doc.rust-lang.
org/nomicon/hrtb.html
help: consider making the bound lifetime-generic with a new `'a` lifetime
  |
8 |     F: for<'a> Fn(&'a str, &'a str) -> &'a str,
  |        +++++++     ++       ++          ++
help: consider making the bound lifetime-generic with a new `'a` lifetime
  |
8 |     for<'a> F: Fn(&'a str, &'a str) -> &'a str,
  |     +++++++        ++       ++          ++
help: consider introducing a named lifetime parameter
  |
6 ~ pub fn split_head<'a, F>(f: F) -> String
7 | where
8 ~     F: Fn(&'a str, &'a str) -> &'a str,
  |
```

This is E0106 again, the error of Chapter 3, with a note pointing to the Rustonomicon's page on higher-ranked polymorphism and three suggestions. The first two make the bound higher-ranked; the third makes `'a` a parameter of `split_head`, which you now know is the wrong kind of lifetime for a callback that will receive borrows of locals.

Look closely at the first two suggestions: they give *both* arguments and the result the same lifetime. That is "candidate B" from Chapter 3, the rule that ties everything together. It always type-checks

the callback, and it moves the cost to the caller of the callback: the result can no longer outlive the *shorter* of the two arguments. If `split_head` keeps the result after the separator is gone, accepting the suggestion verbatim produces a new error (`code/compile_fail/ch09_suggestion_applied.rs`):

```
pub fn split_head<F>(f: F) -> String
where
    F: for<'a> Fn(&'a str, &'a str) -> &'a str, // the compiler's suggestion
{
    let text = String::from("key=value");
    let head;
    {
        let sep = String::from("=");
        head = f(&text, &sep);
    }
    head.to_string()
}
```

```
error[E0597]: `sep` does not live long enough
  --> compile_fail/ch09_suggestion_applied.rs:14:25
   |
13 |         let sep = String::from("=");
   |             --- binding `sep` declared here
14 |         head = f(&text, &sep);
   |                         ^^^^ borrowed value does not live long enough
15 |     }
   |     - `sep` dropped here while still borrowed
16 |     head.to_string()
   |     ---- borrow later used here
```

The bound you want says that the result borrows from the *text* only (`code/examples/ch09_relation.rs`):

```
fn split_head<F>(f: F) -> String
where
    F: for<'t> Fn(&'t str, &str) -> &'t str,
{
    let text = String::from("key=value");
    let head;
    {
        let sep = String::from("=");
        head = f(&text, &sep); // the result borrows `text`, not `sep`
    } // `sep` is dropped here; `head` is still valid
    head.to_string()
}
```

In `for<'t> Fn(&'t str, &str) -> &'t str` the binder introduces `'t` , and the second `&str` is elided *inside* the binder, so it gets a fresh bound lifetime of its own: the bound reads "for all `'t` and all `'s` , `F: Fn(&'t str, &'s str) -> &'t str` ". You can mix named and elided lifetimes this way, exactly as in a function signature. The closure that the caller passes gets this signature from the bound, and its body, which returns a piece of `text` , satisfies it:

```
let head = split_head(|text, sep| text.split(sep).next().unwrap_or(text));
```

```
key
```

When the bounded type contains the lifetime

Suppose you want a function that accepts *any* collection of `i32` (a `Vec` , an array, a `BTreeSet`) and iterates over it twice. You cannot write `C: IntoIterator<Item = i32>` , because iterating a collection by value consumes it. What you want is "a shared borrow of `C` can be iterated, yielding `&i32` ". A borrow

has a lifetime, so the bound has to mention one, and because the function owns `c` and borrows it inside its body, that lifetime must be higher-ranked (`code/examples/ch09_borrowed_iter.rs`):

```
fn stats<C>(c: C) -> (i32, usize)
where
    for<'a> &'a C: IntoIterator<Item = &'a i32>,
{
    let sum = (&c).into_iter().sum();
    let count = (&c).into_iter().count();
    (sum, count)
}
```

Here the binder must come before the whole bound, because the type being bounded, `&'a C`, contains `'a`. The bound reads "for every lifetime `'a`, a `&'a C` can be turned into an iterator over `&'a i32`". The standard library has exactly such implementations for `&'a Vec<T>`, `&'a [T; N]`, `&'a BTreeSet<T>` and many more, each one generic over `'a`, so all of these calls work:

```
println!("{:?}", stats(vec![1, 2, 3]));
println!("{:?}", stats([10, 20]));
println!("{:?}", stats(BTreeSet::from([5, 6])));
println!("{}", sum_all(&vec![1, 2, 3]));
```

```
(6, 3)
(30, 2)
(11, 2)
6
```

Replace the binder by a lifetime parameter and you are back to the problem of the first section (`code/compile_fail/ch09_borrowed_iter_named.rs`):

```
pub fn stats<'x, C: 'x>(c: C) -> (i32, usize)
where
    &'x C: IntoIterator<Item = &'x i32>,
{
    let sum = (&c).into_iter().sum();
    let count = (&c).into_iter().count();
    (sum, count)
}
```

The compiler reports the same error twice, once for each `(&c)`; here is the first:

```
error[E0597]: `c` does not live long enough
  --> compile_fail/ch09_borrowed_iter_named.rs:10:15
   |
 6 | pub fn stats<'x, C: 'x>(c: C) -> (i32, usize)
   |              --         - binding `c` declared here
   |              |
   |              lifetime `'x` defined here
...
10 |     let sum = (&c).into_iter().sum();
   |               ^^^^------------
   |               |
   |               borrowed value does not live long enough
   |               argument requires that `c` is borrowed for `'x`
...
13 | }
   | - `c` dropped here while still borrowed
```

`'x` is picked by the caller and outlives the body; `c` is a local of the body (a parameter passed by value is a local like any other); a borrow of `c` cannot last for `'x`. (The `C: 'x` bound is in the program only

to keep the diagnostic short: without it, rustc additionally reports that `C` itself may not live as long as `'x`, error E0309.)

Contrast this with the last function of the example, which takes the collection *by reference*:

```
// When the caller lends the collection, a named lifetime is enough.
fn sum_all<'x, C>(c: &'x C) -> i32
where
    &'x C: IntoIterator<Item = &'x i32>,
{
    c.into_iter().sum()
}
```

Here a lifetime parameter is right: the caller lends the collection for `'x`, so the borrow already exists before the call and lasts for all of it. This gives a practical rule:

> **TIP**
>
> Use a lifetime parameter when the *caller* lends the data. Use `for<'a>` when the *function* creates or owns the data and lends it out itself: to a callback, or through a bound on `&'a T`.

When a trait has a lifetime parameter

Some traits are parameterized by a lifetime because their methods produce values that may borrow from an input. Here is a small one, modelled on the most famous example of the pattern, serde's `Deserialize<'de>` (`code/examples/ch09_trait_param.rs`):

```
/// Parses a value from text that lives for `'src`.
trait Parse<'src>: Sized {
    fn parse(src: &'src str) -> Option<Self>;
}

// A borrowing implementation: the result points into the source text.
impl<'src> Parse<'src> for &'src str {
    fn parse(src: &'src str) -> Option<Self> {
        src.split_whitespace().next()
    }
}

// An owning implementation: it works for any source lifetime.
impl<'src> Parse<'src> for u32 {
    fn parse(src: &'src str) -> Option<Self> {
        src.trim().parse().ok()
    }
}
```

`Parse<'src>` means "can be parsed from text that lives for `'src`". The implementation for `&'src str` is *borrowing*: it returns a slice of the source, so it exists only for the matching `'src` (`&'x str` implements `Parse<'x>` and nothing else). The implementation for `u32` is *owning*: it implements `Parse<'src>` for every `'src`.

There are two ways to bound such a trait, and they correspond exactly to the two kinds of lifetime in this chapter:

```
/// The caller provides the text, so the result may borrow from it.
fn parse_from<'src, T: Parse<'src>>(src: &'src str) -> Option<T> {
    T::parse(src)
}
```

```
/// The text is a local buffer, so `T` must be parseable from a borrow of any
/// lifetime: in practice, `T` cannot borrow from it.
fn parse_local<T>(bytes: &[u8]) -> Option<T>
where
    T: for<'src> Parse<'src>,
{
    let text = String::from_utf8_lossy(bytes).into_owned();
    T::parse(&text)
}
```

`parse_from` lets the caller provide the text, so it uses a lifetime parameter and accepts borrowing types. `parse_local` parses a buffer it creates itself, so it needs `T: for<'src> Parse<'src>`: "`T` can be parsed from text of any lifetime". Both work for `u32`, and `parse_from` also works for `&str`:

```
let text = String::from("borrowed words");
let word: Option<&str> = parse_from(&text);
let n: Option<u32> = parse_from(" 7 ");
let m: Option<u32> = parse_local(b" 42 ");
println!("{word:?} {n:?} {m:?}");
```

```
Some("borrowed") Some(7) Some(42)
```

Asking `parse_local` for a `&str` cannot work: the result would point into a buffer that `parse_local` drops before returning. The compiler does not phrase it that way, though (`code/compile_fail/ch09_trait_param_borrow.rs`):

```
pub fn caller() {
    let word: Option<&str> = parse_local(b"words");
    println!("{word:?}");
}
```

```
error: implementation of `Parse` is not general enough
  --> compile_fail/ch09_trait_param_borrow.rs:25:30
   |
25 |     let word: Option<&str> = parse_local(b"words");
   |                              ^^^^^^^^^^^^^^^^^^^^^ implementation of `Parse` is not
general enough
   |
   = note: `Parse<'0>` would have to be implemented for the type `&str`, for any lifetime
`'0`...
   = note: ...but `Parse<'1>` is actually implemented for the type `&'1 str`, for some
specific lifetime `'1`
```

This is the message you will see most often with higher-ranked bounds, so it is worth reading slowly. The first note states the requirement: "`Parse<'0>` would have to be implemented for the type `&str`, for any lifetime `'0`". That is the bound `for<'src> Parse<'src>` with the bound lifetime renamed `'0`. The second note states what exists: "`Parse<'1>` is actually implemented for the type `&'1 str`, for some specific lifetime `'1`". The implementation for `&'1 str` covers exactly one lifetime, `'1`, so it is "not general enough" to cover all of them. In rustc's notes, *for any lifetime* marks what the bound requires and *for some specific lifetime* marks what you have.

This is precisely serde's design. A `T: Deserialize<'de>` may borrow from input that lives for `'de`, which is why `serde_json::from_str` can deserialize into `&'a str` fields; a `T: DeserializeOwned` cannot borrow, which is what you need when the input is a buffer the deserializer owns, as in `serde_json::from_reader`.

In serde's own words (https://serde.rs/lifetimes.html), "the `DeserializeOwned` trait is equivalent to the higher-rank trait bound `for<'de> Deserialize<'de>`." The signatures in serde_json (not compiled here: the book's code has no dependencies) are:

```
pub fn from_str<'a, T>(s: &'a str) -> Result<T>
where
    T: Deserialize<'a>,

pub fn from_reader<R, T>(rdr: R) -> Result<T>
where
    R: Read,
    T: DeserializeOwned,
```

`from_str` is `parse_from`; `from_reader` is `parse_local`.

9.4 Higher-ranked types are more general

A higher-ranked type promises something for *every* lifetime, so it can be used wherever the promise is needed for *one* lifetime. For function pointers this is subtyping, and the Reference describes it in one sentence: higher-ranked function pointers "are subtypes of types that are given by substitutions of the higher-ranked lifetimes." Substituting `'static` for `'a` in `for<'a> fn(&'a str) -> usize` gives `fn(&'static str) -> usize`, so the first converts to the second (`code/examples/ch09_hr_subtyping.rs`):

```
// `len` works for every lifetime...
let any: for<'a> fn(&'a str) -> usize = len;
// ...so it can be used where only one specific lifetime is required.
let only_static: fn(&'static str) -> usize = any;
let local = String::from("hello");
println!("{} {}", any(&local), only_static("hi"));
```

```
5 2
"padded" 4
```

The other direction fails, because a function that accepts only `&'static str` cannot accept a borrow of a local (`code/compile_fail/ch09_hr_subtyping.rs`):

```
fn static_only(s: &'static str) -> usize {
    s.len()
}

pub fn caller() {
    let f: fn(&'static str) -> usize = static_only;
    let any: for<'a> fn(&'a str) -> usize = f;
    println!("{}", any("hi"));
}
```

```
error[E0308]: mismatched types
  --> compile_fail/ch09_hr_subtyping.rs:12:45
   |
12 |     let any: for<'a> fn(&'a str) -> usize = f;
   |              ----------------------------   ^ one type is more general than the other
   |              |
   |              expected due to this
   |
   = note: expected fn pointer `for<'a> fn(&'a _) -> _`
              found fn pointer `fn(&'static _) -> _`
```

"one type is more general than the other" is how E0308 (mismatched types) reads when the two types differ only in how their lifetimes are quantified. The notes show both types with the irrelevant parts replaced by `_`: the expected one is higher-ranked (`for<'a> fn(&'a _) -> _`), the one you have fixes the lifetime (`fn(&'static _) -> _`).

Closures whose signature was fixed too early

The same mismatch appears with closures, with a twist: whether a closure's signature is higher-ranked depends on how its parameter types were inferred. In this program, the closure's parameter has no annotation, and its first use fixes the parameter type (`code/compile_fail/ch09_not_general_closure.rs`):

```
pub fn caller() -> usize {
    let count = |s| str::len(s);
    count("literal"); // fixes the parameter type: `&'static str`
    call_on_local(count)
}
```

The error is reported twice, for `Fn` and for its supertrait `FnOnce`; the first copy is:

```
error: implementation of `Fn` is not general enough
  --> compile_fail/ch09_not_general_closure.rs:13:5
   |
13 |     call_on_local(count)
   |     ^^^^^^^^^^^^^^^^^^^^ implementation of `Fn` is not general enough
   |
   = note: closure with signature `fn(&'2 str) -> usize` must implement `Fn<(&'1 str,)>`,
for any lifetime `'1`...
   = note: ...but it actually implements `Fn<(&'2 str,)>`, for some specific lifetime `'2`
help: consider adding an explicit type annotation to the closure's argument
   |
11 |     let count = |s: &str| str::len(s);
   |                  ++++++
```

The first call, `count("literal")`, made the parameter type `&'2 str` for one specific lifetime `'2` (the closure now has "signature `fn(&'2 str) -> usize`"). When the closure is later passed to `call_on_local`, whose bound is `Fn(&str) -> usize`, which is `for<'1> Fn(&'1 str) -> usize`, it is not general enough. A closure gets a higher-ranked signature when, *at the point where it is written*, either its parameters are annotated with reference types or it is checked against an expected `Fn` bound. The compiler's help suggests the first: write `|s: &str|`.

> **NOTE**
>
> "implementation of `Fn` is not general enough" and "one type is more general than the other" are the same diagnosis in two forms. The first appears when a trait must be implemented for every lifetime; the second when two *types* must be equal. In both, the fix is to make the thing you pass as general as the thing that is required.

9.5 Closures that return references

Annotating the parameter makes a closure's *argument* higher-ranked. It does not help with the *result*. A closure that returns part of its argument, stored in a `let` binding, is rejected, and so is the same closure with an annotated return type (`code/compile_fail/ch09_closure_return.rs`):

```
pub fn caller() {
    let first_word = |s: &str| s.split(' ').next().unwrap_or("");
    let annotated = |s: &str| -> &str { s.split(' ').next().unwrap_or("") };
    let text = String::from("hello world");
    println!("{} {}", first_word(&text), annotated(&text));
}
```

```
error: lifetime may not live long enough
 --> compile_fail/ch09_closure_return.rs:8:32
  |
8 |     let first_word = |s: &str| s.split(' ').next().unwrap_or("");
  |                          -   - ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ returning this value
requires that `'1` must outlive `'2`
  |                          |   |
  |                          |   return type of closure is &'2 str
  |                          let's call the lifetime of this reference `'1`

error: lifetime may not live long enough
 --> compile_fail/ch09_closure_return.rs:9:41
  |
9 |     let annotated = |s: &str| -> &str { s.split(' ').next().unwrap_or("") };
  |                          -        -     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ returning this
value requires that `'1` must outlive `'2`
  |                          |        |
  |                          |        let's call the lifetime of this reference `'2`
  |                          let's call the lifetime of this reference `'1`
```

In the first error, `'1` is the lifetime of the argument, which is higher-ranked because of the annotation `&str`. The return type, though, was inferred from the body as `&'2 str` for one fixed lifetime `'2` ("return type of closure is `&'2 str`"). Returning `s` would require `'1: '2` for every possible `'1`, which no single `'2` can satisfy. The second error shows that writing `-> &str` does not tie the two together: the function elision rules of Chapter 3 do not apply to closure annotations, so the `&str` in the return type gets a lifetime of its own, `'2`, unrelated to the argument's `'1`.

There are three reliable fixes (`code/examples/ch09_closure_fixes.rs`):

```
/// Identity function whose only job is to give the closure an expected
/// higher-ranked signature.
fn hr<F: Fn(&str) -> &str>(f: F) -> F {
    f
}

fn first_word(s: &str) -> &str {
    s.split(' ').next().unwrap_or("")
}

fn apply<F: Fn(&str) -> &str>(f: F, s: &str) -> usize {
    f(s).len()
}
```

```
let text = String::from("hello world");
// 1. Pass the closure directly to a function with a higher-ranked bound.
let n = apply(|s| s.trim(), &text);
// 2. Route a stored closure through an identity function with that bound.
let a = hr(|s| s.split(' ').next().unwrap_or(""));
// 3. Use a fn item: it is generic over the (late-bound) lifetime.
let b = first_word;
println!("{n} {} {}", a(&text), b(&text));
```

```
11 hello hello
```

1. **Pass the closure directly** to a function with a higher-ranked `Fn` bound. The closure is checked against `for<'a> Fn(&'a str) -> &'a str` when it is created, and gets exactly that signature. This is what you do most of the time without noticing.
2. **Route a stored closure through an identity function** whose parameter has the bound you want. `hr` does nothing at run time; its only job is to provide the expected signature at the point where the closure is written.
3. **Use a `fn` item.** A function with elided lifetimes is generic over them at each call, so `first_word` already has the signature you want.

Rust has a syntax that would say this directly, a binder on the closure itself: `for<'a> |s: &'a str| -> &'a str { ... }` (RFC 3216, "closure lifetime binder"). As of Rust 1.99 it is unstable (tracking issue https://github.com/rust-lang/rust/issues/97362); you will see the error it produces on stable at the end of this chapter.

9.6 Async callbacks that borrow

Async code makes the callback problem harder. Suppose `with_local` wants a callback that returns a *future*, and the future needs the borrowed string while it runs. Before Rust 1.85, the natural way to write the bound was to give the future type its own parameter (`code/compile_fail/ch09_async_fut_param.rs` ; the small `block_on` function it uses is shown below):

```
fn with_local<F, Fut>(f: F) -> usize
where
    F: Fn(&str) -> Fut,
    Fut: Future<Output = usize>,
{
    let local = String::from("created inside");
    block_on(f(&local))
}

async fn count_words(s: &str) -> usize {
    s.split_whitespace().count()
}

pub fn caller() -> usize {
    with_local(count_words)
}
```

```
error[E0308]: mismatched types
  --> compile_fail/ch09_async_fut_param.rs:34:5
   |
34 |     with_local(count_words)
   |     ^^^^^^^^^^^^^^^^^^^^^^^ one type is more general than the other
   |
   = note: expected opaque type `impl for<'a> Future<Output = usize>`
              found opaque type `impl Future<Output = usize>`
   = note: distinct uses of `impl Trait` result in different opaque types
note: the lifetime requirement is introduced here
  --> compile_fail/ch09_async_fut_param.rs:22:20
   |
22 |     F: Fn(&str) -> Fut,
   |                    ^^^
```

`F: Fn(&str) -> Fut` is `for<'a> Fn(&'a str) -> Fut`, and the binder covers only the arguments: `Fut` is a type parameter of `with_local`, picked once by the caller, the same for every `'a`. But an

`async fn` returns a future that captures its arguments (Chapter 11), so `count_words(&'a str)` returns a *different type* for each `'a`: a type that mentions `'a`. The note says exactly this: it expected "`impl for<'a> Future<Output = usize>`", a future type that works for all `'a`, and found "`impl Future<Output = usize>`" for one particular `'a`. The note "the lifetime requirement is introduced here" points at `Fut`.

There was no good way to write this bound before 1.85 (the usual workaround was to box the future). Rust 1.85 added the `AsyncFn`, `AsyncFnMut` and `AsyncFnOnce` traits, together with async closures, and they solve exactly this problem; the release announcement says it "has not been possible to properly express higher-ranked function signatures with the `Fn` traits returning a `Future`, but you can write this with the `AsyncFn` traits" (https://blog.rust-lang.org/2025/02/20/Rust-1.85.0/). With `AsyncFn` the future is not a separate parameter: it is an associated type of the trait, and the trait is implemented separately for each argument lifetime, so the future may depend on `'a` (`code/examples/ch09_async_fn.rs`):

```
fn with_local<F>(f: F) -> usize
where
    F: AsyncFn(&str) -> usize,
{
    let local = String::from("created inside");
    block_on(f(&local))
}

async fn count_words(s: &str) -> usize {
    s.split_whitespace().count()
}
```

```
let bonus = 100;
println!("{}", with_local(count_words));
println!("{}", with_local(async |s: &str| s.len() + bonus));
```

```
2
114
```

Both an `async fn` and an async closure (`async |s: &str| ...`, which here also uses the captured variable `bonus`) satisfy `AsyncFn(&str) -> usize`, which is sugar for `for<'a> AsyncFn(&'a str) -> usize` like any other `Fn`-style bound.

The examples need a way to run a future without an async runtime, because the book's code uses only the standard library. This is the whole executor:

```
/// Polls a future to completion on the current thread. Good enough for futures
/// that never wait on anything external, like the ones in this example.
fn block_on<T>(fut: impl Future<Output = T>) -> T {
    let mut fut = pin!(fut);
    let mut cx = Context::from_waker(Waker::noop());
    loop {
        if let Poll::Ready(v) = fut.as_mut().poll(&mut cx) {
            return v;
        }
    }
}
```

`Waker::noop()` (stable since 1.85) is a waker that does nothing: it is enough for futures that never wait for anything external, because such futures complete the first time they are polled. Chapter 11 uses the same `block_on` and goes further into lifetimes in async code.

9.7 Early-bound and late-bound lifetimes

You saw that a function item such as `first_word` can be used where a higher-ranked `Fn` bound is required, because it is generic over its lifetime at each call. That is true for most lifetime parameters, but not all, and the difference occasionally produces a baffling error. Here are two functions that differ only in a bound that looks redundant (`code/compile_fail/ch09_early_bound.rs`):

```
fn apply<F: Fn(&[i32]) -> &i32>(f: F) -> i32 {
    let v = vec![3, 1, 2];
    *f(&v)
}

fn first_late<T>(v: &[T]) -> &T {
    &v[0]
}

fn first_early<'a, T: 'a>(v: &'a [T]) -> &'a T {
    &v[0]
}

pub fn caller() -> i32 {
    apply(first_late) + apply(first_early)
}
```

`T: 'a` is implied by `&'a [T]` (Chapter 1), so writing it changes nothing about which calls of `first_early` are valid. Yet `apply(first_late)` compiles and `apply(first_early)` does not (the error is again reported for `Fn` and for `FnOnce` ; the first copy is shown):

```
error: implementation of `Fn` is not general enough
  --> compile_fail/ch09_early_bound.rs:21:25
   |
21 |     apply(first_late) + apply(first_early)
   |                         ^^^^^^^^^^^^^^^^^^ implementation of `Fn` is not general enough
   |
   = note: `fn(&'2 [i32]) -> &'2 i32 {first_early::<'2, i32>}` must implement `Fn<(&'1
[i32],)>`, for any lifetime `'1`...
   = note: ...but it actually implements `Fn<(&'2 [i32],)>`, for some specific lifetime `'2`
```

The note gives the type of the function item: `{first_early::<'2, i32>}` , with the lifetime *already filled in* with a specific `'2` , like the type parameter `i32` . That is the difference:

- A **late-bound** lifetime is chosen at each call. The function item's type stays generic over it, so `first_late::<i32>` has the higher-ranked signature `for<'a> fn(&'a [i32]) -> &'a i32` and satisfies `Fn(&[i32]) -> &i32` .
- An **early-bound** lifetime is chosen when the function is *named*, like a type parameter, so `first_early::<'2, i32>` works for that one `'2` only.

Which lifetimes are early-bound is a rule of rustc rather than something you declare. According to the rustc development guide (https://rustc-dev-guide.rust-lang.org/early-late-parameters.html), a lifetime parameter is early-bound when it appears in a where clause or bound (even a trivial one such

as `'a: 'a`, or `T: 'a` as here), or when it appears only in the return type; an *implied* `T: 'a` keeps it late-bound. In practice: **do not add `T: 'a` bounds to functions when the argument types already imply them.**

Late-bound lifetimes have one more visible consequence: since they are chosen at each call, you cannot fix them with a turbofish (`code/compile_fail/ch09_late_turbofish.rs`):

```
fn late<'a>(s: &'a str) -> &'a str {
    s
}

pub fn caller() -> &'static str {
    let f = late::<'static>;
    f("literal")
}
```

```
error[E0794]: cannot specify lifetime arguments explicitly if late bound lifetime parameters
are present
  --> compile_fail/ch09_late_turbofish.rs:11:20
   |
11 |     let f = late::<'static>;
   |                    ^^^^^^^
   |
note: the late bound lifetime parameter is introduced here
  --> compile_fail/ch09_late_turbofish.rs:6:9
   |
 6 | fn late<'a>(s: &'a str) -> &'a str {
   |         ^^
```

9.8 The limits of `for<..>`

Higher-ranked bounds on stable Rust 1.99 have a few hard edges. This file collects four of them (`code/compile_fail/ch09_binder_limits.rs`):

```
// 1. A bound lifetime used only in the output.
pub fn output_only<F>(f: F)
where
    F: for<'a> Fn() -> &'a str,
{
    let _ = f;
}

// 2. Two nested binders on the same bound.
pub fn nested<F>(f: F)
where
    for<'a> F: for<'b> Fn(&'a str, &'b str),
{
    let _ = f;
}

// 3. A binder over types.
pub fn over_types<F>(f: F)
where
    F: for<T> Fn(T),
{
    let _ = f;
}

// 4. A binder on a closure.
pub fn closure_binder() -> usize {
    let len = for<'a> |s: &'a str| -> usize { s.len() };
```

```
    len("four")
}
```

```
error[E0316]: nested quantification of lifetimes
  --> compile_fail/ch09_binder_limits.rs:16:16
   |
16 |     for<'a> F: for<'b> Fn(&'a str, &'b str),
   |                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^

error[E0658]: `for<...>` binders for closures are experimental
  --> compile_fail/ch09_binder_limits.rs:31:15
   |
31 |     let len = for<'a> |s: &'a str| -> usize { s.len() };
   |               ^^^^^^^
   |
   = note: see issue #97362 <https://github.com/rust-lang/rust/issues/97362> for more
information
   = help: consider removing `for<...>`

error[E0658]: only lifetime parameters can be used in this context
  --> compile_fail/ch09_binder_limits.rs:24:12
   |
24 |     F: for<T> Fn(T),
   |            ^
   |
   = note: see issue #108185 <https://github.com/rust-lang/rust/issues/108185> for more
information

error[E0582]: binding for associated type `Output` references lifetime `'a`, which does not
appear in the trait input types
 --> compile_fail/ch09_binder_limits.rs:8:24
  |
8 |     F: for<'a> Fn() -> &'a str,
  |                        ^^^^^^^
```

Construct	Result in Rust 1.99	What to do
`for<'a> Fn() -> &'a str`	E0582	The result would be valid for any lifetime the caller wants, which only `'static` data can be: write `Fn() -> &'static str`.
`for<'a> F: for<'b> Fn(..)`	E0316	Use one binder: `F: for<'a, 'b> Fn(&'a str, &'b str)`.
`for<T>` (a type in a binder)	E0658	Unstable (`non_lifetime_binders`, issue #108185).
`for<'a> |s: &'a str| ...`	E0658	Unstable (`closure_lifetime_binder`, issue #97362); see *Closures that return references*.
`for<'a where 'a: 'b>`	not valid syntax	No bounds on bound lifetimes; tracked by the 2026 project goal "Next-Gen Region Constraints".

E0582 deserves a sentence more. In `for<'a> Fn() -> &'a str`, `'a` appears in the output but in none of the inputs, so nothing in a call constrains it: the caller of `f` could pick *any* `'a`. The bound would describe a function that can conjure a reference valid for arbitrarily long, which is what `'static` already says. The same E0582 appears for traits with associated types (`for<'a> Tr<Assoc = &'a str>` where `'a` is not in the trait's own parameters).

The last row has a consequence that you will meet in Chapter 10. A binder ranges over *all* lifetimes, including lifetimes much longer than the types involved, and you cannot restrict it to "all `'a` that `T` outlives". When a bound such as `for<'a> L::Item<'a>: Debug` meets a generic associated type declared

with `where Self: 'a`, the only way for the bound to hold for all `'a` is `L: 'static`, and rustc says so in a note: "due to a current limitation of the type system, this implies a `'static` lifetime". Chapter 10 shows the problem in full and two ways around it.

9.9 Reading higher-ranked errors

You see	It means	Usual fix
"`x` does not live long enough ... argument requires that `x` is borrowed for `'a`", where `'a` is a lifetime parameter of the function	A caller-chosen lifetime is being used for a borrow the function creates	Make the bound higher-ranked (`for<'a>`, or the elided `Fn(&T)` form)
"implementation of `Trait` is not general enough ... for any lifetime `'1` ... for some specific lifetime `'2`"	Something implements the trait for one lifetime; the bound needs all of them	Annotate closure parameters; remove early-binding bounds; use an owning type for `for<'de>`-style bounds
E0308 "one type is more general than the other"	A type with a fixed lifetime where a higher-ranked type is required (fn pointers, or a `Fut` parameter for an async callback)	Use the higher-ranked type; for async callbacks use `AsyncFn`
"lifetime may not live long enough ... returning this value requires that `'1` must outlive `'2`" inside a closure	A stored closure returns a reference to its argument	Pass the closure to a bound, use an identity helper, or use a `fn`
E0106 in an `Fn(..)` bound	Elision inside the sugar failed	Write the binder and say which input the output borrows from
E0582, E0316, E0794, E0658	The limits in the previous section	See the table above

9.10 Common confusions

- "**`for<'a>` means `'static`.**" It means *every* lifetime, the shortest ones included. A callback that only works for `'static` data does *not* satisfy a higher-ranked bound.
- "**I need `for<'a>` for every callback.**" Almost never: `Fn(&T) -> U` already is higher-ranked. You write the binder for two-input relationships, bounds on `&'a T`, and traits with a lifetime parameter.
- "**A lifetime on the struct or `impl` works just as well.**" It is a parameter, picked by whoever creates the value. A struct `Handler<'a, F: Fn(&'a str)>` can never pass a borrow of its own temporary data to `F`.
- "**Higher-ranked bounds keep the closure alive longer.**" They say nothing about how long the closure lives; they constrain which arguments it accepts.
- "**The compiler's suggestion is the fix.**" For E0106 in an `Fn` bound, the suggestion ties all lifetimes together. It compiles the callback, but it may be the wrong contract for the code that calls it.

9.11 Summary

- A lifetime parameter is picked by the caller and outlives the whole body, so it can never be the lifetime of a borrow the function creates.
- `for<'a>` moves the choice to the callee: the bound must hold for every lifetime, and each use picks the one it needs. A bound that contains it is a higher-ranked trait bound.
- `Fn(&T) -> U`, `fn(&T) -> U`, `dyn Fn(&T) -> U` and `AsyncFn(&T) -> U` are already higher-ranked; elision inside them follows the function rules.
- Write `for<..>` yourself to relate one of several inputs to the output, to bound a borrowed type (`for<'a> &'a C: IntoIterator`), and to bound a trait with a lifetime parameter (`for<'de> Deserialize<'de>`, i.e. `DeserializeOwned`).
- Higher-ranked types are more general: they convert to any of their instances, never the other way. "not general enough" and "one type is more general than the other" both mean "you passed something less general than required".
- Closures get higher-ranked argument types from annotations or from an expected bound, but a stored closure cannot return a reference tied to its argument; pass it to a bound, use an identity helper, or use a `fn`.
- `AsyncFn` (Rust 1.85) expresses async callbacks whose futures borrow their arguments, which `F: Fn(&T) -> Fut` cannot.
- Lifetimes that appear in bounds are early-bound and make a function item less general; avoid redundant `T: 'a` bounds on functions.

9.12 Exercises

1. Predict whether this function compiles, then check:
   ```
   fn total<'a, F: Fn(&'a str) -> usize>(f: F, items: &[String]) -> usize { items.iter().map(|s| f(s)).sum() }
   ```
   Fix it in two different ways, and explain which fix gives callers more freedom. *Hint:* one fix ties `items` to `'a`; the other removes `'a` from the signature.
2. Which of these bounds need an explicit `for<..>` to compile, and what would you write? `F: Fn(&str) -> String`, `F: Fn(&str, &str) -> bool`, `F: Fn(&str, usize) -> &str`, `F: Fn(&str, &str) -> &str` (the result comes from the second argument). *Hint:* apply the elision rules of Chapter 3 to the arguments of each bound.
3. Change `parse_local` from `code/examples/ch09_trait_param.rs` so that it can return borrowing types such as `&str`. What must change about who owns the buffer? *Hint:* look at `parse_from`, and at the difference between `from_reader` and `from_str`.
4. `let longer = |a: &str, b: &str| if a.len() >= b.len() { a } else { b };` does not compile. Explain the error with the vocabulary of this chapter, then make it compile with an identity helper. *Hint:* the helper's bound needs a binder with one lifetime shared by both arguments and the result.

5. Write `fn retry<F>(mut f: F) -> Option<usize> where F: AsyncFnMut(&str) -> Option<usize>` that builds a local `String` and calls `f` on it up to three times, returning the first `Some`. Run it with the `block_on` of this chapter and an async closure that counts its calls. *Hint:* `AsyncFnMut` lets the closure mutate captured state; the future returned by each call must be awaited (or blocked on) before the next call.
6. In `code/compile_fail/ch09_early_bound.rs`, rewrite `first_late` with a named lifetime (`fn first_late<'a, T>(v: &'a [T]) -> &'a T`) and check that `apply(first_late)` still compiles. Then add `where T: 'a` to it, and finally remove the bound from `first_early`. Explain all three results with the early-bound / late-bound distinction. *Hint:* the note in the diagnostic shows the function item's type with its lifetime filled in.

PART III

Going Further

Part III applies the model to the parts of Rust where lifetimes are hardest to see: generic associated types and lending iterators, async code, interior mutability and its guards, and `unsafe` code, where you take over the job of the borrow checker. It closes with the question every library author eventually faces: when should a type borrow, when should it own, and how many lifetime parameters does it need?

CHAPTER 10

Generic Associated Types and Lending Iterators

Chapter 2 ended with a promise. An iterator whose items borrow from data it holds a reference to, like `slice::Iter<'a, T>` handing out `&'a T`, fits the `Iterator` trait perfectly. An iterator whose items borrow from the *iterator itself* does not fit at all, and no amount of annotation will make it fit. This chapter keeps the promise: it shows exactly where `Iterator` falls short, and what Rust offers instead.

Items that borrow from the iterator are not exotic. A line reader that reuses one buffer for every line hands out string slices pointing into that buffer, and each one is overwritten by the next call. An iterator over overlapping mutable windows of a slice (`[0..2]`, `[1..3]`, ...) cannot let two windows exist at once, because they share an element. In both cases the item is only valid until the next call to `next`, and the type of the item has to say so. Saying so needs an associated type that takes a lifetime parameter: a *generic associated type*, or GAT.

GATs solve the problem, and they bring a few new rules with them: a `where` clause the compiler insists on, `for` loops that stop working, and one real limitation of the type system that shows up as soon as you write generic code over a GAT. We will meet each of them with a program, and read what the compiler says.

In this chapter you will learn:

- why `Iterator` cannot describe items that borrow from the iterator, and what the compiler says when you try;
- how a generic associated type, `type Item<'a> where Self: 'a`, describes them, and why the `where` clause is mandatory;
- how to implement a lending iterator (`WindowsMut` and a buffer-reusing line reader), consume it with `while let`, and why `for` loops and adapters do not apply;
- why a bound such as `for<'a> L::Item<'a>: Debug` turns into `L: 'static`, and two ways around this limitation;
- other uses of GATs, such as borrowing views, and when a lifetime parameter on the trait is the simpler design.

10.1 The problem: items that borrow from the iterator

What `Iterator::next` promises

Here is the heart of the `Iterator` trait, with the elided lifetime of `next` written out (this is an expansion, not compiled code):

```
pub trait Iterator {
    type Item;

    fn next<'n>(&'n mut self) -> Option<Self::Item>;
}
```

The thread of this book applies directly: the signature is the contract. `next` borrows the iterator for some region `'n` that the caller picks, a fresh one for every call. The result has type `Self::Item`, and `Self::Item` is fixed by the `impl`, once for the whole type, before any call happens. It cannot mention `'n`, because `'n` is not in scope where `Item` is defined. So the contract says: *the item you get does not depend on how long you borrowed the iterator.*

Callers rely on that contract all the time. They hold one item while asking for the next (`let a = it.next(); let b = it.next();`), and adapters such as `collect`, `zip`, `peekable` and `max_by` are built on it: `collect` keeps every item alive at the same time, in a vector, long after the borrows that produced them have ended.

When the items borrow from data the iterator merely *points to*, the contract is easy to keep. `slice::Iter<'a, T>` holds a `&'a [T]` and returns `&'a T`: the item borrows the slice, not the iterator, so `Item = &'a T` is a fine choice (Chapter 2 shows how such an `impl` is written). The trouble starts when the item would have to borrow from the iterator itself.

Overlapping mutable windows

The standard library has `slice::windows`, an iterator over overlapping shared windows, but no `windows_mut`. Its documentation explains why:

> Because the Iterator trait cannot represent the required lifetimes, there is no `windows_mut` analog to `windows`; `[0,1,2].windows_mut(2).collect()` would violate the rules of references (though a LendingIterator analog is possible).

Let us try anyway. `WindowsMut` holds the slice as a `&'s mut [T]` and hands out windows of `size` elements, moving the start by one each time (`code/compile_fail/ch10_iterator_windows.rs`):

```
pub struct WindowsMut<'s, T> {
    data: &'s mut [T],
    size: usize,
    start: usize,
}

impl<'s, T> Iterator for WindowsMut<'s, T> {
    type Item = &'s mut [T];

    fn next(&mut self) -> Option<Self::Item> {
        let start = self.start;
        let end = start.checked_add(self.size)?;
        let window = self.data.get_mut(start..end)?;
```

```
        self.start += 1;
        Some(window)
    }
}
```

The only lifetime available for the item is `'s`, the lifetime of the borrowed slice, so we use it. The compiler rejects the body of `next`:

```
error: lifetime may not live long enough
  --> compile_fail/ch10_iterator_windows.rs:20:9
   |
12 | impl<'s, T> Iterator for WindowsMut<'s, T> {
   |      -- lifetime `'s` defined here
...
15 |     fn next(&mut self) -> Option<Self::Item> {
   |             - let's call the lifetime of this reference `'1`
...
20 |         Some(window)
   |         ^^^^^^^^^^^^ method was supposed to return data with lifetime `'s` but it is
returning data with lifetime `'1`
```

Read it label by label:

- "lifetime `'s` defined here" is the lifetime in the `impl` header, which `Item` promised.
- "let's call the lifetime of this reference `'1`" is the elided lifetime of `&mut self`, the `'n` of our expansion. It has no name in the source, so rustc makes one up.
- "method was supposed to return data with lifetime `'s` but it is returning data with lifetime `'1`": `window` was obtained *through* `&'1 mut self`, so it can borrow the slice for `'1` at most.

The last point is the reborrowing rule of Chapter 5. From a `&'1 mut` reference to a `&'s mut [T]` you can reborrow the slice mutably for `'1`, not for `'s`. If you could get the full `'s` out, two calls to `next` would give you two `&'s mut` references whose regions overlap, and the compiler would have no way to stop you from keeping both.

That is not a technicality. With windows of two elements over `[0, 1, 2]`, the first window is `[0, 1]` and the second is `[1, 2]`: both contain the element at index 1. A `collect` into a `Vec<&mut [i32]>` would hold two mutable references to the same element at the same time, which is exactly what aliasing XOR mutation (Chapter 1) forbids. The compiler is right to reject the `impl`; the only question is how to write a *correct* type for `next`.

There is no name for the borrow

The item should have the lifetime of the `&mut self` borrow, so the next idea is to write that lifetime with `'_` (`code/compile_fail/ch10_item_anonymous.rs`):

```
impl<'s, T> Iterator for WindowsMut<'s, T> {
    type Item = &'_ mut [T];
```

```
error[E0637]: `'_` cannot be used here
  --> compile_fail/ch10_item_anonymous.rs:13:18
   |
13 |     type Item = &'_ mut [T];
   |                  ^^ `'_` is a reserved lifetime name
```

E0637 says that `'_` has nothing to stand for here. Chapter 6 explains that `'_` is never a name for an existing lifetime: it asks for a fresh one, or for the one elision picks, and in an associated type there is neither. The lifetime we want belongs to each *call* of `next`, and `Item` is defined outside any call. What we need is an `Item` that takes the lifetime as a parameter.

10.2 Generic associated types

An associated type with a lifetime parameter

A generic associated type is an associated type with generic parameters of its own. Here is the lending iterator trait, as it appears in `code/examples/ch10_windows_mut.rs`:

```
pub trait LendingIterator {
    /// The item type, for each lifetime `'a` of a borrow of the iterator.
    type Item<'a>
    where
        Self: 'a;

    /// Each call borrows `self` and returns an item tied to that borrow.
    fn next(&mut self) -> Option<Self::Item<'_>>;
}
```

`type Item<'a>` is not one type but a family of types, one for each lifetime `'a`: for `WindowsMut` we will define `Item<'a>` as `&'a mut [T]`. In `next`, elision (Chapter 3) gives the `'_` of `Self::Item<'_>` the lifetime of `&mut self`, the only input lifetime. Written out:

```
fn next<'n>(&'n mut self) -> Option<Self::Item<'n>>;
```

Compare it with `Iterator::next`. The lifetime `'n` is still picked by the caller, fresh at every call, but now it appears in the result. The contract has changed: *the item you get is valid for as long as you borrow the iterator, and no longer.* While you hold the item, the iterator stays mutably borrowed, so you cannot call `next` again. That is precisely the guarantee overlapping windows need.

The name "lending iterator" describes this contract: the iterator *lends* you each item, and you give it back before asking for the next. (You may also meet the older name "streaming iterator" for the same idea.)

> **NOTE**
>
> Generic associated types are stable since Rust 1.65 (November 2022). The announcement, "Generic associated types to be stable in Rust 1.65" (https://blog.rust-lang.org/2022/10/28/gats-stabilization/), uses the lending iterator as its main example. GATs can also take type parameters (`type Pointer<T>`); this book needs only lifetime parameters. The standard library has no lending iterator trait as of Rust 1.99, so programs that need one define it, as we do here.

Why the compiler insists on `where Self: 'a`

The `where Self: 'a` clause on `Item` looks like noise. The program `code/compile_fail/ch10_missing_bound.rs` leaves it out:

```
pub trait LendingIterator {
    type Item<'a>;

    fn next(&mut self) -> Option<Self::Item<'_>>;
}
```

and the compiler refuses the trait itself, before any `impl` exists:

```
error: missing required bound on `Item`
 --> compile_fail/ch10_missing_bound.rs:6:5
  |
6 |     type Item<'a>;
  |     ^^^^^^^^^^^^^_
  |                  |
  |                  help: add the required where clause: `where Self: 'a`
  |
  = note: this bound is currently required to ensure that impls have maximum flexibility
  = note: we are soliciting feedback, see issue #87479 <https://github.com/rust-lang/rust/
issues/87479> for more information
```

This error has no code. Its parts:

- "missing required bound on `Item`": the declaration of the GAT is incomplete.
- "help: add the required where clause: `where Self: 'a`": the exact fix.
- "this bound is currently required to ensure that impls have maximum flexibility": the reason, which needs unpacking.
- The last note links to issue #87479, where the Rust project discusses whether the bound should one day be implied instead of written.

Start from what `next` already knows. Its parameter has type `&'n mut Self`, and a reference type `&'n mut Self` is only well formed when `Self: 'n` (the implied bounds of Chapter 1: the referent outlives the reference). So *every* use of `Self::Item<'n>` inside `next` comes with the fact `Self: 'n` for free. The compiler's rule, in the words of the stabilization announcement: "for methods that use the GAT, any bounds that can be proven must also be present on the GAT itself." Writing the bound on the GAT turns a fact that `next` happens to know into a promise of the trait, that `Item<'a>` is only ever used for lifetimes `'a` that `Self` outlives.

That promise is what gives implementations their flexibility. For `Self = WindowsMut<'s, T>`, the bound `WindowsMut<'s, T>: 'a` unpacks into `'s: 'a` and `T: 'a`: the window cannot outlive the slice it is cut from, and the elements outlive the window. With those facts, `&'a mut [T]` is a well-formed type. Without them, the implementation would have to define `Item<'a>` for every lifetime, including lifetimes longer than `'s`, for which `&'a mut [T]` makes no sense.

The rule looks at the methods that use the GAT. If the lifetime of the GAT comes from some other input, no bound is implied and none is required. This trait compiles as it is, without any `where` clause (`code/examples/ch10_parse_gat.rs`):

```
pub trait Parse {
    /// The parsed form of an input that lives for `'a`.
    type Output<'a>;

    fn parse<'a>(input: &'a str) -> Self::Output<'a>;
}
```

```rust
/// Borrows from the input: the words themselves.
pub struct Words;

impl Parse for Words {
    type Output<'a> = Vec<&'a str>;

    fn parse<'a>(input: &'a str) -> Vec<&'a str> {
        input.split_whitespace().collect()
    }
}

/// Borrows nothing: just a count.
pub struct WordCount;

impl Parse for WordCount {
    type Output<'a> = usize;

    fn parse(input: &str) -> usize {
        input.split_whitespace().count()
    }
}
```

```
["listen", "port", "8080"]
3
```

`Words` borrows from the input and `WordCount` borrows nothing, but neither involves `Self`, so `Self: 'a` is not implied anywhere. (Notice also that the `impl` for `WordCount` may write `parse` with elided lifetimes, because `Output<'a>` does not mention `'a`.)

Implementing the lending iterator

Here is `WindowsMut` again, now implementing `LendingIterator`:

```rust
/// Overlapping mutable windows of `size` elements: `[0..2]`, `[1..3]`, ...
pub struct WindowsMut<'s, T> {
    data: &'s mut [T],
    size: usize,
    start: usize,
}

impl<'s, T> WindowsMut<'s, T> {
    pub fn new(data: &'s mut [T], size: usize) -> Self {
        assert!(size > 0, "a window needs at least one element");
        WindowsMut {
            data,
            size,
            start: 0,
        }
    }
}

impl<'s, T> LendingIterator for WindowsMut<'s, T> {
    type Item<'a>
        = &'a mut [T]
    where
        Self: 'a;

    fn next(&mut self) -> Option<&mut [T]> {
        let start = self.start;
        let end = start.checked_add(self.size)?;
        // Reborrow `*self.data` for as long as `&mut self` lasts, no longer.
        let window = self.data.get_mut(start..end)?;
        self.start += 1;
        Some(window)
    }
}
```

Three details deserve a comment.

- The `impl` repeats `where Self: 'a`. It is not decoration: the definition `Item<'a> = &'a mut [T]` is checked with the `impl`'s own `where` clauses, and without the clause nothing says that `T` outlives `'a`.
- `next` writes its return type as `Option<&mut [T]>`, not `Option<Self::Item<'_>>`. An `impl` may spell out the concrete type; the elided lifetime is the one of `&mut self`, as before.
- The body is the same as in the rejected `Iterator` version. Only the contract changed, and the body that broke the old contract keeps the new one.

If you forget the `where` clause in the `impl`, the compiler reports two errors (`code/compile_fail/ch10_impl_without_where.rs`):

```
impl<'s, T> LendingIterator for WindowsMut<'s, T> {
    type Item<'a> = &'a mut [T]; // no `where Self: 'a`

    fn next(&mut self) -> Option<&mut [T]> {
        let start = self.start;
        let end = start.checked_add(self.size)?;
        let window = self.data.get_mut(start..end)?;
        self.start += 1;
        Some(window)
    }
}
```

```
error[E0477]: the type `WindowsMut<'s, T>` does not fulfill the required lifetime
  --> compile_fail/ch10_impl_without_where.rs:21:21
   |
 6 |     type Item<'a>
   |     ------------- definition of `Item` from trait
...
21 |     type Item<'a> = &'a mut [T]; // no `where Self: 'a`
   |                     ^^^^^^^^^^^
   |
note: type must outlive the lifetime `'a` as defined here
  --> compile_fail/ch10_impl_without_where.rs:21:15
   |
21 |     type Item<'a> = &'a mut [T]; // no `where Self: 'a`
   |               ^^
help: copy the `where` clause predicates from the trait
   |
21 |     type Item<'a> = &'a mut [T] where Self: 'a; // no `where Self: 'a`
   |                                 +++++++++++++++

error[E0309]: the parameter type `T` may not live long enough
  --> compile_fail/ch10_impl_without_where.rs:21:5
   |
21 |     type Item<'a> = &'a mut [T]; // no `where Self: 'a`
   |     ^^^^^^^^^^--^^
   |     |         |
   |     |         the parameter type `T` must be valid for the lifetime `'a` as defined
here...
   |     ...so that the reference type `&'a mut [T]` does not outlive the data it points at
   |
help: consider adding an explicit lifetime bound
   |
21 |     type Item<'a> = &'a mut [T] where T: 'a; // no `where Self: 'a`
   |                                 ++++++++++++
```

E0477, "the type `WindowsMut<'s, T>` does not fulfill the required lifetime", says that the `impl` cannot show `Self: 'a`, which the trait's definition of `Item` requires, and its help suggests copying the `where` clause from the trait. E0309, "the parameter type `T` may not live long enough", is the same problem

seen from inside the type: `&'a mut [T]` needs `T: 'a`, and without the clause nothing provides it. Both disappear with `where Self: 'a`. For a type without lifetime or type parameters, such as a struct that owns a `Vec<String>`, `Self: 'a` holds for every `'a` and the `impl` may leave the clause out; you will see one in *Borrowing views* below.

Consuming a lending iterator with `while let`

Here is `WindowsMut` at work. A counter that should never decrease has some dips in its readings, and we repair them by carrying the previous value forward, one window of two readings at a time:

```
// Hourly readings of a counter that should never go down: repair the
// dips by carrying the previous value forward.
let mut readings = vec![3, 5, 4, 8, 7, 9];
let mut windows = WindowsMut::new(&mut readings, 2);
while let Some(pair) = windows.next() {
    if pair[1] < pair[0] {
        pair[1] = pair[0];
    }
}
println!("{readings:?}");
```

```
[3, 5, 5, 8, 8, 9]
```

Each repair is visible to the next window, because the windows overlap: after `[5, 4]` became `[5, 5]`, the next window starts with that second 5. This is the program the standard library's documentation says you cannot write with `Iterator`.

Where are the regions? Each call to `windows.next()` borrows `windows` mutably for a region we can call `'a1`, `'a2`, ..., and the item `pair` has type `&'a1 mut [i32]`. The last use of `pair` is at the end of the loop body, so by non-lexical lifetimes (Chapter 1) the borrow of `windows` ends there, and the next iteration may borrow it again. All of these regions sit inside `'s`, the region in which `windows` borrows `readings`, and `'s` ends after the loop, so `println!` can read `readings` again.

program point	's	item 1	item 2
let mut windows = WindowsMut::new(&mut readings, 2)			
pair = windows.next() (1st call)		'a1	
pair[1] = pair[0]		'a1	
pair = windows.next() (2nd call)			'a2
pair[1] = pair[0]			'a2
println!("{readings:?}")			

The borrow 's of readings spans the loop; each item borrows the iterator for its own region, and two item regions never overlap.

The compiler enforces the "never overlap" part. Holding one item while asking for the next (`code/compile_fail/ch10_two_items.rs`):

```
pub fn first_two<L: LendingIterator>(mut lender: L) {
    let first = lender.next();
    let second = lender.next();
    drop(first);
    drop(second);
}
```

```
error[E0499]: cannot borrow `lender` as mutable more than once at a time
  --> compile_fail/ch10_two_items.rs:16:18
   |
15 |     let first = lender.next();
   |                 ------ first mutable borrow occurs here
16 |     let second = lender.next();
   |                  ^^^^^^ second mutable borrow occurs here
17 |     drop(first);
   |          ----- first borrow later used here
```

This is the ordinary E0499 of Chapter 1, and it reads naturally once you know the signature of `next`: "first mutable borrow occurs here" is the first call, whose result keeps `lender` borrowed; "second mutable borrow occurs here" is the second call; "first borrow later used here" is the use of `first` that keeps the first borrow alive. Notice that this function is generic: the compiler does not know which lending iterator `L` is, and it does not need to. The signature of `next` is enough to reject the code, for every possible `L`.

Items that borrow the iterator's own buffer

Overlapping windows are one reason to lend. The other common reason is reuse: an iterator that keeps its own buffer and hands out views into it. Here is a reader that normalizes the lines of a text (lowercase, runs of whitespace collapsed to one space) into a single `String` that it reuses for every line (`code/examples/ch10_normalized_lines.rs`):

```
/// The lines of a text, lowercased and with runs of spaces collapsed.
pub struct NormalizedLines<'src> {
    lines: std::str::Lines<'src>,
    buffer: String,
}

impl<'src> NormalizedLines<'src> {
    pub fn new(text: &'src str) -> Self {
        NormalizedLines {
            lines: text.lines(),
            buffer: String::new(),
        }
    }
}

impl<'src> LendingIterator for NormalizedLines<'src> {
    type Item<'a>
        = &'a str
    where
        Self: 'a;

    fn next(&mut self) -> Option<&str> {
        let line = self.lines.next()?;
        self.buffer.clear(); // overwrite the previous item
        for word in line.split_whitespace() {
            if !self.buffer.is_empty() {
```

```
                self.buffer.push(' ');
            }
            self.buffer
                .extend(word.chars().flat_map(char::to_lowercase));
        }
        Some(&self.buffer)
    }
}
```

The item borrows `self.buffer`, a field the iterator owns. With `Iterator`, the item type would have to be `String`, and every line would allocate a new string; here the loop allocates once, when the buffer first grows. The price is that an item cannot outlive the iteration that produced it, so a caller that wants to keep something must copy it:

```
let config = String::from("Listen   PORT 8080\n  Log-Level  Debug\nTIMEOUT 30");
let mut lines = NormalizedLines::new(&config);
let mut keys = Vec::new();
while let Some(line) = lines.next() {
    println!("{line}");
    // `line` dies at the end of this iteration; keep an owned copy.
    keys.push(line.split(' ').next().unwrap_or("").to_owned());
}
println!("keys: {keys:?}");
```

```
listen port 8080
log-level debug
timeout 30
keys: ["listen", "log-level", "timeout"]
```

Keeping the item itself instead of a copy is rejected, for the reason we just saw (`code/compile_fail/ch10_keep_line.rs`, with a simpler `TrimmedLines` lender):

```
pub fn keep_all(mut lines: TrimmedLines<'_>) -> usize {
    let mut keys = Vec::new();
    while let Some(line) = lines.next() {
        keys.push(line); // keeps a borrow of `lines.buffer`
    }
    keys.len()
}
```

```
error[E0499]: cannot borrow `lines` as mutable more than once at a time
  --> compile_fail/ch10_keep_line.rs:35:28
   |
35 |     while let Some(line) = lines.next() {
   |                            ^^^^^ `lines` was mutably borrowed here in the previous
iteration of the loop
36 |         keys.push(line); // keeps a borrow of `lines.buffer`
   |         ---- first borrow used here, in later iteration of loop
```

This E0499 has loop-specific labels. "`lines` was mutably borrowed here in the previous iteration of the loop": the call to `next` in iteration one lent an item that the vector still holds. "first borrow used here, in later iteration of loop": `keys.push` uses the vector, and with it that first item, in iteration two, while `next` wants to borrow `lines` again.

The same idea appears in the standard library in a different shape: `BufRead::read_line` writes into a `String` that *you* pass in, so the buffer belongs to the caller and no lending is needed. Lending moves the buffer into the iterator.

Why `for` loops and adapters do not apply

A `for` loop is sugar for a call to `IntoIterator::into_iter` followed by calls to `Iterator::next`. A lending iterator implements neither trait (`code/compile_fail/ch10_for_loop.rs`):

```
pub fn count_items<L: LendingIterator>(lender: L) -> usize {
    let mut count = 0;
    for _item in lender {
        count += 1;
    }
    count
}
```

```
error[E0277]: `L` is not an iterator
  --> compile_fail/ch10_for_loop.rs:16:18
   |
16 |     for _item in lender {
   |                  ^^^^^^ `L` is not an iterator
   |
   = note: required for `L` to implement `IntoIterator`
help: consider further restricting type parameter `L` with trait `Iterator`
   |
14 | pub fn count_items<L: LendingIterator + std::iter::Iterator>(lender: L) -> usize {
   |                                       ++++++++++++++++++++
```

E0277 says "`L` is not an iterator", with the note "required for `L` to implement `IntoIterator`". The help, to add an `Iterator` bound, is the wrong fix here: it would compile, and it would only accept types that are *also* ordinary iterators, which `WindowsMut` cannot be. The honest way to loop is `while let Some(item) = lender.next()`.

The adapters of `Iterator` are not available either, and some of them cannot exist at all. `collect` keeps every item at once, which is exactly what a lending iterator forbids. Others could be written as methods of `LendingIterator`, but their closures must accept an item of *any* lifetime, because each call to `next` lends for a different region. A `for_each` would need `F: FnMut(Self::Item<'_>)`, which is `F: for<'a> FnMut(Self::Item<'a>)`, a higher-ranked bound (Chapter 9). As the next section shows, a higher-ranked bound over a GAT runs into a limitation of today's compiler. And a `map` whose closure returns something borrowed from the item would need its output type to depend on `'a` too, which a plain type parameter `B` cannot do.

10.3 Generic code over a lending iterator

A bound on every item

Suppose we want a function that prints every item of any lending iterator. The items must be `Debug`, for every lifetime the iterator might lend them for, so the natural bound is higher-ranked (`code/compile_fail/ch10_hrtb_static.rs`):

```
pub fn print_all<L>(mut lender: L)
where
    L: LendingIterator,
    for<'a> L::Item<'a>: Debug,
{
    while let Some(item) = lender.next() {
        println!("{item:?}");
```

```
    }
}
```

The function compiles. Calling it with a `NormalizedLines` that reads a `String` does not:

```
pub fn caller() {
    let config = String::from("port 8080\nlog-level debug");
    let lines = NormalizedLines {
        lines: config.lines(),
        buffer: String::new(),
    };
    print_all(lines);
}
```

```
error[E0597]: `config` does not live long enough
  --> compile_fail/ch10_hrtb_static.rs:50:16
   |
48 |     let config = String::from("port 8080\nlog-level debug");
   |         ------ binding `config` declared here
49 |     let lines = NormalizedLines {
50 |         lines: config.lines(),
   |                ^^^^^^ borrowed value does not live long enough
...
53 |     print_all(lines);
   |     ---------------- argument requires that `config` is borrowed for `'static`
54 | }
   | - `config` dropped here while still borrowed
   |
note: due to a current limitation of the type system, this implies a `'static` lifetime
  --> compile_fail/ch10_hrtb_static.rs:38:5
   |
38 |     for<'a> L::Item<'a>: Debug,
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^
note: requirement that the value outlives `'static` introduced here
  --> compile_fail/ch10_hrtb_static.rs:38:26
   |
38 |     for<'a> L::Item<'a>: Debug,
   |                          ^^^^^
```

The first labels are familiar from Chapter 1: `config` is borrowed by `lines`, and dropped at the end of `caller` while still borrowed. The surprise is "argument requires that `config` is borrowed for `'static`": nothing in `print_all` mentions `'static`. The first note explains where it comes from, and it is unusually candid: "due to a current limitation of the type system, this implies a `'static` lifetime", pointing at `for<'a> L::Item<'a>: Debug`.

Here is the reasoning behind it. `for<'a>` means *every* lifetime, including `'static` and lifetimes much longer than the borrow of `config`. But `L::Item<'a>` is only defined when `L: 'a`, the `where` clause of the trait. For the bound to be meaningful for every `'a`, the compiler requires `L: 'a` for every `'a`, and a type that outlives every lifetime is a `'static` type. What we meant was narrower: "for every `'a` such that `L: 'a`, the item is `Debug`". That is a bound *on the binder*, `for<'a where L: 'a>`, and Chapter 9 (*The limits of* `for<..>`) shows that Rust cannot express bounds on binders today. The stabilization announcement of GATs lists this as one of the most limiting known issues, and notes that fixing it needs compiler work that is not a short-term project.

The error is conservative, not a sign of a bug in your code. When the lender really is `'static`, the same `print_all` works. A `NormalizedLines` that reads a string literal holds a `&'static str`, so it is a `'static` type (`code/examples/ch10_print_all_static.rs`):

```
// `NormalizedLines<'static>`: the text is a literal.
let lines = NormalizedLines {
    lines: "port 8080\n  log-level debug".lines(),
    buffer: String::new(),
};
print_all(lines);
```

```
"port 8080"
"log-level debug"
```

The `for_each` adapter from the previous section hits the same wall. As a default method of the trait, it compiles; calling it on a `WindowsMut` over a borrowed vector does not (`code/compile_fail/ch10_for_each_static.rs`):

```
fn for_each<F>(mut self, mut f: F)
where
    Self: Sized,
    F: FnMut(Self::Item<'_>),
{
    while let Some(item) = self.next() {
        f(item);
    }
}
```

```
pub fn reverse_triples(values: &mut Vec<i32>) {
    let windows = WindowsMut {
        data: values,
        size: 3,
        start: 0,
    };
    windows.for_each(|triple| triple.reverse());
}
```

```
error[E0521]: borrowed data escapes outside of function
  --> compile_fail/ch10_for_each_static.rs:53:5
   |
47 | pub fn reverse_triples(values: &mut Vec<i32>) {
   |                        ------  - let's call the lifetime of this reference `'1`
   |                        |
   |                        `values` is a reference that is only valid in the function body
...
53 |     windows.for_each(|triple| triple.reverse());
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |     |
   |     `values` escapes the function body here
   |     argument requires that `'1` must outlive `'static`
   |
note: due to a current limitation of the type system, this implies a `'static` lifetime
  --> compile_fail/ch10_for_each_static.rs:16:12
   |
16 |         F: FnMut(Self::Item<'_>),
   |            ^^^^^^^^^^^^^^^^^^^^
```

This time the borrow comes in through a parameter, so the error is E0521, "borrowed data escapes outside of function" (Chapter 7 reads it in detail), but the note is the same: the `'static` requirement comes from the higher-ranked bound `FnMut(Self::Item<'_>)`. Every generic function whose bound quantifies over the lifetime of a GAT has this problem. There are two ways around it.

Workaround 1: put the bound in the trait

If every item of every lending iterator you care about should be `Debug`, say so in the trait, on the GAT itself (`code/examples/ch10_bound_in_trait.rs`):

```
pub trait LendingIterator {
    type Item<'a>: Debug
    where
        Self: 'a;

    fn next(&mut self) -> Option<Self::Item<'_>>;
}
```

Generic code then needs no bound at all, because the trait guarantees it:

```
pub fn print_all<L: LendingIterator>(mut lender: L) {
    while let Some(item) = lender.next() {
        println!("{item:?}");
    }
}
```

```
let mut readings = vec![3, 5, 4, 8];
print_all(WindowsMut {
    data: &mut readings,
    size: 2,
    start: 0,
});
readings.push(10); // the borrow ended when `print_all` returned
println!("{readings:?}");
```

```
[3, 5]
[5, 4]
[4, 8]
[3, 5, 4, 8, 10]
```

The bound `Item<'a>: Debug` is checked once per `impl`, for the lifetimes the `impl` actually defines (those with `Self: 'a`), so no higher-ranked bound and no `'static` requirement appears. The `impl` pays for it: `WindowsMut` can only implement the trait when its items are `Debug`.

```
impl<'s, T: Debug> LendingIterator for WindowsMut<'s, T> {
    type Item<'a>
        = &'a mut [T]
    where
        Self: 'a;
```

This workaround is the right one when the bound is part of what the trait *means*. The standard library uses it: the trait behind async closures, `AsyncFnMut`, declares `type CallRefFuture<'a>: Future<Output = Self::Output> where Self: 'a;`, a GAT whose every instance must be a future (Chapter 11). It is the wrong one when different callers need different bounds: you cannot add a `Display` requirement for one function without imposing it on every implementation.

Workaround 2: a helper trait

The second workaround avoids the GAT. It comes from Sabrina Jewson's article "The Better Alternative to Lifetime GATs" (https://sabrinajewson.org/blog/the-better-alternative-to-lifetime-gats), and it moves the lifetime from the associated type to a *trait* (`code/examples/ch10_helper_trait.rs`):

```rust
mod sealed {
    pub trait Sealed: Sized {}
    pub struct Bounds<T>(T);
    impl<T> Sealed for Bounds<T> {}
}
use sealed::{Bounds, Sealed};

/// The item type of a lending iterator, for one lifetime `'this`. The second
/// parameter is never written by users: its default, `Bounds<&'this Self>`,
/// only exists to imply `Self: 'this`.
pub trait LendingIteratorLifetime<'this, ImplicitBounds: Sealed = Bounds<&'this Self>> {
    type Item;
}

pub trait LendingIterator: for<'this> LendingIteratorLifetime<'this> {
    fn next(&mut self) -> Option<<Self as LendingIteratorLifetime<'_>>::Item>;
}
```

Read it in two steps.

1. `LendingIteratorLifetime<'this>` is an ordinary trait with an ordinary associated type: for one lifetime `'this`, it says what the item is. `LendingIterator` requires it for every lifetime, with the higher-ranked supertrait `for<'this> LendingIteratorLifetime<'this>`, and `next` returns the item for the lifetime of its `&mut self`.
2. The second parameter, `ImplicitBounds`, is never written by anyone. Its default, `Bounds<&'this Self>`, mentions the type `&'this Self`, and a type that appears in a trait reference brings its implied bounds along. Wherever `LendingIteratorLifetime<'this>` is used, including inside the `for<'this>`, `&'this Self` must be well formed, so `Self: 'this` holds.

The second step is what defeats the limitation. In effect, the implied bound restricts the `for<'this>` to the lifetimes that `Self` outlives, which is exactly the "bound on the binder" that we could not write. More concretely: an implementation never has to state `Self: 'this` as a `where` clause, so there is no clause that the compiler would have to prove "for every lifetime". The private module makes `Sealed` and `Bounds` impossible to name from outside, so no one can override the default.

The implementations look like this:

```rust
impl<'this, 's, T> LendingIteratorLifetime<'this> for WindowsMut<'s, T> {
    type Item = &'this mut [T];
}

impl<'s, T> LendingIterator for WindowsMut<'s, T> {
    fn next(&mut self) -> Option<&mut [T]> {
        let start = self.start;
        let end = start.checked_add(self.size)?;
        let window = self.data.get_mut(start..end)?;
        self.start += 1;
        Some(window)
    }
}
```

There is no `where` clause to repeat. The `impl` of `LendingIteratorLifetime` gets `WindowsMut<'s, T>: 'this`, hence `T: 'this`, as an implied bound of its header, so `&'this mut [T]` is well formed for every lifetime it covers. Generic code names the item as `<L as LendingIteratorLifetime<'a>>::Item`, and both the printing function and a `for_each` now work with borrowed data:

```
pub fn print_all<L>(mut lender: L)
where
    L: LendingIterator,
    for<'a> <L as LendingIteratorLifetime<'a>>::Item: Debug,
{
    while let Some(item) = lender.next() {
        println!("{item:?}");
    }
}

pub fn for_each<L, F>(mut lender: L, mut f: F)
where
    L: LendingIterator,
    F: for<'a> FnMut(<L as LendingIteratorLifetime<'a>>::Item),
{
    while let Some(item) = lender.next() {
        f(item);
    }
}
```

```
let mut readings = vec![3, 5, 4, 8];
print_all(WindowsMut {
    data: &mut readings,
    size: 2,
    start: 0,
});
for_each(
    WindowsMut {
        data: &mut readings,
        size: 3,
        start: 0,
    },
    |triple| triple.reverse(),
);
println!("{readings:?}");
```

```
[3, 5]
[5, 4]
[4, 8]
[4, 8, 3, 5]
```

The defaulted parameter is not an ornament. Remove it, and the `impl` has no implied bound to stand on (`code/compile_fail/ch10_helper_without_bounds.rs`):

```
pub trait LendingIteratorLifetime<'this> {
    type Item;
}

pub struct WindowsMut<'s, T> {
    data: &'s mut [T],
    size: usize,
    start: usize,
}

impl<'this, 's, T> LendingIteratorLifetime<'this> for WindowsMut<'s, T> {
    type Item = &'this mut [T];
}
```

```
error[E0309]: the parameter type `T` may not live long enough
  --> compile_fail/ch10_helper_without_bounds.rs:17:5
   |
16 | impl<'this, 's, T> LendingIteratorLifetime<'this> for WindowsMut<'s, T> {
   |      ----- the parameter type `T` must be valid for the lifetime `'this` as defined
here...
17 |     type Item = &'this mut [T];
```

```
   |     ^^^^^^^^^ ...so that the reference type `&'this mut [T]` does not outlive the data
it points at
   |
help: consider adding an explicit lifetime bound
   |
16 | impl<'this, 's, T: 'this> LendingIteratorLifetime<'this> for WindowsMut<'s, T> {
   |                  +++++++
```

E0309 says that `&'this mut [T]` might outlive the data it points at, because nothing relates `T` to `'this`. The suggested fix, `T: 'this`, is an explicit `where` clause on the `impl`, and an explicit clause is exactly what makes a `for<'this>` bound demand `'static` again, as the GAT's clause did. The implied bound is the whole trick.

Here are the three designs side by side:

Design	Borrowing lenders in generic code	Bounds chosen by	Cost
GAT + `for<'a>` bound	no: forces `'static`	each function	none, until it fails
Bound on the GAT	yes	the trait, for everyone	every `impl` must satisfy it
Helper trait	yes	each function	more syntax, longer error messages

10.4 Other uses of GATs

Borrowing views

Lending iterators are the famous example, but the general pattern is broader: a method that returns *something that may borrow from* `self`, where different implementations borrow differently, or not at all. Here is a source of configuration values (`code/examples/ch10_views.rs`):

```rust
pub trait ConfigSource {
    /// What `get` returns when it borrows `self` for `'a`.
    type Value<'a>: AsRef<str>
    where
        Self: 'a;

    fn get(&self, key: &str) -> Option<Self::Value<'_>>;
}
```

A source that parsed a file owns the strings, and can lend a `&str` pointing into them:

```rust
/// Values parsed from a file: the source owns them and lends `&str`s.
pub struct FileSource {
    entries: Vec<(String, String)>,
}

impl ConfigSource for FileSource {
    type Value<'a> = &'a str;

    fn get(&self, key: &str) -> Option<&str> {
        let (_, value) = self.entries.iter().find(|(k, _)| k == key)?;
        Some(value)
    }
}
```

A source of computed defaults has nothing to lend, and returns owned strings; its `Value<'a>` simply ignores `'a`:

```
/// Values computed on demand: there is nothing to borrow, so they are owned.
pub struct Defaults {
    base_port: u16,
}

impl ConfigSource for Defaults {
    type Value<'a> = String;

    fn get(&self, key: &str) -> Option<String> {
        match key {
            "port" => Some(self.base_port.to_string()),
            "admin-port" => Some((self.base_port + 1).to_string()),
            _ => None,
        }
    }
}
```

Neither `impl` repeats `where Self: 'a : FileSource` and `Defaults` have no lifetime or type parameters, so they outlive every `'a` and the bound holds trivially. Generic code accepts both kinds of source:

```
/// Looks a key up in the first source, then in the second.
pub fn lookup<A, B>(first: &A, second: &B, key: &str) -> Option<String>
where
    A: ConfigSource,
    B: ConfigSource,
{
    if let Some(value) = first.get(key) {
        return Some(value.as_ref().to_owned());
    }
    second.get(key).map(|value| value.as_ref().to_owned())
}
```

```
let file = FileSource {
    entries: vec![
        ("port".into(), "9000".into()),
        ("host".into(), "example.org".into()),
    ],
};
let defaults = Defaults { base_port: 8080 };
for key in ["port", "admin-port", "host", "user"] {
    println!("{key:>10}: {:?}", lookup(&file, &defaults, key));
}
```

```
      port: Some("9000")
admin-port: Some("8081")
      host: Some("example.org")
      user: None
```

Compare the alternatives without a GAT. A trait method returning `&str` cannot be implemented by `Defaults`, which has no string to point to. A method returning `String` forces `FileSource` to allocate on every lookup. A method returning `Cow<'_, str>` (Chapter 14) works, at the cost of a runtime check of which variant you got; the GAT moves that choice to compile time, one choice per implementation. Notice also that `lookup` needed no higher-ranked bound: the `AsRef<str>` requirement sits on the GAT itself, which is workaround 1 used by design.

When a lifetime parameter on the trait is simpler

A GAT ties an item to the borrow of `self`. That is exactly right when the item really borrows from `self`, and needlessly strict when it does not. Before reaching for a GAT, ask: *does the item borrow from the iterator, or from something the iterator borrows?*

A tokenizer over a `&'src str` returns tokens that point into the text, not into the tokenizer. Its trait can carry the lifetime of the text as a parameter (`code/examples/ch10_tokens.rs`):

```
/// A source of tokens that borrow from a text of lifetime `'src`.
pub trait Tokens<'src> {
    fn next_token(&mut self) -> Option<&'src str>;
}
```

Generic code can then keep every token, because a token does not keep the tokenizer borrowed:

```
/// Keeps every token: possible because a token does not borrow `tokens`.
pub fn collect_all<'src, T: Tokens<'src>>(mut tokens: T) -> Vec<&'src str> {
    let mut all = Vec::new();
    while let Some(token) = tokens.next_token() {
        all.push(token);
    }
    all
}
```

```
["listen", "port", "8080"]
```

With `LendingIterator` instead, `collect_all` would be the E0499 program of *Consuming a lending iterator with* `while let`: each token would hold the tokenizer borrowed until the next call. (For this particular case, `Iterator<Item = &'src str>` works just as well; Chapter 2 builds such an iterator. A trait like `Tokens<'src>` is useful when the trait has other methods, or is not an iterator at all.) The lifetime parameter has a cost of its own: generic code that creates the data itself must bound the trait for every lifetime, `for<'src> Tokens<'src>`, as with `for<'de> Deserialize<'de>` in Chapter 9.

A rule of thumb for the three designs:

The item borrows from ...	Design
data the type holds a reference to, for `'s`	`Iterator<Item = &'s T>`, or a trait parameter `'s`
the type itself, differently on each call	a GAT, `type Item<'a> where Self: 'a`
an input of the method other than `self`	a GAT without `where`, or a trait parameter
nothing	a plain associated type

10.5 Common confusions

- "**`where Self: 'a` makes the item live as long as the iterator.**" It does the opposite: it restricts `'a` to lifetimes that `Self` outlives. The item lives at most as long as the borrow of the iterator, never longer.
- "**I can make it work with `for` by implementing `IntoIterator`.**" `IntoIterator` produces an `Iterator`, whose items cannot borrow from it. Use `while let`.

- "**The `'static` requirement means my code would be unsound.**" No. The note says "current limitation of the type system": the compiler cannot express the bound you meant, so it demands a stronger one. Use one of the two workarounds, or a `'static` lender.
- "**The `where` clause in the `impl` is optional, since the trait has it.**" For a type with lifetime or type parameters it is required (E0477, E0309); for a type without parameters it may be omitted.
- "**GATs replace lifetime parameters on traits.**" They answer a different question. If the items borrow from data with a lifetime the type already has, a plain associated type or a trait parameter is simpler and lets callers keep items.
- "**`windows_mut` is missing from the standard library by oversight.**" It is missing because `Iterator` cannot express it. For `Copy` elements, `Cell::from_mut` and `as_slice_of_cells` together with `windows` give an alternative (see the exercises).

10.6 Summary

- `Iterator::Item` is chosen once per type and cannot mention the borrow of `&mut self` made by each call to `next`, so an iterator cannot hand out items that borrow from itself. Trying gives "lifetime may not live long enough" (method "was supposed to return data with lifetime `'s`").
- A generic associated type, `type Item<'a> where Self: 'a`, lets the item depend on the borrow: `fn next(&mut self) -> Option<Self::Item<'_>>`. GATs are stable since Rust 1.65.
- The compiler requires `where Self: 'a` ("missing required bound on `Item`") because `&'a mut self` already implies it; the bound tells implementations that `'a` never outlives `Self`. Implementations of types with parameters repeat it.
- A lending iterator is consumed with `while let`; two items cannot be alive at once (E0499); `for` loops (E0277) and `collect` do not apply, and closures for adapters need higher-ranked bounds.
- A bound like `for<'a> L::Item<'a>: Debug`, or an adapter taking `FnMut(Self::Item<'_>)`, forces `L: 'static` "due to a current limitation of the type system". Workarounds: put the bound on the GAT, or use a helper trait with a defaulted `Bounds<&'this Self>` parameter whose implied bound restricts the `for<'this>`.
- GATs also describe borrowing views (`type Value<'a>`) that some implementations lend and others own. When items borrow from data rather than from `self`, a plain associated type or a lifetime parameter on the trait is simpler.

10.7 Exercises

1. **A numbered line reader.** Write a lending iterator `NumberedLines<'src>` whose items are string slices of the form `"3: text of the third line"`, built in a reused buffer. Use it with `while let` to print a short text, and check that the buffer's capacity stops growing after the longest line. *Hint:* `use std::fmt::Write;` lets you `write!(self.buffer, "{n}: {line}")` into a `String`; `String::capacity` reports the allocation.

2. **Overlapping windows without a lending iterator.** Repair the counter readings of `ch10_windows_mut.rs` without `WindowsMut`, using only `slice::windows`. *Hint:* `Cell::from_mut(&mut readings[..]).as_slice_of_cells()` turns a `&mut [i32]` into a `&[Cell<i32>]`, and shared windows over cells can still be written through `Cell::set`. Which property of `i32` does `Cell::get` rely on?
3. **A `count_if` that works with borrowed data.** Using the helper-trait design of `ch10_helper_trait.rs`, write `fn count_if<L, P>(lender: L, predicate: P) -> usize` that counts the items for which `predicate` returns `true`, and call it on a `WindowsMut` over a local vector. Then write the same function for the GAT design of `ch10_windows_mut.rs`, call it the same way, and read the error. *Hint:* the predicate needs a bound like `P: for<'a> FnMut(&<L as LendingIteratorLifetime<'a>>::Item) -> bool`.
4. **Make the error go away, wrongly.** Change only the *caller* in `ch10_hrtb_static.rs` so that it compiles, without touching `print_all` or the trait. Then explain why this is not a real fix for a program that reads its configuration from a file at run time. *Hint:* the lender must be `'static`; Chapter 7 lists the ways to get a `&'static str`, and one of them works for data created at run time.
5. **A GAT where it does not belong.** Rewrite `Tokens<'src>` from `ch10_tokens.rs` as a `LendingIterator` with `type Item<'a> = &'a str where Self: 'a`, keep `collect_all`, and read the error. Which design would you choose for a tokenizer, and why? *Hint:* the item type now mentions the borrow of the tokenizer, not `'src`.

CHAPTER 11

Lifetimes in Async Code

An `async fn` looks like a function, but calling it does not run its body. The call returns a *future*: a value that holds the arguments and the state of the body, and that runs the body, piece by piece, when something polls it. That one fact explains most lifetime errors in async code. A future is a value like any other, and a value that holds a borrow is constrained by that borrow for as long as the value exists. The borrow does not end when the call returns; it ends when the future is done.

Everything else follows. A future that borrows a local cannot be handed to an executor that may poll it later, which is why `spawn` asks for `'static`. Async callbacks need a way to say "the future may borrow my argument", which plain `Fn` bounds cannot say (Chapter 9) and `AsyncFn` can. And a future that keeps a reference to one of its own locals across an `.await` points into itself, which is why futures are polled through `Pin`.

This chapter works through these consequences with programs that use only the standard library: no async runtime, just a ten-line `block_on` and a toy executor whose `spawn` has the same signature as the real ones.

In this chapter you will learn:

- what the future returned by an `async fn` captures, and why it holds its borrows until it completes;
- how to run futures with the standard library alone (`block_on` with `Waker::noop`);
- why executors require `F: Future + Send + 'static`, what the errors look like when a spawned future borrows, and how to fix them;
- why scoped tasks are hard in async code while `thread::scope` works: futures can be forgotten;
- how `AsyncFn` and `AsyncFnMut` let async callbacks borrow their arguments and their captured state;
- why self-referential futures need `Pin`, just enough to read the messages;
- what `async fn` in traits captures, and how to require `Send` futures.

11.1 What an `async fn` returns

A future that captures every input lifetime

Here are two functions that look up a key in a configuration map, one plain and one async (`code/examples/ch11_async_captures.rs`):

```
fn lookup<'a>(config: &'a Config, key: &str) -> Option<&'a str> {
    config.get(key).map(String::as_str)
}

async fn lookup_async<'a>(config: &'a Config, key: &str) -> Option<&'a str> {
    println!("  (running lookup_async for {key:?})");
    config.get(key).map(String::as_str)
}
```

The signatures look alike, and the bodies are the same apart from a `println!` that will show us *when* the body runs. But `lookup_async` does not return an `Option<&'a str>`. It returns a future whose output is an `Option<&'a str>`, and the compiler reads its signature roughly as follows. The program contains this "desugared" version too, and it behaves the same:

```
/// What the compiler makes of `lookup_async`: a plain function that returns an
/// opaque future capturing both lifetimes.
fn lookup_desugared<'a, 'k>(
    config: &'a Config,
    key: &'k str,
) -> impl Future<Output = Option<&'a str>> + use<'a, 'k> {
    async move {
        println!("  (running lookup_desugared for {key:?})");
        config.get(key).map(String::as_str)
    }
}
```

The return type is an opaque `impl Future` (Chapter 8), and the important part is `use<'a, 'k>`: the future captures *both* input lifetimes. The `async move` block stores `config` and `key` inside the future, because the body has not run yet and will need them later. The output type mentions only `'a`, but the future itself holds a `&'k str`. An `async fn` always captures all of its input lifetimes; the explicit `use<'a, 'k>` here just says so.

Nothing runs at the call. Watch the order of the output:

```
let key = String::from("port");
let future = lookup_async(&config, &key);
println!("future created; nothing has run yet");
println!("result: {:?}", block_on(future));
println!("result: {:?}", block_on(lookup_desugared(&config, &key)));
```

```
future created; nothing has run yet
  (running lookup_async for "port")
result: Some("8080")
  (running lookup_desugared for "port")
result: Some("8080")
sync: Some("8080"), eager: Some("8080")
words: 3
```

The first line is printed before the body of `lookup_async` starts, because the body runs only when `block_on` polls the future. (The last two lines come from code shown later in this chapter.)

The borrow lasts until the future completes

A future that holds a borrow is a value that holds a borrow, so the ordinary rules of Chapter 1 apply to it, with the future's whole existence as the region (`code/compile_fail/ch11_future_holds_borrow.rs`):

```
async fn count_words(text: &str) -> usize {
    text.split_whitespace().count()
}

pub fn caller() -> usize {
    let mut text = String::from("listen port 8080");
    let future = count_words(&text);
    text.push_str(" log-level debug");
    block_on(future)
}
```

```
error[E0502]: cannot borrow `text` as mutable because it is also borrowed as immutable
  --> compile_fail/ch11_future_holds_borrow.rs:27:5
   |
26 |     let future = count_words(&text);
   |                              ----- immutable borrow occurs here
27 |     text.push_str(" log-level debug");
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
28 |     block_on(future)
   |               ------ immutable borrow later used here
```

The labels tell the story in order: "immutable borrow occurs here" is the call, which stores `&text` in the future; "mutable borrow occurs here" is the `push_str` ; "immutable borrow later used here" is `block_on(future)` , where the future, and the borrow inside it, is still alive. With a synchronous `count_words` , the borrow would have ended when the call returned, and the same three lines would compile.

The difference is sharper for the second argument of `lookup_async` . With the plain `lookup` , `key` may die right after the call: the result borrows only `config` . This compiles:

```
let value = {
    let key = String::from("port");
    lookup(&config, &key) // the borrow of `key` ends with the call
};
let future = {
    let key = String::from("port");
    lookup_eager(&config, &key) // the future does not hold `key`
};
println!("sync: {value:?}, eager: {:?}", block_on(future));
```

(`lookup_eager` is explained in a moment.) The same block with `lookup_async` does not (`code/compile_fail/ch11_future_outlives_key.rs`):

```
pub fn caller(config: &Config) -> Option<&str> {
    let future = {
        let key = String::from("port");
        lookup_async(config, &key)
    };
    block_on(future)
}
```

```
error[E0597]: `key` does not live long enough
  --> compile_fail/ch11_future_outlives_key.rs:30:30
   |
28 |     let future = {
```

```
   |         ------ borrow later stored here
29 |         let key = String::from("port");
   |             --- binding `key` declared here
30 |         lookup_async(config, &key)
   |                              ^^^^ borrowed value does not live long enough
31 |     };
   |     - `key` dropped here while still borrowed
```

Read the labels from the bottom up. "`key` dropped here while still borrowed" is the end of the inner block. "borrowed value does not live long enough" points at `&key`. And "borrow later stored here" points at `future`: the borrow of `key` is *stored* in the future, which outlives the block. The output type `Option<&'a str>` has nothing to do with it; what matters is what the future captures.

step	borrow of key: lookup (plain fn)	borrow of key: lookup_async
call with &key	created, used by the body	created, stored in the future
call returns	ended: key is free	held (body not run yet)
other statements		held
future polled		used by the body
future completes		ended: key is free

For a plain function, the borrow of key ends when the call returns; for an async fn it lasts until the future completes.

There are two fixes. The usual one is to await the future while its arguments are alive, which is what `lookup_async(&config, &key).await` written in one expression does. The other is to do the borrowing work *before* creating the future, and capture only what the future needs:

```rust
/// Does the lookup before creating the future, so the future captures only
/// `'a`: `key` may be dropped as soon as this function returns.
fn lookup_eager<'a>(
    config: &'a Config,
    key: &str,
) -> impl Future<Output = Option<&'a str>> + use<'a> {
    let value = config.get(key).map(String::as_str);
    async move { value }
}
```

`lookup_eager` is a plain function that returns a future. Its body runs at the call, uses `key`, and builds an `async move` block that holds only `value`, an `Option<&'a str>`. The `use<'a>` bound promises that the future captures `'a` and nothing else, and the compiler checks the promise against the async block. This is the pattern to reach for when an API should take an argument by reference but must not hold on to it.

Returning a future that borrows a local

A function can return a future that borrows its *parameters*, as `lookup_async` does. It cannot return a future that borrows its *locals*, for the same reason it cannot return a reference to one (Chapter 5) (`code/compile_fail/ch11_return_borrowing_future.rs`):

```
pub fn count_config_words() -> impl Future<Output = usize> {
    let text = String::from("listen port 8080");
    count_words(&text)
}
```

```
error[E0597]: `text` does not live long enough
  --> compile_fail/ch11_return_borrowing_future.rs:13:17
   |
12 |     let text = String::from("listen port 8080");
   |         ---- binding `text` declared here
13 |     count_words(&text)
   |     ------------^^^^^_
   |     |           |
   |     |           borrowed value does not live long enough
   |     argument requires that `text` is borrowed for `'static`
14 | }
   | - `text` dropped here while still borrowed
```

The message asks for `'static` because the function has no lifetime parameters: its opaque return type can capture none, so the future it returns may not hold any borrow that can expire. The fix is to let the future *own* the data, and lend it from inside:

```
fn count_config_words() -> impl Future<Output = usize> {
    let text = String::from("listen port 8080");
    // The async block owns `text`, and lends it to `count_words` from inside.
    async move { count_words(&text).await }
}
```

The `async move` block owns `text`, and `count_words(&text)` borrows it from inside the block. That borrow lives inside the future, next to the data it points to, so it never escapes. We will come back to this "future that borrows from itself" in the section on `Pin`.

11.2 Running futures with the standard library

To run a future you need an executor, and the standard library does not include one. For this book's examples a minimal one is enough:

```
/// Runs a future to completion on the current thread. Enough for futures that
/// never wait for anything external: they make progress at every poll.
fn block_on<F: Future>(future: F) -> F::Output {
    let mut future = pin!(future);
    let mut cx = Context::from_waker(Waker::noop());
    loop {
        if let Poll::Ready(output) = future.as_mut().poll(&mut cx) {
            return output;
        }
    }
}
```

Line by line:

- `Future::poll` takes `self: Pin<&mut Self>` and a `&mut Context<'_>`, and returns `Poll::Ready(output)` or `Poll::Pending`.
- `pin!` (stable since 1.68) pins the future in the current stack frame and gives us a `Pin<&mut F>`; `as_mut` reborrows it for each call to `poll`.
- A `Context` carries a `Waker`, which a pending future uses to say "poll me again, I can make progress now". `Waker::noop()` returns a waker that does nothing.
- The loop polls until the future is ready.

A real executor parks the thread after `Poll::Pending` and wakes it up when the future calls its waker. Ours simply polls again, which is wasteful but correct for futures that never wait for anything outside the program, as is the case for every future in this chapter. Chapter 9 used the same `block_on`.

> **NOTE**
>
> `Waker::noop` is stable since Rust 1.85, like async closures and the `AsyncFn` traits. With an older compiler, `block_on` needs a hand-written waker instead.

11.3 Spawning: why executors require `'static`

A toy executor

`block_on` runs one future and waits for it. An executor also lets you *spawn* tasks: futures that it keeps and polls later, concurrently with other tasks. Here is a toy one (`code/examples/ch11_toy_executor.rs`). First, a future that gives the other tasks a turn, by being pending once:

```
/// A future that is pending the first time it is polled, and ready the
/// second time: a cooperative "let the others run".
struct YieldNow {
    yielded: bool,
}

impl Future for YieldNow {
    type Output = ();

    fn poll(mut self: Pin<&mut Self>, _cx: &mut Context<'_>) -> Poll<()> {
        if self.yielded {
            Poll::Ready(())
        } else {
            self.yielded = true;
            Poll::Pending
        }
    }
}

fn yield_now() -> YieldNow {
    YieldNow { yielded: false }
}
```

Then the executor. Its `spawn` has the same bounds as the `spawn` of multi-threaded executors:

```
type Task = Pin<Box<dyn Future<Output = ()> + Send + 'static>>;

/// Keeps spawned tasks and polls them in turn until all are done.
#[derive(Default)]
pub struct Executor {
    tasks: Vec<Task>,
```

```
}

impl Executor {
    pub fn spawn<F>(&mut self, task: F)
    where
        F: Future<Output = ()> + Send + 'static,
    {
        self.tasks.push(Box::pin(task));
    }

    pub fn run(&mut self) {
        let mut cx = Context::from_waker(Waker::noop());
        while !self.tasks.is_empty() {
            // Poll every task once; keep the ones that are still pending.
            self.tasks
                .retain_mut(|task| task.as_mut().poll(&mut cx).is_pending());
        }
    }
}
```

`spawn` boxes the future and stores it in a vector; `run` polls each stored task in turn until all of them are done. The type of a stored task, `Pin<Box<dyn Future<Output = ()> + Send + 'static>>`, explains the bounds on `spawn`: the vector can hold futures of different types only as trait objects, and a trait object in a `Box` that is kept for an unknown time needs an explicit or default lifetime bound (Chapter 8 explains why `Box<dyn Trait>` defaults to `'static`). Our executor runs on one thread, so `Send` is not strictly needed, but a work-stealing executor moves tasks between worker threads, and the toy keeps the real signature to reproduce the real errors. For comparison, tokio's `spawn` is declared as follows (https://docs.rs/tokio/latest/tokio/task/fn.spawn.html, checked for tokio 1.53):

```
pub fn spawn<F>(future: F) -> JoinHandle<F::Output>
where
    F: Future + Send + 'static,
    F::Output: Send + 'static,
```

Here are two tasks that count the words of the same configuration text, yielding after each word:

```
async fn count_words(label: &str, text: &str) -> usize {
    let mut count = 0;
    for word in text.split_whitespace() {
        println!("{label} reads {word:?}");
        count += 1;
        yield_now().await;
    }
    count
}
```

```
let mut executor = Executor::default();
let config: Arc<str> = Arc::from("listen port 8080");
for label in ["first", "second"] {
    let config = Arc::clone(&config); // each task owns a handle
    executor.spawn(async move {
        let count = count_words(label, &config).await;
        println!("{label} counted {count} words");
    });
}
executor.run();
```

```
first reads "listen"
second reads "listen"
first reads "port"
```

```
second reads "port"
first reads "8080"
second reads "8080"
first counted 3 words
second counted 3 words
```

The tasks interleave: each yield lets the other task run. Each task owns an `Arc<str>` handle to the text, so its type holds no borrow that can expire, and it is `'static` (Chapter 7, *`T: 'static` does not mean "lives forever"*). Inside the task, `count_words(label, &config)` borrows that handle freely: the borrow lives inside the task.

Why `'static`? Because of what `spawn`'s signature cannot know. After `spawn` returns, the executor holds the task and polls it whenever it likes, possibly long after the function that spawned it has returned and its locals have been freed. A borrow of a local would have to last for a region the signature cannot name, so the signature demands the only region that is long enough for any schedule: all of it. It is the same reasoning as for `thread::spawn` in Chapter 7, and the errors are the same too.

An async block that borrows a local

The first attempt spawns an async block that uses a local string (`code/compile_fail/ch11_spawn_borrow_block.rs`, with the same `Executor`):

```
pub fn spawn_count(executor: &mut Executor) {
    let config = String::from("listen port 8080");
    executor.spawn(async {
        println!("{} words", count_words(&config).await);
    });
}
```

```
error[E0373]: async block may outlive the current function, but it borrows `config`, which
is owned by the current function
  --> compile_fail/ch11_spawn_borrow_block.rs:31:20
   |
31 |     executor.spawn(async {
   |                    ^^^^^ may outlive borrowed value `config`
32 |         println!("{} words", count_words(&config).await);
   |                                           ------ `config` is borrowed here
   |
   = note: async blocks are not executed immediately and must either take a reference or
ownership of outside variables they use
help: to force the async block to take ownership of `config` (and any other referenced
variables), use the `move` keyword
   |
31 |     executor.spawn(async move {
   |                           ++++
```

E0373 is the error Chapter 7 shows for a thread's closure, here for an async block: "async block may outlive the current function, but it borrows `config`". The note adds the async-specific reason, "async blocks are not executed immediately", and the help is right: with `async move`, the block owns `config` and borrows it from inside, exactly like the tasks of our example.

A call to an `async fn`

The second attempt spawns the future of an `async fn` call directly; this file uses a free function `spawn` with the same bounds, which drops the task instead of storing it (`code/compile_fail/ch11_spawn_borrow_call.rs`):

```
async fn report(text: &str) {
    println!("{} words", text.split_whitespace().count());
}

pub fn spawn_report() {
    let config = String::from("listen port 8080");
    spawn(report(&config));
}
```

```
error[E0597]: `config` does not live long enough
  --> compile_fail/ch11_spawn_borrow_call.rs:21:18
   |
20 |     let config = String::from("listen port 8080");
   |         ------ binding `config` declared here
21 |     spawn(report(&config));
   |     -------------^^^^^^^--
   |     |            |
   |     |            borrowed value does not live long enough
   |     argument requires that `config` is borrowed for `'static`
22 | }
   | - `config` dropped here while still borrowed
   |
note: requirement that the value outlives `'static` introduced here
  --> compile_fail/ch11_spawn_borrow_call.rs:9:37
   |
 9 |     F: Future<Output = ()> + Send + 'static,
   |                                     ^^^^^^^
```

There is no block here to add `move` to: `report(&config)` is a future that captured `&config` at the call. The labels name the requirement ("argument requires that `config` is borrowed for `'static`") and the note shows where it comes from: the `'static` in `spawn`'s bound. The fix is to give the task its own data, for example `spawn(async move { report(&config).await })`, which moves the `String` into the task and lends it to `report` from inside.

`async move` with a reference

Moving helps only if what you move is owned. When the function receives the data by reference, `async move` moves the reference (`code/compile_fail/ch11_spawn_ref_param.rs`):

```
pub fn spawn_report(config: &str) {
    spawn(async move {
        println!("{} words", config.split_whitespace().count());
    });
}
```

```
error[E0521]: borrowed data escapes outside of function
  --> compile_fail/ch11_spawn_ref_param.rs:16:5
   |
15 |   pub fn spawn_report(config: &str) {
   |                       ------  - let's call the lifetime of this reference `'1`
   |                       |
   |                       `config` is a reference that is only valid in the function body
16 | /     spawn(async move {
17 | |         println!("{} words", config.split_whitespace().count());
18 | |     });
   | |      ^
   | |      |
   | |______`config` escapes the function body here
   |        argument requires that `'1` must outlive `'static`
```

E0521, "borrowed data escapes outside of function", is read label by label in Chapter 7. "`config` escapes the function body here" points at the whole call to `spawn`, and "argument requires that `'1` must outlive `'static`" is the requirement no caller-chosen lifetime can meet. The fixes are the ones for threads: take an owned parameter (`String`, `Arc<str>`), or clone the data into the task before spawning.

Not `Send`: what a future holds across `.await`

The other half of the bound is `Send`, and its error is worth seeing once, because it is phrased in terms of what the future *holds* (`code/compile_fail/ch11_spawn_not_send.rs`):

```
pub fn spawn_shared() {
    spawn(async {
        let shared = Rc::new(String::from("listen port 8080"));
        tick().await;
        println!("{shared}");
    });
}
```

```
error: future cannot be sent between threads safely
  --> compile_fail/ch11_spawn_not_send.rs:18:5
   |
18 | /     spawn(async {
19 | |         let shared = Rc::new(String::from("listen port 8080"));
20 | |         tick().await;
21 | |         println!("{shared}");
22 | |     });
   | |______^ future created by async block is not `Send`
   |
   = help: within `{async block@compile_fail/ch11_spawn_not_send.rs:18:11: 18:16}`, the
trait `Send` is not implemented for `Rc<String>`
note: future is not `Send` as this value is used across an await
  --> compile_fail/ch11_spawn_not_send.rs:20:16
   |
19 |         let shared = Rc::new(String::from("listen port 8080"));
   |             ------ has type `Rc<String>` which is not `Send`
20 |         tick().await;
   |                ^^^^^ await occurs here, with `shared` maybe used later
note: required by a bound in `spawn`
  --> compile_fail/ch11_spawn_not_send.rs:9:30
   |
 7 | pub fn spawn<F>(task: F)
   |        ----- required by a bound in this function
 8 | where
 9 |     F: Future<Output = ()> + Send + 'static,
   |                              ^^^^ required by this bound in `spawn`
```

An async block is `Send` when everything it holds *across an* `.await` is `Send`, because those are the values stored in the future while it is suspended. The note "future is not `Send` as this value is used across an await" points at the `.await` and at `shared`, an `Rc`, which is not `Send`. The same message appears for a `MutexGuard` held across an `.await`. Either use a `Send` type (`Arc` instead of `Rc`), or end the value's life before the `.await`, for example in an inner block.

11.4 Why scoped tasks are hard

Chapter 7 showed the threads version of the borrowing problem and its scoped solution: threads spawned inside `thread::scope` may borrow locals, because `thread::scope` does not return until every

one of them has been joined. The join happens inside the body of `thread::scope`, so no caller can skip it (Chapter 13 tells why the API takes a closure instead of returning a guard).

An async version would have to be an `async fn`, or a function returning a future: the scope waits for its tasks *by being polled* until they are done. And a future is a value that the caller can stop polling at any point, then drop or forget. Forgetting is safe in Rust, and a forgotten future never runs its destructors (`code/examples/ch11_forget_future.rs`):

```
/// Stands for any resource with cleanup: a connection, a lock, a running task.
struct Connection {
    name: &'static str,
}

impl Drop for Connection {
    fn drop(&mut self) {
        println!("closing {}", self.name);
    }
}

async fn session(name: &'static str) {
    let _connection = Connection { name };
    yield_now().await; // suspended here, holding the connection
    println!("{name} finished");
}
```

```
let mut cx = Context::from_waker(Waker::noop());

let mut dropped = Box::pin(session("dropped"));
assert!(dropped.as_mut().poll(&mut cx).is_pending());
drop(dropped); // runs the destructors of the suspended future

let mut forgotten = Box::pin(session("forgotten"));
assert!(forgotten.as_mut().poll(&mut cx).is_pending());
std::mem::forget(forgotten); // safe: no destructor will ever run

println!("end of main");
```

```
closing dropped
end of main
```

Both futures were polled once and suspended at the `.await`, holding a `Connection`. Dropping the first one ran the destructors of everything it held: "closing dropped". The second one was forgotten: its connection is never closed, and nothing reports it.

Now imagine an async scope that spawns tasks on a multi-threaded executor, lets them borrow the caller's locals, and relies on its destructor to cancel or await them if it is dropped early. The caller polls the scope once, so the tasks start and borrow the locals; then it forgets the scope future and returns. The locals are freed, the destructor never runs, and the tasks, still owned by the executor, keep running on other threads with dangling references. Safe code could cause a use-after-free, so such an API would be unsound. This is why async APIs that let spawned tasks borrow have to give something up: they block the thread until the tasks finish (and stop being async), or they are `unsafe` to call, or they do not let the tasks borrow at all, which is the `'static` bound.

Concurrency *inside one task* has no such problem. A `join` that polls two futures in turn can take futures that borrow locals (`code/examples/ch11_join_borrows.rs`):

```
/// Polls two futures in turn until both are done.
async fn join<A: Future, B: Future>(a: A, b: B) -> (A::Output, B::Output) {
    let mut a = pin!(a);
    let mut b = pin!(b);
    let (mut out_a, mut out_b) = (None, None);
    poll_fn(|cx| {
        if out_a.is_none()
            && let Poll::Ready(value) = a.as_mut().poll(cx)
        {
            out_a = Some(value);
        }
        if out_b.is_none()
            && let Poll::Ready(value) = b.as_mut().poll(cx)
        {
            out_b = Some(value);
        }
        if out_a.is_some() && out_b.is_some() {
            Poll::Ready(())
        } else {
            Poll::Pending
        }
    })
    .await;
    (out_a.unwrap(), out_b.unwrap())
}
```

```
let config = String::from("listen 8080\nlog-level debug");
// Both futures borrow `config`, a local; no `'static` is needed.
let (words, lines) = block_on(join(count_words(&config), count_lines(&config)));
println!("{words} words, {lines} lines");
```

```
words: "listen"
lines: "listen 8080"
words: "8080"
lines: "log-level debug"
words: "log-level"
words: "debug"
4 words, 2 lines
```

`join`'s signature has no `'static`: `A` and `B` may borrow anything that outlives the call. This is sound even though the join future could be forgotten, and the reason is ownership. The two child futures live *inside* the join future. If someone forgets it, the children are forgotten with it, and nobody can ever poll them again, so their borrows are never used again either. A spawned task is different: the executor owns it and keeps polling it, whatever happens to the code that spawned it.

11.5 Async callbacks: `AsyncFn` and async closures

Chapter 9 (*Async callbacks that borrow*) showed that `F: Fn(&str) -> Fut` cannot describe a callback whose future borrows its argument: `Fut` is one type, picked by the caller, and it cannot depend on the lifetime of each argument. Rust 1.85 solved this with the `AsyncFn` family of traits and async closures. Here we look at the other borrow an async callback often needs: a borrow of its own captured state.

A callback that records every line it sees mutates a captured vector. With an `FnMut` that returns a future, the future would have to borrow that vector (`code/compile_fail/ch11_fnmut_future_capture.rs`; the callback takes an owned `String` so that only the new problem shows):

```
pub async fn for_each_line<F, Fut>(text: &str, mut callback: F)
where
    F: FnMut(String) -> Fut,
    Fut: Future<Output = ()>,
{
    for line in text.lines() {
        callback(line.to_owned()).await;
    }
}

pub async fn collect_lines(text: &str) -> Vec<String> {
    let mut seen = Vec::new();
    for_each_line(text, |line| async {
        seen.push(line);
    })
    .await;
    seen
}
```

```
error: captured variable cannot escape `FnMut` closure body
  --> compile_fail/ch11_fnmut_future_capture.rs:20:32
   |
19 |       let mut seen = Vec::new();
   |           -------- variable defined here
20 |       for_each_line(text, |line| async {
   |  ________________________________-_^
   | |                                |
   | |                                inferred to be a `FnMut` closure
21 | |         seen.push(line);
   | |         ---- variable captured here
22 | |     })
   | |_____^ returns an `async` block that contains a reference to a captured variable,
which then escapes the closure body
   |
   = note: `FnMut` closures only have access to their captured variables while they are
executing...
   = note: ...therefore, they cannot allow references to captured variables to escape
```

This error has no code. Its labels: "variable captured here" is the use of `seen` inside the async block; "inferred to be a `FnMut` closure" says which trait the closure implements; "returns an `async` block that contains a reference to a captured variable, which then escapes the closure body" is the problem. The two notes give the rule: an `FnMut` closure has access to its captured variables only while it is executing, so whatever it returns cannot borrow them.

The rule comes from the signature of `FnMut`. Its method is `call_mut(&mut self, args) -> Self::Output`, and `Output` is one type for the closure, unrelated to the borrow of `&mut self`. That is exactly the shape of `Iterator::next` from Chapter 10, and it fails for the same reason: the result cannot borrow from the thing being called. The fix is also the same. In the standard library's source, `AsyncFnMut` is declared with a generic associated type (simplified here; these items are unstable to name or call directly, and you use them through the `AsyncFnMut(..)` sugar):

```
pub trait AsyncFnMut<Args: Tuple>: AsyncFnOnce<Args> {
    type CallRefFuture<'a>: Future<Output = Self::Output>
    where
        Self: 'a;

    extern "rust-call" fn async_call_mut(&mut self, args: Args)
        -> Self::CallRefFuture<'_>;
}
```

`CallRefFuture<'a>` is the lending pattern of Chapter 10: the future may borrow the closure for as long as `&mut self` lasts. And the trait is used through a higher-ranked bound, `AsyncFnMut(&str)`, so the future may borrow the argument too. Both borrows at once (`code/examples/ch11_async_fn_mut.rs`):

```
async fn for_each_line(text: &str, mut callback: impl AsyncFnMut(&str)) {
    for line in text.lines() {
        callback(line).await;
    }
}
```

```
let config = String::from("listen   port 8080\n  log-level  debug");
let mut seen = Vec::new();
let collect = async |line: &str| {
    let normalized = normalize(line).await; // borrows `line` across .await
    seen.push(normalized); // mutates the captured `seen`
};
block_on(for_each_line(&config, collect));
println!("{seen:?}");
```

```
["listen port 8080", "log-level debug"]
```

The async closure `async |line: &str| { ... }` keeps `line` across an `.await` (a borrow of the argument) and pushes into `seen` (a mutable borrow of its captured state). Each call returns a future that borrows both, and `for_each_line` awaits it before calling the closure again, which is exactly what the lending signature allows. `AsyncFn` (calls through `&self`) and `AsyncFnOnce` (calls by value, once) complete the family, mirroring `Fn` and `FnOnce`.

11.6 Self-referential futures and `Pin`

The tasks of our toy executor did something that safe Rust does not let you do by hand: they borrowed from themselves. `async move { count_words(label, &config).await }` stores `config` in the future, then creates the future of `count_words`, which holds `&config`, a pointer to another part of the same future. Here is a smaller case (`code/examples/ch11_self_ref_future.rs`):

```
let task = async {
    let keys = ["listen", "port", "log-level"]; // stored in the future
    let first = &keys[0]; // points into the future itself
    yield_now().await; // `keys` and `first` are both kept across this point
    println!("first key: {first}");
};
// Before the first poll, the future is just data: moving it is fine.
let mut queue = vec![task];
let task = queue.pop().unwrap();
block_on(task); // pins the future, then polls it until it is done
```

```
first key: listen
```

`keys` and `first` are both used after the `.await`, so both are stored in the future while it is suspended, and `first` points at `keys`, *inside the same value*. If the future were moved to another address after that point, `first` would still point to the old address. Chapter 13 shows why you cannot write such a struct yourself in safe Rust; the compiler writes it for you in every async block that borrows across an `.await`, and protects it with `Pin`.

The protection is in the signature of `poll`: it takes `self: Pin<&mut Self>`. A `Pin<&mut F>` is a mutable reference with an extra promise: the `F` it points to will not be moved again until it is dropped (unless `F` implements `Unpin`, which says that moving it is harmless). Before the first poll, no reference into the future exists yet, which is why pushing the future into a vector and popping it out was fine. `block_on` pins it with `pin!` before polling, and the toy executor with `Box::pin`; from then on it stays where it is.

If you try to poll a future without pinning it, the compiler stops you (`code/compile_fail/ch11_not_unpin.rs`):

```
pub fn poll_once<F: Future + Unpin>(future: &mut F) -> Poll<F::Output> {
    let mut cx = Context::from_waker(Waker::noop());
    Pin::new(future).poll(&mut cx)
}
```

```
pub fn caller() {
    let mut task = async {
        let keys = ["listen", "port"];
        let first = &keys[0];
        tick().await;
        println!("{first}");
    };
    let _ = poll_once(&mut task);
}
```

```
error[E0277]: `{async block@compile_fail/ch11_not_unpin.rs:20:20: 20:25}` cannot be unpinned
  --> compile_fail/ch11_not_unpin.rs:26:23
   |
26 |     let _ = poll_once(&mut task);
   |             --------- ^^^^^^^^^ the trait `Unpin` is not implemented for `{async
block@compile_fail/ch11_not_unpin.rs:20:20: 20:25}`
   |             |
   |             required by a bound introduced by this call
   |
   = note: consider using the `pin!` macro
           consider using `Box::pin` if you need to access the pinned value outside of the
current scope
note: required by a bound in `poll_once`
  --> compile_fail/ch11_not_unpin.rs:10:30
   |
10 | pub fn poll_once<F: Future + Unpin>(future: &mut F) -> Poll<F::Output> {
   |                              ^^^^^ required by this bound in `poll_once`
```

`poll_once` accepts only `Unpin` futures, because `Pin::new` can pin a plain `&mut F` only when moving `F` is harmless. The async block is not `Unpin` ("cannot be unpinned"), and the notes give the two ways to pin it: `pin!`, which pins in the current stack frame, or `Box::pin`, which pins on the heap. The second note is a lifetime remark in disguise: a `Pin<&mut F>` made by `pin!` is a borrow of a local, so it cannot leave the current function; "if you need to access the pinned value outside of the current scope", the box owns it instead.

That is as much `Pin` as lifetimes require. `Pin` does not make it safe to build self-referential structs by hand; it is the promise that lets the compiler-built ones work.

11.7 `async fn` in traits

What the returned future captures

Since Rust 1.75, traits may declare `async fn` methods. Such a method returns an anonymous future type, one per implementation, which captures every input lifetime and the trait's generic parameters, including `Self` (Chapter 8 explains these captures for `impl Trait` in traits). For

```
pub trait Fetch {
    async fn fetch(&self, key: &str) -> String;
}
```

the future returned by `fetch` borrows `self` and `key` until it completes, exactly like `lookup_async` at the start of this chapter. The lifetime rules hold no surprises. The trouble is `Send`.

`Send` bounds on the returned future

A generic function that spawns the future of `fetch` must know that it is `Send`, and the trait does not say (`code/compile_fail/ch11_trait_future_not_send.rs`):

```
pub fn spawn_fetch<T>(source: Arc<T>)
where
    T: Fetch + Send + Sync + 'static,
{
    spawn(async move {
        let value = source.fetch("port").await;
        println!("port = {value}");
    });
}
```

```
error: future cannot be sent between threads safely
  --> compile_fail/ch11_trait_future_not_send.rs:26:5
   |
26 | /     spawn(async move {
27 | |         let value = source.fetch("port").await;
28 | |         println!("port = {value}");
29 | |     });
   | |______^ future created by async block is not `Send`
   |
   = help: within `{async block@compile_fail/ch11_trait_future_not_send.rs:26:11: 26:21}`,
the trait `Send` is not implemented for `impl Future<Output = String>`
note: future is not `Send` as it awaits another future which is not `Send`
  --> compile_fail/ch11_trait_future_not_send.rs:27:21
   |
27 |         let value = source.fetch("port").await;
   |                     ^^^^^^^^^^^^^^^^^^^^ await occurs here on type `impl Future<Output
= String>`, which is not `Send`
note: required by a bound in `spawn`
  --> compile_fail/ch11_trait_future_not_send.rs:10:30
   |
 8 | pub fn spawn<F>(task: F)
   |        ----- required by a bound in this function
 9 | where
10 |     F: Future<Output = ()> + Send + 'static,
   |                               ^^^^ required by this bound in `spawn`
help: `Send` can be made part of the associated future's guarantees for all implementations
of `Fetch::fetch`
   |
17 -     async fn fetch(&self, key: &str) -> String;
17 +     fn fetch(&self, key: &str) -> impl std::future::Future<Output = String> + Send;
   |
```

`T: Send + Sync` says nothing about the futures that `T`'s methods return: each implementation has its own future type, and generic code sees only "`impl Future<Output = String>`". The note "future is not `Send` as it awaits another future which is not `Send`" points at the `.await` on `fetch`, and the help gives the fix. The same advice appears earlier, as a warning, on the trait itself. The lint `async_fn_in_trait` is warn-by-default for public traits; made an error, it reads (`code/compile_fail/ch11_async_fn_in_trait_lint.rs`):

```
error: use of `async fn` in public traits is discouraged as auto trait bounds cannot be
specified
 --> compile_fail/ch11_async_fn_in_trait_lint.rs:8:5
  |
8 |     async fn fetch(&self, key: &str) -> String;
  |     ^^^^^
  |
  = note: you can suppress this lint if you plan to use the trait only in your own code, or
do not care about auto traits like `Send` on the `Future`
note: the lint level is defined here
 --> compile_fail/ch11_async_fn_in_trait_lint.rs:4:9
  |
4 | #![deny(async_fn_in_trait)]
  |         ^^^^^^^^^^^^^^^^
help: you can alternatively desugar to a normal `fn` that returns `impl Future` and add any
desired bounds such as `Send`, but these cannot be relaxed without a breaking API change
  |
8 -     async fn fetch(&self, key: &str) -> String;
8 +     fn fetch(&self, key: &str) -> impl std::future::Future<Output = String> + Send;
  |
```

The fix on stable Rust is to declare the method as a plain `fn` that returns `impl Future + Send` (`code/examples/ch11_trait_send_bound.rs`):

```rust
pub trait Fetch {
    /// The returned future borrows `self` and `key`, and is `Send`.
    fn fetch(&self, key: &str) -> impl Future<Output = String> + Send;
}
```

Implementations may still write `async fn`; the compiler checks that each of their futures is `Send`:

```rust
pub struct FileSource {
    entries: BTreeMap<String, String>,
}

impl Fetch for FileSource {
    async fn fetch(&self, key: &str) -> String {
        self.entries.get(key).cloned().unwrap_or_default()
    }
}

pub struct Defaults;

impl Fetch for Defaults {
    async fn fetch(&self, key: &str) -> String {
        format!("<default {key}>")
    }
}
```

```rust
pub fn spawn_fetch<T>(executor: &mut Executor, source: Arc<T>, key: &'static str)
where
    T: Fetch + Send + Sync + 'static,
{
    executor.spawn(async move {
        let value = source.fetch(key).await;
```

```
        println!("{key} = {value}");
    });
}
```

```
port = 9000
host = <default host>
```

The trade-off is the one the lint's help states: the `Send` requirement now binds every implementation, and it "cannot be relaxed without a breaking API change". A trait meant for single-threaded executors can keep `async fn`.

What generic code would really like is to require `Send` *at the use site*, for the implementations it is given. That is *return type notation*, and it is unstable (`code/compile_fail/ch11_rtn_unstable.rs`):

```
pub fn spawn_fetch<T>(source: Arc<T>)
where
    T: Fetch + Send + Sync + 'static,
    T::fetch(..): Send,
{
    spawn(async move {
        let value = source.fetch("port").await;
        println!("port = {value}");
    });
}
```

```
error[E0658]: return type notation is experimental
  --> compile_fail/ch11_rtn_unstable.rs:23:13
   |
23 |     T::fetch(..): Send,
   |             ^^^^
   |
   = note: see issue #109417 <https://github.com/rust-lang/rust/issues/109417> for more
information
```

`T::fetch(..): Send` means "the future returned by `T::fetch`, whatever its arguments, is `Send`". With `#![feature(return_type_notation)]`, the nightly compiler of 2026-10-08 accepts this program; on stable Rust 1.99 it is E0658, and issue #109417 tracks it. Check the current status before relying on it.

11.8 Common confusions

- "**Calling an `async fn` runs it.**" It creates a future; the body runs when the future is polled, and not at all if it is dropped first.
- "**The future borrows only what its output borrows.**" It borrows every input of the `async fn` until it completes (`borrow later stored here`). Do the work before an `async move` block to capture less.
- "**`async move` fixes spawn errors.**" It moves what the block captures. If that is a reference, the future still borrows (E0521); own the data instead.
- "**`F: 'static` means the task runs forever.**" It means the task holds no borrow that can expire (Chapter 7). A task that owns an `Arc` is `'static` and finishes whenever it finishes.
- "**`Pin` lets me write self-referential structs.**" `Pin` is a promise not to move a value; it is what makes compiler-generated self-referential futures sound. Writing your own still needs `unsafe` (Chapter 13).

- "**Our `block_on` is a small production executor.**" It spins while a future is pending. It is fine for tests and examples whose futures never wait for anything outside the program.

11.9 Summary

- An `async fn` returns a future that captures every input lifetime. The future holds those borrows until it completes, not until the call returns: a later mutation is E0502, and a dropped argument is E0597 with "borrow later stored here".
- To hold fewer borrows, do the work before an `async move` block and state the captures with `use<..>`. To return a future that uses local data, let the future own the data and borrow it from inside.
- `block_on` with `pin!` and `Waker::noop()` (1.85) runs futures with only the standard library.
- Executors keep spawned tasks and poll them later, so `spawn` requires `F: Future + Send + 'static`. Borrowing a local gives E0373 (async block) or E0597 (async fn call); moving a reference gives E0521. A non-`Send` value held across `.await` gives "future cannot be sent between threads safely".
- Futures can be forgotten safely, so an async scope cannot rely on a destructor to wait for tasks that borrow. Concurrency inside one task (`join`) can borrow, because the child futures live inside the parent.
- `FnMut` closures cannot return futures that borrow their captures ("captured variable cannot escape `FnMut` closure body"). `AsyncFnMut` can: its `CallRefFuture<'a>` is a GAT, a lending call. With async closures (1.85), a callback may borrow its argument and its state across `.await`.
- A future that borrows its own locals across `.await` is self-referential; `poll` takes `Pin<&mut Self>` so it cannot move once polled. Pin it with `pin!` or `Box::pin`.
- `async fn` in traits (1.75) captures all inputs; to make the futures `Send`, declare `fn ... -> impl Future<Output = T> + Send`. Return type notation is unstable.

11.10 Exercises

1. **Two ways to keep `key`.** Make `ch11_future_outlives_key.rs` compile in two different ways: once without changing `lookup_async`, once without changing `caller`'s block structure. *Hint:* await inside the block, or write a function like `lookup_eager` whose future does not capture `key`.
2. **Local tasks.** Add a `spawn_local` method to the toy executor of `ch11_toy_executor.rs` that accepts futures that are not `Send` (for example, one holding an `Rc`). What must change in the type of the stored tasks, and which bound can you *not* drop? *Hint:* a second vector of `Pin<Box<dyn Future<Output = ()> + 'static>>`. Ask whether the executor still keeps the task after `spawn_local` returns.
3. **A retry helper.** Write an async function `retry(times, attempt)` whose `attempt` parameter is an `impl AsyncFnMut(usize) -> Result<String, String>`, and which returns the last result. Call it with an async closure that records every attempt number in a captured `Vec` and succeeds on the third try. Then try the same with an `FnMut` closure returning an `async` block, and compare the errors.

Hint: call `attempt(n).await` in a loop and stop at the first `Ok`. The `FnMut` version fails for the reason shown in `ch11_fnmut_future_capture.rs`.

4. **Scoping a non-`Send` value.** Change `ch11_spawn_not_send.rs` so that it compiles while still creating the `Rc`, and explain the rule you used. *Hint:* only values held *across* an `.await` are stored in the suspended future; put the `Rc` and its last use in an inner block before `tick().await`.
5. **A borrowing task set.** Write a `TaskSet<'a>` that stores `Pin<Box<dyn Future<Output = ()> + 'a>>` values and has `fn run(self)` that polls them all to completion. Show that tasks may borrow locals. Then explain why this is sound even if a `TaskSet` is forgotten after some of its tasks have been polled, while the hypothetical async scope of *Why scoped tasks are hard* is not. *Hint:* who holds the tasks after `mem::forget`, and can anyone poll them?

CHAPTER 12

Interior Mutability, Guards and Lifetimes

The borrow checker enforces one rule above all: at any point, a value has either any number of readers or exactly one writer. Most of the time that rule costs you nothing. Sometimes it is in the way. A cache wants to fill itself in a method that takes `&self`. A command history is appended to by an editor and by an autosave task that both hold a shared reference to it. A counter in a struct is bumped by every read. Several threads push results into one vector.

Rust's answer is **interior mutability**: types such as `Cell`, `RefCell`, `OnceCell`, `Mutex` and `RwLock` allow mutation through a shared reference, and keep it safe by other means: a check at run time, a lock, or an API that never hands out a reference that a mutation could invalidate. It is tempting to think that these types switch lifetimes off. They do not. Every reference into a cell is still a borrow with a region, and most of these types hand out **guards**, values that carry a lifetime parameter and run code when they are dropped. That is exactly the kind of value whose lifetime Chapter 5 showed to be subtle.

This chapter is about the place where the compile-time checks of this book meet the run-time checks of interior mutability.

In this chapter you will learn:

- what `RefCell::borrow` returns: a guard `Ref<'_, T>` that is a compile-time borrow of the cell and a run-time borrow of its contents;
- why a method cannot return `&T` from inside a `RefCell` (E0515), and three ways out: `Ref::map`, a closure, an owned copy;
- why a guard in a `match`, a `while let` or an extended `let` lives longer than you expect, and what edition 2024 changed;
- how `Mutex` and `RwLock` guards follow the same rules, and how to keep the locked region short;
- which conflicts are compile errors and which are panics or deadlocks;
- why `Cell` needs no guard, why it is invariant, and why `OnceCell::get_or_init` can return a plain `&T`;
- how `thread_local!` and `LocalKey::with` lend a reference to a closure that cannot leak it.

12.1 Two kinds of checking

`&T` and `&mut T` encode the readers-or-writer rule in types, and the borrow checker verifies it before the program runs. Interior mutability keeps the rule but enforces it in a different way. At the bottom of every such type is `UnsafeCell<T>`, the one type through which the language allows mutation behind a shared reference; each safe wrapper adds its own discipline on top:

Type	Mutates through `&self` with	Keeps aliasing safe by	Hands out
`Cell<T>`	`set`, `replace`, `take`	never lending a reference to the contents	copies and moved values
`RefCell<T>`	`borrow_mut`	counting borrows at run time	`Ref`, `RefMut` guards
`OnceCell<T>`	`get_or_init`, once	never changing the contents after that	`&T`
`Mutex<T>`	`lock`	a lock that blocks other threads	`MutexGuard`
`RwLock<T>`	`write`	a readers-writer lock	read and write guards

The first three are for a single thread; `Mutex` and `RwLock` are their thread-safe counterparts (as `OnceLock` is for `OnceCell`, Chapter 7). The rest of the chapter takes them in the order that matters for lifetimes: first the guards, then the types that need none.

12.2 `RefCell` and its guards

Mutation through `&self`

Here is a command history that can be appended to through a shared reference. The full program is `code/examples/ch12_refcell_guard.rs`:

```
/// A command history that can be appended to through a shared reference.
struct History {
    entries: RefCell<Vec<String>>,
}

impl History {
    fn new() -> Self {
        History {
            entries: RefCell::new(Vec::new()),
        }
    }

    fn record(&self, entry: &str) {
        self.entries.borrow_mut().push(entry.to_owned()); // `&self` is enough
    }

    fn len(&self) -> usize {
        self.entries.borrow().len()
    }
}
```

`record` takes `&self`, yet it pushes to the vector. Two parts of a program can now hold `&History` at the same time and both record entries:

```
let history = History::new();
let editor = &history;
```

```
let autosave = &history; // two shared references to the same history
editor.record("open notes.txt");
autosave.record("save notes.txt");
println!("{} entries", history.len());
```

```
2 entries
```

What `borrow` returns

The signatures explain everything else. Simplified from the standard library's source (Rust 1.99; attributes, `const` and the `?Sized` bounds omitted), they read:

```
impl<T> RefCell<T> {
    pub fn borrow(&self) -> Ref<'_, T>;
    pub fn borrow_mut(&self) -> RefMut<'_, T>;
    pub fn try_borrow_mut(&self) -> Result<RefMut<'_, T>, BorrowMutError>;
}

pub struct Ref<'b, T> { /* a pointer to the value and to the borrow count */ }
```

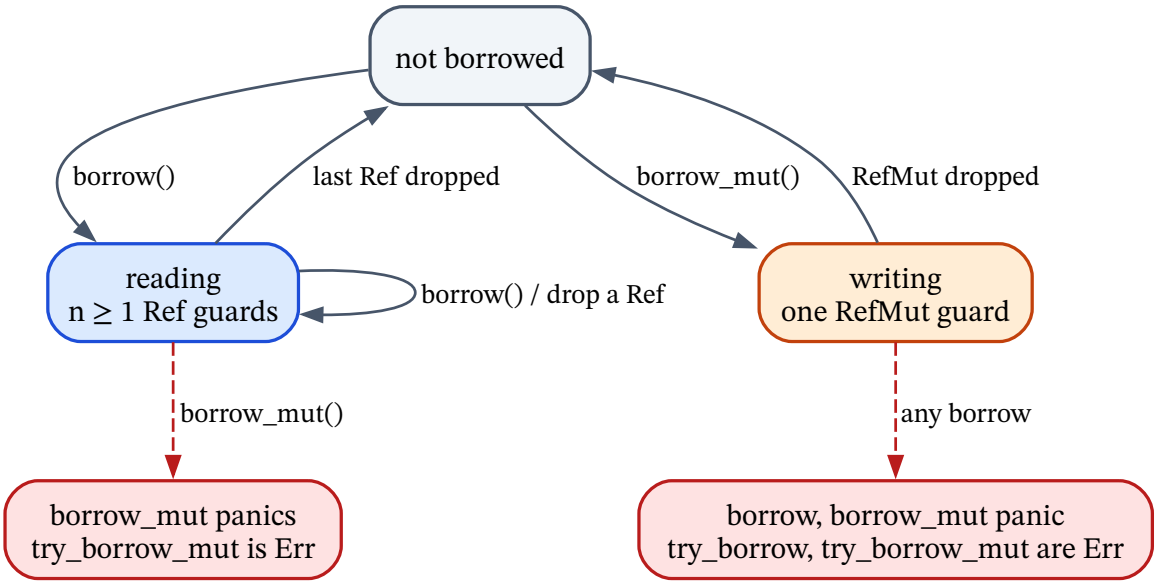
By elision (Chapter 3), the `'_` in the result is the lifetime of `&self`: the guard *borrows the cell*. That gives every guard two layers of checking.

At compile time, a `Ref<'b, T>` holds an ordinary shared borrow of the cell for `'b`. Everything this book has said about borrows applies to it: the cell must outlive the guard, it cannot be moved while a guard exists, and so on. This program, `code/compile_fail/ch12_guard_outlives_cell.rs`, is rejected exactly as if `entries` were a `&Vec<String>`:

```
pub fn entry_count() -> usize {
    let entries;
    {
        let history = RefCell::new(vec![String::from("open")]);
        entries = history.borrow();
    }
    entries.len()
}
```

```
error[E0597]: `history` does not live long enough
  --> compile_fail/ch12_guard_outlives_cell.rs:12:19
   |
11 |         let history = RefCell::new(vec![String::from("open")]);
   |             ------- binding `history` declared here
12 |         entries = history.borrow();
   |                   ^^^^^^^ borrowed value does not live long enough
13 |     }
   |     - `history` dropped here while still borrowed
14 |     entries.len()
   |     ------- borrow later used here
```

At run time, the cell counts its guards. `borrow` succeeds when there is no `RefMut`, and adds one reader; `borrow_mut` succeeds only when there is no guard at all; and each guard's destructor gives its borrow back. The compiler cannot do this part, because `borrow` and `borrow_mut` both take `&self`: to the borrow checker, two calls are two shared borrows of the cell, which never conflict.



The run-time borrow state of a `RefCell`. Guards move the cell between states; a conflicting request panics, or returns an error with the `try_` methods.

`try_borrow_mut` asks for a `RefMut` without panicking, which makes the run-time state visible:

```
let entries = history.entries.borrow(); // a `Ref`: a shared borrow at run time
match history.entries.try_borrow_mut() {
    Ok(_) => println!("with a Ref alive: got a RefMut"),
    Err(error) => println!("with a Ref alive: {error}"),
}
println!("still readable: {:?}", entries.first());
drop(entries); // the run-time borrow ends here, not at the closing brace
match history.entries.try_borrow_mut() {
    Ok(mut entries) => {
        entries.push(String::from("close notes.txt"));
        println!("after drop: got a RefMut");
    }
    Err(error) => println!("after drop: {error}"),
}
```

```
with a Ref alive: RefCell already borrowed
still readable: Some("open notes.txt")
after drop: got a RefMut
```

The `Ref` named `entries` keeps the cell in the "reading" state until `drop(entries)`; the next request succeeds. For a guard, the end of its *region* in the borrow checker's sense (the last use) and the end of the run-time borrow (the drop) are different points. A guard that is no longer used but not yet dropped still blocks writers. With guards, *when a value is dropped* matters far more than *when it was last used*.

Compile error or panic?

Since the run-time layer is invisible to the compiler, a conflict between two guards compiles and panics when it happens. `code/examples/ch12_double_borrow.rs` provokes both conflicts and catches the panics to print their messages:

```
let message = panic_message(|| {
    let reading = words.borrow();
    words.borrow_mut().push(String::from("beta")); // compiles; panics
    println!("never printed: {}", reading.len());
});
println!("borrow_mut while a Ref is alive: {message}");

let message = panic_message(|| {
    let mut writing = words.borrow_mut();
    writing.push(format!("{} words", words.borrow().len())); // panics
});
println!("borrow while a RefMut is alive: {message}");
```

```
borrow_mut while a Ref is alive: RefCell already borrowed
borrow while a RefMut is alive: RefCell already mutably borrowed
after the panics: ["alpha", "gamma"]
```

"RefCell already borrowed" means that `borrow_mut` found a guard; "RefCell already mutably borrowed" means that `borrow` found a `RefMut`. The last line shows that unwinding dropped the guards, so the cell is usable again afterwards.

The rule of thumb is simple. A conflict between two *guards* of the same cell is found at run time. A conflict between a guard and something that needs the cell *itself* (moving it, dropping it, calling a `&mut self` method such as `RefCell::get_mut`) is found at compile time, because the guard is an ordinary borrow of the cell.

Returning a reference from inside a `RefCell` (E0515)

The most common surprise with `RefCell` is a getter that will not compile. A method on `History` that returns the last entry (`code/compile_fail/ch12_return_ref_from_guard.rs`):

```
impl History {
    pub fn last(&self) -> Option<&str> {
        self.entries.borrow().last().map(|entry| entry.as_str())
    }
}
```

```
error[E0515]: cannot return value referencing temporary value
  --> compile_fail/ch12_return_ref_from_guard.rs:14:9
   |
14 |         self.entries.borrow().last().map(|entry| entry.as_str())
   |         ---------------------^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |         |
   |         returns a value referencing data owned by the current function
   |         temporary value created here
```

"temporary value created here" points at `self.entries.borrow()`: the guard is a temporary of the method's body. The `&str` is obtained *through* it, by `Deref::deref(&self) -> &Self::Target`, so its lifetime is that of a borrow of the guard, a value that dies when the method returns. Hence "returns a value referencing data owned by the current function", the E0515 of Chapter 5.

It helps to see why this must be so. If you could get a `&'b str` with the lifetime of the guard's *type* instead, you could drop the guard (resetting the count), keep the `&str`, and then call `borrow_mut` and clear the vector, leaving the `&str` dangling. The run-time count only works if every reference into the cell is tied to a live guard, and the signature of `deref` makes sure it is.

Three ways out

There are three honest signatures for "give the caller access to data inside a `RefCell`". `code/examples/ch12_ref_map.rs` implements all three. The first returns a guard, narrowed to the part the caller asked for:

```
impl History {
    /// The whole list, as a guard that derefs to a slice.
    fn entries(&self) -> Ref<'_, [String]> {
        Ref::map(self.entries.borrow(), |entries| entries.as_slice())
    }

    /// The last entry, if any: `filter_map` gives the guard back on `None`.
    fn last(&self) -> Option<Ref<'_, str>> {
        Ref::filter_map(self.entries.borrow(), |entries| {
            entries.last().map(|entry| entry.as_str())
        })
        .ok()
    }
}
```

`Ref::map` takes a guard and a function that projects the borrowed value to a part of it, and returns a guard for that part; the run-time borrow is transferred, not released. `Ref::filter_map` does the same for a projection that may fail, and gives the original guard back in the `Err` case (stable since Rust 1.63). Both are associated functions, called as `Ref::map(guard, f)` rather than `guard.map(f)`: as the standard library's documentation puts it, "A method would interfere with methods of the same name on the contents of a `RefCell` used through `Deref`."

The return type `Option<Ref<'_, str>>` is honest: it says that the caller holds a run-time borrow of the history until it drops the result. The second design does not let the caller hold anything; it lends the data to a closure. The third gives up on borrowing and returns an owned copy:

```
impl History {
    /// Lends the last entry to `f`; the reference cannot escape the call.
    fn with_last<R>(&self, f: impl FnOnce(Option<&str>) -> R) -> R {
        let entries = self.entries.borrow();
        f(entries.last().map(|entry| entry.as_str()))
    }

    /// Or give up on borrowing: an owned copy has no guard at all.
    fn last_owned(&self) -> Option<String> {
        self.entries.borrow().last().cloned()
    }
}
```

In `with_last`, the closure's parameter is `Option<&str>` with an elided lifetime, so the bound is higher-ranked (Chapter 9): `f` must work for a borrow of *any* lifetime, including one that ends when `with_last` returns. Its result type `R` is chosen by the caller before the call and cannot mention that lifetime, so the `&str` cannot escape. We will meet the same shape again with `thread_local!`. Here is how the three look from the caller's side:

```
if let Some(last) = history.last() {
    println!("last: {}", &*last); // `last` is a `Ref<'_, str>`
} // the guard is dropped here, so recording below is fine
history.entries.borrow_mut().push(String::from("save"));

println!("all: {:?}", &*history.entries());
let shouting = history.with_last(|last| last.map(str::to_uppercase));
```

```
println!("shouting: {shouting:?}");
let owned = history.last_owned();
history.entries.borrow_mut().clear(); // no guard is alive
println!("owned: {owned:?}, left: {}", history.entries().len());
```

```
last: edit
all: ["open", "edit", "save"]
shouting: Some("SAVE")
owned: Some("save"), left: 0
```

The comment after the `if let` is the important one: the guard returned by `last()` is released at the end of the `if let`, so the `borrow_mut` on the next line succeeds. Had the caller kept `last` in a variable until the end of the function, that `borrow_mut` would have panicked. A guard in a return type moves the responsibility for the run-time borrow to the caller; a closure or an owned value keeps it inside the method.

12.3 Guards and temporaries

A guard you create inside an expression and do not bind to a variable is a temporary, and the temporary scope rules of Chapter 5 decide when it is dropped. For most temporaries that timing is invisible. For a guard it decides whether the next `borrow_mut` succeeds. `code/examples/ch12_guard_temporaries.rs` uses a small helper to report the state of a cell without panicking:

```
/// Reports whether the cell is borrowed right now, without panicking.
fn status(words: &RefCell<Vec<String>>) -> &'static str {
    if words.try_borrow_mut().is_ok() {
        "free"
    } else {
        "borrowed"
    }
}
```

`match`: alive for every arm

```
match words.borrow().len() {
    0 => println!("match: empty"),
    n => println!("match: {n} words, cell is {}", status(&words)),
}
```

```
match: 2 words, cell is borrowed
```

The scrutinee is `words.borrow().len()`, a plain `usize`, yet the arm runs with the cell borrowed: the guard is a temporary of the scrutinee, and those live until the end of the `match`, in every edition. Call `words.borrow_mut()` in that arm and the program panics. The fix is to compute the value first, with a `let`:

```
let count = words.borrow().len(); // the guard dies at this `;`
println!("after let: {count} words, cell is {}", status(&words));
```

```
after let: 2 words, cell is free
```

`if let`: released before `else` (edition 2024)

```
if let Some(first) = words.borrow().first().cloned() {
    println!("if let, then-branch: {first}, cell is {}", status(&words));
}
if let Some(tenth) = words.borrow().get(9).cloned() {
    println!("if let, then-branch: {tenth}");
} else {
    println!("if let, else-branch: cell is {}", status(&words));
}
```

```
if let, then-branch: alpha, cell is borrowed
if let, else-branch: cell is free
```

The guard lives through the then-branch, and is released before the else-branch. That second half is new in edition 2024. In edition 2021 the guard lived until the end of the whole `if let`, and a common pattern, "look the key up; if it is missing, insert it in the `else` branch", panicked with "RefCell already borrowed". Chapter 5 shows the edition 2021 rule with a program that the compiler rejects (`ch05_if_let_2021.rs`); with a `RefCell`, the same mistake compiles and fails at run time, which is worse.

Extended `let`: alive until the end of the block

```
{
    let first = &words.borrow()[0]; // the guard is lifetime-extended
    println!("extended: {first}, cell is {}", status(&words));
}
```

```
extended: alpha, cell is borrowed
```

This one is easy to miss. `&words.borrow()[0]` borrows an element of the vector *through* the guard, and the borrow appears in an extending position of the `let` (the operand of `&`, Chapter 5). So the guard is lifetime-extended to the end of the block, as if you had written `let guard = words.borrow();`. Without the extension, `first` would point into a guard that was already dropped. Either way, a reference into a `RefCell` keeps a guard alive, visibly or not.

`while let`: alive for each iteration

```
while let Some(word) = words.borrow_mut().pop() {
    println!("while let body: {word}, cell is {}", status(&words));
}
```

```
while let body: beta, cell is borrowed
while let body: alpha, cell is borrowed
```

The `RefMut` created by `words.borrow_mut()` lives through the whole body of each iteration, not only through the `pop()`. A work queue whose loop body pushes follow-up jobs (`words.borrow_mut().push(...)`) panics on the first iteration. The fix has the same shape as for `match`: pop in a statement of its own, then look at the result (we will write it out for `Mutex` below).

Tail expressions (edition 2024)

The second edition 2024 change of Chapter 5 matters for guards too. A function whose last expression reads a local `RefCell` creates the guard in its tail expression. In edition 2021, that guard was dropped after the local it borrows, and the function did not compile (`code/compile_fail/ch12_tail_guard_2021.rs`, compiled with `edition=2021`):

```
pub fn entry_count() -> usize {
    let history = RefCell::new(vec![String::from("open")]);
    history.borrow().len()
}
```

```
error[E0597]: `history` does not live long enough
  --> compile_fail/ch12_tail_guard_2021.rs:11:5
   |
10 |     let history = RefCell::new(vec![String::from("open")]);
   |         ------- binding `history` declared here
11 |     history.borrow().len()
   |     ^^^^^^^----------
   |     |
   |     borrowed value does not live long enough
   |     a temporary with access to the borrow is created here ...
12 | }
   | -
   | |
   | `history` dropped here while still borrowed
   | ... and the borrow might be used here, when that temporary is dropped and runs the
destructor for type `Ref<'_, Vec<String>>`
   |
   = note: the temporary is part of an expression at the end of a block;
           consider forcing this temporary to be dropped sooner, before the block's local
variables are dropped
help: for example, you could save the expression's value in a new local variable `x` and
then make `x` be the expression at the end of the block
   |
11 |     let x = history.borrow().len(); x
   |     +++++++                       +++
```

The story is the one Chapter 5 read label by label: a temporary with access to the borrow is created in the tail expression, `history` is dropped at the closing brace, and then the borrow "might be used here, when that temporary is dropped and runs the destructor for type `Ref<'_, Vec<String>>`". (Chapter 5's `Reader` had "the `Drop` code for type"; `Ref` has no `Drop` impl of its own, but one of its fields has, so the compiler speaks of its *destructor*.) In edition 2024 the guard is dropped before the locals, and the function compiles unchanged: it is `entry_count` in `ch12_guard_temporaries.rs`, whose last line of output is `entry_count() = 1`.

Here is the whole section in one table:

You write	The guard is released
`let n = cell.borrow().len();`	at the `;`
`let guard = cell.borrow();`	when `guard` goes out of scope, or at `drop(guard)`
`let first = &cell.borrow()[0];`	at the end of the block (extended)
`match cell.borrow().len() { .. }`	after the `match`
`if let .. = cell.borrow().. { } else { }`	before `else` (2024), after it (2021)
`while let .. = cell.borrow_mut().pop() { }`	at the end of each iteration
`cell.borrow().len()` as a tail expression	before the locals (2024), after them (2021)

12.4 `Mutex`, `RwLock` and their guards

A `MutexGuard` is a borrow of the mutex

`Mutex<T>` is the thread-safe relative of `RefCell<T>`, and its signature has the same shape (simplified the same way):

```
impl<T> Mutex<T> {
    pub fn lock(&self) -> LockResult<MutexGuard<'_, T>>;
    pub fn try_lock(&self) -> TryLockResult<MutexGuard<'_, T>>;
}
```

The guard borrows the mutex for `'_`, gives access to the `T` through `Deref` and `DerefMut`, and unlocks the mutex in its destructor. (`LockResult` is a `Result` because of *poisoning*: if a thread panics while holding the guard, later `lock` calls return an error that still contains the guard. `unwrap()` is the usual response.) Everything in the previous two sections applies with one difference: a conflict does not panic, it *waits*, because another thread may release the lock. On the same thread, nobody will. The documentation of `Mutex::lock` says: "The exact behavior on locking a mutex in the thread which already holds the lock is left unspecified. However, this function will not return on the second call (it might panic or deadlock, for example)."

So the examples below use `try_lock`, which returns `Err(TryLockError::WouldBlock)` instead of waiting (`code/examples/ch12_mutex_guards.rs`):

```
/// Reports whether the mutex is locked right now, without blocking.
fn status<T>(mutex: &Mutex<T>) -> &'static str {
    match mutex.try_lock() {
        Ok(_) => "free",
        Err(TryLockError::WouldBlock) => "locked",
        Err(TryLockError::Poisoned(_)) => "poisoned",
    }
}
```

Sharing a mutex between scoped threads

With `thread::scope` (Chapter 7), threads can borrow the mutex directly: the scope guarantees that they finish before `lengths` is dropped, so no `Arc` is needed:

```
let lengths = Mutex::new(Vec::new());
thread::scope(|s| {
```

```
    for word in ["alpha", "beta", "gamma"] {
        let lengths = &lengths; // a plain shared borrow: no `Arc` needed
        s.spawn(move || {
            let len = word.len(); // work without holding the lock
            lengths.lock().unwrap().push((word, len)); // lock, push, unlock
        });
    }
});
let mut lengths = lengths.into_inner().unwrap();
lengths.sort();
println!("{lengths:?}");
```

```
[("alpha", 5), ("beta", 4), ("gamma", 5)]
```

Each thread does its work (here, `word.len()`) *before* taking the lock, and the guard is a temporary of the statement `lengths.lock().unwrap().push(...)`, so the lock is held for the push and released at the `;`. Keeping the locked region short is the main design rule for mutexes, and the temporary scope rules are how you control it.

Scoping a guard

When you need the lock for several statements, bind the guard and give it a block:

```
let queue = Mutex::new(vec!["parse", "check"]);
{
    let mut jobs = queue.lock().unwrap();
    jobs.push("emit");
    println!("inside the block: {}", status(&queue));
} // `jobs` is dropped: unlocked
println!("after the block: {}", status(&queue));
```

```
inside the block: locked
after the block: free
```

`drop(jobs)` would release it just as well, earlier than the end of the block. What does *not* work is `let _ = queue.lock().unwrap();` to "hold the lock for the rest of the function": `_` binds nothing, so the guard is dropped at the end of the statement (Chapter 5). The mistake is common enough that rustc has a deny-by-default lint for it (`code/compile_fail/ch12_let_underscore_lock.rs`):

```
pub fn append(queue: &Mutex<Vec<String>>, job: &str) {
    let _ = queue.lock().unwrap(); // meant to hold the lock
    println!("appending {job}");
}
```

```
error: non-binding let on a synchronization lock
  --> compile_fail/ch12_let_underscore_lock.rs:10:9
   |
10 |     let _ = queue.lock().unwrap(); // meant to hold the lock
   |         ^ this lock is not assigned to a binding and is immediately dropped
   |
   = note: `#[deny(let_underscore_lock)]` (part of `#[deny(let_underscore)]`) on by default
help: consider binding to an unused variable to avoid immediately dropping the value
   |
10 |     let _unused = queue.lock().unwrap(); // meant to hold the lock
   |          ++++++
help: consider immediately dropping the value
   |
10 -     let _ = queue.lock().unwrap(); // meant to hold the lock
```

```
10 +     drop(queue.lock().unwrap()); // meant to hold the lock
   |
```

The first help is the fix you meant: bind the guard to a name that starts with an underscore, such as `_guard`, which *is* a binding and keeps the guard until the end of the scope.

The `while let` trap with a mutex

The `while let` rule from the previous section turns into a deadlock with a mutex:

```
while let Some(job) = queue.lock().unwrap().pop() {
    // Still locked here: calling `queue.lock()` would never return.
    println!("while let, job {job}: {}", status(&queue));
}
```

```
while let, job emit: locked
while let, job check: locked
while let, job parse: locked
```

The body runs with the queue locked. If it called `queue.lock()` to push a follow-up job, the thread would wait for itself forever (or panic: the behavior is unspecified). The fix is to take the job out in a statement of its own, so that the guard dies at the `;` before the body runs:

```
loop {
    let next = queue.lock().unwrap().pop(); // the guard dies at this `;`
    let Some(job) = next else { break };
    println!("loop, job {job}: {}", status(&queue));
}
```

```
loop, job check: free
loop, job parse: free
```

The same applies to `match queue.lock().unwrap().pop() { .. }` in every edition, and to `if let` in edition 2021.

> **WARNING**
>
> A `RefCell` conflict panics, which you will notice in your tests. A `Mutex` conflict on the same thread may deadlock, and a lock held longer than needed only makes other threads wait, which you may not notice at all. Review every `match`, `while let` and extended `let` whose scrutinee or initializer calls `lock()`.

Readers and writers: `RwLock`

`RwLock<T>` allows many readers or one writer, with `read()` and `write()` returning `RwLockReadGuard<'_, T>` and `RwLockWriteGuard<'_, T>`. The rules for the guards are the same (`code/examples/ch12_rwlock_static.rs`):

```
fn rwlock_demo() {
    let config = RwLock::new(String::from("verbose=false"));
    {
        let first = config.read().unwrap();
        let second = config.read().unwrap(); // many readers at once
        let can_write = config.try_write().is_ok(); // a writer must wait
        println!("readers: {first} and {second}; can write: {can_write}");
```

```
    } // both read guards dropped
    config.write().unwrap().replace_range(8.., "true");
    println!("after writing: {}", config.read().unwrap());
}
```

```
readers: verbose=false and verbose=false; can write: false
after writing: verbose=true
```

Two read guards coexist, and a writer must wait until both are dropped. The write happens through a temporary guard in a statement of its own, and the final read guard is a temporary of the `println!` statement.

Guards of a `static`

The lifetime in a guard's type is the lifetime of the borrow of the lock. When the lock is a `static`, the borrow can be `'static`, and so can the guard:

```
static LOG: Mutex<Vec<String>> = Mutex::new(Vec::new());

fn log(line: &str) {
    LOG.lock().unwrap().push(line.to_owned());
}

/// The mutex is a `static`, so its guard can be `MutexGuard<'static, _>` and
/// can be returned from a function without any lifetime parameter.
fn log_entries() -> MutexGuard<'static, Vec<String>> {
    LOG.lock().unwrap()
}
```

```
log: ["started", "loaded config"]
```

`log_entries` returns a `MutexGuard<'static, Vec<String>>` with no lifetime parameter in sight, because `LOG` lives for the whole program. Note what `'static` does *not* mean here: the lock is held exactly as long as the guard lives, which should still be short. `'static` describes the *mutex*, not the critical section.

One tool you might look for is missing on stable Rust: there is no `MutexGuard::map` to narrow a guard the way `Ref::map` does. In Rust 1.99, `MappedMutexGuard` and the `map` functions of the lock guards are unstable (feature `mapped_lock_guards`, tracking issue rust-lang/rust#117108). For a `Mutex`, use the closure or the owned-copy design from the `RefCell` section, or return the whole guard.

12.5 `Cell`, `OnceCell` and invariance

`Cell`: no references, no guards

`Cell<T>` takes the opposite approach to `RefCell`. It never lends a reference to its contents through `&self`: you can copy the value out (`get`, for `T: Copy`), put a value in (`set`), swap one in (`replace`) or take it out (`take`, for `T: Default`). Since no reference into a `Cell` can exist while someone holds only `&Cell<T>`, a `set` can never invalidate one, and nothing needs to be counted. Here is a document that counts how often its summary is read (`code/examples/ch12_cell_oncecell.rs`):

```
struct Document {
    text: String,
    reads: Cell<u32>,          // a statistic updated through `&self`
    summary: OnceCell<String>, // computed on first use, then fixed
}

impl Document {
    fn new(text: &str) -> Self {
        Document {
            text: text.to_owned(),
            reads: Cell::new(0),
            summary: OnceCell::new(),
        }
    }

    /// The returned `&str` borrows `self`: no guard, no run-time count.
    fn summary(&self) -> &str {
        self.reads.set(self.reads.get() + 1);
        self.summary.get_or_init(|| {
            println!("  (computing the summary)");
            let first: Vec<&str> = self.text.split_whitespace().take(3).collect();
            first.join(" ")
        })
    }
}
```

`reads` is updated in `summary(&self)` with `get` and `set`: no guard, no run-time check, no possible panic. The price is the restricted API: you cannot call a method on the value *inside* the cell, only move whole values in and out.

`Cell<&'a T>` and invariance

A `Cell` can hold a borrowed value, which makes its lifetime parameter part of the picture. This cursor remembers the last token it saw, through `&self`:

```
/// A `Cell` can hold a borrowed value too: `Cell<&'src str>` is fine as long
/// as every slice stored in it comes from the same `'src` text.
struct Cursor<'src> {
    last_token: Cell<&'src str>,
}

impl<'src> Cursor<'src> {
    fn remember(&self, token: &'src str) {
        self.last_token.set(token);
    }
}
```

This works because every token comes from the same source text, `'src`. Chapter 4 showed why `Cell<T>` is *invariant* in `T`, and here is the consequence in a setting of this chapter. The compiler rejects this program (`code/compile_fail/ch12_cell_invariance.rs`):

```
fn remember<'a>(slot: &Cell<&'a str>, token: &'a str) {
    slot.set(token);
}

pub fn caller(slot: &Cell<&'static str>) {
    let local = String::from("short-lived");
    remember(slot, &local); // would leave a dangling `&str` in `slot`
}
```

```
error[E0597]: `local` does not live long enough
  --> compile_fail/ch12_cell_invariance.rs:14:20
```

```
   |
13 |     let local = String::from("short-lived");
   |         ----- binding `local` declared here
14 |     remember(slot, &local); // would leave a dangling `&str` in `slot`
   |     ---------------^^^^^^_
   |     |              |
   |     |              borrowed value does not live long enough
   |     argument requires that `local` is borrowed for `'static`
15 | }
   | - `local` dropped here while still borrowed
```

`remember` asks for a slot and a token with the same `'a`. For a plain `&&'static str`, the compiler could shrink the slot's type to `&&'a str` (covariance), pick `'a` as short as `local`, and accept the call. For a `Cell`, it may not: `slot` would then let `remember` store a short-lived `&str` in a cell that the caller still believes to hold `&'static str`, and the caller could read it after `local` is gone. Since the cell's type cannot change, `'a` must be `'static`, and the label says so: "argument requires that `local` is borrowed for `'static`". Invariance is how the type system protects every type that can be written through a shared reference.

`OnceCell`: plain references, no guard

`OnceCell<T>` is a cell that can be written once. Its key method has a signature that may surprise you after the `RefCell` section:

```
impl<T> OnceCell<T> {
    pub fn get_or_init<F>(&self, f: F) -> &T
    where
        F: FnOnce() -> T;
}
```

It returns a plain `&T` with the lifetime of `&self`: no guard. The `summary` method of `Document` above uses it to compute the summary on first use and to return a `&str` that borrows the document:

```
let doc = Document::new("lifetimes are regions of the program");
let first = doc.summary();
let second = doc.summary(); // both references are alive at once
println!("{first:?} == {second:?}");
println!("summary read {} times", doc.reads.get());
```

```
  (computing the summary)
"lifetimes are regions" == "lifetimes are regions"
summary read 2 times
```

The summary is computed once, and two references to it are alive at the same time as the `Cell` counter is updated. This is sound because of what `OnceCell` does *not* offer: once the cell is full, nothing that takes `&self` changes the contents. A second `get_or_init` returns the existing value. The methods that can change or remove it (`take`, `get_mut`, `into_inner`) need `&mut self` or ownership, and those conflict with every outstanding `&T` in the ordinary, compile-time way. This program is rejected (`code/compile_fail/ch12_oncecell_take.rs`):

```
pub fn reset(summary: &mut OnceCell<String>) {
    let current = summary.get_or_init(|| String::from("draft"));
    summary.take(); // would free the `String` that `current` points to
    println!("{current}");
}
```

```
error[E0502]: cannot borrow `*summary` as mutable because it is also borrowed as immutable
  --> compile_fail/ch12_oncecell_take.rs:11:5
   |
10 |     let current = summary.get_or_init(|| String::from("draft"));
   |                   ------- immutable borrow occurs here
11 |     summary.take(); // would free the `String` that `current` points to
   |     ^^^^^^^^^^^^^ mutable borrow occurs here
12 |     println!("{current}");
   |                ------- immutable borrow later used here
```

So `OnceCell` needs only one bit of run-time state (empty or full). The aliasing rule for the contents is left to the borrow checker, which can enforce it because the dangerous operations require `&mut`. That is the design to aim for when you build your own types: the more of the aliasing rule you express with `&` and `&mut` in signatures, the less you have to check at run time.

12.6 `thread_local!` and closure-scoped borrows

A `thread_local!` declares a value of which each thread has its own copy. The declaration creates a `LocalKey<T>`, and the value is reached through methods of the key (`code/examples/ch12_thread_local.rs`):

```
thread_local! {
    static LOG: RefCell<Vec<String>> = const { RefCell::new(Vec::new()) };
    static DEPTH: Cell<u32> = const { Cell::new(0) };
}
```

```
fn log(line: &str) {
    LOG.with(|log| log.borrow_mut().push(line.to_owned()));
}

fn log_len() -> usize {
    LOG.with_borrow(|log| log.len()) // the result is a `usize`: nothing borrowed
}

fn first_entry() -> Option<String> {
    LOG.with_borrow(|log| log.first().cloned()) // an owned copy leaves the closure
}
```

```
log("main: start");
DEPTH.set(DEPTH.get() + 1);
thread::scope(|s| {
    s.spawn(|| {
        log("worker: start");
        println!("worker: log length {}, depth {}", log_len(), DEPTH.get());
    });
});
log("main: done");
println!("main: log length {}, depth {}", log_len(), DEPTH.get());
println!("main: first entry {:?}", first_entry());
```

```
worker: log length 1, depth 0
main: log length 2, depth 1
main: first entry Some("main: start")
```

The worker thread sees only its own entry and its own `DEPTH`, which is still 0; the main thread sees its two entries. `with_borrow` and `with_borrow_mut` (for `LocalKey<RefCell<T>>`) and `get` and `set` (for `LocalKey<Cell<T>>`) are shortcuts, stable since Rust 1.73.

Why `with` takes a closure

Every access goes through a closure. The signature of `LocalKey::with`, verbatim from the standard library, is:

```
pub fn with<F, R>(&'static self, f: F) -> R
where
    F: FnOnce(&T) -> R,
```

The key is `&'static`, but the value is not: each thread has its own, and it is destroyed when that thread exits. A `&'static T` to it could outlive its thread. So `with` lends the reference to `f` instead. `FnOnce(&T) -> R` elides the lifetime of `&T`, which makes the bound higher-ranked (Chapter 9): `for<'a> FnOnce(&'a T) -> R`. `with` picks `'a` inside its body, and `R` is fixed by the caller before that, so `R` cannot contain `'a`. Trying to return the reference anyway is rejected (`code/compile_fail/ch12_thread_local_escape.rs`):

```
thread_local! {
    static GREETING: String = String::from("hello");
}

pub fn greeting() -> &'static str {
    GREETING.with(|text| text.as_str())
}
```

```
error: lifetime may not live long enough
  --> compile_fail/ch12_thread_local_escape.rs:11:26
   |
11 |     GREETING.with(|text| text.as_str())
   |                    -----  ^^^^^^^^^^^^^ returning this value requires that `'1` must
outlive `'2`
   |                    |   |
   |                    |   return type of closure is &'2 str
   |                    has type `&'1 String`
```

This error has no code; it is the "lifetime may not live long enough" of closures. The labels name two anonymous lifetimes: `'1` is the lifetime of the closure's parameter `text` ("has type `&'1 String`"), and `'2` is the lifetime in the closure's return type ("return type of closure is `&'2 str`"). Returning `text`'s contents requires `'1` to outlive `'2`. But `'1` is chosen anew for each call, inside `with`, and `'2` is part of the type `R`, chosen outside. Nothing can make one outlive the other, which is the point: the borrow is confined to the closure.

Even inside the closure, `with` makes no promise for the time while the thread is being torn down: its documentation says that it "will `panic!()` if the key currently has its destructor running, and it **may** panic if the destructor has previously been run for this thread". The fallible variant `try_with` returns an error instead.

Lending instead of returning

`LocalKey::with` is one instance of a pattern you have now seen three times: `History::with_last` in this chapter, `thread::scope` in Chapter 7, and `LocalKey::with`. When the validity of a reference depends on something that must happen *after* its last use (a guard released, a thread joined, a thread-local torn down), the API cannot return the reference, because the caller could keep it past that point. It lends the reference to a closure, and a higher-ranked bound guarantees that the closure gives it back.

The cost is a less direct calling style; the gain is that misuse is a compile error instead of a panic, a deadlock or a dangling pointer.

12.7 Common confusions

"**`RefCell` turns off the borrow checker.**" It moves one check to run time: the conflict between guards of the same cell. Every guard is still an ordinary borrow of the cell, and the cell is still subject to every compile-time rule (`ch12_guard_outlives_cell.rs`).

"**A `Ref<'b, T>` can give me a `&'b T`.**" It gives you a reference tied to the guard *value*. That is what keeps the run-time count honest, and it is why getters return `Ref<'_, T>`, take a closure, or return an owned copy.

"**`match cell.borrow().len()` releases the guard, since `len()` returned a number.**" The guard is a temporary of the scrutinee and lives until the end of the `match`, whatever the type of the scrutinee's value. The same holds for `while let`; `if let` releases it before `else` in edition 2024.

"**`let _ = mutex.lock()` holds the lock.**" It releases it immediately, and rustc rejects it for that reason. `let _guard = mutex.lock().unwrap();` holds it.

"**A `MutexGuard<'static, T>` holds the lock forever.**" It holds the lock until it is dropped. The `'static` is the lifetime of the borrow of the mutex, possible because the mutex is a `static`.

"**`Cell` is a cheaper `RefCell`.**" It is a different contract: no references to the contents at all. When you need to call a method on the contents in place, you need `RefCell` (or a lock); when you only move values in and out, `Cell` is simpler and cannot panic.

12.8 Summary

- Interior mutability allows mutation through `&T`. It keeps the readers-or-writer rule, enforced at run time (`RefCell`, `Mutex`, `RwLock`) or by an API that cannot violate it (`Cell`, `OnceCell`). It does not remove lifetimes.
- `RefCell::borrow(&self) -> Ref<'_, T>`: the guard is a compile-time borrow of the cell and a run-time borrow of its contents. Conflicts between guards panic ("RefCell already borrowed", "RefCell already mutably borrowed"); conflicts with the cell itself are compile errors.
- References obtained through a guard live no longer than the guard. A getter cannot return `&T` from a `RefCell` (E0515); return a guard narrowed with `Ref::map` or `Ref::filter_map`, lend the data to a closure, or return an owned copy.
- A guard that is not bound to a variable is a temporary: it lives to the end of a `match`, of each `while let` iteration, or of the block when extended; in edition 2024 an `if let` releases it before `else`, and a tail expression before the block's locals.
- `MutexGuard` and the `RwLock` guards follow the same rules; a conflict waits instead of panicking, and on the same thread never returns. Keep critical sections short with statement-level temporaries, blocks or `drop`; never `let _ =` a guard. Guards of a `static` lock are `'static`.

- `Cell` never lends references, so it needs no guard; `Cell<T>` is invariant in `T`. `OnceCell::get_or_init` returns `&T` tied to `&self` because the contents never change through `&self` once set.
- `LocalKey::with` lends `&T` to a closure with a higher-ranked bound, so the reference cannot escape the call. Lending instead of returning is the general pattern when a reference must not outlive a cleanup step.

12.9 Exercises

1. Add to `History` (from `ch12_ref_map.rs`) a method `fn longest(&self) -> Option<Ref<'_, str>>` that returns the longest entry. Then write a caller that holds the result in a variable while calling `history.entries.borrow_mut()`, and observe the panic. Change the caller, not the method, so that it works. *Hint:* `Ref::filter_map` with `iter().max_by_key(..)`; the caller needs a shorter scope for the guard.
2. A work queue is a `RefCell<Vec<String>>`. Write a loop that pops jobs and, for every job that ends with `!`, pushes a follow-up job without the `!`. Write it first with `while let Some(job) = queue.borrow_mut().pop()` and run it; then fix it. *Hint:* the guard of a `while let` scrutinee lives for the whole body; pop in a `let` statement inside a `loop`.
3. Rewrite `History` with a `Mutex<Vec<String>>` so that it can be shared by threads in a `thread::scope`. Which of `entries`, `last`, `with_last` and `last_owned` can you keep with the same shape, and what must change? *Hint:* there is no stable `MutexGuard::map`; a method can still return the whole `MutexGuard<'_, Vec<String>>`.
4. Declare a thread-local `RefCell<Vec<String>>` of configuration lines and write `fn with_config<R>(f: impl FnOnce(&[String]) -> R) -> R`. Then try to write `fn config() -> &'static [String]` on top of it, read the error, and explain in one sentence which lifetime is chosen by whom. *Hint:* compare with `ch12_thread_local_escape.rs`; the closure's parameter lifetime is chosen inside `with`.
5. Give `Document` a field `words: OnceCell<Vec<String>>` filled on first use, and a method `fn words(&self) -> &[String]`. Explain why this needs no guard, then explain why `fn reset(&self)` that empties the cell cannot be written, while `fn reset(&mut self)` can (with `OnceCell` alone). *Hint:* look at which `OnceCell` methods take `&self` and which take `&mut self`.

CHAPTER 13

Lifetimes and `unsafe` Code

Everywhere else in this book, the borrow checker checks your lifetimes for you. In `unsafe` code there are operations it does not check: dereferencing a raw pointer, calling an `unsafe fn` such as `slice::from_raw_parts`, implementing an `unsafe` trait. Lifetimes do not disappear when you use them. They move to the boundary. The function you build on top of `unsafe` code has a signature, safe callers are checked against that signature and nothing else, and so the signature must tell the truth about every borrow. *The signature is the contract* is never more literal than here: in safe code a wrong signature is rejected, while in `unsafe` code a wrong signature compiles and lets safe callers create dangling references.

This chapter shows where lifetimes enter and leave `unsafe` code. A raw pointer has no lifetime, and the reference you make from it gets whatever lifetime the surrounding code asks for, unless a signature bounds it. A struct that holds raw pointers needs `PhantomData` to tell the compiler which borrow it represents and how it varies. Some designs that look natural, such as a struct that borrows from itself, cannot be expressed at all. The drop check has an `unsafe` escape hatch that `Vec` uses and you cannot (yet). And one fact shapes several APIs: leaking a value is safe, so no safe API may depend on a destructor running.

Every `unsafe` block in this chapter's programs is sound, and says why in a `// SAFETY:` comment. Two programs contain an *unsound* function on purpose, to show that it compiles; they are marked as such and never call it.

In this chapter you will learn:

- why raw pointers carry no lifetime, how dereferencing one creates an *unbounded* lifetime, and how a signature bounds it;
- who promises the lifetime in APIs such as `slice::from_raw_parts`;
- what `PhantomData` is for: unused lifetime parameters (E0392), tying a raw pointer to a borrow, and choosing variance;
- why safe Rust rejects self-referential structs, what to do instead, and how `Pin` relates;
- how the drop check treats your types and `Vec`, and what the unstable `#[may_dangle]` promises;
- why leaking is safe, and why that is the reason `thread::scope` takes a closure;
- what Miri can and cannot tell you about your `unsafe` code.

13.1 Raw pointers carry no lifetime

A pointer is an address, not a borrow

A reference `&'a T` carries a region: the borrow checker knows where it is valid and checks every use against that. A raw pointer, `*const T` or `*mut T`, has no lifetime parameter at all. Creating one from a reference is safe, keeping it is safe, and the borrow checker stops tracking it the moment it is created: the pointer may outlive the value it points to, and nothing complains. Only dereferencing it requires `unsafe`, because only then does it matter whether the memory is still there. This compiles and runs, as part of `code/examples/ch13_raw_buf.rs`:

```
let pointer: *const String = {
    let temporary = String::from("temporary");
    &temporary // a `&String` coerced to a raw pointer: the borrow ends here
}; // `temporary` is dropped; `pointer` dangles, and that is allowed
assert!(!pointer.is_null()); // safe: this does not dereference it
```

That is exactly why raw pointers are useful for building data structures whose borrowing pattern the borrow checker cannot see, and exactly why they are dangerous: all the information that `'a` would have carried now lives only in your head, and in the `SAFETY:` comments you write down.

Unbounded lifetimes

Going back from a raw pointer to a reference, with `&*ptr` or with functions such as `slice::from_raw_parts`, produces a reference whose lifetime nothing constrains. The Rustonomicon calls it *unbounded* (https://doc.rust-lang.org/nomicon/unbounded-lifetimes.html): "Such a lifetime becomes as big as context demands." If the context wants `'static`, it gets `'static`.

Here is a buffer of bytes held as a raw pointer and a length, the shape you get from a C library or a custom allocator. (With only the standard library, the memory comes from a `Box<[u8]>`, so that the `Drop` impl can free it again.) The rest of `code/examples/ch13_raw_buf.rs` defines it:

```
/// An owned byte buffer kept as a raw pointer and a length, as you might get
/// from a C library or a custom allocator.
pub struct RawBuf {
    ptr: *mut u8,
    len: usize,
}

impl RawBuf {
    pub fn new(bytes: &[u8]) -> RawBuf {
        let boxed: Box<[u8]> = bytes.into();
        let len = boxed.len();
        // `into_raw` gives up ownership; `Drop` below takes it back.
        let ptr = Box::into_raw(boxed).cast::<u8>();
        RawBuf { ptr, len }
    }
}

impl Drop for RawBuf {
    fn drop(&mut self) {
        let slice = std::ptr::slice_from_raw_parts_mut(self.ptr, self.len);
        // SAFETY: `ptr` and `len` came from `Box::into_raw` in `new`, and this
        // is the only place that turns them back into a box, exactly once.
        drop(unsafe { Box::from_raw(slice) });
    }
}
```

Now two ways to look at the bytes. The first has an elided signature, which ties the slice to the borrow of `self` (Chapter 3):

```
impl RawBuf {
    /// The elided signature reads `fn as_bytes<'a>(&'a self) -> &'a [u8]`.
    pub fn as_bytes(&self) -> &[u8] {
        // SAFETY: `ptr` points to `len` initialized bytes owned by `self`.
        // They stay allocated while `self` is alive, and no method mutates
        // them, so they are valid and unchanged for the lifetime of `&self`,
        // which is the lifetime the signature gives the slice.
        unsafe { slice::from_raw_parts(self.ptr, self.len) }
    }
}
```

The second names a lifetime that appears only in the output, so the *caller* picks it, as for any lifetime parameter (Chapter 9, *Who picks the lifetime?*):

```
impl RawBuf {
    /// UNSOUND: the caller picks `'a`, and nothing ties it to `self`.
    #[allow(dead_code)] // only used by `keep_forever`, which is never called
    pub fn as_bytes_unbounded<'a>(&self) -> &'a [u8] {
        // SAFETY: (incomplete) the bytes are valid while `self` is alive, but
        // this signature lets the caller keep the slice longer than that.
        unsafe { slice::from_raw_parts(self.ptr, self.len) }
    }
}

/// Compiles, and returns a dangling slice: `buf` is freed at the end of the
/// function. Never called: calling it is undefined behavior.
#[allow(dead_code)]
fn keep_forever() -> &'static [u8] {
    let buf = RawBuf::new(b"temporary");
    buf.as_bytes_unbounded()
}
```

The bodies are identical, and both compile, because `from_raw_parts` will return a slice of any lifetime you like. The difference is entirely in the contract. With `as_bytes_unbounded`, the function `keep_forever` compiles: it asks for `&'static [u8]`, the signature grants it, and the buffer is freed when `buf` goes out of scope at the end of the function. Any use of the returned slice reads freed memory. That is undefined behavior, so the program never calls `keep_forever`; it exists to show that the compiler accepts it.

With the bounded `as_bytes`, the same function is rejected (`code/compile_fail/ch13_bounded_slice.rs`):

```
pub fn keep_forever() -> &'static [u8] {
    let buf = RawBuf::new(b"temporary");
    buf.as_bytes()
}
```

```
error[E0515]: cannot return value referencing local variable `buf`
  --> compile_fail/ch13_bounded_slice.rs:37:5
   |
37 |     buf.as_bytes()
   |     ---^^^^^^^^^^
   |     |
   |     returns a value referencing data owned by the current function
   |     `buf` is borrowed here
```

E0515 is the ordinary "returning a reference to a local" error of Chapter 5, "returns a value referencing data owned by the current function". The `unsafe` block inside `as_bytes` did not weaken anything:

once the signature says `&'a self -> &'a [u8]`, the borrow checker enforces it on every caller. The correct program uses the slice only while the buffer lives:

```
let buf = RawBuf::new(b"listen port 8080");
let bytes = buf.as_bytes(); // borrows `buf`
let first_word = bytes.split(|&b| b == b' ').next().unwrap_or(&[]);
println!("{}", String::from_utf8_lossy(first_word));
drop(buf); // fine: `bytes` and `first_word` are no longer used
```

```
listen
```

The Rustonomicon's advice is the lesson of this section: "The easiest way to avoid unbounded lifetimes is to use lifetime elision at the function boundary." Bound the lifetime as soon as possible, ideally in the same function that dereferences the pointer, by giving that function a signature that ties the result to the owner of the memory.

Who promises the lifetime?

Several standard library functions return references with unbounded lifetimes. Here are four, from the 1.99 source, without their `const` qualifiers:

Function	Signature	Lifetime of the result
`slice::from_raw_parts`	`unsafe fn from_raw_parts<'a, T>(data: *const T, len: usize) -> &'a [T]`	picked by the caller
`<*const T>::as_ref`	`unsafe fn as_ref<'a>(self) -> Option<&'a T>`	picked by the caller
`NonNull::as_ref`	`unsafe fn as_ref<'a>(&self) -> &'a T`	picked by the caller, not tied to `&self`
`str::from_utf8_unchecked`	`unsafe fn from_utf8_unchecked(v: &[u8]) -> &str`	elided: tied to `v`

In the first three, `'a` appears only in the result. The compiler cannot check it, so the documentation turns it into a precondition that the caller promises. For `from_raw_parts`, one of the safety requirements reads: "The memory referenced by the returned slice must not be mutated for the duration of lifetime `'a`, except inside an `UnsafeCell`." And a *Caveat* section gives the same advice as the Rustonomicon:

> The lifetime for the returned slice is inferred from its usage. To prevent accidental misuse, it's suggested to tie the lifetime to whichever source lifetime is safe in the context, such as by providing a helper function taking the lifetime of a host value for the slice, or by explicit annotation.

`RawBuf::as_bytes` is such a helper: its host value is `self`. The fourth row shows the other style. `from_utf8_unchecked` takes a reference, not a pointer, so elision bounds its result to `v`, and the only promise left to the caller is that the bytes are UTF-8.

Note the third row: `NonNull::as_ref` takes `&self` and still returns an unbounded lifetime. The `&self` is a borrow of the *pointer*, which says nothing about how long the memory it points to stays valid; tying the result to it would be wrong in both directions. Read signatures of `unsafe` functions with this in mind: an elided-looking `&self` is not always where the lifetime comes from.

13.2 `PhantomData`

Unused lifetime parameters: E0392

Suppose we want our own slice iterator, built from raw pointers the way the standard library's `slice::Iter` is. It hands out `&'a T`, so the type needs a lifetime parameter `'a`; but its fields are a raw pointer and a count, which carry no lifetime (`code/compile_fail/ch13_unused_lifetime.rs`):

```
pub struct RawIter<'a, T> {
    ptr: *const T,
    remaining: usize,
}
```

```
error[E0392]: lifetime parameter `'a` is never used
 --> compile_fail/ch13_unused_lifetime.rs:6:20
  |
6 | pub struct RawIter<'a, T> {
  |                    ^^ unused lifetime parameter
  |
  = help: consider removing `'a`, referring to it in a field, or using a marker such as
`PhantomData`
```

E0392 says "lifetime parameter `'a` is never used". The compiler insists for a good reason: the meaning of a lifetime parameter, and the *variance* of the type in it (Chapter 4), are computed from the fields. With no field mentioning `'a`, there is nothing to compute from: `RawIter<'a, T>` could be shrunk, grown, or neither, and the compiler would have to guess. The help lists the options, and the third one is the tool for this job: "a marker such as `PhantomData`".

Tying a raw pointer to a borrow

`PhantomData<X>` is a zero-sized type that holds no data. It tells the compiler to treat the containing type *as if* it held a value of type `X`: for variance, for auto traits such as `Send`, for the drop check, and for implied bounds. Here is the iterator with a marker that says "this type borrows values of type `T` for `'a`" (`code/examples/ch13_raw_iter.rs`):

```
/// Iterates over a slice with a raw pointer, like `slice::Iter` does.
pub struct RawIter<'a, T> {
    ptr: *const T,
    remaining: usize,
    /// "This type borrows `T`s for `'a`": no data, only a lifetime and a
    /// variance for the compiler.
    _borrow: PhantomData<&'a T>,
}

impl<'a, T> RawIter<'a, T> {
    pub fn new(slice: &'a [T]) -> Self {
        RawIter {
            ptr: slice.as_ptr(),
            remaining: slice.len(),
            _borrow: PhantomData,
        }
    }
}

impl<'a, T> Iterator for RawIter<'a, T> {
    type Item = &'a T;

    fn next(&mut self) -> Option<&'a T> {
        if self.remaining == 0 {
            return None;
```

```
        }
        // SAFETY: `ptr` points into the slice passed to `new`, at an element
        // not yet returned (`remaining > 0`). The slice is borrowed for `'a`,
        // so it is neither freed nor mutated while `'a` lasts, and returning
        // `&'a T` is exactly what that borrow allows.
        let item = unsafe { &*self.ptr };
        // SAFETY: there are `remaining` elements from `ptr` on, so `ptr + 1`
        // stays within the slice or one past its end.
        self.ptr = unsafe { self.ptr.add(1) };
        self.remaining -= 1;
        Some(item)
    }
}
```

```
let words = vec![String::from("listen"), String::from("port")];
let longest = RawIter::new(&words).max_by_key(|w| w.len());
println!("{longest:?}");
```

```
Some("listen")
```

The `SAFETY:` comment of `next` relies on one fact: the slice stays borrowed, and therefore alive and unmodified, for `'a`. The signature of `new` is what makes that fact true. It takes a `&'a [T]` and returns a `RawIter<'a, T>`, so the borrow checker keeps the slice borrowed for as long as the iterator, or any item it returned, is in use (`code/compile_fail/ch13_raw_iter_borrow.rs`):

```
pub fn caller() {
    let mut words = vec![String::from("listen"), String::from("port")];
    let first = RawIter::new(&words).next();
    words.push(String::from("8080")); // may reallocate: `first` would dangle
    println!("{first:?}");
}
```

```
error[E0502]: cannot borrow `words` as mutable because it is also borrowed as immutable
  --> compile_fail/ch13_raw_iter_borrow.rs:43:5
   |
42 |     let first = RawIter::new(&words).next();
   |                              ------ immutable borrow occurs here
43 |     words.push(String::from("8080")); // may reallocate: `first` would dangle
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
44 |     println!("{first:?}");
   |                ----- immutable borrow later used here
```

This is the E0502 of Chapter 1, and that is the point: a type built on raw pointers gets the same protection as `slice::Iter`, because its signature and its marker say the same thing a `&'a T` field would. The standard library's `slice::Iter<'a, T>` has exactly this field, `_marker: PhantomData<&'a T>`, next to its raw pointers.

Choosing variance

The marker also decides the variance of the type, which is the part that is easy to get wrong. Here are the common choices, with what they mean for a lifetime `'a` and a type `T` (the auto-trait column was checked with rustc 1.99):

Marker	Variance	Also
`PhantomData<&'a T>`	covariant in `'a` and `T`	requires `T: 'a`; `Send` and `Sync` like `&T`
`PhantomData<&'a mut T>`	covariant in `'a`, invariant in `T`	requires `T: 'a`
`PhantomData<T>`	covariant in `T`	"owns a `T`" for the drop check
`PhantomData<*const T>`	covariant in `T`	not `Send`, not `Sync`
`PhantomData<*mut T>`	invariant in `T`	not `Send`, not `Sync`
`PhantomData<fn(T)>`	contravariant in `T`	always `Send` and `Sync`
`PhantomData<fn() -> T>`	covariant in `T`	always `Send` and `Sync`; owns nothing
`PhantomData<fn(&'a ()) -> &'a ()>`	invariant in `'a`	always `Send` and `Sync`
`PhantomData<Cell<&'a ()>>`	invariant in `'a`	not `Sync`

To see the difference, here are three handle types that differ only in their marker (`code/examples/ch13_variance_markers.rs`):

```
/// Covariant in `'a`, like `&'a T`.
pub struct Shared<'a> {
    _marker: PhantomData<&'a ()>,
}

/// Contravariant in `'a`, like the argument of `fn(&'a ())`.
pub struct Sink<'a> {
    _marker: PhantomData<fn(&'a ())>,
}

/// Invariant in `'a`: the argument and the result of a function at once.
pub struct Exact<'a> {
    _marker: PhantomData<fn(&'a ()) -> &'a ()>,
}
```

```
/// Covariance: a handle for a longer region can be used where a shorter one
/// is expected.
fn shrink<'long: 'short, 'short>(handle: Shared<'long>) -> Shared<'short> {
    handle
}

/// Contravariance: the other way around.
fn grow<'long: 'short, 'short>(sink: Sink<'short>) -> Sink<'long> {
    sink
}

/// Invariance: only the same region.
fn same<'a>(exact: Exact<'a>) -> Exact<'a> {
    exact
}
```

```
covariant, contravariant and invariant handles all compile
```

All three conversions compile. Trying to shrink the invariant one does not (`code/compile_fail/ch13_invariant_marker.rs`):

```
pub fn shrink<'long: 'short, 'short>(exact: Exact<'long>) -> Exact<'short> {
    exact
}
```

```
error: lifetime may not live long enough
  --> compile_fail/ch13_invariant_marker.rs:13:5
   |
12 | pub fn shrink<'long: 'short, 'short>(exact: Exact<'long>) -> Exact<'short> {
   |               -----          ------ lifetime `'short` defined here
   |               |
   |               lifetime `'long` defined here
13 |     exact
   |     ^^^^^ function was supposed to return data with lifetime `'long` but it is
returning data with lifetime `'short`
   |
   = help: consider adding the following bound: `'short: 'long`
   = note: requirement occurs because of the type `Exact<'_>`, which makes the generic
argument `'_` invariant
   = note: the struct `Exact<'a>` is invariant over the parameter `'a`
   = help: see <https://doc.rust-lang.org/nomicon/subtyping.html> for more information about
variance
```

The main message, "function was supposed to return data with lifetime `'long` but it is returning data with lifetime `'short`", sounds backwards until you remember what invariance means: `Exact<'long>` and `Exact<'short>` are unrelated types, so returning one where the other is expected would need the two regions to be *equal*, and the help offers the missing half, `'short: 'long`. The notes then name the cause: "the struct `Exact<'a>` is invariant over the parameter `'a`". When you see this note about a type of your own, look at its `PhantomData`.

When variance matters: a write handle on `NonNull`

Which marker to choose follows the rule of Chapter 4: a type through which you can *write* a `T` must be invariant in `T`, or a caller could shrink the type, write something short-lived, and leave a long-lived owner holding a dangling reference. `Cell<&'a str>` is invariant for this reason. A raw-pointer type gets no such protection automatically, and the documentation of `NonNull` warns about it:

> Unlike `*mut T`, `NonNull<T>` is covariant over `T`. This is usually the correct choice for most data structures and safe abstractions, such as `Box`, `Rc`, `Arc`, `Vec`, and `LinkedList`.
>
> In rare cases, if your type exposes a way to mutate the value of `T` through a `NonNull<T>`, and you need to prevent unsoundness from variance (for example, if `T` could be a reference with a shorter lifetime), you should add a field to make your type invariant, such as `PhantomData<Cell<T>>` or `PhantomData<&'a mut T>`.

Here is such a type, written first with the wrong marker (`code/examples/ch13_covariant_slot.rs`). `CovariantSlot<'s, 'v>` is a handle that overwrites a `&'v str` owned by someone else:

```rust
/// UNSOUND: covariant in `'v` (through `NonNull`), yet it writes a `&'v str`.
pub struct CovariantSlot<'s, 'v> {
    target: NonNull<&'v str>,
    _borrow: PhantomData<&'s ()>, // mentions `'s` only
}
```

Its marker mentions only `'s`, so its variance in `'v` comes from `NonNull<&'v str>`: covariant. This function compiles:

```rust
/// Compiles, and returns a dangling `&'static str`. Never called: calling it
/// is undefined behavior.
#[allow(dead_code)]
```

```
fn dangling() -> &'static str {
    let mut name: &'static str = "static";
    {
        let mut slot = CovariantSlot::new(&mut name);
        let temporary = String::from("temporary");
        slot.set(&temporary); // accepted: `'v` shrinks to fit `temporary`
    }
    name // points to the freed `temporary`
}
```

`name` is a `&'static str`, so the slot starts as a `CovariantSlot<'_, 'static>`. Covariance lets the compiler shrink `'v` to the lifetime of `temporary`, `set` accepts `&temporary`, and `name` ends up pointing to a freed string, while its type still says `'static`. As before, the program never calls this function.

The correct type uses the marker the documentation recommends (`code/examples/ch13_out_slot.rs`):

```
/// A handle that can overwrite a `&'v str` owned by someone else, for `'s`.
/// (In safe code you would store the `&'s mut &'v str` itself; the raw pointer
/// stands for one that came from C, or from a node of a linked structure.)
pub struct OutSlot<'s, 'v> {
    target: NonNull<&'v str>,
    /// Covariant in `'s`, invariant in `'v`, like `&'s mut &'v str`.
    _borrow: PhantomData<&'s mut &'v str>,
}

impl<'s, 'v> OutSlot<'s, 'v> {
    pub fn new(target: &'s mut &'v str) -> Self {
        OutSlot {
            target: NonNull::from(target),
            _borrow: PhantomData,
        }
    }

    pub fn set(&mut self, value: &'v str) {
        // SAFETY: `target` came from a `&'s mut &'v str` that this handle
        // keeps borrowed for `'s`, so the slot is valid and nobody else can
        // access it. `value` lives for `'v`, the lifetime the slot's owner
        // expects; invariance in `'v` stops anyone from shrinking `'v` first.
        unsafe { *self.target.as_ptr() = value }
    }
}
```

```
let mut setting: &str = "unset";
{
    let mut slot = OutSlot::new(&mut setting);
    slot.set("listen");
}
println!("{setting}");
```

```
listen
```

`PhantomData<&'s mut &'v str>` makes `OutSlot` behave like the `&'s mut &'v str` it stands for: covariant in `'s`, invariant in `'v`. The dangerous function is now rejected (`code/compile_fail/ch13_out_slot_variance.rs`):

```
pub fn dangling() -> &'static str {
    let mut name: &'static str = "static";
    {
        let mut slot = OutSlot::new(&mut name);
        let temporary = String::from("temporary");
        slot.set(&temporary);
    }
```

```
    name
}
```

```
error[E0597]: `temporary` does not live long enough
  --> compile_fail/ch13_out_slot_variance.rs:33:18
   |
29 |     let mut name: &'static str = "static";
   |                   ------------ type annotation requires that `temporary` is borrowed
for `'static`
...
32 |         let temporary = String::from("temporary");
   |             --------- binding `temporary` declared here
33 |         slot.set(&temporary);
   |                  ^^^^^^^^^^ borrowed value does not live long enough
34 |     }
   |     - `temporary` dropped here while still borrowed
```

The label "type annotation requires that `temporary` is borrowed for `'static`" points at `name`'s type. With invariance, `'v` must stay exactly `'static`, so `set` demands a `&'static str`, and `&temporary` cannot be one. The `SAFETY:` comment of `set` says that invariance is part of the argument; with the wrong marker, the same comment would have been a false statement that no compiler could catch.

13.3 Self-referential structs

Why safe Rust rejects them

A document that keeps its text and a reference to its title line *inside that text* is a natural thing to want. Here is the attempt everyone writes once (`code/compile_fail/ch13_self_ref_new.rs`; it produces two errors):

```
pub struct Document<'a> {
    text: String,
    title: &'a str, // meant to point into `text`
}

impl<'a> Document<'a> {
    pub fn new(text: String) -> Document<'a> {
        let title = text.lines().next().unwrap_or("");
        Document { text, title }
    }
}
```

```
error[E0515]: cannot return value referencing function parameter `text`
  --> compile_fail/ch13_self_ref_new.rs:14:9
   |
13 |         let title = text.lines().next().unwrap_or("");
   |                     ---- `text` is borrowed here
14 |         Document { text, title }
   |         ^^^^^^^^^^^^^^^^^^^^^^^^ returns a value referencing data owned by the current
function

error[E0505]: cannot move out of `text` because it is borrowed
  --> compile_fail/ch13_self_ref_new.rs:14:20
   |
11 | impl<'a> Document<'a> {
   |      -- lifetime `'a` defined here
12 |     pub fn new(text: String) -> Document<'a> {
   |                ---- binding `text` declared here
13 |         let title = text.lines().next().unwrap_or("");
   |                     ---- borrow of `text` occurs here
```

```
14 |         Document { text, title }
   |         -----------^^^^---------
   |         |          |
   |         |          move out of `text` occurs here
   |         returning this value requires that `text` is borrowed for `'a`
```

E0515 says that `new` returns a value referencing its parameter `text`, which the function owns. E0505 adds that `text` is moved into the struct while it is still borrowed by `title`. Both are the compiler's way of saying that `'a` has no correct value. It would have to be "the lifetime of the `Document` that contains this very field", and no such lifetime exists: lifetime parameters are picked by whoever *uses* the type, before the value exists. (Writing `&'self str` does not help either; `'self` is not a lifetime name Rust knows.)

Curiously, the same struct can be built in place (`code/compile_fail/ch13_self_ref_move.rs`):

```
pub fn load() {
    let mut doc = Document {
        text: String::from("Config\nport 8080"),
        title: "",
    };
    doc.title = doc.text.lines().next().unwrap_or("");
    println!("{}", doc.title); // fine: used in place
    let moved = doc; // a move would leave `title` pointing at the old place
    println!("{}", moved.title);
}
```

```
error[E0505]: cannot move out of `doc` because it is borrowed
  --> compile_fail/ch13_self_ref_move.rs:18:17
   |
12 |     let mut doc = Document {
   |         ------- binding `doc` declared here
...
16 |     doc.title = doc.text.lines().next().unwrap_or("");
   |                 -------- borrow of `doc.text` occurs here
17 |     println!("{}", doc.title); // fine: used in place
18 |     let moved = doc; // a move would leave `title` pointing at the old place
   |                 ^^^
   |                 |
   |                 move out of `doc` occurs here
   |                 borrow later used here
```

The assignment to `doc.title` and the first `println!` are accepted: `doc` is borrowed by one of its own fields, and the borrow checker treats it like any other borrow. The price is that `doc` is frozen. The move on the next line is E0505, with an unusual pair of labels on the same spot: "move out of `doc` occurs here" and "borrow later used here", because moving `doc` also moves the borrow stored in `doc.title`. A mutation such as `doc.text.push_str(...)` followed by a use of `doc.title` is rejected too (as E0502).

Both restrictions protect real invariants. A move copies the bytes of the struct to a new place, so a reference into the old place would dangle whenever the referenced data lives *inside* the struct (an array field, say). For a `String`, the characters live on the heap and do not move, but a `push_str` may reallocate them, which would leave `title` dangling just the same. The borrow checker does not know which case it is looking at, so it forbids both.

Indices instead of references

The standard alternative stores *positions* instead of references, and borrows on demand (`code/examples/ch13_document_ranges.rs`):

```
pub struct Document {
    text: String,
    title: Range<usize>, // byte positions in `text`, not a reference
}

impl Document {
    pub fn new(text: String) -> Document {
        let end = text.find('\n').unwrap_or(text.len());
        Document {
            text,
            title: 0..end,
        }
    }

    /// Elided: `fn title<'a>(&'a self) -> &'a str`, a borrow of the document.
    pub fn title(&self) -> &str {
        &self.text[self.title.clone()]
    }

    /// Mutation is allowed, and must keep the positions valid.
    pub fn append(&mut self, more: &str) {
        self.text.push_str(more); // may reallocate: a `&str` field would dangle
    }
}
```

```
let mut doc = Document::new(String::from("Config\nport 8080"));
println!("{}", doc.title());
doc.append("\nlog-level debug");
let moved = doc; // moving is fine: a range does not point anywhere
println!("{} ({} bytes)", moved.title(), moved.text.len());
```

```
Config
Config (32 bytes)
```

A `Range<usize>` does not point anywhere, so moving the document is free, and `append` may reallocate the text without invalidating anything. The borrow appears only in `title(&self) -> &str`, where elision ties it to the borrow of the document: exactly the bounded lifetime of the previous section, without any `unsafe`. The cost is a bounds check when slicing, and the duty to keep the positions consistent with the text (here, `append` only adds at the end). The same idea scales up: arenas with index handles, graph nodes stored in a `Vec` and referred to by number, string interners that hand out ids. Chapter 14 returns to the alternative of keeping the owner and the borrowing view in two separate types.

How `Pin` relates

Chapter 11 showed that the compiler *does* build self-referential values: an async block that keeps a reference to its own local across an `.await`. They are sound because they are polled through `Pin<&mut Self>`, which promises that the value will not move again. `Pin` is that promise and nothing more: it does not give safe code a way to create a self-reference. Building a self-referential type by hand still means storing a raw pointer into the value, setting it after the value has reached its final place, preventing moves with `Pin`, and writing the `SAFETY:` arguments for every access. That is possible,

and it is a job for a small, carefully reviewed library type, not for application code. Before you start, check whether indices, or splitting the owner from the view, would do.

13.4 The drop check and `#[may_dangle]`

Chapter 5 introduced the drop check: when a value is dropped, any borrow its destructor might use must still be valid. Here is the rule seen from the side of a type author (`code/compile_fail/ch13_dropck_log.rs`):

```
pub struct Log<T> {
    entries: Vec<T>,
}

impl<T> Drop for Log<T> {
    fn drop(&mut self) {
        println!("dropping a log of {} entries", self.entries.len());
    }
}

pub fn record() {
    let mut log = Log {
        entries: Vec::new(),
    };
    let line = String::from("GET /index.html");
    log.entries.push(&line);
}
```

```
error[E0597]: `line` does not live long enough
  --> compile_fail/ch13_dropck_log.rs:21:22
   |
20 |     let line = String::from("GET /index.html");
   |         ---- binding `line` declared here
21 |     log.entries.push(&line);
   |                      ^^^^^ borrowed value does not live long enough
22 | }
   | -
   | |
   | `line` dropped here while still borrowed
   | borrow might be used here, when `log` is dropped and runs the `Drop` code for type
`Log`
   |
   = note: values in a scope are dropped in the opposite order they are defined
```

Locals are dropped in the reverse order of declaration, so `line` is dropped before `log`. When `log` is dropped, its `Drop` impl runs, and for all the compiler knows, `drop` could read the `&String` entries, after `line` is gone. The label says precisely that: "borrow might be used here, when `log` is dropped and runs the `Drop` code for type `Log`". Our `drop` only reads the length, but the drop check does not look into the body of `drop`. A generic `Drop` impl is a signature like any other, and it says "may use `T`".

Yet a plain `Vec<&String>` in the same position is fine (`code/examples/ch13_dropck.rs`):

```
fn record_in_vec() -> usize {
    let mut entries = Vec::new();
    let line = String::from("GET /index.html");
    entries.push(&line);
    entries.len()
} // `line` is dropped first, then `entries`: accepted
```

`Vec` has a `Drop` impl too. The difference is a promise written into it. In the standard library's source, it reads like this (simplified: the 1.99 source also carries `const` annotations):

```
unsafe impl<#[may_dangle] T, A: Allocator> Drop for Vec<T, A> {
    fn drop(&mut self) { /* drops the elements, then the buffer is freed */ }
}
```

`#[may_dangle] T` tells the drop check: "when this `drop` runs, values of type `T` may already be dangling; I will not use them, except to drop them". The impl must be `unsafe impl`, because the compiler takes the promise on trust: a `drop` that printed its elements would break it. Together with the promise, the standard library keeps a `PhantomData<T>` in the buffer type behind `Vec`, which tells the drop check that dropping a `Vec<T>` drops `T` values. That matters when `T` itself has a destructor that uses borrows: then the strict rule applies again, through `T`. For a `Vec<&String>`, dropping a `&String` does nothing, so the borrow may dangle.

The attribute is unstable. On stable Rust 1.99 it is E0658 (`code/compile_fail/ch13_may_dangle_unstable.rs`):

```
// SAFETY: `drop` never reads or writes a `T`; it only reads the length.
unsafe impl<#[may_dangle] T> Drop for Log<T> {
    fn drop(&mut self) {
        println!("dropping a log of {} entries", self.entries.len());
    }
}
```

```
error[E0658]: the `may_dangle` attribute has unstable semantics and may be removed in the
future
  --> compile_fail/ch13_may_dangle_unstable.rs:10:15
   |
10 | unsafe impl<#[may_dangle] T> Drop for Log<T> {
   |               ^^^^^^^^^^
   |
   = note: see issue #34761 <https://github.com/rust-lang/rust/issues/34761> for more
information
```

With `#![feature(dropck_eyepatch)]`, the nightly compiler of 2026-10-08 accepts this impl, and with it the `record` function of `ch13_dropck_log.rs`. On stable, the usual workaround is to keep the `Drop` impl away from `T`: put the cleanup in a field whose type does not mention `T`, and give the generic type no `Drop` impl of its own:

```
/// The `Drop` lives in a field that holds no `T`.
struct Report {
    count: usize,
}

impl Drop for Report {
    fn drop(&mut self) {
        println!("dropping a log of {} entries", self.count);
    }
}

pub struct Log<T> {
    entries: Vec<T>,
    report: Report,
}

impl<T> Log<T> {
    pub fn new() -> Self {
```

```
        Log {
            entries: Vec::new(),
            report: Report { count: 0 },
        }
    }

    pub fn push(&mut self, entry: T) {
        self.entries.push(entry);
        self.report.count = self.entries.len();
    }
}

fn record_in_log() {
    let mut log = Log::new();
    let line = String::from("GET /index.html");
    log.push(&line);
} // `Log<&String>` has no `Drop` impl of its own: accepted
```

```
vec: 1 entries
dropping a log of 1 entries
```

Dropping a `Log<&String>` now drops its fields: a `Vec<&String>`, which is lenient thanks to `#[may_dangle]`, and a `Report`, which holds no borrow. Nothing in the drop glue can use the dangling reference, and the compiler can see it. As a user of collections, the rule to remember is short: standard containers (`Vec`, `Box`, `HashMap`, ...) may hold borrows that die just before them; your own generic type with a `Drop` impl may not, unless the `Drop` is moved out of its way.

13.5 Leaking is safe

`mem::forget`, `Box::leak` and cycles

Safe Rust lets you skip a destructor in several ways (`code/examples/ch13_leaks.rs`):

```
struct Connection {
    name: &'static str,
}

impl Drop for Connection {
    fn drop(&mut self) {
        println!("closing {}", self.name);
    }
}
```

```
/// A node that may point to another node.
struct Node {
    _connection: Connection,
    next: RefCell<Option<Rc<Node>>>,
}
```

```
drop(Connection { name: "dropped" }); // runs `drop`
mem::forget(Connection { name: "forgotten" }); // never runs it

let leaked: &'static mut Connection =
    Box::leak(Box::new(Connection { name: "leaked" }));
println!("{} lives until the end of the program", leaked.name);

let first = Rc::new(Node {
    _connection: Connection { name: "in a cycle" },
    next: RefCell::new(None),
});
```

```
*first.next.borrow_mut() = Some(Rc::clone(&first)); // points to itself
drop(first); // the count drops from 2 to 1: never 0, never freed
println!("end of main");
```

```
closing dropped
leaked lives until the end of the program
end of main
```

Only the first connection is closed. `mem::forget` takes ownership and never runs `drop`. `Box::leak` turns a box into a reference that may have any lifetime, `'static` included, because the memory is simply never freed (Chapter 7 uses it as a source of `&'static`); its documentation says "the type `T` must outlive the chosen lifetime `'a`", which is the only requirement left. And a reference cycle of `Rc` handles keeps its own count above zero after the last outside handle is dropped, so its destructor never runs either, with no special function involved at all.

None of this needs `unsafe`, and the documentation of `mem::forget` explains why it does not:

> `forget` is not marked as `unsafe`, because Rust's safety guarantees do not include a guarantee that destructors will always run.

and draws the consequence for everyone who writes `unsafe` code:

> Because forgetting a value is allowed, any `unsafe` code you write must allow for this possibility. You cannot return a value and expect that the caller will necessarily run the value's destructor.

Why `thread::scope` takes a closure

This rule was learned the hard way. In the months before Rust 1.0, the standard library had a function `thread::scoped` that returned a `JoinGuard`, described in the 1.0 documentation as "An RAII-style guard that will block until thread termination when dropped" (https://doc.rust-lang.org/1.0.0/std/thread/index.html). The scoped thread could borrow the caller's locals, and the guard's destructor joined it before those locals could die. RFC 1066, "safe `mem::forget`" (https://rust-lang.github.io/rfcs/1066-safe-mem-forget.html), records what went wrong: "It was recently discovered by @arielb1 that the `thread::scoped` API was unsound", because "the safety of `thread::scoped` relied on the fact that the `Drop` implementation for `JoinGuard` was *always* run", and a cycle of `Rc` handles could leak the guard without any `unsafe` code. Here is the idea as a sketch (not the historical code, and not compiled):

```
let data = vec![1, 2, 3];
let guard = thread::scoped(|| println!("{data:?}")); // the thread borrows `data`
mem::forget(guard); // the destructor that would join never runs
drop(data);         // the thread may still be reading `data`
```

The RFC made `mem::forget` safe and stated that not running destructors is not unsafe. `thread::scoped` shipped in Rust 1.0 only as an unstable function behind a feature gate, and today's standard library no longer has it. Scoped threads came back, stable, in Rust 1.63, with a design that does not depend on any destructor:

```
pub fn scope<'env, F, T>(f: F) -> T
where
    F: for<'scope> FnOnce(&'scope Scope<'scope, 'env>) -> T,
```

The join is not in a guard that the caller holds. It is in the body of `scope`, which calls `f`, then waits for every thread spawned through the `Scope`, then returns. There is no value the caller could forget, because the caller does not get control back until the threads are done (`code/examples/ch13_scope.rs`):

```
let words = vec!["listen", "port", "8080"];
let mut total = 0;
thread::scope(|s| {
    let lengths = s.spawn(|| words.iter().map(|w| w.len()).sum::<usize>());
    // This handle is dropped without `join`: the scope still waits for it.
    s.spawn(|| println!("a scoped thread sees {} words", words.len()));
    total = lengths.join().unwrap();
});
println!("after the scope: {total} bytes, and `words` is ours again: {words:?}");
```

```
a scoped thread sees 3 words
after the scope: 14 bytes, and `words` is ours again: ["listen", "port", "8080"]
```

The second thread's handle was never joined, and the scope waited for it anyway. The design is the callback pattern you have seen throughout this book: the callee, not the caller, controls when the region ends. The higher-ranked `for<'scope>` (Chapter 9) adds the last piece: `'scope` is picked inside `scope`, so nothing that mentions it can leave the closure (`code/compile_fail/ch13_scope_escape.rs`):

```
pub fn escape() -> usize {
    let words = vec!["listen", "port"];
    let handle = thread::scope(|s| s.spawn(|| words.len()));
    handle.join().unwrap()
}
```

```
error: lifetime may not live long enough
  --> compile_fail/ch13_scope_escape.rs:10:36
   |
10 |     let handle = thread::scope(|s| s.spawn(|| words.len()));
   |                                 -- ^^^^^^^^^^^^^^^^^^^^^^^ returning this value
requires that `'1` must outlive `'2`
   |                                 ||
   |                                 |return type of closure is ScopedJoinHandle<'2, usize>
   |                                 has type `&'1 Scope<'1, '_>`
   |
help: consider adding 'move' keyword before the nested closure
   |
10 |     let handle = thread::scope(|s| s.spawn(move || words.len()));
   |                                             ++++
```

The closure's return type, `ScopedJoinHandle<'2, usize>`, would have to outlive the `Scope` it came from, `'1`. The help is a false lead: with `move`, the closure owns `words`, but the handle still carries `'scope`, and the error stays (`code/compile_fail/ch13_scope_escape_move.rs`):

```
error: lifetime may not live long enough
  --> compile_fail/ch13_scope_escape_move.rs:10:36
   |
10 |     let handle = thread::scope(|s| s.spawn(move || words.len()));
   |                                 -- ^^^^^^^^^^^^^^^^^^^^^^^^^^^ returning this value
requires that `'1` must outlive `'2`
   |                                 ||
   |                                 |return type of closure is ScopedJoinHandle<'2, usize>
   |                                 has type `&'1 Scope<'1, '_>`
```

The standard library's implementation is written with leaks in mind, too. Its counter of running threads checks for overflow, and the source comment explains when that could happen: "This can

only reasonably happen by mem::forget()'ing a lot of ScopedJoinHandles." Chapter 11 explains why an async version of the same design is much harder: there, the scope would itself be a future, and a future can be forgotten.

13.6 Checking `unsafe` code with Miri

The borrow checker cannot check `unsafe` code, and tests only check that a program produces the right output, not that it avoids undefined behavior. Miri fills part of that gap. Its README (https://github.com/rust-lang/miri) describes it as "an Undefined Behavior detection tool for Rust": it runs your program in an interpreter that tracks every allocation and every pointer, and stops with an error when an execution does something the language forbids. The kinds of errors it lists include:

- out-of-bounds memory accesses and use-after-free;
- invalid use of uninitialized data;
- not sufficiently aligned memory accesses and references;
- violation of basic type invariants (a `bool` that is not 0 or 1, for example);
- data races;
- experimentally, violations of the Stacked Borrows aliasing rules, or of the Tree Borrows rules as an alternative.

Miri is distributed for nightly toolchains only, as a `rustup` component. The README gives these commands:

```
rustup +nightly component add miri
cargo +nightly miri test
cargo +nightly miri run
```

`cargo miri` wraps Cargo, so for an example of this book you would add the usual `--example ch13_raw_iter`. Miri's own flags go in the `MIRIFLAGS` environment variable; one of them, `-Zmiri-tree-borrows`, "replaces Stacked Borrows with the Tree Borrows rules", which the README calls "even more experimental".

Two limits matter. First, Miri checks the executions it runs, and nothing else. Using the slice returned by `keep_forever` is a use-after-free, the first kind of error in the list, but only an execution that calls `keep_forever` contains it; a function that is never called, or a path that no test takes, is never checked. Second, it checks the *behavior* of your code, not its *contract*. An execution of `CovariantSlot` that only stores string literals has no undefined behavior at all, so there is nothing to find; the unsoundness shows only when a caller uses the covariance, and a test has to be written to do that. The README is explicit: "Miri does not catch every violation of the Rust specification in your program", and "Miri fundamentally cannot ensure that your code is sound." Use it as you would use a sanitizer: run your tests under it, especially tests that exercise the edges of your `unsafe` code, and keep writing the `SAFETY:` arguments that explain why the code is right.

> **NOTE**
>
> Miri was not installed on the machine used for this book (checked on 2026-10-12 with the nightly of 2026-10-08), so the commands above come from the README and were not run. Check the README for the current instructions.

13.7 Common confusions

- "**`unsafe` turns off the borrow checker.**" It does not. Inside an `unsafe` block every reference is still checked; `unsafe` only allows a few extra operations, such as dereferencing raw pointers. What escapes the checker is the lifetime of a reference made from a raw pointer, until a signature bounds it.
- "**`as_bytes_unbounded` is fine as long as callers are careful.**" A safe function must be safe for *every* caller the compiler accepts. If a safe caller can create a dangling reference without writing `unsafe`, the function is unsound, however unlikely the misuse.
- "**`PhantomData` is just there to silence E0392.**" It decides variance, auto traits and drop behavior. A marker that silences the error but gives the wrong variance compiles, and can be unsound (`CovariantSlot`).
- "**`Pin` makes self-referential structs safe to write.**" It keeps a value from moving; creating the self-reference is still up to you, with `unsafe`.
- "**A destructor is a guarantee.**" It is a strong habit, not a guarantee: `mem::forget`, `Box::leak` and `Rc` cycles all skip it in safe code. Soundness must never depend on it.
- "**Miri passed, so my code is sound.**" Miri found no undefined behavior in the executions it ran. Soundness is a property of every possible caller.

13.8 Summary

- Raw pointers carry no lifetime. A reference made from one (`&*ptr`, `slice::from_raw_parts`, `ptr.as_ref()`) has an *unbounded* lifetime that becomes whatever the context asks for, even `'static`.
- Bound it immediately with a signature: `fn as_bytes(&self) -> &[u8]` ties the result to `self`, and the borrow checker then enforces it on every caller (E0515 for the dangling version). `fn f<'a>(&self) -> &'a [u8]` compiles and is unsound.
- In the standard library's raw-pointer APIs, a lifetime that appears only in the result is a promise of the caller, stated in the safety documentation.
- A lifetime parameter must be used by a field (E0392). `PhantomData<X>` makes a type behave as if it held an `X`: `PhantomData<&'a T>` for a borrowing iterator, `PhantomData<&'a mut T>` or `PhantomData<Cell<T>>` for invariance, `PhantomData<fn(T)>` for contravariance. `NonNull<T>` is covariant, so a type that writes through it needs an invariant marker.

- Safe Rust cannot express a struct that borrows from itself: the constructor is E0515/E0505, and a value built in place is frozen (E0505 on a move). Store indices or ranges and borrow on demand. `Pin` protects the self-referential futures the compiler builds; it does not create self-references.
- A generic type with a `Drop` impl makes the drop check assume that `drop` uses its borrows ("borrow might be used here, when `log` is dropped"). `Vec` avoids this with `unsafe impl<#[may_dangle] T> Drop`, which is unstable (E0658); on stable, move the `Drop` into a field that does not mention `T`.
- Leaking is safe (`mem::forget`, `Box::leak`, `Rc` cycles). The pre-1.0 `thread::scoped` relied on a guard's destructor and was unsound; `thread::scope` (1.63) joins inside its own body and uses `for<'scope>` so that nothing escapes.
- Miri (nightly) detects undefined behavior in the executions it runs, including use-after-free and aliasing violations; it cannot prove soundness.

13.9 Exercises

1. **Bound it with a helper.** For `ch13_raw_buf.rs`, write an `unsafe fn bytes_at` that takes `owner: &'a RawBuf`, a start and a length, and returns a `&'a [u8]`. Give it a `# Safety` section in its documentation comment that says what the caller must promise, and a `SAFETY:` comment at the call to `from_raw_parts`. Which of the preconditions of `from_raw_parts` does the signature take care of, and which remain the caller's? *Hint:* the lifetime and the "not mutated for `'a`" requirement come from `owner`; the range must still be checked or promised.
2. **A mutable raw iterator.** Write `RawIterMut<'a, T>`, which yields `&'a mut T`. Which `PhantomData` marker gives it the variance of `slice::IterMut`, and why would `PhantomData<&'a T>` be wrong? *Hint:* `&'a mut T` is invariant in `T` (Chapter 4). Try to write a function that turns a `RawIterMut<'a, &'static str>` into a `RawIterMut<'a, &'short str>` with each marker.
3. **Variance from two fields.** In `ch13_out_slot_variance.rs`, replace the marker with `PhantomData<(&'s (), fn(&'v str))>`. Is `dangling` still rejected? Work out the variance of `OutSlot` in `'v` before you compile, then check your answer. *Hint:* `NonNull<&'v str>` is covariant in `'v` and `fn(&'v str)` is contravariant; Chapter 4 explains how the variances of several fields combine.
4. **A document without a `Range`.** Rewrite `Document` from `ch13_document_ranges.rs` so that it owns its text and offers `fn lines(&self) -> impl Iterator<Item = &str>` and `fn title(&self) -> &str`, without storing any position at all. When is recomputing better than storing an index? *Hint:* `str::lines` and `Iterator::next` on a borrow of `self.text`.
5. **A leak-proof guard?** Someone proposes a scoped-thread API that returns a guard, but makes the guard `!Send` and the constructor `unsafe`. Does this fix the problem described in *Why `thread::scope` takes a closure*? What must the caller of the `unsafe` constructor promise? *Hint:* an `Rc` cycle does not need to send anything anywhere. The promise is about every path by which the guard could be leaked.

CHAPTER 14

Designing APIs with Lifetimes

The previous chapters explained what lifetimes mean and how the compiler checks them. This one is about choosing them. Every reference in a public signature is a promise, and the signature is the contract: your function's body is checked against it once, but every caller is checked against it at every call, for as long as the API exists. A lifetime that is easy to write can be hard to live with. A struct with three lifetime parameters compiles today; tomorrow every type that contains it needs three lifetime parameters too, and none of them can be sent to a thread.

Good API design with lifetimes is mostly about *where* to put them. Borrowing is what makes Rust fast: a tokenizer that hands out slices of its input allocates nothing. Owning is what makes Rust values easy to move around: a report that owns its strings can be stored, returned, cached or sent anywhere. The art is to borrow where the work happens and to own where the results are kept, and to keep the remaining lifetimes few, honest and well named.

In this chapter you will learn:

- how to choose between borrowing and owning for parameters, return values and fields, and what `impl AsRef<str>` and `impl Into<String>` add;
- when `Cow<'a, str>`, `Arc<str>` or `Rc<str>` is a better answer than a lifetime parameter;
- how to build a zero-copy tokenizer with a borrowed token type, and an owned twin for storage;
- how many lifetime parameters a struct needs, and how to name them;
- how to recognize and fix the `&'a mut T<'a>` "borrowed forever" antipattern;
- how lifetimes shape callbacks, iterators that return references, errors and builders;
- how to refactor "lifetime soup" into a design with few, well-named lifetimes.

14.1 Own or borrow?

For every reference you are about to write in a signature, ask one question: does the code on the other side need the data *after the call returns*? If not, borrow. If it does, own. "Borrow for the call, own for storage" settles most cases; the rest of this section applies it to parameters, return values and fields. The program is `code/examples/ch14_parameters.rs`.

Parameters

A function that only reads its argument during the call should borrow it:

```
/// Reads its argument during the call only: borrow it.
fn count_words(text: &str) -> usize {
    text.split_whitespace().count()
}
```

`&str` is the most flexible string parameter there is. It accepts a literal, a `&String` (through deref coercion), and a slice of either. A parameter of type `&String` would accept only the second; the same goes for `&[T]` versus `&Vec<T>`. The lifetime is elided, and no caller ever has to think about it: the borrow lasts for the call, and every borrow a caller can produce lasts at least that long.

When callers hold many different string types, a generic parameter can accept them all, by value or by reference:

```
/// Also reads only, but accepts anything that can lend a `&str`, by value or by
/// reference.
fn count_lines(text: impl AsRef<str>) -> usize {
    text.as_ref().lines().count()
}
```

`impl AsRef<str>` accepts `String`, `&String`, `&str`, `Cow<str>`, `Box<str>`, `Arc<str>` and more; the standard library uses the same idea for paths (`std::fs::read_to_string` takes `P: AsRef<Path>`). The function is still a borrower: it calls `as_ref()` to get a `&str` that is valid during the call. The cost is that the function is generic, so it is compiled once per argument type; for most functions plain `&str` is the better default, and `AsRef` is a convenience for APIs that are called with owned values often.

When the function *keeps* its argument, the signature should say so by taking ownership:

```
/// Keeps its argument, so it takes ownership. With `impl Into<String>` the
/// caller passes a literal (one allocation here) or a `String` (moved, no copy).
struct Label {
    name: String,
}

impl Label {
    fn new(name: impl Into<String>) -> Self {
        Label { name: name.into() }
    }
}
```

With `name: &str`, `Label::new` would have to call `name.to_string()`, and a caller who already had a `String` it no longer needed would pay for a copy anyway. With `impl Into<String>`, the caller decides: a literal is converted (one allocation, which was unavoidable), and a `String` is moved in without copying. Chapter 7 met the same choice from the other side: a function that needs to keep a string and asks for `&'static str` instead forces its callers to produce literals or leaked data, while one that asks for ownership accepts anything.

Return values

A function returns a reference when its result is a *part* of an input, and an owned value when it creates something new:

```
/// Returns a view into its input: the result borrows from `text` (elision).
fn first_word(text: &str) -> &str {
    text.split_whitespace().next().unwrap_or("")
}
```

```
/// Returns new data: the result is owned, and the signature has no lifetime.
fn shout(text: &str) -> String {
    text.to_uppercase()
}
```

```
let owned = String::from("lifetimes are regions\nnot durations");
println!("words: {}", count_words("a literal")); // `&'static str`
println!("words: {}", count_words(&owned)); // `&String` coerces to `&str`
println!("words: {}", count_words(&owned[..9])); // a slice of a `String`
println!("lines: {}", count_lines(&owned)); // a reference...
println!("lines: {}", count_lines(String::from("one line"))); // ...or a value

let fixed = Label::new("fixed"); // allocates a `String`
let built = Label::new(format!("label-{}", 2)); // moves the `String` in
println!("labels: {}, {}", fixed.name, built.name);

let first = first_word(&owned); // borrows from `owned`
println!("first: {first}, shouted: {}", shout(first));
```

```
words: 2
words: 5
words: 1
lines: 2
lines: 1
labels: fixed, label-2
first: lifetimes, shouted: LIFETIMES
```

`first_word` returns a view: by elision (Chapter 3) the result borrows from `text`, so `first` keeps `owned` borrowed for as long as `first` is used. That is the right contract for a view, and it costs nothing at run time. `shout` creates a new string, so it returns a `String` and its signature has no lifetime at all. A function cannot return a reference to something it created (E0515, Chapter 5), so the choice is usually made for you; the interesting case is a function whose result is *sometimes* a part of the input and sometimes new, which is what `Cow` is for (next section).

Fields

A struct that is kept, such as a configuration, a record or a result, should own its fields. It then has no lifetime parameter, and it can be stored in a collection, returned from any function, cached, or sent to another thread.

A struct with a reference field is a *view*: it has a lifetime parameter (Chapter 2), it is cheap to create, and it can never outlive the data it borrows. The lifetime also spreads. Every struct that contains a `View<'a>` needs a lifetime parameter of its own, every function that returns one ties its result to an input, and none of them satisfies `T: 'static` unless it borrows `'static` data (Chapter 7). Views are the right tool for short-lived things: tokenizers, iterators, parsed tokens, the result of a lookup. They are the wrong tool for anything that is meant to be kept.

Situation	Choose
read during the call only	`&str`, `&[T]`, `&T`
read during the call, many argument types	`impl AsRef<str>`
kept after the call	`String`, `Vec<T>`, `impl Into<String>`
return a part of an input	`&str`, tied to that input
return new data	`String`
return a part or new data	`Cow<'a, str>`
a short-lived view	a struct with a lifetime parameter
a value that is stored, shared or sent	owned fields, `Arc<str>`

14.2 Alternatives to a lifetime parameter

Between "borrow" and "own" there are types that do a bit of both. They let you avoid a lifetime parameter, or avoid a copy, in places where neither plain choice fits. The programs are `code/examples/ch14_cow.rs` and `code/examples/ch14_shared_strings.rs`.

`Cow<'a, str>`: borrow when you can, own when you must

`std::borrow::Cow` ("clone on write") is an enum with two variants. The real definition is generic over any type that can be borrowed and owned; for strings, `Cow<'a, str>` amounts to this (not compiled, and not the real definition):

```
enum CowStr<'a> {
    Borrowed(&'a str), // `Cow::Borrowed`
    Owned(String),     // `Cow::Owned`
}
```

It dereferences to `&str` in both cases, so the caller can use it like a string. It shines as the return type of a function that usually returns its input unchanged but sometimes has to build a new string:

```
/// Escapes the three characters that are special in markup. Most text contains
/// none of them, and then the result borrows the input.
fn escape(input: &str) -> Cow<'_, str> {
    if !input.contains(['<', '>', '&']) {
        return Cow::Borrowed(input);
    }
    let mut escaped = String::with_capacity(input.len() + 8);
    for c in input.chars() {
        match c {
            '<' => escaped.push_str("&lt;"),
            '>' => escaped.push_str("&gt;"),
            '&' => escaped.push_str("&amp;"),
            _ => escaped.push(c),
        }
    }
    Cow::Owned(escaped)
}
```

```
for input in ["plain text", "a < b && c"] {
    let escaped = escape(input);
    println!("{:<9} {escaped}", describe(&escaped));
}
```

```
let kept: String = escape("x > 0").into_owned(); // detach from the input
println!("kept      {kept}");
```

```
borrowed  plain text
owned     a &lt; b &amp;amp;&amp;amp; c
kept      x &gt; 0
```

Most text contains no `<`, `>` or `&`, and for that text `escape` returns the input itself, without allocating. The signature, `fn escape(input: &str) -> Cow<'_, str>`, tells the caller the truth: the result may borrow from `input`. A caller who wants to keep the result after `input` is gone calls `into_owned()`, which returns the `String`, either the one already inside or a fresh copy of the borrowed text.

`Cow<'static, str>` in a field

With `'static` as the lifetime, `Cow` becomes a field type that holds either a literal or a run-time string, and has no lifetime parameter to spread:

```
/// A column name is usually a literal, sometimes built at run time.
#[derive(Debug)]
struct Column {
    name: Cow<'static, str>,
}

impl Column {
    fn new(name: impl Into<Cow<'static, str>>) -> Self {
        Column { name: name.into() }
    }
}
```

```
let columns = vec![Column::new("id"), Column::new(format!("score_{}", 2026))];
for column in &columns {
    println!("{:<9} {}", describe(&column.name), column.name);
}
// `Column: 'static`, so the columns can move to another thread.
let handle = thread::spawn(move || columns.len());
println!("{} columns sent to a thread", handle.join().unwrap());
```

```
borrowed  id
owned     score_2026
2 columns sent to a thread
```

`Column::new("id")` stores the literal (no allocation); the formatted name is stored as an owned `String`. Both kinds of `Column` are the same type, and since `Cow<'static, str>` holds no borrow that can expire, `Column: 'static`: the vector moves to another thread without complaint. This is rung 3 of the decision ladder in Chapter 7, "mix". It fits well for names, labels and messages that are usually literals.

`Arc<str>` and `Rc<str>`: shared, immutable text

When the same immutable string is needed in many places, such as a name that is both a field of a record and a key of an index, copying it into each place wastes memory, and borrowing it from one place ties every other place to it with a lifetime. `Arc<str>` is the third option: a reference-counted string that several owners share. Cloning it copies a pointer and increments a counter.

```
/// A file entry. Its name is also a key of the index below.
struct Entry {
```

```
    name: Arc<str>,
    size: u64,
}

/// Entries plus an index by name, sharing one allocation per name.
struct Catalog {
    entries: Vec<Entry>,
    by_name: BTreeMap<Arc<str>, usize>,
}

impl Catalog {
    fn new(files: &[(&str, u64)]) -> Self {
        let mut catalog = Catalog {
            entries: Vec::new(),
            by_name: BTreeMap::new(),
        };
        for &(name, size) in files {
            let name: Arc<str> = Arc::from(name); // one allocation
            let index = catalog.entries.len();
            catalog.by_name.insert(Arc::clone(&name), index); // a second owner
            catalog.entries.push(Entry { name, size });
        }
        catalog
    }

    fn size_of(&self, name: &str) -> Option<u64> {
        // `Arc<str>: Borrow<str>`, so a `&str` is enough to look up a key.
        self.by_name.get(name).map(|&i| self.entries[i].size)
    }
}
```

```
let catalog = Catalog::new(&[("notes.txt", 120), ("todo.md", 30)]);
println!("todo.md: {:?} bytes", catalog.size_of("todo.md"));
let first = &catalog.entries[0].name;
println!("{first}: {} owners", Arc::strong_count(first)); // entry + index

// No lifetime parameter, and `'static`: the names can go to a thread.
let names: Vec<Arc<str>> = catalog.entries.iter().map(|e| e.name.clone()).collect();
let handle = thread::spawn(move || names.join(", "));
println!("from a thread: {}", handle.join().unwrap());

// `Rc<str>` is the same idea for data that stays on one thread.
let tag: Rc<str> = Rc::from("draft");
let tags = [Rc::clone(&tag), Rc::clone(&tag)];
println!("{}: {} owners", tags[1], Rc::strong_count(&tag));
```

```
todo.md: Some(30) bytes
notes.txt: 2 owners
from a thread: notes.txt, todo.md
draft: 3 owners
```

Each name is allocated once, by `Arc::from(name)`, and shared by the entry and the index (the strong count is 2). Lookups take a plain `&str`, because `Arc<str>` implements `Borrow<str>`. Neither `Entry` nor `Catalog` has a lifetime parameter, and `Arc<str>` is `Send` and `'static`, so names can move to another thread. `Rc<str>` is the same idea without the atomic counter, for data that stays on one thread (it is not `Send`). Prefer `Arc<str>` to `Arc<String>`: it is one allocation instead of two, and the text cannot be changed anyway.

Choosing among them

Type	Lifetime parameter	Cost of a clone	`'static`
`&'a str`	yes	copies a pointer	only if `'a` is
`String`	no	copies the text	yes
`Cow<'a, str>`	yes (none with `'static`)	copies if owned	if `'a` is `'static`
`Arc<str>`, `Rc<str>`	no	increments a counter	yes

14.3 Zero-copy parsing: borrowed tokens, owned tokens

A tokenizer produces many small strings. If each token owned its text, every identifier and number would cost an allocation and a copy. If each token borrows its text from the input, producing a token costs nothing. The price is the usual one: a borrowed token cannot outlive the input. The standard design keeps both options open: a borrowed token type for the work, and an owned twin for the tokens you keep. The program is `code/examples/ch14_tokenizer.rs`.

The borrowed token

```
/// A token whose text is a slice of the source it was read from.
#[derive(Debug, Clone, Copy, PartialEq)]
enum Token<'src> {
    Ident(&'src str),
    Number(&'src str),
    Str(&'src str), // the contents, without the quotes
    Punct(char),
}
```

```
struct Tokenizer<'src> {
    rest: &'src str,
}

impl<'src> Tokenizer<'src> {
    fn new(src: &'src str) -> Self {
        Tokenizer { rest: src }
    }
}

impl<'src> Iterator for Tokenizer<'src> {
    type Item = Token<'src>; // items borrow from the source, not from the tokenizer

    fn next(&mut self) -> Option<Token<'src>> {
        let rest: &'src str = self.rest.trim_start(); // copy the `&'src str` out
        let first = rest.chars().next()?;
        let word_end =
            |pred: fn(char) -> bool| rest.find(|c| !pred(c)).unwrap_or(rest.len());
        let (token, len) = if first.is_alphabetic() || first == '_' {
            let len = word_end(|c| c.is_alphanumeric() || c == '_');
            (Token::Ident(&rest[..len]), len)
        } else if first.is_ascii_digit() {
            let len = word_end(|c| c.is_ascii_digit());
            (Token::Number(&rest[..len]), len)
        } else if first == '"' {
            let close = 1 + rest[1..].find('"')?; // an unterminated string ends the input
            (Token::Str(&rest[1..close]), close + 1)
        } else {
            (Token::Punct(first), first.len_utf8())
        };
        self.rest = &rest[len..];
```

```
        Some(token)
    }
}
```

Two details matter. First, `type Item = Token<'src>`: the items borrow from the *source*, not from the tokenizer, so a caller can collect them, keep them after the tokenizer is gone, and hold several at once (Chapter 2 explains why `next(&mut self)` may return a `Token<'src>`). Second, the first line of `next` copies the `&'src str` out of `self` into a local `rest`. All the slices are taken from that local, so they are `&'src str` and do not borrow `self`.

```
let source = String::from(r#"let total = price * 3 + "tax";"#);
let tokens: Vec<Token<'_>> = Tokenizer::new(&source).collect();
println!("{tokens:?}");

// Zero-copy: the text of every token points into `source`.
let range = source.as_bytes().as_ptr_range();
for token in &tokens {
    if let Token::Ident(text) | Token::Number(text) | Token::Str(text) = token {
        assert!(range.contains(&text.as_ptr()));
    }
}
```

```
[Ident("let"), Ident("total"), Punct('='), Ident("price"), Punct('*'), Number("3"),
Punct('+'), Str("tax"), Punct(';')]
```

The `assert!` checks the "zero-copy" claim: the text of every token points into the bytes of `source`.

When the input goes away

Borrowed tokens are a problem as soon as the input is temporary. A typical case is reading a file line by line into one reused buffer, as `BufRead::read_line` does, and keeping the tokens of every line. The program `code/compile_fail/ch14_tokens_outlive_buffer.rs` tries it with a smaller tokenizer:

```
pub fn caller(input: &[&str]) -> usize {
    let mut line = String::new();
    let mut history: Vec<Token<'_>> = Vec::new();
    for text in input {
        line.clear(); // needs `&mut line`...
        line.push_str(text);
        history.extend(tokenize(&line)); // ...but `history` borrows `line`
    }
    history.len()
}
```

The compiler rejects it with two errors, one for `line.clear()` and one for `line.push_str(text)`; here is the first (the second is identical in shape and omitted):

```
error[E0502]: cannot borrow `line` as mutable because it is also borrowed as immutable
  --> compile_fail/ch14_tokens_outlive_buffer.rs:19:9
   |
19 |         line.clear(); // needs `&mut line`...
   |         ^^^^^^^^^^^^ mutable borrow occurs here
20 |         line.push_str(text);
21 |         history.extend(tokenize(&line)); // ...but `history` borrows `line`
   |         -------                 ----- immutable borrow occurs here
   |         |
   |         immutable borrow later used here
```

Read the labels in the order of execution, across two iterations of the loop. In the first iteration, `&line` is borrowed ("immutable borrow occurs here") and the tokens holding that borrow are stored in `history`. In the second iteration, `line.clear()` needs a mutable borrow ("mutable borrow occurs here"), while the first iteration's borrow is still alive, because `history` is used again on the same line ("immutable borrow later used here"). Clearing the buffer would destroy the text that the stored tokens point to.

The owned twin

The fix is to convert the tokens you keep into a type that owns its text:

```
/// The same token, owning its text: it can outlive the source.
#[derive(Debug, Clone, PartialEq)]
enum OwnedToken {
    Ident(String),
    Number(String),
    Str(String),
    Punct(char),
}

impl Token<'_> {
    /// Copies the borrowed text, so the result no longer depends on the source.
    fn into_owned(self) -> OwnedToken {
        match self {
            Token::Ident(s) => OwnedToken::Ident(s.to_owned()),
            Token::Number(s) => OwnedToken::Number(s.to_owned()),
            Token::Str(s) => OwnedToken::Str(s.to_owned()),
            Token::Punct(c) => OwnedToken::Punct(c),
        }
    }
}
```

```
// Reading line by line into one reused buffer, as `BufRead::read_line` does.
let input = ["x = 1", "y = x + 2"];
let mut line = String::new();
let mut history: Vec<OwnedToken> = Vec::new();
for text in input {
    line.clear();
    line.push_str(text);
    history.extend(Tokenizer::new(&line).map(Token::into_owned));
}
let last = history.last();
println!("kept {} tokens, the last is {last:?}", history.len());
```

```
kept 8 tokens, the last is Some(Number("2"))
```

The tokenizer is unchanged and still allocates nothing; the copies happen only for the tokens that are kept, at the point where they are kept. The method is called `into_owned`, after `Cow::into_owned`. It cannot be called `to_owned`: `Token` derives `Clone`, so it already has a `to_owned` method (from the blanket `ToOwned` implementation for `Clone` types), and that one returns another `Token<'src>`.

Two other designs are worth knowing. A token can hold `Cow<'src, str>` fields, so that one type covers both cases and the owned version is `Token<'static>` (exercise 1). Or a token can store a byte range (`Range<usize>`) into the input instead of a slice: it then has no lifetime at all, and whoever still has the input can slice it again. The error type later in this chapter uses the second idea.

14.4 How many lifetime parameters does a struct need?

Most borrowing structs need exactly one lifetime parameter, even when they hold several references. One parameter can serve several fields because shared references are covariant (Chapter 4): when you pass two borrows with different lifetimes, the compiler picks the shorter one for the parameter, and the longer borrow is shortened to fit. The struct is then usable while *both* borrows are valid, which is usually all you need.

"Usually" is the important word. The program `code/compile_fail/ch14_one_lifetime.rs` defines a field splitter with one lifetime parameter for the line and the separator:

```
pub struct Fields<'a> {
    rest: Option<&'a str>,
    sep: &'a str, // the same lifetime as the line
}

impl<'a> Fields<'a> {
    pub fn new(line: &'a str, sep: &'a str) -> Self {
        Fields {
            rest: Some(line),
            sep,
        }
    }
}
```

Its `Iterator` implementation returns `&'a str` fields, slices of the line. Now a convenience function builds the separator as a local `String` and returns the first field:

```
pub fn first_field(line: &str, sep: char) -> &str {
    let sep = sep.to_string();
    Fields::new(line, &sep).next().unwrap_or("")
}
```

```
error[E0515]: cannot return value referencing local variable `sep`
  --> compile_fail/ch14_one_lifetime.rs:42:5
   |
42 |     Fields::new(line, &sep).next().unwrap_or("")
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |     |                  |
   |     |                  `sep` is borrowed here
   |     returns a value referencing data owned by the current function
```

"`sep` is borrowed here", and the value returned is "referencing data owned by the current function". The field returned is a slice of `line`, which outlives the call, but the type does not say so. With a single parameter, `'a` had to be short enough for *both* arguments, so it ended when the local `sep` is dropped, and the item type `&'a str` inherited that limit. The struct forgot which of its two borrows the results come from.

Giving each borrow its own parameter keeps that information (`code/examples/ch14_two_lifetimes.rs`):

```
/// Splits a line into fields at a separator. The fields are slices of the
/// line; the separator is only consulted.
struct Fields<'line, 'sep> {
    rest: Option<&'line str>,
    sep: &'sep str,
}

impl<'line, 'sep> Fields<'line, 'sep> {
```

```
    fn new(line: &'line str, sep: &'sep str) -> Self {
        assert!(!sep.is_empty(), "the separator must not be empty");
        Fields {
            rest: Some(line),
            sep,
        }
    }
}

impl<'line> Iterator for Fields<'line, '_> {
    type Item = &'line str; // fields borrow from the line only

    fn next(&mut self) -> Option<&'line str> {
        let rest = self.rest?;
        match rest.find(self.sep) {
            Some(i) => {
                self.rest = Some(&rest[i + self.sep.len()..]);
                Some(&rest[..i])
            }
            None => {
                self.rest = None;
                Some(rest)
            }
        }
    }
}
```

```
/// The separator is built inside the function and dropped at its end; the
/// result may borrow only from `line`.
fn first_field(line: &str, sep: char) -> &str {
    let sep = sep.to_string();
    Fields::new(line, &sep).next().unwrap_or("")
}
```

```
let record = "ada;lovelace;1815";
let fields: Vec<&str> = Fields::new(record, ";").collect();
println!("{fields:?}");
println!("{:?}", first_field(record, ';'));
```

```
["ada", "lovelace", "1815"]
"ada"
```

The item type mentions only `'line`, and the `Iterator` implementation writes the separator's lifetime as `'_` (Chapter 6), because nothing in it depends on that lifetime. So `first_field` may return a field that outlives its local separator.

The rule of thumb: start with one lifetime parameter, and add a second one when

- the struct borrows from two sources and its results borrow from only one of them, which may outlive the other (as here); or
- the struct holds a `&mut` to something that itself holds borrows (the next section shows why).

Otherwise, one is enough, and every extra parameter is something every user of the struct has to carry.

Name lifetimes after what they borrow

`'a` and `'b` say nothing. `'line` and `'sep` say which borrow is which, and so do `'src`, `'text`, `'doc` and `'out` in this chapter. The standard library does the same in `thread::Scope<'scope, 'env>`, and serde's `Deserialize<'de>` names the lifetime of the data being deserialized (Chapter 9). Diagnostics benefit too: a message such as "lifetime `'src` defined here" points you at the right borrow immediately.

A single `'a` is fine in a short generic function with one borrow; structs, traits and anything public deserve names.

14.5 The `&'a mut T<'a>` antipattern: borrowed forever

Chapter 2 introduced this trap, Chapter 3 met it as the consequence of a tempting elision rule ("make every lifetime the same"), and Chapter 4 explained the invariance behind it. Here it is as an API design mistake, because that is where it usually appears: in a method that seemed to need it. The program `code/compile_fail/ch14_borrowed_forever.rs` collects the distinct words of a text:

```
pub struct WordSet<'text> {
    words: Vec<&'text str>,
}

impl<'text> WordSet<'text> {
    pub fn new() -> Self {
        WordSet { words: Vec::new() }
    }

    pub fn add_all(&'text mut self, text: &'text str) {
        for word in text.split_whitespace() {
            if !self.words.contains(&word) {
                self.words.push(word);
            }
        }
    }

    pub fn len(&self) -> usize {
        self.words.len()
    }
}

pub fn caller() -> usize {
    let text = String::from("the cat saw the dog");
    let mut set = WordSet::new();
    set.add_all(&text);
    set.len() // the set is still borrowed by `add_all`
}
```

The type compiles, and so does `add_all`. The caller does not:

```
error[E0502]: cannot borrow `set` as immutable because it is also borrowed as mutable
  --> compile_fail/ch14_borrowed_forever.rs:33:5
   |
32 |     set.add_all(&text);
   |     --- mutable borrow occurs here
33 |     set.len() // the set is still borrowed by `add_all`
   |     ^^^
   |     |
   |     immutable borrow occurs here
   |     mutable borrow later used here
```

The labels contain the hallmark of this antipattern. "Mutable borrow occurs here" points at `add_all`, and at the very next use of `set`, the compiler reports both "immutable borrow occurs here" and "mutable borrow later used here", on the same expression. The mutable borrow taken by `add_all` is still alive at every later use of `set`, so *any* later use conflicts with it.

Why? `add_all` takes `&'text mut WordSet<'text>`: the borrow of the set and the words inside it have the same lifetime. `WordSet<'text>` sits behind `&mut`, where it is invariant (Chapter 4), so `'text` cannot

be shortened to make the borrow short. The borrow of `set` therefore lasts for all of `'text`, and `'text` must cover every point where `set` is used, since `set`'s type mentions it. After the first call, the set is mutably borrowed for the rest of its life.

The author probably wrote `&'text mut self` because `add_all` stores `&'text str` values in the set. But storing them requires only that the *argument* lives for `'text`, not the borrow of the set. The fix, in `code/examples/ch14_borrowed_forever_fixed.rs`, lets the receiver's borrow have its own, anonymous lifetime:

```
struct WordSet<'text> {
    words: Vec<&'text str>,
}

impl<'text> WordSet<'text> {
    fn new() -> Self {
        WordSet { words: Vec::new() }
    }

    fn add_all(&mut self, text: &'text str) {
        for word in text.split_whitespace() {
            if !self.words.contains(&word) {
                self.words.push(word);
            }
        }
    }

    fn len(&self) -> usize {
        self.words.len()
    }
}
```

```
let text = String::from("the cat saw the dog");
let mut set = WordSet::new();
set.add_all(&text);
println!("{} distinct words", set.len());
set.add_all("the end"); // a second mutable borrow: fine now
println!("{} distinct words: {:?}", set.len(), set.words);
```

```
4 distinct words
5 distinct words: ["the", "cat", "saw", "dog", "end"]
```

Now each call borrows the set only for the call, and the set can be used and mutated again afterward.

The same shape in a field

The antipattern also hides in struct fields. A `Collector` that pushes words into a vector owned by someone else is natural to write with one lifetime (`code/compile_fail/ch14_collector_forever.rs`):

```
/// Pushes words into a vector owned by someone else.
pub struct Collector<'a> {
    out: &'a mut Vec<&'a str>,
}

impl<'a> Collector<'a> {
    pub fn push(&mut self, word: &'a str) {
        self.out.push(word);
    }
}

pub fn caller() -> usize {
    let text = String::from("to be or not to be");
```

```
    let mut words: Vec<&str> = Vec::new();
    {
        let mut collector = Collector { out: &mut words };
        for word in text.split_whitespace() {
            collector.push(word);
        }
    } // the collector is gone, but `words` is still mutably borrowed
    words.len()
}
```

```
error[E0502]: cannot borrow `words` as immutable because it is also borrowed as mutable
  --> compile_fail/ch14_collector_forever.rs:26:5
   |
21 |         let mut collector = Collector { out: &mut words };
   |                                              ---------- mutable borrow occurs here
...
26 |     words.len()
   |     ^^^^^
   |     |
   |     immutable borrow occurs here
   |     mutable borrow later used here
```

The field type `&'a mut Vec<&'a str>` is `&'a mut T<'a>` again: the borrow of `words` and the strings inside it share `'a`, and `Vec<&'a str>` is invariant behind `&mut`. The strings must stay valid as long as `words` is used, so the mutable borrow of `words` must too, even after the collector is gone. Here a struct needs two lifetime parameters, one for each borrow (`code/examples/ch14_collector.rs`):

```
/// `'out`: how long we may write into the vector.
/// `'text`: how long the words we push stay valid.
struct Collector<'out, 'text> {
    out: &'out mut Vec<&'text str>,
}

impl<'text> Collector<'_, 'text> {
    fn push(&mut self, word: &'text str) {
        self.out.push(word);
    }
}
```

```
let text = String::from("to be or not to be");
let mut words: Vec<&str> = Vec::new();
{
    let mut collector = Collector { out: &mut words };
    for word in text.split_whitespace() {
        collector.push(word);
    }
} // the borrow `'out` ends here; `'text` goes on
words.sort(); // `words` is ours again: we can mutate it
words.dedup();
println!("{} distinct words: {words:?}", words.len());
```

```
4 distinct words: ["be", "not", "or", "to"]
```

To spot the antipattern, look for a lifetime that appears both on a `&mut` and inside the type it points to: `&'a mut self` in `impl<'a> T<'a>`, `&'a mut T<'a>`, `&'a mut Vec<&'a str>`. The fix is always the same: give the outer borrow its own lifetime, which in a method usually means eliding it.

14.6 Callbacks

A callback parameter is a small signature inside a signature, and its lifetimes decide what the callback may do with its arguments. The program is `code/examples/ch14_callbacks.rs`.

Callbacks that are called, not stored

When a function only calls its callback before returning, take it as a generic `impl Fn...` parameter:

```
/// Calls `f` on every non-blank line. The callback must accept a borrow of any
/// lifetime (the bound is higher-ranked, Chapter 9), so it cannot keep it.
fn for_each_line(text: &str, mut f: impl FnMut(&str)) {
    for line in text.lines().filter(|line| !line.trim().is_empty()) {
        f(line);
    }
}
```

The bound `impl FnMut(&str)` has an elided lifetime in the callback's argument, which makes it higher-ranked: it means `impl for<'l> FnMut(&'l str)`, "a callback that accepts a `&str` of *any* lifetime" (Chapter 9 explains `for<'l>` in detail). That gives the function freedom. It may pass the callback slices of `text`, or borrows of a buffer it creates and drops itself, and no caller can tell the difference.

The other side of that freedom is that the callback cannot keep what it receives. The program `code/compile_fail/ch14_callback_keeps_borrow.rs` tries to collect the lines:

```
pub fn caller(text: &str) -> usize {
    let mut kept: Vec<&str> = Vec::new();
    for_each_line(text, |line| kept.push(line));
    kept.len()
}
```

```
error[E0521]: borrowed data escapes outside of closure
  --> compile_fail/ch14_callback_keeps_borrow.rs:14:32
   |
13 |     let mut kept: Vec<&str> = Vec::new();
   |         --------- `kept` declared here, outside of the closure body
14 |     for_each_line(text, |line| kept.push(line));
   |                          ----  ^^^^^^^^^^^^^^^ `line` escapes the closure body here
   |                          |
   |                          `line` is a reference that is only valid in the closure body
   |
   = note: requirement occurs because of a mutable reference to `Vec<&str>`
   = note: mutable references are invariant over their type parameter
   = help: see <https://doc.rust-lang.org/nomicon/subtyping.html> for more information about
variance
```

E0521 again, this time "borrowed data escapes outside of closure". "`line` is a reference that is only valid in the closure body": as far as the closure knows, the borrow could end as soon as the call returns. "`line` escapes the closure body here": pushing it into `kept`, which lives outside the closure. The notes explain why the compiler cannot simply pick a longer lifetime: `kept` is reached through a mutable reference, and `Vec<&str>` is invariant behind it (Chapter 4).

If the API wants callers to be able to keep the lines, it must promise that the lines come from `text`, and say so in the signature:

```
/// The same, but the signature promises that every line borrows from `text`,
/// so a callback may keep the lines for as long as `text` lives.
```

```
fn for_each_line_kept<'text>(text: &'text str, mut f: impl FnMut(&'text str)) {
    for line in text.lines().filter(|line| !line.trim().is_empty()) {
        f(line);
    }
}
```

```
let mut total = 0;
for_each_line(&text, |line| total += line.len()); // borrows `total` mutably
println!("total length: {total}");

let mut kept: Vec<&str> = Vec::new();
for_each_line_kept(&text, |line| kept.push(line)); // keeps the lines
println!("kept: {kept:?}");
```

```
total length: 40
kept: ["INFO start", "WARN disk almost full", "INFO stop"]
```

The two signatures are two different contracts. `for_each_line` promises less, so its implementation may change (to trim lines into a reused buffer, say) without breaking any caller. `for_each_line_kept` promises more, which lets callers keep the lines, but it can never be changed to pass anything that does not borrow from `text`. Choose the second only when keeping the arguments is a real use case.

Callbacks that are stored

A callback that is stored lives as long as the struct that stores it. Chapter 7 showed the default: `Box<dyn Fn(&str)>` means `Box<dyn Fn(&str) + 'static>`, so stored callbacks must own what they capture. That is the right choice for a long-lived registry. For a dispatcher that is used within one function, a lifetime parameter lets the callbacks borrow local variables:

```
/// Stores callbacks that may borrow anything that outlives `'a`.
struct Dispatcher<'a> {
    handlers: Vec<Box<dyn FnMut(&str) + 'a>>,
}

impl<'a> Dispatcher<'a> {
    fn new() -> Self {
        Dispatcher {
            handlers: Vec::new(),
        }
    }

    fn on_line(&mut self, handler: impl FnMut(&str) + 'a) {
        self.handlers.push(Box::new(handler));
    }

    fn dispatch(&mut self, line: &str) {
        for handler in &mut self.handlers {
            handler(line);
        }
    }
}
```

```
let mut count = 0;
let mut warnings = Vec::new();
{
    let mut dispatcher = Dispatcher::new();
    dispatcher.on_line(|_| count += 1);
    dispatcher.on_line(|line| {
        if line.starts_with("WARN") {
            warnings.push(line.to_owned()); // the line is borrowed: copy it
```

```
        }
    });
    for_each_line(&text, |line| dispatcher.dispatch(line));
} // the dispatcher is dropped: `count` and `warnings` are free again
println!("{count} lines, warnings: {warnings:?}");
```

```
3 lines, warnings: ["WARN disk almost full"]
```

The two closures borrow `count` and `warnings` mutably, and the dispatcher holds those borrows (`+ 'a`) until it is dropped at the end of the block. After that, both variables can be read. A `Dispatcher<'a>` cannot be stored in a global or sent to `thread::spawn`, but within its scope it accepts any closure. Note also that the second closure copies the line (`to_owned`): the dispatcher passes a `&str` that is valid only during the call, so keeping it would be the E0521 above.

14.7 Iterators that return references

A method that returns an iterator over borrowed data is easy to write with `impl Iterator`. The program is `code/examples/ch14_iterators.rs`:

```
/// The items borrow from `self`; in edition 2024 the opaque type captures
/// the elided lifetime of `&self`, so no `+ '_` is needed.
fn words(&self) -> impl Iterator<Item = &str> {
    self.body.split_whitespace()
}

fn long_words(&self, min_len: usize) -> impl Iterator<Item = &str> {
    self.words().filter(move |word| word.len() >= min_len)
}
```

The item type `&str` borrows from `&self` (elision rule 3), and the iterator itself borrows `self` as well. In edition 2024, a return-position `impl Trait` captures every lifetime in scope, including the elided lifetime of `&self`, so nothing more is needed. Edition 2021 captured only the lifetimes named in the bounds: this signature worked there too, because `&str` names the lifetime, but an iterator over the `char`s of `self.body` needed `+ '_` (error E0700 without it), which you will still see in older code. Chapter 8 has the complete rules.

An `impl Trait` type has no name, so callers cannot write it in a struct field. When callers may want to store the iterator, return a named type instead:

```
/// Returns a named type, which callers can store in a field.
fn lines(&self) -> BodyLines<'_> {
    BodyLines {
        inner: self.body.lines(),
    }
}
```

```
/// The non-blank lines of a document, trimmed.
struct BodyLines<'doc> {
    inner: std::str::Lines<'doc>,
}

impl<'doc> Iterator for BodyLines<'doc> {
    type Item = &'doc str;
```

```
    fn next(&mut self) -> Option<&'doc str> {
        self.inner
            .by_ref()
            .map(str::trim)
            .find(|line| !line.is_empty())
    }
}

/// A reader that keeps its position in a document: possible because the
/// iterator type has a name.
struct Reader<'doc> {
    lines: BodyLines<'doc>,
    line_no: usize,
}
```

`Reader<'doc>` keeps an iterator and a line counter together, which is possible only because `BodyLines<'doc>` can be named. This is why the standard library returns named types such as `Lines<'a>` and `SplitWhitespace<'a>` from `str` methods.

The capture rule of edition 2024 has one consequence for API design. When the items borrow from a *source* held by `self` rather than from `self`, capturing the lifetime of `&self` ties the iterator to `self` for no reason. Here is the problem with a temporary view, in `code/compile_fail/ch14_iterator_overcaptures.rs` (in edition 2021 this program compiles, because there the bound `Item = &'src str` decides what is captured):

```
pub struct Index<'src> {
    src: &'src str,
}

impl<'src> Index<'src> {
    pub fn words(&self) -> impl Iterator<Item = &'src str> {
        self.src.split_whitespace()
    }
}

pub fn words_of(src: &str) -> impl Iterator<Item = &str> {
    Index { src }.words()
}
```

```
error[E0716]: temporary value dropped while borrowed
  --> compile_fail/ch14_iterator_overcaptures.rs:17:5
   |
16 | pub fn words_of(src: &str) -> impl Iterator<Item = &str> {
   |                       - let's call the lifetime of this reference `'1`
17 |     Index { src }.words()
   |     ^^^^^^^^^^^^^^^^^^^^^
   |     |                   |
   |     |                   temporary value is freed at the end of this statement
   |     creates a temporary value which is freed while still in use
   |     argument requires that borrow lasts for `'1`
```

The items are `&'src str`, slices of the source, which outlives the call. But the opaque iterator type also captured the anonymous lifetime of `&self`, so it borrows the temporary `Index { src }` ("creates a temporary value which is freed while still in use"), which is dropped at the end of the statement. The fix is a *precise capture* list, `use<'src>`, which says that the iterator captures only `'src`:

```
/// A short-lived view of some source text.
struct Index<'src> {
    src: &'src str,
}
```

```
impl<'src> Index<'src> {
    /// The items borrow from the source, not from the index: `use<'src>` says
    /// that the iterator captures only `'src` (Chapter 8).
    fn words(&self) -> impl Iterator<Item = &'src str> + use<'src> {
        self.src.split_whitespace()
    }
}

/// The `Index` is a temporary, dropped at the end of the function; the
/// iterator survives it because it does not borrow it.
fn words_of(src: &str) -> impl Iterator<Item = &str> {
    Index { src }.words()
}
```

```
let doc = Document {
    body: String::from("  borrow checkers\n\n  check borrows \n"),
};
println!("{:?}", doc.words().collect::<Vec<_>>());
println!("{:?}", doc.long_words(7).collect::<Vec<_>>());

let mut reader = Reader {
    lines: doc.lines(),
    line_no: 0,
};
while let Some(line) = reader.lines.next() {
    reader.line_no += 1;
    println!("{}: {line}", reader.line_no);
}

println!("{:?}", words_of("one two three").last());
```

```
["borrow", "checkers", "check", "borrows"]
["checkers", "borrows"]
1: borrow checkers
2: check borrows
Some("three")
```

Chapter 8 explains `use<..>` in full. For API design, the point is to decide what the returned iterator borrows (`self`, or the data behind it) and to make the signature say exactly that.

14.8 Errors that borrow and errors that own

An error value describes what went wrong, and it is tempting to make it point at the offending input. The program `code/examples/ch14_errors.rs` starts with such an error:

```
/// Cheap to create, but tied to the input it points into.
#[derive(Debug)]
struct ParseError<'src> {
    line_no: usize,
    line: &'src str,
}

/// Parses `key = value` lines; the pairs and the error borrow from `src`.
fn parse(src: &str) -> Result<Vec<(&str, &str)>, ParseError<'_>> {
    let mut pairs = Vec::new();
    for (i, line) in src.lines().enumerate() {
        match line.split_once('=') {
            Some((key, value)) => pairs.push((key.trim(), value.trim())),
            None => {
                return Err(ParseError {
                    line_no: i + 1,
                    line,
```

```
                });
            }
        }
    }
    Ok(pairs)
}
```

For a caller that owns the input, `ParseError<'src>` is fine: it is cheap, and the caller can print the line. But errors travel. They are returned up the stack, converted by `?`, boxed into `Box<dyn Error>`, logged later, sent to other threads. Each of those trips makes a borrowed error harder to use. The first problem appears as soon as a function creates its own input. In `code/compile_fail/ch14_error_borrows_local.rs`, `load` reads the text and then parses it:

```
pub fn load(name: &str) -> Result<usize, ParseError<'_>> {
    let text = read_source(name);
    parse(&text) // the error would borrow `text`, which dies here
}
```

```
error[E0515]: cannot return value referencing local variable `text`
  --> compile_fail/ch14_error_borrows_local.rs:28:5
   |
28 |     parse(&text) // the error would borrow `text`, which dies here
   |     ^^^^^^-----^
   |     |     |
   |     |     `text` is borrowed here
   |     returns a value referencing data owned by the current function
```

"`text` is borrowed here", and the error returned would be "referencing data owned by the current function". Look again at the signature: by elision, the `'_` in `ParseError<'_>` is the lifetime of `name`, the only reference parameter. The signature promises an error that borrows from `name`, and no body could keep that promise with an error that points into `text`. There is no lifetime the author could write instead, because `text` does not exist outside the function.

The fix is an owned error at the boundary where the input is created:

```
/// Owns its data: it can outlive the input, cross threads, and become a
/// `Box<dyn Error>` (which means `Box<dyn Error + 'static>`).
#[derive(Debug)]
struct ConfigError {
    source_name: String,
    line_no: usize,
    line: String,
}

impl fmt::Display for ConfigError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        let (name, line_no, line) = (&self.source_name, self.line_no, &self.line);
        write!(f, "{name}:{line_no}: not a `key = value` line: {line:?}")
    }
}

impl Error for ConfigError {}
```

```
/// Reads the text itself, so nothing it returns may borrow from that text.
fn load(name: &str) -> Result<Vec<(String, String)>, ConfigError> {
    let text = read_source(name);
    let pairs = parse(&text).map_err(|e| ConfigError {
        source_name: name.to_owned(),
        line_no: e.line_no,
```

```
            line: e.line.to_owned(),
        })?;
        Ok(pairs
            .into_iter()
            .map(|(k, v)| (k.to_owned(), v.to_owned()))
            .collect())
}
```

```
// The caller owns the input: a borrowed error is fine here.
let input = String::from("a = 1\nb 2");
if let Err(e) = parse(&input) {
    println!("line {}: {:?}", e.line_no, e.line);
}

println!("{:?}", load("good.conf"));
let error: Box<dyn Error> = load("bad.conf").unwrap_err().into();
println!("{error}");
```

```
line 2: "b 2"
Ok([("name", "indexer"), ("workers", "4")])
bad.conf:2: not a `key = value` line: "workers 4"
```

`load` converts the borrowed `ParseError` into an owned `ConfigError` with `map_err`, copying one line of text, and only on the error path. The owned error can then go anywhere: here it becomes a `Box<dyn Error>`, which requires `'static` (Chapter 7; Chapter 8 shows what happens with a borrowed error and `?`).

As a rule, errors in public APIs should own their data. When copying text into the error is too expensive, store *positions* instead (a line number, a byte range): the error has no lifetime, and whoever holds the input can show the text (exercise 2). Low-level, performance-sensitive parsers may return borrowed errors, ideally with an `into_owned` method like the tokens above.

14.9 Builders

Builders configure a value step by step. They rarely need lifetime parameters, but the way their methods take `self` decides how callers can use them. The program is `code/examples/ch14_builders.rs`.

The simplest style takes the builder by value and returns it:

```
/// Each method takes the builder by value and returns it.
struct RequestBuilder {
    request: Request,
}

impl RequestBuilder {
    fn new(url: impl Into<String>) -> Self {
        let request = Request {
            url: url.into(),
            headers: Vec::new(),
            timeout_ms: 30_000,
        };
        RequestBuilder { request }
    }

    fn header(mut self, name: impl Into<String>, value: impl Into<String>) -> Self {
        self.request.headers.push((name.into(), value.into()));
        self
    }
```

```
    fn timeout_ms(mut self, timeout_ms: u64) -> Self {
        self.request.timeout_ms = timeout_ms;
        self
    }

    fn build(self) -> Request {
        self.request
    }
}
```

```
let builder = RequestBuilder::new("https://example.com/items").timeout_ms(500);
let builder = builder.header("accept", "text/plain"); // stored, then continued
let request = builder.build();
println!(
    "{} {:?} {}ms",
    request.url, request.headers, request.timeout_ms
);
```

There are no borrows here at all: each call consumes the builder and returns it, so it can be chained, stored in a variable, and continued later.

The other style, used by `std::process::Command` (whose `arg`, `env` and `current_dir` take `&mut self` and return `&mut Command`), borrows the builder:

```
/// Each method borrows the builder mutably and returns the same borrow.
struct QueryBuilder {
    table: String,
    filters: Vec<String>,
}

impl QueryBuilder {
    fn new(table: impl Into<String>) -> Self {
        QueryBuilder {
            table: table.into(),
            filters: Vec::new(),
        }
    }

    fn filter(&mut self, condition: impl Into<String>) -> &mut Self {
        self.filters.push(condition.into());
        self
    }

    fn build(&self) -> String {
        if self.filters.is_empty() {
            return format!("SELECT * FROM {}", self.table);
        }
        format!(
            "SELECT * FROM {} WHERE {}",
            self.table,
            self.filters.join(" AND ")
        )
    }
}
```

```
// One statement: the temporary builder lives until the end of the statement.
let all = QueryBuilder::new("users").filter("age > 30").build();
println!("{all}");

// Several statements: bind the builder to a variable first.
let only_active = true; // imagine a command-line flag
let mut query = QueryBuilder::new("users");
query.filter("age > 30");
if only_active {
```

```
    query.filter("active");
}
println!("{}", query.build());
```

```
https://example.com/items [("accept", "text/plain")] 500ms
SELECT * FROM users WHERE age > 30
SELECT * FROM users WHERE age > 30 AND active
```

Its advantages are that a builder can be configured conditionally without rebinding it, and reused to build several values (`build(&self)`). Its pitfall is the return type: `filter` returns a borrow of the builder. In a single statement that is fine, because a temporary lives until the end of the statement it is created in (Chapter 5). Binding the borrow to a variable is not, as `code/compile_fail/ch14_builder_temporary.rs` shows:

```
pub fn caller() -> String {
    let query = QueryBuilder::new("users").filter("age > 30");
    query.filter("active");
    query.build()
}
```

```
error[E0716]: temporary value dropped while borrowed
  --> compile_fail/ch14_builder_temporary.rs:34:17
   |
34 |     let query = QueryBuilder::new("users").filter("age > 30");
   |                 ^^^^^^^^^^^^^^^^^^^^^^^^^^                    - temporary value is freed
at the end of this statement
   |                 |
   |                 creates a temporary value which is freed while still in use
35 |     query.filter("active");
   |     ----- borrow later used here
   |
help: consider using a `let` binding to create a longer lived value
   |
34 ~     let mut binding = QueryBuilder::new("users");
35 ~     let query = binding.filter("age > 30");
   |
```

E0716, "temporary value dropped while borrowed". The temporary is the builder created by `QueryBuilder::new("users")` ("creates a temporary value which is freed while still in use"); `query` is a `&mut QueryBuilder` pointing into it; the temporary is freed at the semicolon; and `query` is used on the next line ("borrow later used here"). The help shows the fix: bind the builder itself to a variable, as the working program does.

Both builders own their data (`impl Into<String>`), which is why neither has a lifetime parameter. A builder may borrow its inputs when it is always used within one expression, but the value it builds is usually kept, so it should own.

14.10 A refactoring walkthrough: from lifetime soup to a clear design

Lifetime problems in real code rarely come one at a time. A common history is this: a struct borrows something, the compiler complains, a lifetime is added, the compiler complains elsewhere, the same lifetime is added there too, and so on, until the code compiles. The result is "lifetime soup": many

parameters, each added to silence one error, with relationships nobody designed. This section takes one such design, a small log analyzer, and improves it in three steps. Every version is a complete program, and the three that compile print the same report.

Version 1: lifetime soup

Here is the starting point, `code/compile_fail/ch14_refactor_v1.rs`:

```
pub struct Config<'c> {
    pub min_level: Level,
    pub ignore: Vec<&'c str>,
}

pub struct Analyzer<'a, 'b, 'c> {
    config: &'b Config<'c>,
    input: &'a str,
    counts: BTreeMap<Level, usize>,
    errors: Vec<&'a str>,
}

impl<'a, 'b, 'c> Analyzer<'a, 'b, 'c> {
    pub fn new(config: &'b Config<'c>, input: &'a str) -> Analyzer<'a, 'b, 'c> {
        Analyzer {
            config,
            input,
            counts: BTreeMap::new(),
            errors: Vec::new(),
        }
    }

    pub fn run(&'a mut self) {
        for line in self.input.lines() {
            let Some((level, message)) = parse_line(line) else {
                continue;
            };
            let ignored = self.config.ignore.iter().any(|word| message.contains(word));
            if level < self.config.min_level || ignored {
                continue;
            }
            *self.counts.entry(level).or_insert(0) += 1;
            if level == Level::Error {
                self.errors.push(message);
            }
        }
    }

    pub fn counts(&'a self) -> &'a BTreeMap<Level, usize> {
        &self.counts
    }

    pub fn errors(&'a self) -> &'a [&'a str] {
        &self.errors
    }
}
```

Three lifetime parameters: `'a` for the input, `'b` for the borrow of the configuration, `'c` for the strings inside the configuration. The receivers `&'a mut self` and `&'a self` were added because the methods deal with `'a` data. The library compiles. Its first caller does not:

```
pub fn caller(input: &str) {
    let config = Config {
        min_level: Level::Warn,
        ignore: vec!["heartbeat"],
    };
    let mut analyzer = Analyzer::new(&config, input);
```

```
    analyzer.run();
    println!("{:?}", analyzer.counts());
}
```

```
error[E0502]: cannot borrow `analyzer` as immutable because it is also borrowed as mutable
  --> compile_fail/ch14_refactor_v1.rs:82:22
   |
81 |     analyzer.run();
   |     -------- mutable borrow occurs here
82 |     println!("{:?}", analyzer.counts());
   |                      ^^^^^^^^
   |                      |
   |                      immutable borrow occurs here
   |                      mutable borrow later used here
```

The labels are the hallmark from the previous section: the mutable borrow of `run` is "later used" at the very next use of `analyzer`. `run(&'a mut self)` in `impl<'a, 'b, 'c> Analyzer<'a, 'b, 'c>` is `&'a mut T<'a>`, and the analyzer is borrowed forever after its first run. The worst part is *where* the error appears: not in the library, which compiled, but in the code of its users.

Version 2: fix the receivers

The minimal fix gives every receiver its own, elided lifetime (`code/examples/ch14_refactor_v2.rs`):

```
// `run` is now `fn run(&mut self)`: a short borrow, chosen at each call.
fn counts(&self) -> &BTreeMap<Level, usize> {
    &self.counts
}

fn errors(&self) -> &[&'a str] {
    &self.errors
}
```

```
let input = String::from(LOG);
let config = Config {
    min_level: Level::Warn,
    ignore: vec!["heartbeat"],
};
let mut analyzer = Analyzer::new(&config, &input);
analyzer.run();
for (level, count) in analyzer.counts() {
    println!("{level:?}: {count}");
}
println!("errors: {:?}", analyzer.errors());
```

```
Warn: 1
Error: 2
errors: ["db timeout after 30s", "disk full"]
```

`errors(&self) -> &[&'a str]` shows the two borrows clearly: the slice borrows the analyzer (briefly, elided), and the strings in it borrow the input (`'a`). This version is correct. But look at what remains: three lifetime parameters, and two of them buy nothing. The configuration is a few bytes, built once; borrowing it forces every analyzer to live inside the configuration's scope, and `'c` makes the configuration itself a view of string literals. Anyone who stores an `Analyzer` in a struct must now write three lifetimes.

Version 3: own what is small, borrow what is big

The input can be large, and borrowing it is the point of the design. The configuration is small: let it own its strings, and borrow it only for the call that needs it (`code/examples/ch14_refactor_v3.rs`):

```
/// Small, built once, read often: it owns its data.
struct Config {
    min_level: Level,
    ignore: Vec<String>,
}

/// Borrows the log it analyzes, and nothing else.
struct Analyzer<'src> {
    input: &'src str,
    counts: BTreeMap<Level, usize>,
    errors: Vec<&'src str>,
}

impl<'src> Analyzer<'src> {
    fn new(input: &'src str) -> Self {
        Analyzer {
            input,
            counts: BTreeMap::new(),
            errors: Vec::new(),
        }
    }

    /// The configuration is needed only during the call: borrow it there.
    fn run(&mut self, config: &Config) {
        for line in self.input.lines() {
            let Some((level, message)) = parse_line(line) else {
                continue;
            };
            let ignored = config
                .ignore
                .iter()
                .any(|word| message.contains(word.as_str()));
            if level < config.min_level || ignored {
                continue;
            }
            *self.counts.entry(level).or_insert(0) += 1;
            if level == Level::Error {
                self.errors.push(message);
            }
        }
    }

    fn counts(&self) -> &BTreeMap<Level, usize> {
        &self.counts
    }

    fn errors(&self) -> &[&'src str] {
        &self.errors
    }
}
```

```
Warn: 1
Error: 2
errors: ["db timeout after 30s", "disk full"]
```

One lifetime parameter is left, named after what it borrows: `'src` , the log. `Config` has no lifetime, and `run(&mut self, config: &Config)` borrows it for the duration of the call, so one configuration can serve any number of analyzers.

Version 4: views for the work, owned values for the results

One question remains: why does the *analyzer* hold borrows at all? The input is needed only while the analysis runs; what is kept afterward is the result. The final version separates the two (`code/examples/ch14_refactor_v4.rs`). The work uses a zero-copy view of each line:

```
/// One parsed line: a zero-copy view into the log.
#[derive(Clone, Copy, Debug)]
struct Entry<'src> {
    level: Level,
    message: &'src str,
}

impl<'src> Entry<'src> {
    fn parse(line: &'src str) -> Option<Self> {
        let (level, message) = line.split_once(' ')?;
        let level = match level {
            "INFO" => Level::Info,
            "WARN" => Level::Warn,
            "ERROR" => Level::Error,
            _ => return None,
        };
        Some(Entry { level, message })
    }
}

/// The entries of a log, borrowed from it.
fn entries(input: &str) -> impl Iterator<Item = Entry<'_>> {
    input.lines().filter_map(Entry::parse)
}
```

and the result owns its data:

```
struct Config {
    min_level: Level,
    ignore: Vec<String>,
}

impl Config {
    fn accepts(&self, entry: &Entry<'_>) -> bool {
        let ignored = self
            .ignore
            .iter()
            .any(|word| entry.message.contains(word.as_str()));
        entry.level >= self.min_level && !ignored
    }
}

/// What we keep: it owns its data, so it can outlive the log.
#[derive(Default)]
struct Report {
    counts: BTreeMap<Level, usize>,
    errors: Vec<String>,
}

/// Borrows the log and the configuration for the duration of the call only.
fn analyze(input: &str, config: &Config) -> Report {
    let mut report = Report::default();
    for entry in entries(input).filter(|entry| config.accepts(entry)) {
        *report.counts.entry(entry.level).or_insert(0) += 1;
        if entry.level == Level::Error {
            report.errors.push(entry.message.to_owned());
        }
    }
    report
}
```

```
let input = String::from(LOG);
let config = Config {
    min_level: Level::Warn,
    ignore: vec![String::from("heartbeat")],
};
let report = analyze(&input, &config);
drop(input); // the report does not borrow the log
for (level, count) in &report.counts {
    println!("{level:?}: {count}");
}
println!("errors: {:?}", report.errors);

// `Report: Send + 'static`: it can be handed to another thread.
let handle = thread::spawn(move || report.errors.len());
assert_eq!(handle.join().unwrap(), 2);
```

```
Warn: 1
Error: 2
errors: ["db timeout after 30s", "disk full"]
```

`analyze` is the boundary. It borrows the log and the configuration for the duration of one call, works with borrowed `Entry<'src>` values that never leave it, and returns a `Report` that owns everything it contains. The `main` function drops the input before printing the report, and sends the report to another thread afterward: neither would compile if the report borrowed the log. The price is a copy of each error message, a few bytes; if it ever mattered, the report could store line numbers instead.

What changed

Version	Lifetime parameters	What it means for users
1	three, and `&'a` receivers	the analyzer is unusable after `run` (E0502)
2	three	works; every user writes three lifetimes
3	one, `'src`	the analyzer borrows the log only
4	none on kept types	`Report` can be stored, returned, sent

The general recipe, in the order the walkthrough applied it:

1. Remove lifetimes from receivers: `&self` and `&mut self` almost never need a named lifetime. A lifetime on both a `&mut` and the type behind it is a bug.
2. Own small data, especially configuration. Borrow it per call if needed.
3. Name the lifetimes that remain after what they borrow.
4. Borrow in the functions that do the work; own in the types that hold the results. A function call is a natural boundary between the two.

14.11 Common confusions

"**More lifetime annotations make code faster or more precise.**" Lifetimes are erased before code generation (Chapter 1); they never make code faster. Each annotation is a constraint, and a wrong one, like `&'a mut self`, only rejects more programs.

"**Owning is slow, so borrow everywhere.**" Copying a configuration value or an error message at a boundary costs almost nothing. Borrowing pays off where data is large or produced in great numbers, such as the tokens of a big input. Borrow there, own elsewhere, and measure before trading clarity for speed.

"**A method that stores `&'a` data needs `&'a mut self`.**" It needs the *data* to be `&'a`, which is a requirement on the argument. The receiver's borrow should stay short and elided.

"**One lifetime parameter is always simpler.**" Simpler to write, yes. When a struct's results borrow from only one of its sources, or when it holds a `&mut` to a container of borrows, one parameter is wrong, and the error shows up in the callers.

"**`Cow` everywhere avoids every decision.**" `Cow` adds a lifetime parameter (or a `'static` restriction) and a branch at every use. It is the right type when both cases, borrowed and owned, are common; otherwise pick one.

"**A callback that receives `&str` can keep it if the data lives long enough.**" Only if the signature says so. With `impl Fn(&str)`, the callback must work for any lifetime, so it can never keep the borrow, however long the data actually lives.

14.12 Summary

- Borrow for the call, own for storage. Parameters that are only read are `&str`, `&[T]` or `impl AsRef<str>`; parameters that are kept are `String` or `impl Into<String>`, which lets callers move a `String` in without a copy.
- Return a reference for a part of an input, an owned value for new data, and `Cow<'a, str>` when it can be either. Structs that are kept should own their fields; structs with reference fields are views, for short-lived work.
- `Cow<'static, str>` holds a literal or a run-time string without a lifetime parameter. `Arc<str>` and `Rc<str>` share immutable text among many owners, with no lifetime parameter and cheap clones.
- Zero-copy designs pair a borrowed type (`Token<'src>`) for the work with an owned twin (`OwnedToken`, via `into_owned`) for what is kept.
- One lifetime parameter is usually enough, thanks to covariance. Use two when results borrow from only one of two sources, or when a field is a `&mut` to a container of borrows. Name lifetimes after what they borrow.
- `&'a mut T<'a>` borrows a value for the rest of its life ("borrowed forever"). It appears as `&'a mut self` in `impl<'a> T<'a>` and as `&'a mut Vec<&'a str>` in fields. Give the outer borrow its own lifetime.
- `impl Fn(&T)` callbacks receive borrows they cannot keep (E0521); a signature with a named lifetime lets them keep the borrow, at the cost of a stronger promise. Stored callbacks are `'static` by default; `dyn Fn() + 'a` lets them borrow.

- `impl Iterator<Item = &T>` captures the `&self` lifetime in edition 2024; use a named iterator type when callers need to store it, and `use<'src>` when the items borrow from a source rather than from `self`.
- Public errors should own their data (or store positions). Builders should own their data; `&mut self` builders must be bound to a variable before being configured over several statements (E0716).

14.13 Exercises

1. In a copy of `ch14_tokenizer.rs`, give `Token<'src>` fields of type `Cow<'src, str>` instead of `&'src str`, and a method `fn into_owned(self) -> Token<'static>`. Then remove `OwnedToken`: the history becomes a `Vec<Token<'static>>`. *Hint:* each variant becomes `Ident(Cow<'src, str>)`; the tokenizer creates `Cow::Borrowed(..)`, and `into_owned` rebuilds each variant with `Cow::Owned(text.into_owned())`.
2. In a copy of `ch14_errors.rs`, make `ParseError` store the byte range of the offending line, a `std::ops::Range<usize>`, instead of a `&'src str`. Add a method `line` that takes the source and returns the text of the line. Can `load` now return a `ParseError` directly? Is that a good idea? *Hint:* the method is `fn line<'s>(&self, src: &'s str) -> &'s str`. The error has no lifetime parameter any more, so the E0515 of `ch14_error_borrows_local.rs` disappears. But the caller of `load` never sees the text, so the range alone is not very useful to it.
3. Change `for_each_line` in `ch14_callbacks.rs` so that it passes each line in lowercase, built in a `String` buffer that is reused for every line. Then try the same change in `for_each_line_kept`. *Hint:* the first signature allows it, because the callback accepts a borrow of any lifetime, including a borrow of a local buffer. The second promised `&'text str`, and a borrow of the buffer is not one: the compiler rejects `f(&buffer)` (E0597, "`buffer` does not live long enough").
4. Give `WordSet` from `ch14_borrowed_forever_fixed.rs` a field `counts: BTreeMap<&'text str, usize>` and a method `fn most_common(&self) -> Option<&'text str>`. Show that the result can be used after the `WordSet` is dropped. *Hint:* the result is copied out of the map with the lifetime `'text`, not the lifetime of `&self`; write `self.counts.iter().max_by_key(..).map(|(w, _)| *w)`.
5. Extend version 4 of the log analyzer: add `fn merge(&mut self, other: Report)` to `Report`, and analyze two logs on two threads with `thread::scope`, merging the reports in `main`. Which of the four versions could you have done this with `thread::spawn` instead? *Hint:* scoped threads may borrow the logs and the configuration (Chapter 7). With `thread::spawn`, the threads must own their inputs, and only version 4's `Report` can come back from a thread on its own.

Appendices

The appendices are for reference: how to run and verify the examples, a catalogue of the lifetime-related compiler errors with pointers to the chapters that explain them, a one-page quick reference, a glossary and a list of further reading.

APPENDIX A

Setting Up and Verifying the Examples

All the code in this book lives in the `code/` directory of the book's repository: a Cargo workspace with no dependencies. This appendix explains how to install the right compiler, run the examples, compile the programs that are supposed to fail, and use the verification script, including how to use it to see what changes when you try a newer compiler.

In this appendix you will:

- install the pinned toolchain with rustup
- run an example and compile a program that must be rejected
- verify the whole book with `tools/run_examples.py`
- compare the book's diagnostics with those of another compiler version

A.1 Installing the toolchain

Install Rust with rustup if you have not already. The workspace pins its compiler in `code/rust-toolchain.toml`:

```
# The book shows real compiler diagnostics, so the exact compiler is pinned.
# rustup installs it automatically the first time you run cargo here.
# rust-src is the standard library's source: rust-analyzer needs it, and with it
# installed rustc quotes standard-library code in some notes, which the
# diagnostic snapshots contain.
[toolchain]
channel = "1.99.0"
components = ["rustfmt", "rust-src"]
```

You do not need to install Rust 1.99 by hand: the first time you run `cargo` or `rustc` inside `code/`, rustup reads this file and installs the toolchain with the two listed components. `rust-src` is the source of the standard library. Editors need it to navigate into the standard library, and with it installed rustc quotes standard-library code in a few notes, which is what the book's diagnostics show. To check which compiler is active:

```
cd code
rustup show active-toolchain   # 1.99.0-... (overridden by '.../code/rust-toolchain.toml')
rustc --version                # rustc 1.99.0 (b940084d7 2026-09-28)
```

If you use rust-analyzer, note that the workspace is one directory below the repository root. The repository contains a `.vscode/settings.json` that points rust-analyzer at `code/Cargo.toml`; for other editors, open `code/` as the project, or configure the linked project the same way.

A.2 Running the examples

Runnable examples are single files in `code/examples/`, named after the chapter that uses them: `ch09_callee_picks.rs` belongs to Chapter 9. Each one starts with a line such as `// harness: run` (for the verification script, see below) and a doc comment that says what it shows and how to run it:

```
cd code
cargo run --example ch09_callee_picks
```

Every example builds without warnings and prints the output that the book shows. Change them freely: they are small, and most of the exercises start from one.

A.3 Compiling the programs that must be rejected

The programs in `code/compile_fail/` are not part of the Cargo package (they would stop it from building). Each one is a small library, with no `main` function, that the compiler must reject. Its first line lists the error codes or messages it must produce, for example:

```
// harness: compile-fail E0597
```

and its diagnostics are stored next to it in a file with the extension `.stderr`. To see the errors yourself, compile one with `rustc` from the `code/` directory, the same way the verification script does:

```
cd code
rustc --edition 2024 --crate-type lib --emit=metadata -A warnings \
      -o /tmp/out.rmeta compile_fail/ch09_caller_picks.rs
```

`--crate-type lib` says that there is no `main` to look for, `--emit=metadata` stops after type and borrow checking (no code is generated), and `-A warnings` hides warnings that are not part of the lesson, such as unused variables. A few programs declare an older edition in their first line (`edition=2021`); compile those with `--edition 2021` to see what the book shows.

A.4 Verifying everything

`tools/run_examples.py` is the script that keeps the book honest. From the root of the repository:

```
python3 tools/run_examples.py                 # every example and compile-fail program
python3 tools/run_examples.py --only ch09     # one chapter
python3 tools/run_examples.py --rust          # only the runnable examples
python3 tools/run_examples.py --compile-fail  # only the programs that must fail
```

For each runnable example, it builds the example (failing on any warning), runs it, and compares its standard output with the snapshot in `code/output/`, which is what the book includes. For each compile-fail program, it compiles the file, checks that compilation fails with the declared codes or messages, and compares the complete diagnostics with the `.stderr` snapshot. A line of output per program and a final count tell you whether everything matches:

```
ok   rust          ch09_callee_picks                    0.4s
ok   compile-fail  ch09_caller_picks                    0.0s
```

```
...
2 passed, 0 failed
```

When something differs, the script prints a diff between the snapshot and what the compiler produced. Two options change what it does with differences:

- `--bless` rewrites the snapshots from the current results. Authors use it after changing an example; you can use it after an experiment, to see the new diagnostics in the files.
- `--codes-only` checks only that each compile-fail program fails with its declared codes and messages, not that the full text is identical.

A.5 Trying a newer (or older) compiler

Diagnostics change from release to release, usually for the better. To see how your compiler differs from the one the book was verified with, override the pinned toolchain for one run with the `RUSTUP_TOOLCHAIN` environment variable, which takes precedence over `rust-toolchain.toml`:

```
RUSTUP_TOOLCHAIN=stable python3 tools/run_examples.py --codes-only
RUSTUP_TOOLCHAIN=stable python3 tools/run_examples.py --compile-fail
```

The first command answers the important question: does every program that the book says is rejected still fail for the same reason, and does every example still compile and print the same output? The second shows the exact differences in wording, which are usually harmless (a reworded label, an extra help line). If a program that should fail now compiles, the language has changed. The most likely candidate is the example of Chapter 5 that today's borrow checker rejects and Polonius accepts (see the next section).

A.6 Nightly-only experiments

A few places in the book mention features that are only available on the nightly compiler, most importantly Polonius, the borrow checker that is expected to replace today's NLL checker, which accepts some programs that stable Rust rejects (Chapter 5). To try them, install a nightly toolchain and compile the program with it explicitly:

```
rustup toolchain install nightly
cd code
rustc +nightly --edition 2024 --crate-type lib --emit=metadata \
      -o /tmp/out.rmeta compile_fail/ch05_nll_case3.rs
```

On stable Rust 1.99 this program is rejected with E0502 (that is why it is in `compile_fail/`); with the nightly of 2026-10-08 it compiles. Adding `-Zpolonius=off` to the nightly command switches Polonius off again, and the error comes back.

The book's statements about nightly behavior were checked with `rustc 1.101.0-nightly (2026-10-08)`. Nightly changes every day: if the result differs, check the status of the feature in its tracking issue (Appendix E lists them).

A.7 Building the book

You do not need to build the book to use it, but if you want to rebuild the PDFs after changing an example, the repository's README lists the requirements (Python 3.11, pandoc 3, Typst and Graphviz) and the commands. The short version:

```
python3 tools/build_book.py check   # includes, anchors and image links
python3 tools/build_book.py all     # diagrams, per-chapter Markdown and PDFs, the book PDF
```

A.8 Summary

- The compiler is pinned in `code/rust-toolchain.toml` (Rust 1.99.0, edition 2024); rustup installs it on first use.
- Run an example with `cargo run --example chNN_name` from `code/`.
- Compile a rejected program with `rustc --edition 2024 --crate-type lib --emit=metadata -A warnings <file>`.
- `python3 tools/run_examples.py` verifies every program, output and diagnostic; `--only`, `--bless` and `--codes-only` select, update and relax the checks.
- `RUSTUP_TOOLCHAIN=<toolchain>` runs the verification with another compiler.

APPENDIX B

Error Catalogue

This appendix lists every lifetime-related diagnostic that appears in the book: errors with a code, errors without one, lints, and the two run-time panics of `RefCell`. For each one it gives what the diagnostic usually means, the typical fixes, and where the book explains it, with the file that reproduces it. Every file named here is in `code/compile_fail/` (or `code/examples/` for the panics), and its `.stderr` snapshot shows the full diagnostic as rustc 1.99 prints it.

Two habits make any of these diagnostics easier to read (Chapter 1):

- Read the labels as a timeline: where a borrow is created, what conflicts with it or ends its value, which later use keeps it alive, and where a requirement comes from ("argument requires that ...", "returning this value requires that ...", "type annotation requires that ...").
- Treat a suggestion as one way to make *this* error disappear, not as the contract you want. Several suggestions in this book (noted below) compile and are still the wrong design.

`rustc --explain E0597` prints the long explanation of any code.

B.1 Errors with a code

E0106 – missing lifetime specifier

Means: a reference (or a type with a lifetime parameter) needs a lifetime that elision cannot provide.

- In a return type: more than one parameter has lifetimes and there is no `self` reference, or the only parameter has two lifetimes (`&[&str] -> &str`). Name the lifetime the result borrows from: `fn f<'a>(x: &'a str, y: &str) -> &'a str`. Writing `'_` in the output does not help. (Chapter 3; Chapter 6 for `'_`; `ch03_two_inputs`, `ch03_named_and_elided`, `ch03_rule2_shared_name`, `ch03_rule2_slice_of_refs`, `ch03_self_by_value`, `ch03_self_two_refs`, `ch06_ambiguous`)
- In a struct or enum field: fields are never elided. Declare the parameter, `struct S<'a> { r: &'a str }`, or own the data. (Chapters 2 and 3; `ch02_missing_lifetime`, `ch03_struct_field_elided`)
- In an `Fn(..)` bound, a function pointer type or a trait method: the same rules apply as for functions. In an `Fn` bound, the suggestion ties every lifetime together, which may be the wrong contract. (Chapters 3 and 9; `ch03_fn_pointer_two_inputs`, `ch03_trait_no_body`, `ch09_sugar_two_inputs`)

E0227 – ambiguous lifetime bound, explicit lifetime bound required

Means: a trait object's default lifetime bound would come from the trait, but the trait has two different non-`'static` lifetime bounds. Write the bound (`dyn Trait + 'a`). (Chapter 8; `ch08_two_trait_bounds`)

E0228 – cannot deduce the lifetime bound for this trait object type from context

Means: the type that contains a trait object has more than one lifetime bound on that parameter, so there is no default. Write the bound. (Chapter 8; `ch08_two_bounds`)

E0261 – use of undeclared lifetime name

Means: a lifetime such as `'a` is used without being declared in `<...>` on the function, `impl`, struct or trait. A `'_` in an `impl` header cannot be referred to by a name later. Declare it: `impl<'src> Parser<'src>`. (Chapters 3 and 6; `ch03_struct_field_placeholders`, `ch06_impl_header_name`)

E0277 – trait bound not satisfied

Two forms in this book:

- "`L` is not an iterator": a `for` loop over a lending iterator, which does not implement `Iterator`. Use `while let Some(item) = it.next()`. (Chapter 10; `ch10_for_loop`)
- "`{async block@...}` cannot be unpinned": an `Unpin` bound applied to a future that may be self-referential. Pin it first with `pin!` or `Box::pin`. (Chapter 11; `ch11_not_unpin`)

E0308 – mismatched types: "one type is more general than the other"

Means: a type with a fixed lifetime is used where a higher-ranked type is required: a `fn(&'static str)` where `for<'a> fn(&'a str)` is expected, or a future type parameter `Fut` for an async callback that borrows its argument. Use the higher-ranked type; for async callbacks use `AsyncFn`. (Chapter 9; `ch09_hr_subtyping`, `ch09_async_fut_param`)

E0308 also reports that coercions do not pass through containers: a `Vec<&String>` is not a `Vec<&str>`. (Chapter 4; `ch04_no_coercion_in_vec`)

E0309 – the parameter type `T` may not live long enough

Means: a bound or type requires `T: 'a` and nothing guarantees it. Add the bound (`T: 'a`), or avoid the requirement: in edition 2024, an `impl Trait` return type that captures `T` does not need `+ 'a`. (Chapters 8 and 9; `ch08_outlives_type_param`)

E0310 – the parameter type `T` may not live long enough (`'static`)

Means: generic code passes `T` to an API that requires `'static` (`thread::spawn`, `Box<dyn Any>`, a `'static` trait object). Add the bound to your signature (`T: Send + 'static`), so that callers are checked against it. (Chapter 7; `ch07_generic_spawn`)

E0316 – nested quantification of lifetimes

Means: a bound has two `for<..>` binders, as in `for<'a> F: for<'b> Fn(&'a str, &'b str)`. Use one binder with both lifetimes. (Chapter 9; `ch09_binder_limits`)

E0373 – closure may outlive the current function, but it borrows `x`

Means: a closure that borrows a local is passed to an API that requires `'static` (usually `thread::spawn`). Add `move` so that the closure owns its captures, or use `thread::scope`. The same error exists for async blocks. (Chapter 7; `ch07_spawn_borrow`, `ch07_static_table_no`; for an async block spawned on an executor, Chapter 11: `ch11_spawn_borrow_block`)

E0382 – borrow of moved value / use of moved value

Means, in this book: a `&mut T` (or an `Option<&mut T>`) was moved, because mutable references are not `Copy`, and then used again. Let the compiler reborrow (pass it directly to a function), or reborrow explicitly with `&mut *r` or `.as_deref_mut()`. (Chapter 5; `ch05_move_mut_ref`, `ch05_generic_moves`, `ch05_option_mut_moves`)

E0392 – lifetime parameter `'a` is never used

Means: a type declares a lifetime (or type) parameter that none of its fields uses, typically a wrapper around a raw pointer. Add a `PhantomData` field that says what the type behaves as if it held, such as `PhantomData<&'a T>`; the choice also sets the type's variance. (Chapter 13; `ch13_unused_lifetime`)

E0477 – the type does not fulfill the required lifetime

Means, in this book: an `impl` of a trait with a generic associated type does not repeat the trait's `where Self: 'a` on the associated type. Copy the where clause into the impl. (Chapter 10; `ch10_impl_without_where`, which also shows E0309 for the generic parameter)

E0499 – cannot borrow `x` as mutable more than once at a time

Means: two mutable borrows of the same place are live at the same point.

- Sequence the borrows so that the first is no longer used when the second starts. (Chapter 1; `ch01_two_mutable`)
- A method returns a reference with the elided `&mut self` lifetime, so every result keeps the value mutably borrowed. Return the data's lifetime instead (`-> Option<&'src str>`). (Chapter 2; `ch02_parser_elided`, `ch02_source_elided`)
- A method takes `&'a mut self` in `impl<'a> S<'a>`: "borrowed forever". Use the elided `&mut self`. (Chapters 2 and 4; `ch02_borrowed_forever`, `ch04_mut_self_lock`)
- Two methods on `&mut self` cannot split a value into disjoint parts, and indexing cannot prove two indices differ: borrow fields directly, or use `split_at_mut` / `get_disjoint_mut`. (Chapter 5; `ch05_split_methods`, `ch05_split_index`, `ch05_reborrow_frozen`)
- Two items of a lending iterator alive at once, or one item kept across loop iterations ("mutably borrowed here in the previous iteration of the loop"): each item borrows the iterator until it is dropped. (Chapter 10; `ch10_two_items`, `ch10_keep_line`)

E0502 – cannot borrow `x` as mutable because it is also borrowed as immutable (or the reverse)

Means: a shared and a mutable borrow of the same place are live at the same point.

- The classic: a reference into a collection is still used after a call that mutates it. Move the use, or copy the value out. (Chapter 1; `ch01_nll_conflict`, `ch01_nll_derived`, `ch01_mutable_then_shared`, `ch01_branch_conflict`, `ch01_loop_conflict`)
- An argument that borrows the receiver, where two-phase borrows do not apply. (Chapter 5; `ch05_two_phase_explicit`, `ch05_two_phase_shared`)
- A temporary in a `match` scrutinee, or (in edition 2021) an `if let` scrutinee, keeps a borrow alive in the arms. Bind it with `let` first. (Chapters 5 and 12; `ch05_match_scrutinee`, `ch05_if_let_2021`)
- A conditionally returned borrow (NLL problem case #3): accepted by Polonius on nightly; on stable, restructure (look up first, then insert). (Chapter 5; `ch05_nll_case3`)
- "Borrowed forever" (`&'a mut T<'a>`): the immutable borrow and "mutable borrow later used here" are on the same expression. (Chapters 3 and 14; `ch03_rule_unify_push`, `ch14_borrowed_forever`, `ch14_collector_forever`)
- Tokens borrowed from a buffer that is reused; an `impl Trait` in a trait that captures `&self`. (Chapters 8, 12 and 14; `ch14_tokens_outlive_buffer`, `ch08_rpitit_captures`, `ch12_oncecell_take`, `ch02_shared_self_keep`)
- A future holds the borrows of its arguments until it completes; a raw-pointer iterator holds a borrow through `PhantomData`. (Chapters 11 and 13; `ch11_future_holds_borrow`, `ch13_raw_iter_borrow`)

E0505 – cannot move out of `x` because it is borrowed

Means: a value is moved while a borrow of it is still going to be used. Use the borrow before the move, or clone the value. (Chapter 5; `ch05_move_while_borrowed`)

A self-referential value cannot be moved while it borrows from itself: "move out of `doc` occurs here" and "borrow later used here" point at the same spot. Store indices or ranges instead of references. (Chapter 13; `ch13_self_ref_move`, `ch13_self_ref_new`)

E0506 – cannot assign to `x` because it is borrowed

Means: an assignment would drop the old value while a borrow of it is live. Assign after the last use of the borrow. (Chapter 1; `ch01_assign_borrowed`)

E0515 – cannot return reference to local variable / value referencing local variable / reference to temporary value

Means: a function returns a borrow of something it owns, which is dropped when it returns.

- Return owned data (`String`, `Vec`), or let the caller own the input. (Chapters 5 and 14; `ch05_return_local`, `ch14_error_borrows_local`, `ch08_error_escapes`)
- A compiler suggestion tied the result to a local collection. (Chapter 3; `ch03_slice_suggested`)
- `&String::new()` or `&Cell::new(0)` is not promoted to a `'static` constant. (Chapter 7; `ch07_not_promoted`)
- A reference into a `RefCell` would outlive its guard: return the guard, `Ref::map`, or take a closure. (Chapter 12; `ch12_return_ref_from_guard`)

- With one lifetime parameter for two inputs, the result is tied to the shorter-lived one: use two parameters. (Chapter 14; `ch14_one_lifetime`)
- A correctly bounded `unsafe` API refuses to let a slice outlive its buffer. (Chapter 13; `ch13_bounded_slice`)

E0521 – borrowed data escapes outside of function (or closure)

Means: data borrowed from a parameter flows into a place that must outlive the function, typically `thread::spawn` or an `impl dyn Trait` block with an implicit `'static`. `move` does not help when the captured value is itself a reference. Own the data (clone, `Arc`), use `thread::scope`, or write `impl dyn Trait + '_`. For closures: a callback that stores its argument in an outer collection. The same error appears when an `async move` block captures a reference parameter and is spawned, and with the "current limitation" note of Chapter 10. (Chapters 6, 7, 10, 11 and 14; `ch07_spawn_param`, `ch06_impl_dyn_static`, `ch14_callback_keeps_borrow`, `ch11_spawn_ref_param`, `ch10_for_each_static`)

E0582 – binding for associated type `Output` references lifetime `'a`, which does not appear in the trait input types

Means: a higher-ranked lifetime appears only in an output, as in `for<'a> Fn() -> &'a str`. Nothing constrains it; use `'static` or tie it to an input. (Chapter 9; `ch09_binder_limits`)

E0597 – `x` does not live long enough

Means: a borrow would have to stay live after its value is dropped. The labels tell you why it must stay live:

- "borrow later used here": a use after the value's scope ended. Use the result earlier, move the value to an outer scope, or own the result. (Chapter 1; `ch01_longest_dangling`, `ch01_signature_contract`)
- "argument requires that `x` is borrowed for `'a`", where `'a` is a lifetime parameter of the function: a caller-chosen lifetime used for a local; make the bound higher-ranked. (Chapter 9; `ch09_caller_picks`, `ch09_borrowed_iter_named`)
- "… borrowed for `'static`" from an argument, a type annotation or a coercion: a `'static` requirement (a `T: 'static` bound, `&'static`, a default `Box<dyn Trait>` bound). Pass owned data. (Chapters 1, 3, 7 and 8; `ch01_annotation_static`, `ch03_static_field_local`, `ch07_static_bound_borrow`, `ch07_callback_borrow`, `ch08_trait_bounds_explicit`)
- A contract that ties the result to a short-lived input: give the inputs separate lifetimes. (Chapter 3; `ch03_rule_first`, `ch03_rule_unify`, `ch03_contract_v2`)
- Invariance: the type behind `&mut` or `Cell` cannot shrink, so a short borrow must be stored as a long one. (Chapter 4; `ch04_mut_invariance`, `ch04_cell_invariance`)
- "borrow might be used here, when `x` is dropped and runs the `Drop` code": the drop check; declare the borrowed value first. (Chapter 5; `ch05_drop_check`)
- Edition 2021 tail-expression temporaries ("… when that temporary is dropped"); edition 2024 fixes them. (Chapters 5 and 12; `ch05_tail_expr_2021`, `ch12_tail_guard_2021`)
- Edition 2024 over-capturing in `impl Trait`, with the note "this call may capture more lifetimes than intended": use `use<..>`. (Chapter 8; `ch08_overcapture`, `ch08_async_captures`)

- "borrow later stored here": an `async fn` future captures every input, even one its output does not mention; a future returned or spawned while it borrows a local. (Chapter 11; `ch11_future_outlives_key`, `ch11_return_borrowing_future`, `ch11_spawn_borrow_call`)
- A higher-ranked bound over a GAT, with the note "due to a current limitation of the type system, this implies a `'static` lifetime". (Chapter 10; `ch10_hrtb_static`)
- An invariant `PhantomData` marker refuses a short-lived value, or a generic `Drop` impl keeps a borrow alive until the drop. (Chapter 13; `ch13_out_slot_variance`, `ch13_dropck_log`)

E0637 – `'_` cannot be used here

Means: `'_` appears where a lifetime must be declared, such as an associated type or a where-clause bound. Use a named lifetime; for an associated type that should borrow from `&self`, use a generic associated type. (Chapters 6 and 10; `ch06_assoc_type`, `ch06_where_clause`, `ch10_item_anonymous`)

E0658 – unstable feature

Five unstable features in this book produce E0658 on stable Rust 1.99: closure lifetime binders (`for<'a> |x: &'a str| ...`), binders over types (`for<T>`), elided lifetimes inside argument-position `impl Trait`, return type notation (`T::method(..): Send`) and `#[may_dangle]`. (Chapters 8, 9, 11 and 13; `ch09_binder_limits`, `ch08_apit_anonymous`, `ch11_rtn_unstable`, `ch13_may_dangle_unstable`)

E0700 – hidden type captures lifetime that does not appear in bounds

Means: an `impl Trait` return type's hidden type uses a lifetime that the opaque type does not capture. In edition 2021 this is the default for elided lifetimes (the old fix was `+ '_`); in edition 2024, it happens when a `use<..>` list leaves out a lifetime the hidden type needs. Add the lifetime to `use<..>`. (Chapter 8; `ch08_rpit_2021`, `ch08_use_nothing`)

E0716 – temporary value dropped while borrowed

Means: a reference to a temporary outlives the statement that created the temporary. Temporary lifetime extension applies only to certain `let` positions, not through method or function calls. Bind the value with `let` first. (Chapter 5; `ch05_temporary_dropped`; Chapter 14: `ch14_builder_temporary`, and `ch14_iterator_overcaptures` for the edition 2024 variant)

E0794 – cannot specify lifetime arguments explicitly if late bound lifetime parameters are present

Means: a turbofish names a lifetime that is chosen at each call. Remove it. (Chapter 9; `ch09_late_turbofish`)

B.2 Diagnostics without a code

lifetime may not live long enough

The most common lifetime diagnostic. It means that a region constraint between two lifetimes cannot be proven. The primary label says which:

- "returning this value requires that `'1` must outlive `'static`": a borrowed value is returned as a trait object with the default `'static` bound (`Box<dyn Fn ...>`, `Box<dyn Error>`). Add `+ '_`, or own the data. (Chapters 6 and 8; `ch06_dyn_static`, `ch08_error_box_static`)
- "function was supposed to return data with lifetime `'a` but it is returning data with lifetime `'b`": the body breaks the signature; the help suggests a bound `'b: 'a`. (Chapters 1, 2 and 6; `ch01_body_vs_signature`, `ch02_lending_promise`, `ch06_impl_unrelated`)
- "returning this value requires that `'1` must outlive `'2`" inside a closure: a stored closure returns a reference to its argument. Pass it to a function with an `Fn` bound, route it through an identity helper, or use a `fn`. (Chapters 9 and 12; `ch09_closure_return`, `ch12_thread_local_escape`)
- "coercion requires that `'src` must outlive `'static`": a borrowing type converted to `Box<dyn Any>`. The help ("add an explicit `'src` lifetime bound") moves the error to the callers. (Chapter 7; `ch07_any_borrowed`)
- Notes "requirement occurs because of a mutable reference to ..." and "mutable references are invariant over their type parameter": a variance problem. (Chapter 4; `ch04_invariant_witnesses`, `ch04_invariant_cells`, `ch04_struct_invariant`)
- Two `'_` assumed equal; a `+ 'a` used as if it were a capture list. (Chapters 6 and 8; `ch06_never_equal`, `ch08_outlives_two`)
- "method was supposed to return data with lifetime `'s` ..." in an `Iterator` impl whose items would borrow from `&mut self`: a lending iterator, which needs a GAT. (Chapter 10; `ch10_iterator_windows`)
- Notes "… makes the generic argument ... invariant": an invariant `PhantomData` marker. A `ScopedJoinHandle` returned out of `thread::scope`; here the help "consider adding 'move' keyword" does not fix anything. (Chapter 13; `ch13_invariant_marker`, `ch13_scope_escape`, `ch13_scope_escape_move`)

implementation of `Trait` is not general enough

Means: something implements a trait (often `Fn` or `FnOnce`) for one specific lifetime, and a higher-ranked bound requires it for every lifetime. The notes name both: "for any lifetime `'1`" is the requirement, "for some specific lifetime `'2`" is what exists. Annotate closure parameters, remove bounds that make a lifetime early-bound, or use an owning type for `for<'de>`-style bounds. (Chapter 9; `ch09_trait_param_borrow`, `ch09_not_general_closure`, `ch09_early_bound`)

missing required bound on `Item`

Means: a generic associated type is used with `&'a self` or `&'a mut self` in a trait method, and the compiler requires the declaration to say `where Self: 'a`. The help gives the exact clause to add. (Chapter 10; `ch10_missing_bound`)

captured variable cannot escape `FnMut` closure body

Means: an `FnMut` closure returns a future that borrows the closure's own captures. Use an async closure with an `AsyncFnMut` bound. (Chapter 11; `ch11_fnmut_future_capture`)

future cannot be sent between threads safely

Means: a future that must be `Send` holds a non-`Send` value (such as an `Rc`) across an `.await` (note "this value is used across an await"), or a trait's `async fn` gives generic code no way to know its future is `Send`. Use `Arc`, drop the value before the await, or declare the trait method as `fn ... -> impl Future<Output = T> + Send`. (Chapter 11; `ch11_spawn_not_send`, `ch11_trait_future_not_send`)

associated type `Iterator::Item` is declared without lifetime parameters

Means: an attempt to implement `Iterator` with items that borrow from the iterator itself (a lending iterator). Use a separate type that borrows the data, yield owned items, or use a GAT-based lending trait. (Chapters 2 and 10; `ch02_lending_iterator`)

`impl Trait` must mention all type parameters in scope in `use<...>` / must mention the `Self` type of the trait

Means: a `use<..>` list may leave out lifetimes but not type parameters, and in a trait it must list `Self`. (Chapter 8; `ch08_use_all_types`, `ch08_rpitit_use_self`)

due to a current limitation of the type system, this implies a `'static` lifetime

Means (a note): a higher-ranked bound over a generic associated type with `where Self: 'a` can only hold if the type is `'static`. Put the bound on the GAT in the trait, or use a helper trait. (Chapters 9 and 10)

B.3 Lints

Lint	Default	Fires on	Chapter
`mismatched_lifetime_syntaxes` (Rust 1.89)	warn	one lifetime written two ways: "hiding a lifetime that's elided elsewhere is confusing", "eliding a lifetime that's named elsewhere is confusing"	3, 6 (`ch06_hidden_return`)
`elided_lifetimes_in_paths` (in `rust_2018_idioms`)	allow	any hidden lifetime parameter: "hidden lifetime parameters in types are deprecated"	6 (`ch06_hidden_param`)
`async_fn_in_trait`	warn	`async fn` in a public trait: "auto trait bounds cannot be specified"	11 (`ch11_async_fn_in_trait_lint`)
`let_underscore_lock`	deny	`let _ = mutex.lock()` , which drops the guard at once: "non-binding let on a synchronization lock"	12 (`ch12_let_underscore_lock`)
`if_let_rescope` , `tail_expr_drop_order` , `impl_trait_overcaptures` (in `rust_2024_compatibility`)	allow	code whose meaning changes in edition 2024: drop order of temporaries, `impl Trait` captures	5, 8

B.4 Run-time panics

`RefCell` moves the borrow rules to run time. Breaking them panics:

Message	Means	Chapter
"RefCell already borrowed"	`borrow_mut` while a `Ref` or `RefMut` is alive	12 (`ch12_double_borrow`)
"RefCell already mutably borrowed"	`borrow` while a `RefMut` is alive	12 (`ch12_double_borrow`)

`try_borrow` and `try_borrow_mut` return an error instead of panicking.

APPENDIX C

Quick Reference

This appendix collects the book's answers in the form you need them while writing code. Each row points to the chapter that explains it.

C.1 I want to...

I want to...	Write	Chapter
return a reference derived from the only reference input	elide: `fn f(x: &T) -> &U`	3
return a reference derived from one of several inputs	name it: `fn f<'a>(x: &'a T, y: &U) -> &'a V`	3
say that the result may come from either input	`fn f<'a>(x: &'a T, y: &'a T) -> &'a T`	1, 3
store a reference in a struct	`struct S<'a> { r: &'a T }`	2
return data from a struct's referent, not from the `&mut self` borrow	`impl<'a> S<'a> { fn f(&mut self) -> &'a T }`	2
make a hidden lifetime visible	`Iter<'_, T>`, `Formatter<'_>`	6
implement a trait for a borrowing type	`impl Trait for Wrapper<'_>`	6
return an iterator over `&self`	`-> impl Iterator<Item = &T>` (captures `self` in edition 2024)	8
return `impl Trait` that must not borrow some argument	`-> impl Trait + use<'a>`	8
return a boxed closure or trait object that borrows an argument	`Box<dyn Fn(..) -> .. + '_>`	6, 8
store a trait object in a long-lived struct	`Box<dyn Trait>` (`'static`) or `struct S<'a> { b: Box<dyn Trait + 'a> }`	8
return an error that borrows its input	`Box<dyn Error + '_>`, or own the data in the error	8
run a closure on another thread with borrowed data	`thread::scope(|s| { s.spawn(|| ..); })`	7
make generic code spawnable	`T: Send + 'static`	7
accept a callback for borrows the function itself creates	`F: Fn(&T) -> U` (that is, `for<'a> Fn(&'a T) -> U`)	9
relate a callback's result to one of two arguments	`F: for<'a> Fn(&'a T, &U) -> &'a V`	9
accept an async callback whose future borrows the argument	`F: AsyncFn(&T) -> U`	9, 11
iterate an owned generic collection by reference	`where for<'a> &'a C: IntoIterator<Item = &'a T>`	9
accept only types that do not borrow from the input	`T: for<'de> Trait<'de>` (serde: `DeserializeOwned`)	9
yield items that borrow from the iterator itself	a GAT: `type Item<'a> where Self: 'a;`	10
hand out a reference into a `RefCell`	`Ref::map(cell.borrow(), |v| &v.field)`	12
tie a raw-pointer wrapper to a borrow	`PhantomData<&'a T>`	13
hold text that is usually borrowed, sometimes owned	`Cow<'a, str>`	14
share immutable text without a lifetime parameter	`Arc<str>` / `Rc<str>`	14

C.2 What the compiler writes for you

You write	The compiler reads	Chapter
`fn f(x: &T) -> &U`	`fn f<'a>(x: &'a T) -> &'a U`	3
`fn m(&self, k: &K) -> &V`	the output borrows from `self`	3
`fn f(x: &T, y: &T) -> &U`	error E0106: nothing to choose from	3
`static S: &str`, `const C: &str`	`&'static str`	3
`impl S { const C: &str = ...; }`	`&'static str` (also in `impl`s and traits with lifetime parameters since 1.99)	3
`&'a T` in a signature	also the bound `T: 'a`	1
`fn f(x: &str) -> impl Trait` (edition 2024)	the opaque type captures the lifetime of `x`	8
`Box<dyn Trait>`	`Box<dyn Trait + 'static>`	8
`&'a dyn Trait`	`&'a (dyn Trait + 'a)`	8
`F: Fn(&T) -> &U`	`F: for<'a> Fn(&'a T) -> &'a U`	9
`fn(&T) -> &U`	`for<'a> fn(&'a T) -> &'a U`	9

C.3 Rules of thumb

1. A lifetime is a region of program points, not a duration. Annotations describe relationships between borrows; they never make anything live longer.
2. Only signatures need lifetimes; inside a body the compiler infers them.
3. The signature is the contract: the body is checked against it, and callers are checked against it and nothing else.
4. Elision uses the lifetime of the `self` reference if there is one, else the lifetime of the only parameter that has exactly one, else fails. It never looks at the body.
5. The `&mut self` borrow and a struct's own lifetime parameter are different lifetimes; return the one you mean.
6. `'_` means "a lifetime I do not need to name", never "the same as the other one".
7. `+ '_` lets a trait object borrow. An `impl Trait` return type captures every lifetime in scope (edition 2024); `use<..>` narrows it.
8. Shared references are covariant; `&mut T` and `Cell<T>` are invariant in `T`.
9. `&'static T` is a reference to data that lives until the end of the program; `T: 'static` is a bound that every owned type satisfies.
10. `Box<dyn Trait>` means `+ 'static`; `&'a dyn Trait` means `+ 'a`.
11. A lifetime parameter is picked by the caller and outlives the whole body. When the function itself lends data out, the bound must be higher-ranked (`for<'a>`, or the elided `Fn(&T)` form).
12. When tempted to add `'static`, first try owning, scoping, or `Cow`.
13. `&'a mut T<'a>` borrows a value for the rest of its life. Use two lifetimes.

APPENDIX D

Glossary

Terms are listed alphabetically; symbols come first. The chapter in parentheses is where the term is introduced or explained in depth.

D.1 Symbols

`'_` — the anonymous lifetime: "a lifetime exists here, I do not need to name it". Its meaning depends on its position: a fresh lifetime in a parameter, the elided output lifetime in a return type, an inference variable in a body. Two occurrences are never forced to be equal. (Chapter 6)

`'1`, `'2`, **...** — names that rustc invents in diagnostics for lifetimes that have no name in the source, such as the lifetime of an elided `&str` parameter. (Chapter 1)

`'a: 'b` — "`'a` outlives `'b`": the region `'a` contains every point of `'b`. Every lifetime outlives itself. (Chapters 1 and 4)

`'static` — the region that contains every point of the program. `&'static T` is a reference to data that lives until the program ends; the bound `T: 'static` says that `T` contains no borrow that can expire, which every owned type satisfies. (Chapter 7)

`for<'a>` — a binder: "for all `'a`". In a bound, it makes the bound higher-ranked. (Chapter 9)

`T: 'a` — "`T` outlives `'a`": every reference and lifetime parameter inside `T` outlives `'a`, so values of type `T` remain valid throughout `'a`. (Chapters 1 and 7)

`use<..>` — precise capturing: the list of generic parameters, lifetimes included, that an `impl Trait` type may capture. Stable since Rust 1.82. (Chapter 8)

D.2 A–D

Argument-position `impl Trait` (APIT) — `fn f(x: impl Trait)`, an anonymous type parameter. It cannot be named, so it cannot be listed in `use<..>`. (Chapter 8)

Aliasing XOR mutation — the rule behind borrow conflicts: at every point, a value may have any number of shared borrows or exactly one mutable borrow, but not both. (Chapter 1)

`AsyncFn`, `AsyncFnMut`, `AsyncFnOnce` — the traits of async callables (Rust 1.85). Unlike `F: Fn(&T) -> Fut`, a bound `F: AsyncFn(&T) -> U` lets the returned future borrow the argument. (Chapters 9 and 11)

Borrow — the creation of a reference to a place: shared (`&x`) or mutable (`&mut x`). The borrow is *live* while the reference, or anything derived from it, may still be used. (Chapter 1)

Borrowing view — an associated type with a lifetime, such as `type View<'a> where Self: 'a`, that some implementations fill with a borrow and others with an owned value. (Chapter 10)

Borrow checker — the part of the compiler that checks, function by function, that no borrow outlives its value and that no live borrow conflicts with another access. Since 2018 it is based on non-lexical lifetimes. (Chapter 1)

Borrowed forever — the effect of a mutable borrow annotated with the lifetime of the borrowed value's own contents, `&'a mut T<'a>`: because `&mut` is invariant, the borrow must last as long as the value, which can never be used again. (Chapters 2, 3, 4 and 14)

Closure-scoped borrow — an API that lends a reference to a callback instead of returning it (`LocalKey::with`, `thread::scope`), so that the reference cannot escape the call. (Chapters 7 and 12)

Conflict — an access (a borrow, read, write, assignment, move or drop) that is incompatible with a live borrow of the same place. (Chapter 1)

Constant promotion — taking `&` of a constant expression with no destructor and no interior mutability (`&42`, `&[1, 2, 3]`) stores the value in static memory and gives a `&'static` reference. (Chapter 7)

Contract — the signature of a function, seen as a promise: the body is checked against it, and callers are checked against it and nothing else. (Chapter 1)

Contravariant — a type constructor is contravariant in a parameter if subtyping passes through it reversed: `fn(&'short str)` is a subtype of `fn(&'long str)`. Function arguments are the main example. (Chapter 4)

Control-flow graph — the body of a function seen as a graph whose nodes are program points and whose edges lead from each point to every point that can run next. Regions are sets of its points. (Chapter 1)

Covariant — a type constructor is covariant in a parameter if subtyping passes through it in the same direction: `&'long T` is a subtype of `&'short T`, and so is `Vec<&'long T>` of `Vec<&'short T>`. (Chapter 4)

`Cow<'a, B>` — "clone on write": a value that is either borrowed (`&'a B`) or owned; a common alternative to choosing one at the type level. (Chapter 14)

Default object lifetime bound — the lifetime bound the compiler writes for a trait object when you leave it out: `Box<dyn Trait>` means `Box<dyn Trait + 'static>`, `&'a dyn Trait` means `&'a (dyn Trait + 'a)`. (Chapter 8)

Drop check — dropping a value whose type implements `Drop` counts as a use of every borrow the value holds, so those borrows must be live until the drop. (Chapters 5 and 13)

D.3 E-L

Early-bound lifetime — a lifetime parameter that is fixed when a function is *named*, like a type parameter; it happens when the lifetime appears in a where clause or bound, or only in the return type. Function items are not higher-ranked over early-bound lifetimes. (Chapter 9)

Elision — the fixed, syntactic rewrite that fills in the lifetimes you leave out of a signature: a fresh lifetime for each elided input; the input's lifetime for the outputs when exactly one parameter has exactly one lifetime; otherwise the lifetime of the `self` reference. It never looks at the body. (Chapter 3)

Erasure — lifetimes are removed before code generation; they have no run-time representation and cannot be inspected by a running program. (Chapter 1)

Executor — the part of an async runtime that stores futures (*tasks*) and polls them; *spawning* a future hands it to an executor, which is why spawned futures must usually be `Send + 'static`. (Chapter 11)

Generic associated type (GAT) — an associated type with its own generic parameters, such as `type Item<'a> where Self: 'a;`. Stable since Rust 1.65; it makes lending iterators expressible. (Chapter 10)

Guard — a value that borrows a cell or a lock and releases it when dropped: `Ref`, `RefMut`, `MutexGuard`, `RwLockReadGuard`, `RwLockWriteGuard`. (Chapter 12)

Helper-trait pattern — a workaround for higher-ranked bounds over GATs: the lifetime becomes a parameter of a separate trait, with a defaulted parameter whose implied bound restricts `for<'this>` to lifetimes that `Self` outlives. (Chapter 10)

Hidden lifetime parameter — a lifetime parameter of a type that is left out of a path, as in `SplitWhitespace` for `SplitWhitespace<'_>`. Legal, but flagged by `mismatched_lifetime_syntaxes` when it hides a borrow in a return type, and by the allow-by-default `elided_lifetimes_in_paths` everywhere. (Chapter 6)

Hidden type — the concrete type behind an `impl Trait` return type, chosen by the function's body and invisible to callers. (Chapter 8)

Higher-ranked trait bound (HRTB) — a bound that contains a binder, such as `F: for<'a> Fn(&'a str) -> usize`: it must hold for every lifetime, and each use picks the lifetime it needs. `Fn(&str) -> usize` is already one. (Chapter 9)

Higher-ranked type — a type that binds its own lifetimes, such as the function pointer type `for<'a> fn(&'a str) -> &'a str`. It is a subtype of each of its instances. (Chapters 4 and 9)

`#[may_dangle]` — an unstable, `unsafe` promise on a `Drop` impl that the destructor does not use values of a type parameter, which relaxes the drop check; used by `Box`, `Vec` and `HashMap`. Also called the *drop check eyepatch*. (Chapter 13)

Implied bound — an outlives relation that follows from the types in a signature and need not be written: `&'a T` implies `T: 'a`, and `&'a &'b str` implies `'b: 'a`. (Chapter 1)

Interior mutability — mutation through a shared reference, built on `UnsafeCell`: `Cell`, `RefCell`, `Mutex`, atomics. (Chapter 12)

Invariant — a type constructor is invariant in a parameter if no subtyping passes through it: `&mut T` and `Cell<T>` are invariant in `T`, because they allow writes. (Chapter 4)

Late-bound lifetime — a lifetime parameter that is chosen at each call; a function item is higher-ranked over its late-bound lifetimes. Late-bound lifetimes cannot be given with a turbofish (E0794). (Chapter 9)

Leaking — turning an owned value into a reference that lives until the end of the program by never freeing it (`Box::leak`, `String::leak`). Safe, but the memory is never reclaimed. (Chapters 7 and 13)

Lending iterator — an iterator whose items borrow from the iterator itself (for example, overlapping mutable windows into a buffer). It cannot implement `Iterator`; it can be written with a GAT. Also called a *streaming iterator*. (Chapters 2 and 10)

Lifetime — in the borrow checker's model, a region of a function body; in the source, a name such as `'a` (or an elided position) that stands for one. (Chapter 1)

Lifetime parameter — a lifetime declared on a function, type, trait or `impl` (`<'a>`). On a function, it is chosen by the caller at each call and outlives the whole body. (Chapters 1 and 9)

Lifetime soup — a design in which lifetime parameters were added one at a time to silence errors, so that nobody designed the relationships between them. (Chapter 14)

Live — a borrow is live at a point if the reference, or anything derived from it, may be used later along some path from that point. (Chapter 1)

D.4 N–R

Non-lexical lifetimes (NLL) — the borrow checker introduced with the 2018 edition (Rust 1.31): a borrow's region is computed from the uses of the reference, not from the blocks of the source text. (Chapters 1 and 5)

Miri — an interpreter for Rust's intermediate representation, run on nightly Rust, that detects many kinds of undefined behavior in `unsafe` code. (Chapter 13)

NLL problem case #3 — a pattern that NLL rejects although it is safe: returning a borrow on one path and mutating the borrowed value on another. Polonius accepts it. (Chapter 5)

Opaque type — the type behind `impl Trait` in return position: callers know only the traits it implements and the generic parameters it captures. (Chapter 8)

Outlives — see `'a: 'b` and `T: 'a`.

`Pin` / `Unpin` — `Pin<P>` promises that the value behind the pointer `P` will not move again, which self-referential futures need; `Unpin` marks types for which moving after pinning is harmless. (Chapter 11)

`PhantomData<T>` — a zero-sized marker field that makes a type behave as if it contained a `T`: it uses an otherwise unused lifetime or type parameter and determines the type's variance in it. (Chapters 4 and 13)

Polonius — the location-sensitive reformulation of the borrow checker that accepts programs such as NLL problem case #3. On by default in nightly Rust ("Polonius Alpha") since August 2026; check its current status. (Chapter 5)

Over-capturing — an `impl Trait` return type that captures a lifetime its hidden type never uses, which keeps the corresponding argument borrowed for as long as the result lives; the edition 2024 default can cause it, and `use<..>` fixes it. (Chapter 8)

Precise capturing — see `use<..>`.

Raw pointer — `*const T` or `*mut T`: a pointer with no lifetime and no borrow checking; dereferencing it is `unsafe`. (Chapter 13)

Return type notation — the unstable syntax `T::method(..): Send` for bounding the future returned by a trait's `async fn`. (Chapter 11)

Reborrow — a new reference to the place an existing reference points to (`&mut *r`, or implicitly at a call), with a region that may be shorter. While a mutable reborrow is live, the original reference is frozen. (Chapter 5)

Receiver — the `self` parameter of a method, in any of its forms (`self`, `&self`, `&mut self`, `self: Box<Self>`, `self: Pin<&mut Self>`, ...). (Chapters 2 and 3)

Region — a set of points in the control-flow graph of a function body: the borrow checker's notion of a lifetime. (Chapter 1)

Return-position `impl Trait` (RPIT) — `fn f() -> impl Trait`, which returns an opaque type. In edition 2024 it captures every generic parameter in scope, lifetimes included, unless `use<..>` says otherwise. In traits it is called RPITIT, and it captures every parameter in all editions. (Chapter 8)

D.5 S–Z

Self-referential value — a value that holds a reference to its own data, such as a future that keeps a reference to one of its locals across an `.await`. Safe Rust cannot build or move one without `Pin`; ordinary code stores indices instead. (Chapters 11 and 13)

Scope of a value — from the creation of a value until it is dropped or moved away. A borrow's region must fit inside it. (Chapter 1)

Scope of a variable — the block in which a variable is declared; a syntactic notion that, since NLL, does not determine how long a borrow held by the variable is live. (Chapter 1)

Scoped thread — a thread spawned through `thread::scope`, which is joined before `scope` returns and may therefore borrow from the enclosing function. (Chapter 7)

Scrutinee — the expression a `match`, `if let` or `while let` examines. Temporaries created in it can live longer than you expect. (Chapters 5 and 12)

Soundness — a safe API is sound if no safe code can use it to cause undefined behavior, whatever the arguments. (Chapter 13)

Subtype — `S` is a subtype of `T` if a value of type `S` can be used wherever a `T` is expected. For lifetimes, `&'long T` is a subtype of `&'short T` when `'long: 'short`. (Chapter 4)

Tail expression — the final expression of a block, without a semicolon, which gives the block its value. Edition 2024 drops its temporaries before the block's local variables. (Chapters 5 and 12)

Temporary — an unnamed value that the compiler creates to evaluate an expression, such as the result of a function call that is borrowed immediately. It is dropped at the end of its *temporary scope*, usually the end of the statement. (Chapter 5)

Temporary lifetime extension — a temporary borrowed in certain positions of a `let` initializer (`let r = &make();`) lives until the end of the enclosing block instead of the end of the statement. (Chapter 5)

Two-phase borrow — a mutable borrow that is first *reserved*, acting as a shared borrow, and *activated* only when the call begins; it is why `v.push(v.len())` compiles. (Chapter 5)

Unbounded lifetime — a lifetime that nothing constrains, as when a raw pointer is turned into a reference: it can become any lifetime the caller wants, including `'static`. Bound it with a signature. (Chapter 13)

Universal region — rustc's name for a lifetime parameter of the function being checked: it is chosen outside the function and contains every point of the body. (Chapter 9)

Unsizing coercion — the conversion of a pointer to a concrete type into a pointer to a dynamically sized type, such as `Box<Circle>` to `Box<dyn Shape>` or `&[T; N]` to `&[T]`. When the target is a trait object, the object's lifetime bound must be satisfied by the concrete type. (Chapter 8)

Waker — the handle a pending future uses to ask to be polled again; `Waker::noop()` (Rust 1.85) does nothing and is enough for the book's minimal `block_on`. (Chapters 9 and 11)

Variance — how subtyping between lifetimes (or types) passes through a type constructor: covariant, contravariant or invariant. (Chapter 4)

Well-formed type — a type whose implied requirements hold: `&'a T` is well-formed only if `T: 'a`. The compiler assumes that the types in a signature are well-formed, which is where implied bounds come from. (Chapter 1)

Witness function — a function whose body only returns its argument at a different type, such as `fn f<'s>(x: &'static str) -> &'s str { x }`; it compiles exactly when the subtyping holds. (Chapter 4)

Zero-copy parsing — parsing in which the results (tokens, fields, entries) are slices of the input, so that producing them allocates nothing. (Chapter 14)

APPENDIX E

Further Reading

Every link below was opened and checked on 2026-10-12. Documentation moves, and pages about unstable features change quickly; if a link is broken, search for the title.

E.1 Official documentation

The Rust Programming Language ("the Book"), chapter "Validating References with Lifetimes": the introduction most readers start from. https://doc.rust-lang.org/book/ch10-03-lifetime-syntax.html

The Rust Reference describes the language precisely. The pages most relevant to this book:

- Lifetime elision, including default trait-object lifetimes (Chapters 3, 6 and 8): https://doc.rust-lang.org/reference/lifetime-elision.html — the section on trait-object defaults is known to be inaccurate about precedence (see https://github.com/rust-lang/reference/issues/1407, the open pull request https://github.com/rust-lang/reference/pull/2281 that rewrites it, and Chapter 8).
- Subtyping and variance (Chapter 4): https://doc.rust-lang.org/reference/subtyping.html
- Trait and lifetime bounds, including higher-ranked trait bounds (Chapter 9): https://doc.rust-lang.org/reference/trait-bounds.html#higher-ranked-trait-bounds
- `impl Trait`, including precise capturing with `use<..>` (Chapter 8): https://doc.rust-lang.org/reference/types/impl-trait.html

The Rustonomicon is the guide to `unsafe` Rust, and its chapters on lifetimes are the classic deeper treatment:

- Lifetimes: https://doc.rust-lang.org/nomicon/lifetimes.html
- Subtyping and variance: https://doc.rust-lang.org/nomicon/subtyping.html
- Higher-rank trait bounds: https://doc.rust-lang.org/nomicon/hrtb.html
- Unbounded lifetimes (Chapter 13): https://doc.rust-lang.org/nomicon/unbounded-lifetimes.html

The Edition Guide explains what each edition changed. For edition 2024:

- RPIT lifetime capture rules (Chapter 8): https://doc.rust-lang.org/edition-guide/rust-2024/rpit-lifetime-capture.html
- Tail expression temporary scope (Chapter 5): https://doc.rust-lang.org/edition-guide/rust-2024/temporary-tail-expr-scope.html
- `if let` temporary scope (Chapters 5 and 12): https://doc.rust-lang.org/edition-guide/rust-2024/temporary-if-let-scope.html

The rustc book lists every lint, including `mismatched_lifetime_syntaxes` (Chapter 6): https://doc.rust-lang.org/rustc/lints/listing/warn-by-default.html#mismatched-lifetime-syntaxes and the lint groups such as `rust-2018-idioms` : https://doc.rust-lang.org/rustc/lints/groups.html

The release notes list every change, including the small lifetime fixes mentioned in Chapter 3 (associated consts in 1.99): https://doc.rust-lang.org/stable/releases.html

The error index has an entry for every error code; `rustc --explain E0597` prints the same text locally. For example, E0794 (Chapter 9): https://doc.rust-lang.org/error_codes/E0794.html

E.2 The compiler

The rustc development guide documents how the compiler works. Two of its pages are references for this book:

- Two-phase borrows (Chapter 5): https://rustc-dev-guide.rust-lang.org/borrow-check/two-phase-borrows.html
- Early-bound and late-bound parameters (Chapter 9): https://rustc-dev-guide.rust-lang.org/early-late-parameters.html

The pull request that changed default trait-object lifetimes in Rust 1.98 (Chapter 8): https://github.com/rust-lang/rust/pull/129543

The pull request that refined elision for receivers that contain `Self` (Chapter 3, Rust 1.81): https://github.com/rust-lang/rust/pull/117967

E.3 RFCs

The design documents behind the features in this book:

- RFC 1066, making `mem::forget` safe, the reason `thread::scope` takes a closure (Chapter 13): https://rust-lang.github.io/rfcs/1066-safe-mem-forget.html
- RFC 2094, non-lexical lifetimes, including "Problem case #3" (Chapters 1 and 5): https://rust-lang.github.io/rfcs/2094-nll.html
- RFC 3216, closure lifetime binders (Chapter 9, unstable): https://rust-lang.github.io/rfcs/3216-closure-lifetime-binder.html
- RFC 3498, lifetime capture rules for edition 2024 (Chapter 8): https://rust-lang.github.io/rfcs/3498-lifetime-capture-rules-2024.html
- RFC 3617, precise capturing with `use<..>` (Chapter 8): https://rust-lang.github.io/rfcs/3617-precise-capturing.html

E.4 The Rust blog

- Announcing Rust 1.31 and Rust 2018, the release that brought non-lexical lifetimes (Chapter 1): https://blog.rust-lang.org/2018/12/06/Rust-1.31-and-rust-2018/

- Generic associated types to be stable in Rust 1.65 (Chapter 10): https://blog.rust-lang.org/2022/10/28/gats-stabilization/
- Announcing Rust 1.85.0 and Rust 2024, with async closures and the `AsyncFn` traits (Chapters 9 and 11): https://blog.rust-lang.org/2025/02/20/Rust-1.85.0/
- The longer explanation of async closures and why `F: Fn(&T) -> Fut` cannot express borrowing callbacks: https://blog.rust-lang.org/inside-rust/2024/08/09/async-closures-call-for-testing/
- Enabling Polonius Alpha on nightly (Chapter 5): https://blog.rust-lang.org/2026/08/04/enabling-polonius-alpha-on-nightly/
- Enabling the next-generation trait solver on nightly: https://blog.rust-lang.org/2026/08/21/enabling-next-solver-on-nightly/

E.5 Unstable features mentioned in the book

Tracking issues are the place to check the current status:

- `closure_lifetime_binder`: https://github.com/rust-lang/rust/issues/97362
- `non_lifetime_binders`: https://github.com/rust-lang/rust/issues/108185
- `return_type_notation`: https://github.com/rust-lang/rust/issues/109417
- Polonius: https://github.com/rust-lang/rust/issues/160456
- `dropck_eyepatch` (`#[may_dangle]`): https://github.com/rust-lang/rust/issues/34761
- Defaults for `where Self: 'a` on GATs: https://github.com/rust-lang/rust/issues/87479
- The 2026 project goal on region constraints (bounds on higher-ranked lifetimes, implied bounds): https://goals.rust-lang.org/2026/next-gen-region-constraints.html

E.6 Beyond the official documentation

- serde, "Understanding deserializer lifetimes": the `Deserialize<'de>` / `DeserializeOwned` design of Chapter 9 explained by its authors. https://serde.rs/lifetimes.html
- The documentation of tokio's `spawn`, a typical executor signature (`F: Future + Send + 'static`; Chapter 11): https://docs.rs/tokio/latest/tokio/task/fn.spawn.html
- Miri, the undefined-behavior detector for `unsafe` code (Chapter 13): https://github.com/rust-lang/miri
- Sabrina Jewson, "The Better Alternative to Lifetime GATs": the helper-trait pattern of Chapter 10. https://sabrinajewson.org/blog/the-better-alternative-to-lifetime-gats